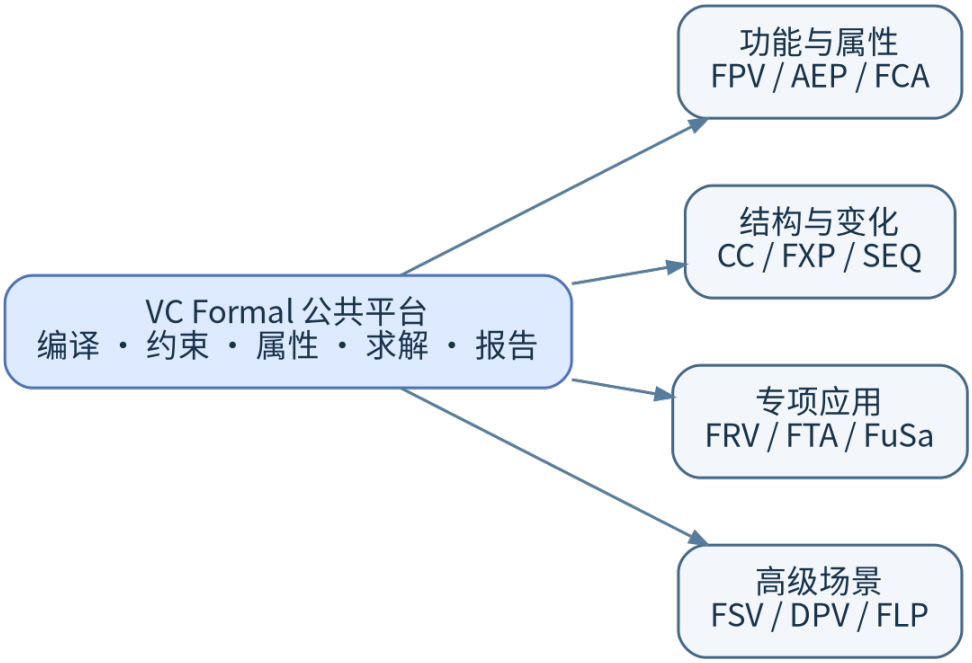
VC Formal

中文形式验证手册

原理 · SVA · 命令详解 · 完整案例



文档基线 X-2025.06-SP1

手册修订 1.3 · 2026 年 10 月

非官方中文教程；工具执行结果须在授权环境中取得

目录

阅读指南 **1**
1. 内容组织 . . . 1
2. 分篇目录 . . . 1
3. 代码与语法的读法 . . . 2
4. 资料基线和验证状态 . . . 2
5. 阅读格式 . . . 2

形式验证：从数学原理到芯片工程 **3**
1. 为什么需要形式验证 . . . 3
2. 有限状态模型：形式验证究竟检查什么 . . . 3
3. 工具如何找到反例或完成证明 . . . 4
4. 断言、假设与覆盖目标 . . . 5
5. 结果的工程含义 . . . 5
6. 与仿真、静态检查、ATPG、等价检查的区别 . . . 6
7. 一个完整的思考例子：仲裁器 . . . 6
8. 从规格到验证的推荐过程 . . . 7
9. 初学者常见误解 . . . 7
10. 本手册学习索引与参考依据 . . . 7

VC Formal 平台、环境与命令执行 **8**
1. 平台定位与基本结构 . . . 8
2. 应用名称与 Tcl 模式的区别 . . . 8
3. 环境准备 . . . 9
4. Tcl 外壳与帮助系统 . . . 11
5. 读入、编译与展开 RTL . . . 11
6. 时钟与复位定义 . . . 12
7. 检查准备状态 . . . 12
8. 属性管理、运行、输出 . . . 13
9. 计算资源与回归 . . . 13
10. 会话、图形调试与工作区 . . . 13
11. 命令出现错误时的检查顺序 . . . 13
12. 官方资料索引 . . . 14

形式验证工程组织与结果判读 **15**
1. 从设计需求建立验证对象 . . . 15
2. 推荐项目目录 . . . 15
3. 完整执行顺序 . . . 15
4. 读懂反例：逐周期定位 . . . 16

5. 证明状态与报告编写 . . . 16
6. 时钟、复位与环境建模审查 . . . 16
7. 小型设计推荐的脚本骨架 . . . 17
8. 状态回归和证明性能 . . . 19
9. 验收条件建议 . . . 19
10. 参考 . . . 19

SystemVerilog Assertions（SVA）完整中文教程 **20**
1. SVA 解决什么问题 . . . 20
2. 先理解三种角色 . . . 20
3. 并发断言的时钟、采样与失效条件 . . . 20
4. 重叠蕴含与非重叠蕴含 . . . 21
5. sequence 和 property 的职责 . . . 22
6. 序列连接与延迟运算符 . . . 22
7. 序列的组合 . . . 23
8. 属性运算符：时间上的持续和终止 . . . 24
9. 常用采样函数 . . . 24
10. 立即断言和并发断言的区别 . . . 25
11. 属性局部变量、`let` 和参数化 . . . 25
12. `bind`：无需改动设计源码注入 SVA . . . 26
13. 常见系统级断言模板 . . . 26
14. 空洞性、覆盖目标与不恰当假设 . . . 27
15. 常见错误及修正方法 . . . 28
16. 多时钟属性、X 值与复杂语法的限制 . . . 29
17. 完整示例：有界计数器及 VC Formal 工程 . . . 29
18. 工程练习（建议自行运行与审查） . . . 31
19. 语法检索与选择 . . . 31
20. 参考与责任界定 . . . 32

VC Formal 命令分类与通用命令详解 **33**
1. 怎样阅读命令语法 . . . 33
2. 按工作阶段选择命令 . . . 33
3. 命令语法、参数与用例 . . . 34
4. 命令语法、参数与用例 . . . 44
5. 完整计数器工程 . . . 52

FPV：功能属性验证应用使用指南 **54**
1. 定位、能力与不适用范围 . . . 54
2. 标准流程与输入输出 . . . 54
3. 单时钟计数器：完整 RTL、SVA 与运行流程 . . . 54
4. 如何组织断言 . . . 57
5. 属性与运行命令详解 . . . 58

6. 四个逐步进阶的用例 . . . 61
7. 如何分析结果 . . . 63
8. 优化证明效率而不损害结论 . . . 63
9. 常见故障与排查 . . . 64
10. 进阶主题 . . . 64
11. 参考定位 . . . 64

AEP：自动提取属性应用使用指南 65
1. AEP 的角色与应用价值 . . . 65
2. 检查对象与生成条件 . . . 65
3. 入门 AEP 案例：数组越界 . . . 66
4. 标准运行步骤 . . . 66
5. 状态机检查：死锁、活锁与公平性 . . . 67
6. 命令语法、参数与用例 . . . 67
7. 常见问题与处理方案 . . . 70
8. 推荐的 AEP + FPV 双阶段工作方法 . . . 70
9. 官方参考 . . . 70

FXP：形式未知值传播验证使用指南 71
1. 为什么未知值传播值得单独验证 . . . 71
2. 注入点与观察点的概念 . . . 71
3. 自动提取对象：来源与观察点 . . . 71
4. 标准脚本与运行顺序 . . . 72
5. 命令语法、参数与用例 . . . 72
6. 教学用例 A：未复位的数据寄存器 . . . 74
7. 教学用例 B：显式 X 赋值与越界 . . . 75
8. 教学用例 C：有意屏蔽不确定数据 . . . 76
9. 调试：由下游症状回溯到上游来源 . . . 76
10. 设计规模很大时怎么做 . . . 76
11. 常见问题 . . . 76
12. FXP 与 AEP、FPV 的分工 . . . 77
13. 官方依据与版本提醒 . . . 77

CC：形式连通性检查使用指南 78
1. 这项应用解决的问题 . . . 78
2. 连通性规格包含哪些信息 . . . 78
3. 最小实例：多路选择器 . . . 79
4. CSV 导入：大规模 SoC 的推荐方法 . . . 79
5. 源目的别名与设计层级 . . . 80
6. 命令语法、参数与用例 . . . 80
7. 三种常见用例 . . . 83
8. 失败分类与排查 . . . 84

9. 对大型项目的建议 . . . 84
10. 参考 . . . 84

FCA：形式覆盖率分析使用指南 85
1. FCA 与 FPV 的区别 . . . 85
2. 覆盖对象与每种结果的含义 . . . 85
3. 第一个 FCA 用例：输入条件与代码分支可达性 . . . 86
4. 主要流程：完整 Tcl . . . 86
5. 从仿真数据库开始 . . . 87
6. 命令语法、参数与用例 . . . 87
7. Auto Scale 及资源策略 . . . 90
8. 排除文件需要审查 . . . 90
9. 常见排查 . . . 90
10. 命名差异和官方参考 . . . 90

SEQ：顺序等价检查使用指南 91
1. 为什么不能只比较组合逻辑 . . . 91
2. 初学者例子：两种计数器写法 . . . 91
3. 编译两个独立设计库并运行 . . . 92
4. 对应关系是什么 . . . 93
5. 顺序优化类型与对应关系 . . . 93
6. 命令语法、参数与用例 . . . 93
7. 如何解释不等价结果 . . . 97
8. 高级优化策略 . . . 97
9. 教学练习 . . . 97
10. 参考 . . . 98

FRV：形式寄存器验证使用指南 99
1. 典型目标和输入资料 . . . 99
2. 核心概念 . . . 99
3. 寄存器规格与 RTL 的对照 . . . 101
4. 一个最小脚本模板 . . . 103
5. 命令语法、参数与用例 . . . 104
6. 结果分析的方法 . . . 106
7. 与 UVM RAL、仿真和 FPGA/SoC 寄存器校验的配合 . . . 106
8. 工程清单 . . . 107
9. 参考资料 . . . 107

FTA：形式测试平台分析使用指南 108
1. FTA 解决的不是 RTL 正确性，而是属性环境的质量 . . . 108
2. 工作过程 . . . 108
3. 不能直接从 `-fmode FTA` 启动 . . . 108
4. 示范 Tcl：多阶段流程的轮廓 . . . 108

5. 故障分类与结果含义 . . . 109
6. 命令语法、参数与用例 . . . 109
7. 两个评估示例 . . . 113
8. FTA 与故障仿真的区别 . . . 116
9. 参考 . . . 116

FuSa：功能安全形式分析使用指南 117
1. 场景与关键概念 . . . 117
2. 前置要求和故障输入 . . . 117
3. 四类分析 . . . 117
4. 标准脚本结构 . . . 117
5. 逐步提高分析精度 . . . 118
6. 命令语法、参数与用例 . . . 118
7. 案例：带奇偶校验的安全寄存器 . . . 122
8. 与 ISO 26262 / FMEDA 的关系 . . . 124
9. 常见错误 . . . 124
10. 官方资料 . . . 124

FSV：形式安全信息流验证使用指南 125
1. 目标：检查本不应出现的信息传播 . . . 125
2. 什么是信息流与非干扰 . . . 125
3. 典型命令结构 . . . 125
4. 可控的允许传播 . . . 126
5. 案例：保护调试输出的完整工程 . . . 126
6. 信息流安全与普通功能断言相互补充 . . . 128
7. 黑盒与抽象的风险 . . . 128
8. 命令语法、参数与用例 . . . 128
9. 常见错误及处理 . . . 129
10. 官方参考 . . . 130

DPV：数据路径验证与 C/C++—RTL 事务等价使用指南 131
1. 解决的问题与 SEQ 的区别 . . . 131
2. 典型应用示例：8 位加法器 . . . 131
3. C++ 事务接口的结构 . . . 131
4. 创建双设计环境 . . . 132
5. 通过阶段对应建立事务关系 . . . 134
6. 证明与报告 . . . 136
7. 完整可验证事务的六个核对项 . . . 136
8. 命令语法、参数与用例 . . . 136
9. 数据路径证明困难时的策略 . . . 141
10. 官方参考 . . . 142

FLP / LPC：低功耗连通性形式验证使用指南 143

1. 核心问题 . . . 143
2. 重要对象 . . . 143
3. 该版本的工作模式 . . . 143
4. 最小系统案例 . . . 144
5. UPF 可信性与 PMC . . . 146
6. FLP CC 和普通 CC 的重要区别 . . . 146
7. 命令语法、参数与用例 . . . 146
8. 三个需要分别检查的场景 . . . 149
9. 调试方法 . . . 149
10. 参考 . . . 149

设计查询、对象集合与属性筛选 **150**
1. 查询对象不是修改 RTL . . . 150
2. 名称、集合与 Tcl 列表 . . . 150
3. 命令语法、参数与用例 . . . 150
4. 将查询结果用于验证范围审查 . . . 157

Assertion IP 与分解证明 **158**
1. 两类复用方式 . . . 158
2. 命令语法、参数与用例 . . . 158
3. 组合证明的逻辑要求 . . . 158
4. 命令语法、参数与用例 . . . 159

反例调试、证明性能与复杂性控制 **161**
1. 为什么形式验证会困难 . . . 161
2. 报告状态决定不同的调试路径 . . . 161
3. 失败属性：逐周期回溯 . . . 162
4. 约束检查：最应优先防止的错误 . . . 162
5. 查找状态空间问题 . . . 162
6. 改善证明性能的合理顺序 . . . 162
7. `fvtask`：按性质组织工作 . . . 163
8. 抽象及裁剪驱动器 . . . 163
9. 困难证明的辅助分析 . . . 163
10. Proof Architect：组合式证明思路 . . . 163
11. 有规律地建立证明性能实验 . . . 164
12. 命令语法、参数与用例 . . . 164
13. 官方参考 . . . 171

覆盖率评估、验证质量与结果验收 **172**
1. 为什么不能只报告“全部属性通过” . . . 172
2. 建立规格—属性对应表 . . . 172
3. 验收需要保留哪些文件 . . . 172
4. 对“Proven”的独立复核 . . . 173

5. 不完成或豁免的处理要求 173
6. 组合使用 FCA 与 FTA 173
7. 质量评审问题 173
8. 自动化结果摘要 173
9. 命令语法、参数与用例 174
10. 参考 175

术语表、应用选择与官方资料定位 **176**
1. 术语表 176
2. 遇到问题先选哪个应用 177
3. 官方 UG 章节与页码 177
4. 确认过的重要版本差异 178
5. 文档范围与版权说明 178

阅读指南

本手册从形式验证原理开始，依次介绍 VC Formal 环境、SVA、通用命令及专用应用。软件文档基线为 **X-2025.06-SP1**，手册修订为 **1.3**。短条目用汇总表比较，使用条件和原理在表后展开；含竖线的语法放在代码块中，完整案例源码直接放在各篇正文。

1. 内容组织

阅读阶段	内容	建议目标
原理与模型	形式验证、平台、工程与结果	理解证明在什么前提下成立
语言与操作	SVA、通用命令	独立建立一个完整验证工程
专用应用	FPV 至 FLP / LPC	按具体问题选择输入、目标和流程
查询与进阶	集合、AIP、分解、调试和验收	审查范围、分析困难和保存证据

第一次使用时，先读原理、平台和 SVA，再完成计数器正确版与错误版的对照。已有仿真经验的读者应重点复核 SVA 采样、历史值和复位处理，而不是直接把仿真激励变成形式假设。SoC 集成场景可在通用基础之后按需阅读 CC、FRV、FXP 和低功耗章节。

2. 分篇目录

文章	内容
形式验证	状态模型、证明方法、断言责任和结论限制
VC Formal 平台与环境	应用模式、编译、时钟复位和初始化
验证工程与结果判读	规格分解、工程目录、反例和状态解释
SVA 语法合集	时序语法、采样函数、完整检查器与常见错误
通用命令详解	按阶段分类的启动、环境、属性和运行命令
FPV	手写属性、计数器与 FIFO 完整案例
AEP	自动规则、生成条件、状态机检查及豁免
FXP	未知值来源、观察点、传播与定位
CC	连接规则、CSV、别名和连通性覆盖
FCA	覆盖对象、数据库、不可达分析和排除结果
SEQ	双设计编译、对应关系和不等价调试
FRV	寄存器规格、自动断言、访问覆盖和桥接
FTA	属性检错能力、故障配置和数据库
FuSa	故障激活、功能观察、安全检测和结果保存
FSV	信息来源、目标、允许传播条件及黑盒风险
DPV	参考计算、事务阶段、设计组合和证明运行
FLP LPC	电源意图、低功耗连通性与隔离钳位
设计查询与集合	对象类别、查询、集合运算和属性筛选
AIP 与分解证明	协议检查复用与子证明依赖
调试与证明性能	约束冲突、反例、复杂度、抽象和资源
覆盖与验证验收	规格覆盖、形式核心和交付证据
术语表与资料索引	中英文概念和官方资料页码

每篇应用文章把用途、输入、执行顺序、命令定义和案例放在一起。共享的启动、编译、任务和资源控制在通用命令章集中详解，FPV 章另保留核心属性命令全文。需要按命令名检索时，可直接使用命令逐项详解，其中汇集 200 个命令及应用用法条目；同名命令在不同应用中可能具有不同参数，不能跨应用直接套用。

3. 代码与语法的读法

标有“保存为”的代码框给出完整文件内容，连同文件列表、绑定文件和 Tcl 运行步骤可直接组成对应案例。全书 16 篇文章包含 81 处完整源码框，覆盖 58 份源码和配置；同一案例在不同独立文章中出现时，必要代码重复列出，便于单篇阅读。独立工程目录只是可编辑副本，不是阅读正文的前置条件。

命令的“语法”使用尖括号表示应替换参数、方括号表示可选项、竖线表示可选值。它不是待原样执行的脚本；随后给出的示例才是具体调用形式。SVA 运算符的竖线则是语言本身的一部分，不能删掉或加反斜杠。所有含这类字符的完整语法均保留在代码块中。

4. 资料基线和验证状态

命令解释依据所提供的 **VC Formal User Guide**（1,168 页）及 **VC Formal Quick Reference Guide**（34 页），各条目保留来源页码。两份文件都对应 X-2025.06-SP1，但存在少数命令名称或参数拼写差异；相应条目说明采用哪一种定义及其限制。UG 将部分命令的完整参数转交给 VC Static Platform 或 DPV 专用手册时，本书只解释已提供资料中能够核实的用法，不自行补造参数。

当前环境没有可授权执行的 VC Formal，**没有运行工具编译或形式证明**。本文的预期结果、反例分析与代码检查不能替代真实工具报告。需要寄存器规格、SFF/FDB、HECTOR 包装或 UPF 的专用流程保留明确的配置要求；配套 FPV 练习不冒充相应原生应用的证明。

5. 阅读格式

Markdown 合订本便于搜索和编辑；离线 HTML内嵌图片和样式；PDF 合订本用于连续阅读和打印。插图提供 SVG、PNG 与 DOT 工程。原始参考 PDF 不重复打包，资料权利归其原作者和权利人；本手册不是 Synopsys 官方译本。

形式验证：从数学原理到芯片工程

1. 为什么需要形式验证

仿真回答“设计在**本次输入激励**下如何运行”；形式验证尝试回答“对于**满足约束的所有合法输入行为**，设计是否始终符合指定规则”。两者并不互相替代。仿真善于运行长时间系统场景、软件交互和模拟接口；形式验证善于证明局部控制逻辑、协议约束、寄存器访问以及难以用随机激励命中的组合条件。工具搜索到一个能够违反规则的输入与状态序列时，会给出反例；如果建立了有效的数学证明，则给出该属性在所建模型和假设条件下成立的结论。

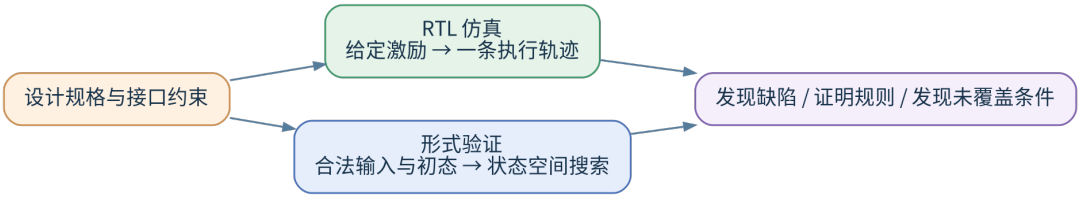


图 1: 形式验证与仿真的关系

典型问题包括：先进先出缓冲区是否发生上溢和下溢；多路仲裁器能否同时授权两个请求方；复位后控制状态是否正确；握手请求能否在指定期限得到响应；中断是否会在清除之后错误保持；重构后的控制逻辑是否与原设计一致；安全数据能否流向公开端口；未知值是否泄漏到关键控制输出。

不适合单独依赖形式验证的问题包括：没有可形式化规格说明的复杂系统行为、第三方模拟器件的模拟特性、未经正确建模的软件运行环境，以及大量没有抽象处理的存储阵列或复杂数据处理导致的计算难题。形式验证的结论取决于模型、约束、属性和工具求解状态。

2. 有限状态模型：形式验证究竟检查什么

把一个同步数字模块抽象为状态转移系统：

$$\mathcal{M} = (S, I, T, S_0), \qquad s_{t+1} = T(s_t, i_t)$$

其中，S 为所有寄存器状态的集合，I 为可用输入，T 为当前状态与输入决定下一状态的逻辑，S_0 为**允许的初始状态**。一个安全性属性可以写成：

$$\forall t \geq 0,\ \forall (s_0, i_0, \ldots, i_t) \in \text{LegalTraces}, \quad P(s_t, i_t) = 1.$$

这里的 `LegalTraces` 极其重要，它由复位初始条件、输入协议假设、时钟关系和外部环境模型共同限定。属性被证明，不代表“在任何想象中的物理条件下都正确”，只代表**在被建模的所有合法执行轨迹中**成立。如果约束不合理地排除了错误场景，证明结果就可能失去工程价值。

2.1 安全性与活性

类别	要表达的规则	示例	常见求解关注点
安全性（Safety）	坏事情永不发生	缓冲区满时不得额外写入	是否存在有限长度反例

类别	要表达的规则	示例	常见求解关注点
有界响应	事件必须在有限窗口响应	请求发生后 1 ~ 3 周期出现应答	计数范围、复位与连续请求
活性（Liveness）	好事情终将发生	请求最终被服务	公平性、无限执行、死锁和活锁
可达性（Reachability）	某状态是否能够到达	仲裁状态是否被访问	覆盖证明或可达见证
等价性（Equivalence）	两个系统的可观测行为一致	改写后的设计与参考设计一致	输入、状态与延迟关系

安全性通常可以由一段有限反例推翻，例如在某一拍出现非法写入；活性失败则可能表现为无限等待，需要考虑环境是否最终允许前进。有界响应把完成期限写入规格，更便于得到有限反例，但不能为降低求解难度而擅自把无期限要求改成任意的周期上限。

有界响应往往容易写成安全性检查；无限期“最终应答”涉及环境是否持续提供条件。例如，如果从设备永远可以拒绝服务，那么主设备无法保证最终完成。不要用无法由本模块控制的假设来证明本模块“有活性”。

2.2 关于初始状态与复位

未经初始化的寄存器在形式模型中可能代表任意初始取值。`create_reset` 用于声明复位信号及其极性，`sim_run`、`sim_save_reset` 等命令用于建立与保存可供证明使用的初始化状态，但必须确认复位真正驱动到目标寄存器，而不能假定声明命令自然令所有寄存器变为零。复位后的第一拍、同步/异步复位、部分复位寄存器需要区别处理。参见《VC Formal User Guide》§4.11 ~ 4.13（约第 132 ~ 154 页）。

3. 工具如何找到反例或完成证明

3.1 可满足性求解与有界模型检查

对于最大展开深度 k，可构造：

$$S_0(s_0) \wedge \bigwedge_{t=0}^{k-1} T(s_t, i_t, s_{t+1}) \wedge \bigvee_{t=0}^{k} \neg P(s_t, i_t).$$

若可满足，满足赋值对应一条反例；若不可满足，仅能表明在该展开深度内未找到违反，**不能由此推出无限深度已证明**。有界模型检查（Bounded Model Checking，BMC）常适合较短缺陷路径。

3.2 数学归纳与增强不变量

归纳证明要求初始时刻性质成立，并证明“假设性质对当前状态成立，则对下一状态也成立”。简单归纳可能因为包含不可达状态而失败，这时可引入更强的不变量（例如缓冲区占用计数与读写指针之间的关系）。归纳深度增加不必然解决证明困难；关键在于性质与状态约束是否充分。

3.3 属性引导可达性与其他证明引擎

工业工具可结合 SAT/SMT、归纳、插值、PDR/IC3、抽象及问题分解等多种方法；不同属性和设计结构的性能差异很大。VC Formal 的实际引擎组合与配置由软件版本、应用模式、许可证和资源控制决定，不应把某个求解方法当作该工具唯一的实现。

3.4 抽象、分解与可能的误报

只保留某个属性关心的逻辑集合可降低求解规模，这被称为影响锥（Cone of Influence，COI）约简。进一步将乘法器、存储器或内部网络抽象成不完全受原逻辑约束的变量，通常形成更宽泛的行为集合，因此可能产生原始设计并不可能发生的反例。遇到这种反例应检查被抽象的信号与约束，而不是直接修改 RTL。



图 2: 从设计与属性到证明结论

4. 断言、假设与覆盖目标

角色	表达的责任	结果的含义
断言	设计必须满足的规则	证明所有允许执行都符合规则，或提供反例
假设	环境保证的合法行为	限制引擎允许搜索的行为集合
覆盖	希望确认可发生的场景	找到至少一条见证路径，或分析其不可达性

三种角色可以使用相近的 SVA 语法，但逻辑责任完全不同。把断言改成假设并不修复设计，而是排除所有违反该规则的执行；把断言改成覆盖，则只要求某一条执行符合规则，不能继续声称对全部行为进行了证明。

```
// 外部协议确实要求请求期间输入数据有效，才可作为环境前提。
as_request_data: assume property (
    @(posedge clk) disable iff (!rst_n) req |-> valid_input);
// 队列占用计数是设计自身责任，不能把这条关系改成 assume。
ap_occupancy: assert property (
    @(posedge clk) disable iff (!rst_n) count <= DEPTH);
// 确认当前环境允许请求真正出现。
cp_request: cover property (
    @(posedge clk) disable iff (!rst_n) req);
```

如果假设请求永不出现，所有以请求为前件的响应规则都可能空通过。触发覆盖能够揭示这种问题，但覆盖成功仍不足以证明输入假设全部合理：还需要审查请求重叠、背压、错误输入和运行中复位等其他合法情况。有效验证要同时保持真实环境允许的行为，并排除真实环境禁止的行为。

5. 结果的工程含义

状态	能得出的结论	不能得出的结论	后续动作
Proven	对当前模型、初始状态和假设已经证明	系统级规格自动正确	审查属性范围、前提和空洞性
Falsified	存在可触发该属性失败的候选反例	一定存在芯片功能缺陷	检查反例、假设、复位和抽象
Inconclusive	在当前资源、策略或时间内尚无明确结论	该属性正确或错误	缩小问题、调优、延长资源
Covered	找到了满足覆盖目标的路径	设计全部功能受到验证	检查路径是否真实且有意义
Uncoverable	在当前模型和约束下不可到达	目标在真实系统里也不可到达	检查约束与复位、用例定义
Not run / Disabled	属性未被验证	属性已通过验证	检查启用列表和任务配置

证明状态只对本次构造的数学模型有效。判断结果时，应把设计版本、初始状态、输入前提、抽象设置与属性正文一起看。特别是覆盖目标的不可达，可能是设计固有行为，也可能是环境约束限制了场景；未完成则表示现有时间和资源还不足以确定结果，不能归入已证明或已失败。

状态名称依应用、版本及报告格式略有变化。不能把“检查结束”、“没有失败条目”、“达到某个深度”直接写成“全部已证明”。工具输出还可能包括 `checking`、`unknown`、`waived` 等管理状态。参见 User Guide §7.1（第 247 页起）。

6. 与仿真、静态检查、ATPG、等价检查的区别

技术	主要目标	典型输入	典型输出
RTL 仿真	在给定激励下观察行为	Testbench、激励、RTL	波形、断言失败、覆盖率
规则检查/静态检查	查找结构与编码问题	RTL、规则集	报错或告警
属性形式验证	证明逻辑规则或找反例	RTL、SVA、形式约束	证明、反例、未完成
等价检查	比较两个版本	参考与实现设计、对应关系	等价或差异证据
ATPG	为制造缺陷模型生成测试向量	网表、故障模型、扫描结构	测试图样、故障覆盖率
MBIST	在存储器上运行算法发现制造缺陷	存储宏、控制器、March 测试	测试结果、故障定位信息

这些方法相互补充而不是彼此取代。仿真便于检查复杂软件场景和真实激励，形式验证擅长系统地寻找属性反例，等价检查适合验证实现变换前后的一致性，制造测试向量则针对制造故障模型。选择方法时先说明要排除的错误类型，再决定所需输入和接受条件。

特别说明：形式验证与 ATPG 都使用逻辑求解技术，但其目标并不相同。RTL 断言证明不能替代制造测试；ATPG 高故障覆盖率也不能证明功能协议正确。

7. 一个完整的思考例子：仲裁器

设计：两个请求 `req[1:0]`，输出授权 `gnt[1:0]`。初步规格：

1. 同一时刻最多有一个授权；
2. 未收到请求的端口不得得到授权；
3. 在持续请求且另一端口有公平竞争条件时，请求不会无限期饥饿；
4. 复位之后授权清零。

最先写的属性通常是：

```
ap_no_double_grant: assert property (
  @(posedge clk) disable iff (!rst_n) $onehot0(gnt));
ap_grant_requires_request: assert property (
  @(posedge clk) disable iff (!rst_n) ((gnt & ~req) == 2'b00));
cp_contention: cover property (
  @(posedge clk) disable iff (!rst_n) req == 2'b11);
```

以上第二条**只适用于同周期请求—授权协议**；如果授权允许滞后一拍，则应与 `$past(req)` 或挂起请求记录相匹配。第三条用于防止所有验证都发生在“根本没有竞争”的场景。公平性属性必须考虑仲裁策略、请求是否持续、时钟是否运行以及端口是否能响应，因此不应脱离协议就写出无条件的 `eventually`。

8. 从规格到验证的推荐过程

1. 阅读接口规格，列出模块能保证的行为、外部必须保证的条件，以及排除的非法使用方式。
2. 把规则分成复位、功能安全性、进展性、交叉接口、异常处理和参数化条件。
3. 每条规则编写可阅读的自然语言解释、SVA 表达式、触发示例与应失败的反例。
4. 建立时钟、复位、初始状态和必要的输入假设，不要提前把内部状态约束成预期结果。
5. 检查属性是否生效，并利用覆盖目标验证关键触发场景可达。
6. 证明安全性；记录未完成的原因；修正设计后回归全部属性。
7. 以版本化的 RTL、SVA、Tcl、日志、结果报告和豁免说明组成可复现的结果包。

详见 验证工程与结果判读。

9. 初学者常见误解

“我有 100% 代码覆盖率，所以所有功能都正确。” 代码覆盖率描述哪些代码结构被执行，不等同于正确性。**“模型只要能跑出 Proven 就代表覆盖完整。”** 属性可能遗漏关键规格或触发条件不可达。**“形式验证不需要 Testbench。”** 虽然不必使用传统仿真激励，仍需通过假设、状态初始化和环境模型定义合法行为。**“加入假设后证明更快一定是好事。”** 如果假设屏蔽关键场景，速度提高反而意味着可信度下降。**“每个未完成都表示工具有问题。”** 更多时候是属性范围、运算规模、状态空间或外部环境模型仍需优化。

10. 本手册学习索引与参考依据

- SVA 系统教材：SVA 语法合集.md。
- 工具共性：VC_Formal_ 平台与环境.md、验证工程与结果判读.md。
- 首次动手：FPV.md、案例工程/README.md。
- 实战进阶：调试与证明性能.md、覆盖与验证验收.md。
- 官方版本参考：用户提供的 VC Formal User Guide, X-2025.06-SP1，§1、§3、§4、§6、§7、§17、§18；以及 VC Formal Quick Reference Guide 同版本。版权属于 Synopsys，本文为独立撰写的中文学习教程，不是官方文件的逐页翻译。

VC Formal 平台、环境与命令执行

1. 平台定位与基本结构

VC Formal 是 Synopsys 的形式验证平台，以共同的编译、展开、属性管理、求解、报告及调试环境提供不同应用。它支持以传统 RTL 与约束出发的功能属性验证，也支持自动提取问题、基于结构连接列表的检查、顺序等价分析、安全性分析等专用任务。

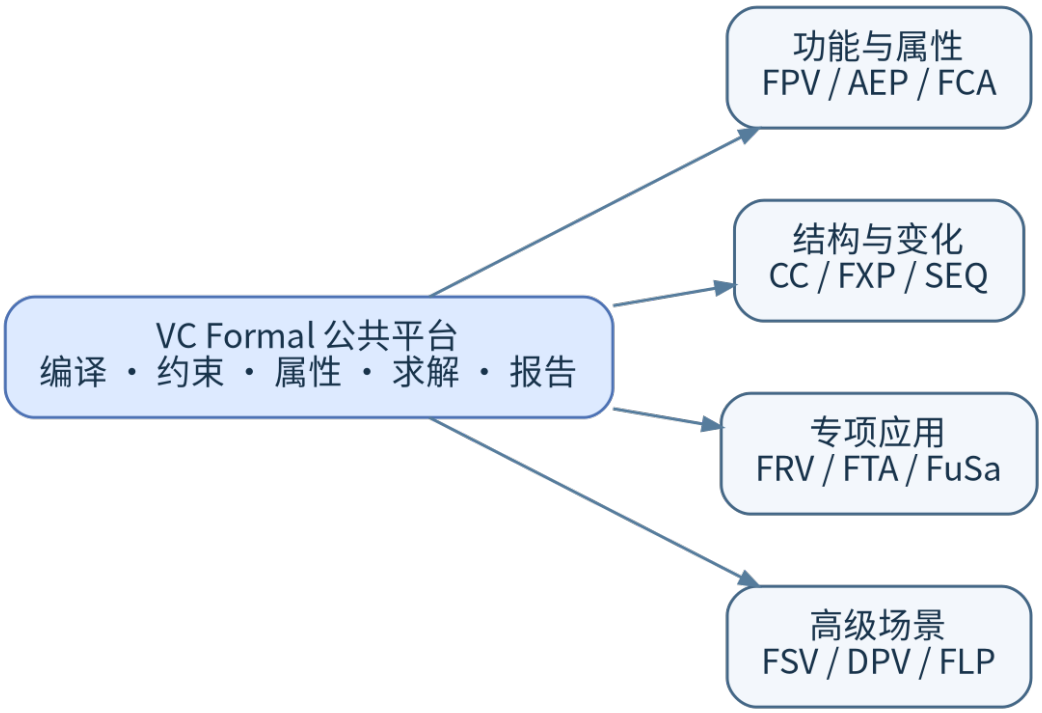


图 3: VC Formal 应用布局

一个验证任务一般由六个部分构成：RTL 或参考模型；SVA/CSV/寄存器描述/故障模型等规格输入；时钟复位与环境假设；证明引擎与资源策略；反例及结果报告；必要时的 Verdi 图形调试。不同应用共享 Tcl 外壳，但**应用模式、专业命令与输入数据类型并不统一**。

2. 应用名称与 Tcl 模式的区别

中文用途	文档中的应用名称	Tcl 模式	专用资料
功能属性证明	FPV	`FPV`	FPV.md
自动提取设计检查	AEP	`AEP`	AEP.md
连通性	CC	`CC`	CC.md
形式覆盖率分析	FCA	`COV`	FCA.md
未知值传播	FXP	`FXP`	FXP.md
顺序等价	SEQ	`SEQ`	SEQ.md
寄存器验证	FRV	`FRV`	FRV.md

中文用途	文档中的应用名称	Tcl 模式	专用资料
测试平台分析	FTA	`FTA`（通常由 FPV 会话切换）	FTA.md
功能安全	FuSa	`FUSA`	FuSa.md
信息流安全	FSV	`FSV`	FSV.md
数据路径对照	DPV	`DPV`	DPV.md
低功耗连通性	FLP/LPC	`CC` + 低功耗命令（该使用流程）	FLP_LPC.md

2025 年 9 月快速指南的 FCA 命令行示例使用 `set_fml_appmode FCA`，但同版本 1,168 页 User Guide 在第 65、395、430 页明确展示 `COV`。本教程的 FCA 脚本以 `COV` 为首选，启动后通过 `get_fml_appmode` 确认。FTA 模式在 User Guide 第 65 页明确指出不应直接使用 `vcf -fmode FTA` 启动，而应在 VC Formal 会话内用 `set_fml_appmode FTA`。这两处必须与实际安装版本核对。

3. 环境准备

正式环境通常需要 Linux 主机、已安装的 VC Formal / VC Static 相关组件、相应许可证，以及可用的 Verdi（仅在图形调试需要时）。具体支持的平台、许可证名称、最低内存与批量计算系统应以单位内部安装说明及实际版本的安装指南为准，不能从本文推断许可证一定已开通。

首先在终端确认：

```
which vcf
vcf -help
pwd
```

工具可以从 `-f` 读取 Tcl 脚本。推荐从**项目根目录**执行，以使源文件列表和日志位置一致：

```
vcf -f scripts/run_fpv.tcl -batch
vcf -f scripts/run_fpv.tcl -verdi
vcf -f scripts/run_fpv.tcl
```

典型授权选项示例：`-fml_elite`、`-fml_token`；只有在内部许可证策略支持且确实有需要时再使用。实际提示和参数通过 `vcf -help` 检查，不要把 shell 指令原样放入 Tcl 脚本。

3.1 首次运行所需的全部文件

环境准备后，可以先保存以下最小工程。本节给出 RTL、检查器、文件列表和 Tcl 的完整内容。后续章节再解释这些命令和属性的细节。

3.1.1 被验证的计数器 RTL

规格：复位后计数为零；使能时从 0 计到 9 后回到 0；没有使能时保持。RTL 文件只包含设计，不把待验证结论写成环境假设。

保存为 案例工程/rtl/counter.sv：

```
module counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       enable,
    output logic [3:0] count
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)             count <= 4'd0;
    else if (enable) begin
      if (count == 4'd9)    count <= 4'd0;
```

```
      else                  count <= count + 4'd1;
    end
  end
endmodule
```

3.1.2 完整断言与绑定文件

history_valid 在复位后建立历史有效标志。ap_range 检查范围，ap_hold 检查保持，ap_increment 检查普通递增，ap_wrap 检查回绕；两条 cover 检查最高计数值及回绕场景是否能够出现。末尾 bind 将检查器接入同名设计信号，不需要修改 RTL。

保存为 案例工程/formal/counter_props.sv：

```
module counter_props (
    input logic       clk, rst_n, enable,
    input logic [3:0] count
);
  logic history_valid;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) history_valid <= 1'b0;
    else        history_valid <= 1'b1;

  ap_range: assert property (@(posedge clk) disable iff (!rst_n)
      count <= 4'd9);

  ap_hold: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(!enable) |-> count == $past(count));

  ap_increment: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count < 4'd9)
      |-> count == $past(count) + 4'd1);

  ap_wrap: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count == 4'd9)
      |-> count == 4'd0);

  cp_reaches_nine: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9);
  cp_wrap: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9 ##1 count == 4'd0);
endmodule

bind counter counter_props u_counter_props (.*);
```

3.1.3 编译文件列表

先列 RTL，再列包含 bind 的属性文件；路径相对于运行 VC Formal 的 案例工程/ 目录，而不是 Tcl 文件所在目录。

保存为 案例工程/filelist_counter.f：

```
./rtl/counter.sv
./formal/counter_props.sv
```

3.1.4 完整运行脚本

依次选择应用、读入设计与 SVA、定义时钟复位、保存初始状态、检查设置、列出属性并开始证明。初始化步骤需要在实际安装环境核对复位状态，不能把声明复位等同于已经完成复位。

保存为 案例工程/scripts/fpv_counter.tcl：

```
# Run from the example project root with: vcf -f scripts/fpv_counter.tcl -batch
set_fml_appmode FPV
read_file -top counter -sva -format sverilog -vcs {-f filelist_counter.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup -block
report_fv_setup -list
report_fv -list
set_fml_var fml_max_time 1H
set_fml_var fml_max_memory 8GB
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_counter.tcl -batch 2>&1 | tee logs/fpv_counter.log
```

结果阅读。在初始状态正确的前提下，正确版预期满足这些断言，并存在到达 9 和回绕的覆盖轨迹。这不是本次 VC Formal 的实际运行结果。这些断言分别覆盖范围、保持、普通递增和回绕；复位值及运行中重新复位仍需结合初始化模型和专用复位检查，不能仅凭范围属性成立就认定全部功能正确。

4. Tcl 外壳与帮助系统

```
help
help report*
report_fv -help
man report_fv
view_help report_fv
report_fml_var fml_max_*
report_app_var *fml*
get_fml_appmode
```

`help` 与 `-help` 用于核对命令接受的选项；`man`、`view_help` 调出较长帮助；`report_fml_var` 用于查询形式变量，`report_app_var` 用于应用变量。变量归属不能随便互换：有的需要 `set_fml_var`，有的需要 `set_app_var`。**优先执行安装版本的帮助而不是直接复制网络脚本**。

5. 读入、编译与展开 RTL

下列两种是供学习的可选路径，通常选其中一种，而不是把 `read_file` 和 `analyze/elaborate` 在同一工程里无条件重复：

```
# 一步读入并完成展开
read_file -top counter -sva -format sverilog -vcs {-f filelist.f}

# 或者分步编译
analyze -format sverilog -vcs {-f filelist.f}
elaborate counter -sva
```

`read_file` 负责读入源与创建形式设计；`analyze` 处理 HDL 分析；`elaborate` 指定顶层并进行展开；`-sva` 表示需要处理断言；`-vcs` 允许传递 VCS 风格的选项；`-format` 应与 HDL 类型对应。`-f filelist.f` 指向相对路径的设计列表，必须处理宏、包含目录、编译顺序及需要的库。参见 UG §4.2（第 118 页起）。

建议文件列表写为：

```
+incdir+./rtl/include
./rtl/counter.sv
./formal/counter_properties.sv
```

有些示例直接把 `bind` 文件放在最后一行；当属性用包中的定义或其他模块时应按语言编译要求安排顺序。遇到语法或层级错误，先检查源文件展开成功再分析证明失败。

6. 时钟与复位定义

```
create_clock clk -period 100
create_reset rst_n -sense low
set_constant test_mode -value 1'b0
set_change_at -clock clk -default
```

`create_clock` 声明逻辑时钟；这里的 `-period 100` 为形式时钟配置中的周期单位，不应未经说明就解释成目标芯片的物理时间；`create_reset` 描述复位信号及有效电平；`set_constant` 建模测试模式等稳定输入；`set_change_at` 用于约束输入在哪一时钟边沿或时序位置允许变化（特别注意多时钟设计）。这些命令不等同于 STA 中完成时序约束，也不自动描述完整的上电过程。

6.1 建立复位后的初始化状态

```
sim_run -stable
sim_run 2 -clk clk
sim_save_reset
view_trace -reset
```

这是操作顺序示意，不是对任何 DUT 都成立的复位过程。如果复位需要由外部拉高或拉低，必须按真实复位时序添加 `sim_force`、`sim_run` 等操作；保存前应使用 `sim_get` 或可视化追踪检查寄存器值。不要因为示例写了两拍就将所有芯片复位程序固定成两拍。参见 UG §4.13。

6.2 多时钟处理

对每一个独立时钟域分别确定时钟定义、输入变化时机和复位释放规则。不要把异步时钟误设成固定整数倍的同步时钟来获得快速的证明。对于异步握手，接口级假设应来自 CDC 规格，而不是简单假定采样不会遇到亚稳态。形式分析的二值逻辑模型一般不直接证明模拟亚稳态的物理概率。

6.3 初始复位与运行中复位是不同任务

`create_reset` 与初始化仿真主要用于建立证明的起始状态。初始化完成后，复位信号在常见流程中保持无效，使引擎从复位后的状态探索正常运行。若规格还要求处理中途复位、某个电源域单独复位或复位交错，必须另行建立允许这些事件的模型和属性，不能从一次初始化成功推断全部复位行为正确。

`disable iff` 的作用也不同：它描述哪些时刻中止属性尝试，不是在验证环境中生成复位波形。把所有重要属性都在复位期间禁用，只能避免用正常运行规则检查复位过程，并没有证明复位值、释放顺序或未复位寄存器的有效性。来源：User Guide，第 138–154 页。

7. 检查准备状态

```
report_blackbox
report_formal_setup
report_fv_complexity
check_fv_setup -block
report_fv_setup -list
```

核对无意中的黑盒、未定义实例、时钟/复位检测、设计复杂度及组合反馈等结构问题。`check_fv_setup` 的结果属于**环境检查**，不等于完成功能证明。过大的黑盒范围可能令断言看起来更容易，也可能产生不实际的反例。

8. 属性管理、运行、输出

```
# 下面是运行阶段的常用命令
report_fv -list
set_fml_var fml_max_time 2H
set_fml_var fml_max_memory 16GB
check_fv -block
report_fv
report_fv -verbose
```

`fvassert`、`fvassume`、`fvcover` 可以在 Tcl 环境中创建或改变属性角色。优先用 SV 文件管理复杂时序 SVA，便于维护、测试、版本审查。属性状态需要按分组与任务核对；`check_fv` 未指定 `-block` 时可能处于后台运行方式，输出报告时要确认检查完成。运行后的文字报告与反例都应随任务保存。

9. 计算资源与回归

```
set_fml_var fml_effort default
set_fml_var fml_max_time 4H
set_fml_var fml_max_memory 32GB
report_fml_engines
report_fml_jobs
report_fml_hosts
# 如果本机与网格环境支持，可配置调度方式：
# set_grid_usage -type RSH=4
```

`fml_effort` 是求解努力程度的预置策略。更高的资源投入不能保证所有属性最终完成；过度并行也可能受单机内存和许可证数量限制。大型项目推荐按模块、属性类别和运行目的组织任务，并在同等硬件条件下比较优化效果。可选的 `fvlearn_config -local_dir rma_db` 适用于符合条件的回归加速流程，详见 UG §5。

10. 会话、图形调试与工作区

```
save_session -session my_session
restore_session -session my_session
view_trace -property my_assertion
```

工具重启时可采用 `vcf -restore -session my_session`。注意会话文件和相应数据库是否一起备份，设计或约束更改后不应直接将旧结果当作新版本的证明。Verdi 的属性列表、源代码定位、反例波形和结构视图可用于追踪失败属性；对于没有图形环境的服务器可以先保留文字报告再移交调试。

11. 命令出现错误时的检查顺序

1. 命令不识别：先查 `get_fml_appmode`、该应用专属帮助以及版本差异，尤其注意 `COV`/`FCA`。
2. 文件未找到：确认执行目录，检查 `filelist.f` 中的相对路径、`+incdir+`、宏定义和权限。
3. 属性列表为空：检查 `-sva`、`bind` 编译是否生效、顶层、属性是否被禁用。
4. 大量未知初始状态：重新确认复位/初始化保存，避免把初始状态直接强制为期望结果。
5. 反例路径异常：检查外部假设、时钟关系、非法环境输入及黑盒。
6. 显示已证明但无法触发：补充 `cover`、空洞性分析和约束可满足性检查。
7. 证明超时：先缩小验证目标和影响锥，再调整引擎、内存和运行时限。

12. 官方资料索引

- User Guide §2 第 53 ~ 65 页：环境、启动与模式。
- §3 第 66 ~ 114 页：约束、属性、SVA 与属性管理。
- §4 第 115 ~ 170 页：编译、时钟、复位、初始状态及设计检查。
- §5 ~ 8 第 171 ~ 344 页：回归加速、运行控制、结果调试与工作区。
- Quick Reference Guide 第 1 ~ 34 页：各应用最常用命令的快速定位。**快速指南并不是完整命令参考书**。

形式验证工程组织与结果判读

1. 从设计需求建立验证对象

建议为每条规则建立一行审查记录：**需求编号、自然语言规则、采样时钟、复位条件、触发条件、允许的延迟、输入假设、属性名称、覆盖见证、报告状态、评审人**。自然语言规则应足够具体，使不接触 SVA 的设计工程师也可以判断反例究竟违反了什么。

例如，规则 **REQ-03：接受请求以后最迟在第 3 个时钟上升沿给出应答**，并不意味着 `req` 一高就要出现 `ack`。首先确认接受发生在 `req && ready`，请求是否允许撤回，`ack` 是脉冲还是电平，以及响应期间能否接受第二个请求。只有这些定义明确后才能编写正确的 SVA。

2. 推荐项目目录

```
project/
├── rtl/                    # 待验证设计，不得含隐含形式假设
├── formal/
│   ├── properties.sv       # 断言、假设、覆盖目标
│   ├── bind.sv             # 可选的非侵入式绑定
│   └── waivers/            # 带原因、审核与到期条件的豁免
├── scripts/
│   ├── setup.tcl           # 通用编译、时钟、复位
│   ├── run_fpv.tcl         # 应用专属检查
│   └── filelist.f
├── reports/                # 文字和机器可读报告
├── sessions/               # 会话与可视化调试数据
└── README.md               # 软件版本、运行命令与结论
```

不要把运行日志与 RTL 混在一起，也不要提交可能包含许可证、客户路径或公司机密的整个工具工作目录到公开代码仓库。

3. 完整执行顺序



图 4: 可复现验证流程

3.1 编译通过不代表环境正确

第一步执行 `read_file` 或 `analyze/elaborate`，核对顶层、模块实例、语言、参数值。第二步执行时钟与复位定义，保存初始化状态。第三步执行 `report_blackbox`、`check_fv_setup`，确认没有未预期的黑盒和关键错误。第四步查看属性数量与类型；如果有 20 条预期断言但报告只列出 3 条，必须在求解前调查。

3.2 先验证假设，再验证 DUT

优先运行简单覆盖目标：复位退出、请求出现、握手成功、缓冲区填满、缓冲区清空和错误条件能够触发。若关键事件覆盖目标不可达，先检查假设是否过度限制输入。输入假设应有规格出处；特别禁止为了获得 `Proven` 把 DUT 输出或内部状态直接约束为正确取值。

3.3 先短后长的证明计划

第一次运行用较短时间验证环境完整性与可发现的浅层缺陷；第二次把剩余难属性按功能块分组，按重要性分配时间、内存与计算节点；最终运行固定输入版本并记录工具版本、命令与状态。这样既能早发现工程错误，又能减少在错误环境里长时间求解的成本。

4. 读懂反例：逐周期定位

反例至少应该包含：属性触发周期、导致失败的周期、触发前的设计状态、相关输入变化、reset 状态、接受信号与相关时钟。检查时按顺序回答：

1. 是否处于允许的运行状态？复位是否已结束，起始状态是否合理？
2. 是否违反接口对输入的要求？如果是，应该将其写成环境假设还是 DUT 应处理的非法输入？
3. 失败的准确周期是什么？`|->` 与 `|=>` 是否写反？`$past` 是否在未初始化时使用？
4. 内部状态在此周期之前如何变化？是否存在 NBA 更新与 SVA 采样的时间差？
5. 是否人为加入抽象/黑盒而导致不可行状态？
6. 修改 RTL、修正规格描述还是修改属性？必须区分并留下审查记录。

4.1 反例说明模板

```
属性：AP_FIFO_NO_OVERFLOW
模块版本：rtl_commit=...
实际检查范围：posedge clk; disable iff(!rst_n)
失败位置：反例第 17 周期
输入：push=1, pop=0, full=1
状态：occupancy=8（DEPTH=8）
期望：不得接受写入，occupancy 保持 8
实际：occupancy 变为 9
处置：修复计数更新条件，并添加满状态覆盖目标
```

以上代码块只是一条**假设性的教学反例**，并非本手册实际运行 VC Formal 得到的结果。

5. 证明状态与报告编写

必须报告属性总数与分布，包括 `Proven`、`Falsified`、`Inconclusive`、`Covered`、`Uncoverable`、`Disabled`、`Waived` 及其他输出状态。对每条断言写清时钟、复位、有效前提和设计版本。`report_fv -list` 适合总览；`report_fv -verbose` 适合排查；对单条失败使用 `view_trace -property <name>`。`report_fv` 字段随应用和版本有区别，不能把若干种状态强行压成“通过/失败”二选一。

5.1 “Proven”与“覆盖完整”分别检查

强断言 `assert property(req |-> ack)` 被证明，但 `req` 从未出现，其结论仍可能是空洞的。一般需要独立的触发覆盖目标。验证完成至少检查：断言是否被启用、关键 antecedent 可达、是否存在矛盾假设、证明是否包含完整设计范围、所有豁免是否有审查依据。

5.2 为什么未完成不等于失败

`Inconclusive` 仅说明当前求解资源与方法不足；可能是大整数运算、内部存储阵列、长时间依赖、跨多个控制状态或过大影响锥导致。应尝试简化断言、拆分属性、建立合法的不变量和合理抽象，而不应直接增加一个使属性易证但不合理的假设。

6. 时钟、复位与环境建模审查

对象	必须核对的问题	常见错误
主时钟	哪一边沿更新/采样，是否有门控	SVA 在错误边沿采样
异步复位	生效极性和释放关系	`disable iff` 之外仍使用非法 `$past`
多时钟	相位与允许变化时刻	错将异步时钟固定同步
总线数据	`valid && ready` 时何时稳定	把每拍数据都假设不变
存储器	上电初值与读写冲突规则	将未初始化读数据强制为零
黑盒	哪些信号成为任意值	意外屏蔽实现错误
测试端口	正常模式/测试模式的关系	为了便于证明错误强制测试端口

形式工具不会自动知道板级时钟、电源启动顺序或软件配置过程。工程师应把这些外部条件转成可以审查的模型，并把“真实环境保证的行为”与“希望设计满足的行为”分别写成假设和断言。对未初始化状态作额外固定、把输出当成输入约束，都会缩小可探索的行为范围。

7. 小型设计推荐的脚本骨架

```
# 一条教学用途的 FPV 批量运行示例
set_fml_appmode FPV
read_file -top counter -sva -format sverilog -vcs {-f filelist.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup -block
report_fv -list
set_fml_var fml_max_time 1H
check_fv -block
report_fv -list
report_fv -verbose
```

使用条件：DUT 的初始状态在该复位配置下正确建立、路径从执行目录解析、属性确实已被编译；本节随后直接给出完整示例；实际使用时仍应补充适合目标设计的复位激励并核验状态。如果缺少许可证，本示例仅是经文档对照的参考脚本，不是已经执行成功的日志。

7.1 将骨架落实为完整计数器工程

上一段的 `dut/filelist.f` 是通用名称。下面给出实际顶层、文件列表和全部源码，作为结果判读练习的固定输入。

7.1.1 被验证的计数器 RTL

规格：复位后计数为零；使能时从 0 计到 9 后回到 0；没有使能时保持。RTL 文件只包含设计，不把待验证结论写成环境假设。

保存为 案例工程/rtl/counter.sv：

```
module counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       enable,
    output logic [3:0] count
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)             count <= 4'd0;
    else if (enable) begin
      if (count == 4'd9)    count <= 4'd0;
```

```
      else                  count <= count + 4'd1;
    end
  end
endmodule
```

7.1.2 完整断言与绑定文件

history_valid 在复位后建立历史有效标志。ap_range 检查范围，ap_hold 检查保持，ap_increment 检查普通递增，ap_wrap 检查回绕；两条 cover 检查最高计数值及回绕场景是否能够出现。末尾 bind 将检查器接入同名设计信号，不需要修改 RTL。

保存为 案例工程/formal/counter_props.sv：

```
module counter_props (
    input logic       clk, rst_n, enable,
    input logic [3:0] count
);
  logic history_valid;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) history_valid <= 1'b0;
    else        history_valid <= 1'b1;

  ap_range: assert property (@(posedge clk) disable iff (!rst_n)
      count <= 4'd9);

  ap_hold: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(!enable) |-> count == $past(count));

  ap_increment: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count < 4'd9)
      |-> count == $past(count) + 4'd1);

  ap_wrap: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count == 4'd9)
      |-> count == 4'd0);

  cp_reaches_nine: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9);
  cp_wrap: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9 ##1 count == 4'd0);
endmodule

bind counter counter_props u_counter_props (.*);
```

7.1.3 编译文件列表

先列 RTL，再列包含 bind 的属性文件；路径相对于运行 VC Formal 的 案例工程/ 目录，而不是 Tcl 文件所在目录。

保存为 案例工程/filelist_counter.f：

```
./rtl/counter.sv
./formal/counter_props.sv
```

7.1.4 完整运行脚本

依次选择应用、读入设计与 SVA、定义时钟复位、保存初始状态、检查设置、列出属性并开始证明。初始化步骤需要在实际安装环境核对复位状态，不能把声明复位等同于已经完成复位。

保存为 案例工程/scripts/fpv_counter.tcl：

```
# Run from the example project root with: vcf -f scripts/fpv_counter.tcl -batch
set_fml_appmode FPV
read_file -top counter -sva -format sverilog -vcs {-f filelist_counter.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup -block
report_fv_setup -list
report_fv -list
set_fml_var fml_max_time 1H
set_fml_var fml_max_memory 8GB
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_counter.tcl -batch 2>&1 | tee logs/fpv_counter.log
```

结果阅读。在初始状态正确的前提下，正确版预期满足这些断言，并存在到达 9 和回绕的覆盖轨迹。这不是本次 VC Formal 的实际运行结果。这些断言分别覆盖范围、保持、普通递增和回绕；复位值及运行中重新复位仍需结合初始化模型和专用复位检查，不能仅凭范围属性成立就认定全部功能正确。

8. 状态回归和证明性能

相同的属性在不同的 RTL、SVA、时钟约束或工具版本下不应直接比较结果。推荐记录固定的 run manifest：源码校验值、文件列表、Tcl 参数、许可证环境、CPU 与内存资源、运行时间、属性状态汇总、工具报告路径。对于复杂设计，建立按功能分类的任务：复位初始化、寄存器读写、主状态机、协议握手、数据通路和异常控制。

9. 验收条件建议

一个适合作为版本交付的形式验证结果包通常应满足：所有必检断言均明确列出；失败已定位和评审；未完成属性标出后续验证措施；所有豁免有理由与责任人；约束集合经过独立审查；关键触发条件有覆盖见证；代码/时钟/复位/工具版本可以重现。这里的“验收”指项目质量要求，而不是 VC Formal 某个按钮自动代表的最终结论。

详见 调试与证明性能.md 及 覆盖与验证验收.md。

10. 参考

VC Formal User Guide, X-2025.06-SP1，§3 ~ 8、§18 ~ 20；Quick Reference Guide，FPV 与各应用的运行及报告部分。

SystemVerilog Assertions（SVA）完整中文教程

1. SVA 解决什么问题

SVA 是用 SystemVerilog 描述时间相关规则的一组语言结构。它不仅可以检查单个信号是否为正确值，还能表达“请求以后数拍内必须应答”“在握手完成之前数据必须保持稳定”等跨周期关系。SVA 可以用于仿真，亦可以在形式验证时被工具转化为逻辑属性。SVA 本身并不让环境自动合法；外部输入要求仍然需要明确的假设和复位模型。



图 5: SVA 触发与周期关系

2. 先理解三种角色

```
ap_safety: assert property (@(posedge clk) disable iff (!rst_n)
                            !(write && full && !read));
as_input:  assume property (@(posedge clk) disable iff (!rst_n)
                            req |-> input_valid);
cp_full:   cover  property (@(posedge clk) disable iff (!rst_n)
                            full);
```

`assert property` 定义需要被验证的设计责任；`assume property` 描述环境向设计提供的约束；`cover property` 请求寻找满足某条行为的输入轨迹。不要把设计错误的结论改写为 `assume` 以获得更多的 `Proven`。

3. 并发断言的时钟、采样与失效条件

3.1 标准结构

```
property p_example;
  @(posedge clk) disable iff (!rst_n)
  trigger |-> ##[1:3] response;
endproperty

ap_example: assert property (p_example);
```

- `@(posedge clk)`：属性在时钟上升沿采样。
- `disable iff (!rst_n)`：复位低有效期间关闭或中止该属性的活动求值。
- `trigger`：时序要求的触发部分；称为前件（antecedent）。
- `|->`：重叠蕴含；后件从前件序列匹配结束的那个采样周期开始。
- `##[1:3] response`：允许未来第 1、2 或 3 个采样周期响应。

SVA 并发断言在时钟事件的采样值上判断表达式，通常会看到 **NBA 非阻塞赋值更新之前**采样到的值。因此，在 `always_ff @(posedge clk)` 中用非阻塞赋值更新的寄存器，其新值通常在下一次采样沿才被断言看到。检查“状态更新后”的规则时往往需要 `|=>` 或 `$past`。

3.2 复位并不是采样函数历史自动清零

`disable iff` 是属性失效条件；不能将其理解为 `$past` 自动替换成复位值。比如初次进入正常工作状态时，上一拍可能仍处于复位。如果规则依赖上一拍已经处于有效工作状态，应增加 `past_valid` 寄存器或适当的 `$past(rst_n)` 前提。

```
logic past_valid;
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) past_valid <= 1'b0;
  else        past_valid <= 1'b1;

ap_hold: assert property (@(posedge clk) disable iff (!rst_n)
  past_valid && hold |-> data == $past(data));
```

上例的 `past_valid` 为工具提供可信的前一采样点。在不同时间点立即释放/断言复位时，异步 `disable iff` 和时钟采样的交互仍应按 DUT 的实际复位语义审查。

3.3 default clocking / default disable iff

```
default clocking cb @(posedge clk); endclocking
default disable iff (!rst_n);

ap_onehot: assert property ($onehot0(grant));
```

适用于同一时钟下的属性模块。多时钟或不同复位域，建议为每条关键属性显式指定时钟和失效条件，避免默认时钟覆盖不匹配的接口。

4. 重叠蕴含与非重叠蕴含

写法编号	后件起点	前件在第 5 拍成立时检查哪一拍
A：重叠蕴含	前件结束的同一拍	第 5 拍
B：非重叠蕴含	前件结束后的下一拍	第 6 拍
C：重叠后延迟两拍	从同一拍再计两拍	第 7 拍
D：非重叠后延迟两拍	从下一拍再计两拍	第 8 拍

```
// A：同拍；B：下一拍；C：两拍后；D：三拍后。
a |-> b
a |=> b
a |-> ##2 b
a |=> ##2 b
```

上表以单周期布尔前件 `a` 为例。重叠与非重叠决定后件从哪里开始，再由后件自身的延迟决定实际检查周期。因此非重叠后再写两拍延迟，合计是前件结束后三拍，不是两拍。对长度大于一拍的前件，起点是前件**完成匹配的那一拍**，而不是前件开始的那一拍。

```
// req 在第 5 拍、accept 在第 7 拍完成前件。
// ap_same 检查第 7 拍的 busy；ap_next 检查第 8 拍的 busy。
ap_same: assert property (@(posedge clk) disable iff (!rst_n)
    (req ##2 accept) |-> busy);
ap_next: assert property (@(posedge clk) disable iff (!rst_n)
    (req ##2 accept) |=> busy);
```

这里的简单单时钟布尔后件可以用一拍显式延迟改写非重叠蕴含。复杂序列可能有多个结束匹配或空匹配，多时钟属性还有专门的时钟衔接规则，不能只根据符号外观机械替换。每条关键规则应写清触发周期、前件结束周期和预计失败周期。

5. sequence 和 property 的职责

```
sequence s_accept;
  valid && ready;
endsequence

sequence s_finish_within_3;
  ##[1:3] done;
endsequence

property p_finish;
  @(posedge clk) disable iff (!rst_n)
  s_accept |-> s_finish_within_3;
endproperty

ap_finish: assert property(p_finish);
```

sequence 描述如何匹配一个有限时序片段；property 组合时序规则、蕴含、停止和其他属性运算符。复杂序列可以接受参数和局部变量，用于协议检测、延迟记录及数据关联。推荐给可重复使用的时序模式起易读的名称。

6. 序列连接与延迟运算符

6.1 固定延迟 ##n

```
sequence s_two_cycles;
  a ##2 b;
endsequence
```

这要求 a 后第二个采样时刻 b 成立。a ##0 b 要求二者在同一采样时刻满足。当 a 是多周期序列时，##n 从 a 匹配结束处计算。

6.2 范围延迟 ##[m:n]

```
ap_ack_1_4: assert property (@(posedge clk) disable iff (!rst_n)
  $rose(req) |-> ##[1:4] ack);
```

ack 可以在第 1 ~ 4 拍任一时刻满足。要注意**逐拍重新发起的并行检查**：如果 req 连续高 4 拍，使用 req 作为前件就会发起 4 条独立的响应义务。使用 $rose(req) 仅表示检测上升沿，仍不一定等价于总线“接受请求”的定义。

6.3 连续重复 [*]

```
// busy 必须持续 3 拍
ap_busy: assert property (@(posedge clk) disable iff (!rst_n)
  start |-> ##1 busy[*3]);

// ready 可以延迟出现，示意有界等待
ap_wait: assert property (@(posedge clk) disable iff (!rst_n)
  start |-> (!ready)[*0:3] ##1 ready);
```

[*3] 表示连续 3 次匹配；[*1:4] 表示连续 1 ~ 4 次；[*0:$] 允许零次且上限无限，和后续表达式连接时可能形成意料外的空匹配。尤其是空匹配与蕴含组合，必须通过覆盖和波形验证真正在检查的规则。

6.4 非连续重复 [=] 与 goto 重复 [->]

```
sequence s_three_events;
  event_a[=3];
```

```
endsequence

sequence s_second_hit;
  event_a[->2];
endsequence
```

[=3] 描述在一段匹配范围内出现三次非连续的 event_a，可在最后一次命中后继续匹配非事件周期；[->2] 直到第二次事件命中处结束。两者区别会改变和后续 ##1 b 的关系，复杂协议建议先写小型波形测试确认其末端匹配位置。

7. 序列的组合

7.1 and / or / intersect

```
sequence s_both;
  (a ##1 b) and (c ##2 d);
endsequence

sequence s_one;
  (a ##1 b) or (a ##2 b);
endsequence

sequence s_same_window;
  (a ##1 b) intersect (c ##1 d);
endsequence
```

and 要求两个序列都匹配，但它们的结束位置不要求相同；intersect 要求匹配区间的开始和结束一致。or 允许任意一个序列匹配。不要把序列 and 与普通同周期位运算 && 混淆。

7.2 throughout：范围内持续成立

```
ap_hold_during_busy: assert property (@(posedge clk) disable iff (!rst_n)
  start |-> (enable throughout (busy[*1:5] ##1 done)));
```

expr throughout sequence 表示布尔表达式在被匹配序列涉及的全部采样位置保持真。对于简单请求保持协议，通常 req && !ack |=> req 更直接、更易审查。

7.3 within：一个时序片段位于另一个片段范围中

```
sequence s_short;
  a ##1 b;
endsequence
sequence s_outer;
  start ##[1:5] stop;
endsequence
sequence s_inside;
  s_short within s_outer;
endsequence
```

within 可以表达一个事件序列完整发生在更大序列匹配区间之内。涉及可变时长时要特别警惕匹配歧义。使用工具的 cover 检查预期的代表性路径。

7.4 first_match

```
sequence s_first_ack;
  first_match(##[1:8] ack);
endsequence
```

用于从候选匹配中选取首次匹配，减少后续序列对不同匹配长度的歧义。不是所有需要“最早应答”的

规则都要使用 first_match；如果只要求区间内任一次应答，则直接使用范围延迟更清楚。

8. 属性运算符：时间上的持续和终止

8.1 until 与 until_with

```
ap_keep_valid: assert property (@(posedge clk) disable iff (!rst_n)
  req |-> (valid until_with ack));
```

until 不要求条件在终止条件发生的那个周期仍成立；until_with 要求在终止周期也成立。弱形式可能不要求终止条件真的出现，因此如果要求“必须最终完成”，还需要强形式或独立的有界完成检查。

8.2 strong 与 weak

```
ap_eventual: assert property (@(posedge clk) disable iff (!rst_n)
  req |-> strong(##[1:$] ack));
```

strong 表示时序匹配最终必须结束（具体可证明性视形式应用、活性支持以及公平性而定）；weak 允许一些有限轨迹或无限未完成场景下的宽松解释。无条件无限等待往往困难，实际项目应先考虑规范是否可以给出合理的最大响应延迟。强活性属性通常需要假设外部持续提供时钟与资源。

8.3 accept_on / reject_on

属性可在规定事件发生时被提前接受或拒绝。例如，一次事务在取消信号到来后不再要求原始应答，是否“取消即成功”必须符合协议而不能仅为了避免失败。disable iff 通常用于复位；accept_on 可用于协议意义上的正常取消；reject_on 可用于协议明确规定的提前失败条件。两者与采样时刻有关，项目需要按工具实现验证。

9. 常用采样函数

函数	含义	典型用途	容易忽略的地方
$past(expr, n)	读取前 n 次采样的值	下一周期状态关系	起始历史值未定义
$rose(expr)	检测从非真到真的变化	捕获请求新事务	不等同于 req && ready
$fell(expr)	检测从真到非真的变化	撤销请求	异步与 X 变化要谨慎
$stable(expr)	当前值与上一采样相同	等待期间数据保持	初始样本的比较问题
$changed(expr)	与上一采样不同	寻找状态翻转	重复周期持续匹配
$isunknown(expr)	表达式包含 X 或 Z	仿真与 X 检查	二值形式模型解释可不同
$onehot(expr)	恰有一位为 1	必须存在且唯一授权	空闲时全零会失败
$onehot0(expr)	最多一位为 1	允许空闲的授权	不能保证至少有一位
$countones(expr)	统计为 1 的位数	配额/令牌检查	大位宽求解可能昂贵
$initstate	初始时间检测	初始化相关规则	工具、上下文支持需核对

9.1 $past 的正确使用

```
ap_count_inc: assert property (@(posedge clk) disable iff (!rst_n)
  past_valid && $past(inc && !clear)
    |-> count == $past(count) + 1'b1);
```

这是在**当前采样周期**检查上一周期已满足加一条件后的寄存器值。若计数器会饱和或回绕，还必须补上对应前提。需要特别确认 past_valid 的第一个有效周期与设计运行时序一致。

9.2 `$stable` 的常见错误

对 `valid && !ready |=> $stable(data)` 的解释要看接口在下一拍是否仍继续 `valid`。若数据在 `valid && ready` 发生后下一拍可以自由变化，则这个规则可能过度严格。更常用且直观的条件是：

```
ap_stable_when_stalled: assert property (@(posedge clk) disable iff (!rst_n)
  valid && !ready |=> valid && $stable(data));
```

这个写法要求：上一拍未被接受的传输，下一拍仍保持 `valid` 和数据，恰好符合很多 ready/valid 接口的要求。允许在下一拍被接受并不改变“采样的当前数据仍应保持”的要求。

10. 立即断言和并发断言的区别

```
always_comb begin
  assert (!(grant[0] && grant[1]))
    else $error("two simultaneous grants");
end

ap_grant: assert property (@(posedge clk) disable iff (!rst_n)
  $onehot0(grant));
```

立即断言在过程执行时检查瞬时条件，可能受组合逻辑的调度次序、暂态变化及 X 值影响；并发断言按事件采样，便于表达时序关系。形式验证中要遵循该工具对即时/延迟断言支持的说明；不能把所有仿真调试语句当成可被形式引擎直接证明的属性。

11. 属性局部变量、`let` 和参数化

11.1 `let`：抽象复杂布尔条件

```
let accepted(v, r) = (v && r);

ap_no_drop: assert property (@(posedge clk) disable iff (!rst_n)
  accepted(valid, ready) |-> !error);
```

`let` 表达式可复用，减少同一接口条件在几十条属性中被不同工程师写出微小差异。是否使用 `let` 由编译器支持情况决定。

11.2 sequence 局部变量：关联请求与响应数据

```
property p_data_return;
  logic [7:0] saved_id;
  @(posedge clk) disable iff (!rst_n)
    (req, saved_id = req_id) |-> ##[1:4]
      (rsp && rsp_id == saved_id);
endproperty
ap_data_return: assert property (p_data_return);
```

此属性为**每一次匹配的请求**保存其标识并要求窗口内匹配应答。若允许多个并发请求且同一标识重复使用，仍需根据协议处理事务编号、队列深度、重排规则和独立超时。局部变量并非 RTL 中真的增加了寄存器，工具可能为断言引入匹配状态，过度复杂的动态序列会增加形式求解成本。

11.3 参数化属性

```
module ready_valid_checker #(
  parameter int DATA_W = 32,
  parameter int RESP_MAX = 8
) (
  input logic clk, rst_n, valid, ready,
  input logic [DATA_W-1:0] data
```

```
);
  ap_stall: assert property (@(posedge clk) disable iff(!rst_n)
    valid && !ready |=> valid && $stable(data));
  cp_transfer: cover property (@(posedge clk) disable iff(!rst_n)
    valid && ready);
endmodule
```

参数化断言适合复用在多个实例；但 RESP_MAX 等参数如果在属性中用于常量延迟，必须遵守工具对常量表达式的要求。

12. bind：无需改动设计源码注入 SVA

```
module counter_checker (
  input logic clk, rst_n, inc,
  input logic [3:0] count
);
  ap_range: assert property (@(posedge clk) disable iff(!rst_n)
    count <= 4'd9);
  cp_full: cover property (@(posedge clk) disable iff(!rst_n)
    count == 4'd9);
endmodule

bind counter counter_checker u_checker (
  .clk(clk), .rst_n(rst_n), .inc(inc), .count(count)
);
```

bind 在指定设计模块或实例上附加检查器，不必改动交付 RTL。但要确保端口名与被绑定模块一致，检查器的编译顺序正确，且顶层编译开启断言支持（例如相应流程中的 -sva）。**不要在形式测试文件里用 assign 强行驱动 DUT 的内部状态来代替外部约束**。

13. 常见系统级断言模板

13.1 复位值

```
ap_reset_state: assert property (@(posedge clk)
  !rst_n |-> state == IDLE);
```

此示例适用于复位在该采样时钟上足够早生效，且状态在这一采样点应已为 IDLE 的协议模型。若同步复位在 posedge clk 的 NBA 才生效，应换成正确的下一拍检查；异步复位应另外确认有效瞬间与采样要求。

13.2 一热与互斥

```
ap_mutex: assert property (@(posedge clk) disable iff(!rst_n)
  $onehot0(grant));
ap_one_grant_when_pending: assert property (@(posedge clk) disable iff(!rst_n)
  pending |-> $onehot(grant));
```

第二条仅适用于协议要求有请求时必须立刻授权，否则应改成有界最终授权，并定义公平条件。

13.3 请求—应答

```
ap_ack: assert property (@(posedge clk) disable iff(!rst_n)
  (req && accept) |-> ##[1:3] ack);
cp_req: cover property (@(posedge clk) disable iff(!rst_n)
  req && accept);
```

该写法有一个明确的受理事件；如果 ack 可能与受理同周期，则窗口改成 ##[0:3]。若协议允许接受

多个请求，需要采用事务关联检查。

13.4 背压期间保持

```
ap_hold: assert property (@(posedge clk) disable iff(!rst_n)
  valid && !ready |=> valid && $stable(payload));
```

对于标准 ready/valid 流控制，后一拍可以 ready 变为 1，此时仍须在握手时提供原来的 payload。

13.5 FIFO 不上溢、不下溢与计数范围

```
ap_depth: assert property (@(posedge clk) disable iff(!rst_n)
  occupancy <= DEPTH);
ap_no_empty_pop: assert property (@(posedge clk) disable iff(!rst_n)
  pop_accept |-> !empty);
ap_no_full_push: assert property (@(posedge clk) disable iff(!rst_n)
  push_accept |-> (!full || pop_accept));
```

这里假定 FIFO 允许满状态同时读写（具体需看实现），push_accept/pop_accept 是**已接受操作**而不是原始请求。若 push 为请求而硬件可以拒绝，那么 push && full 并非必然错误。计数器溢出的检查应与接受操作和旁路条件一致。

13.6 超时与重试

```
ap_timeout: assert property (@(posedge clk) disable iff(!rst_n)
  launch |-> ##[1:8] (done || timeout));
```

本例只保证在第 1 ~ 8 拍的某一拍看到完成或超时，没有保证二者互斥；如果规定互斥，另外增加 !(done && timeout) 断言。

13.7 有限状态机状态合法性

```
ap_state_legal: assert property (@(posedge clk) disable iff(!rst_n)
  state inside {ST_IDLE, ST_REQ, ST_WAIT, ST_DONE});
```

inside 中的常量值必须与 RTL 一致。枚举类型有时可以使用更清晰的编码检查；该属性只能证明取值合法，不能证明所有转移符合协议。

13.8 安全权限控制

```
ap_secure_write: assert property (@(posedge clk) disable iff(!rst_n)
  write_accept && secure_reg_sel |-> privileged);
```

还需要检查非法请求是否被阻止、读路径是否泄漏以及复位后的权限状态。单条写入权限断言不代表已经证明没有信息泄漏。

14. 空洞性、覆盖目标与不恰当假设

```
ap_req_ack: assert property (@(posedge clk) disable iff(!rst_n)
  req |-> ##[1:3] ack);
cp_req_seen: cover property (@(posedge clk) disable iff(!rst_n)
  req);
cp_req_ack: cover property (@(posedge clk) disable iff(!rst_n)
  req ##[1:3] ack);
```

反例：如果在其他文件里写了 assume property(!req)，ap_req_ack 可能轻松被证明，但是 cp_req_seen 不可达。**验证环境需要既让合法关键场景可达，又排除真实接口所禁止的输入行为**。

工程里建议同时维护三类检查：设计断言；环境前提的注释与审查；关键规格场景覆盖。形式覆盖与仿真覆盖不是同一概念，应分别分析。

15. 常见错误及修正方法

错误类别	直接后果	修正方向
同拍与下一拍混淆	检查过早或过晚	先画采样周期，再选蕴含形式
非重叠后重复计算延迟	比规格多等待一拍	以完整前件结束时刻为零点
无效历史值	首拍错误失败或错误通过	建立历史有效标志
无界弱等待	未表达必须最终响应	使用有界响应或明确强活性
前件从未触发	属性空通过	添加触发和完成覆盖目标
接收方始终就绪假设	屏蔽背压缺陷	只保留真实协议前提
一热规则不允许全零	合法空闲状态失败	区分恰好一个与最多一个
把设计责任写成假设	直接排除了错误状态	对设计责任使用断言
用复位禁用代替复位验证	复位值从未被检查	分别检查初始化与复位行为
以一次完成覆盖代替进展性	遗漏永久等待请求	对每次合法请求检查完成

先把时间关系写正确。下一拍应答与两拍后应答应使用不同的序列。以下属性都显式给出时钟和复位，其中第二条等价于第一条的简单布尔情形；第三条要求两拍后响应，而非三拍后响应。

```
ap_next_1: assert property (@(posedge clk) disable iff (!rst_n)
    req |=> ack);
ap_next_2: assert property (@(posedge clk) disable iff (!rst_n)
    req |-> ##1 ack);
ap_two_cycles: assert property (@(posedge clk) disable iff (!rst_n)
    req |-> ##2 ack);
```

历史值与复位分别处理。`$past(data)` 读取的是过去采样点的值，`disable iff` 中止属性尝试，却不会自动把所有历史采样变成期望值。需要一拍历史时，可以设置复位后才拉高的 `past_valid`；需要多拍历史时，有效标志也要覆盖足够长的历史窗口。运行中再次复位是否允许，应与工具对初始复位的处理区分。

```
logic past_valid;
always_ff @(posedge clk or negedge rst_n)
  if (!rst_n) past_valid <= 1'b0;
  else        past_valid <= 1'b1;

ap_hold_when_stalled: assert property (
    @(posedge clk) disable iff (!rst_n)
    past_valid && $past(valid && !ready)
    |-> valid && $stable(data));
```

通过必须有合适的触发场景。不能把 `ready` 固定为 1 后声称已经验证背压，也不能把计数范围直接写成假设后再证明同一个范围。对于允许空闲的授权向量，应检查最多一位有效；覆盖目标分别确认请求出现和一次完整事务可发生，但不能代替每次请求的完成断言。

```
ap_grant_exclusive: assert property (
    @(posedge clk) disable iff (!rst_n) $onehot0(grant));
ap_response_deadline: assert property (
    @(posedge clk) disable iff (!rst_n) req |-> ##[1:3] ack);
cp_request: cover property (
    @(posedge clk) disable iff (!rst_n) req);
cp_transaction: cover property (
    @(posedge clk) disable iff (!rst_n) req ##[1:3] ack);
```

无限等待需要明确的活性要求。不应把带无界延迟的弱序列直接解释为“最终一定应答”。真实规格有超时上限时，用有界窗口；没有上限而又要求最终完成时，需要强活性表达以及可解释的环境公平性。以下写法要求存在一个有限的匹配终点，但仍要核对所使用工具对相应活性形式的支持，不能为了证明方便随意增加“外部必然响应”的假设。

```
ap_eventual_response: assert property (
    @(posedge clk) disable iff (!rst_n)
    req |-> strong(##[0:$] ack));
```

16. 多时钟属性、X 值与复杂语法的限制

多时钟属性在 SystemVerilog 标准中有专门的采样与序列连接规则，不能简单将两种时钟事件放入单时钟模板。不同 EDA 引擎对多时钟交叉属性支持可能有限；参考 VC Formal User Guide §3.9 的具体限制。跨时钟同步器、握手与异步 FIFO 通常需分别定义每个时钟域的责任及跨时钟的数据一致性。

`$isunknown` 在四值仿真中很直观，但传统形式证明的二值布尔编码可能与仿真中的 X 传播语义不同；检查 RTL 中 X 传播风险更适合配合 VC Formal FXP 和明确的注入/观察模型。写有 `casex`、`casez`、`unique case` 的 RTL 时，还应结合综合与仿真对 X 的不同处理方式审查。

17. 完整示例：有界计数器及 VC Formal 工程

前文的短代码块用于解释单个语法；本节则给出可保存成文件的完整设计、检查器、编译列表和 Tcl。代码与 FPV 章的计数器案例保持一致。

17.1 被验证的计数器 RTL

规格：复位后计数为零；使能时从 0 计到 9 后回到 0；没有使能时保持。RTL 文件只包含设计，不把待验证结论写成环境假设。

保存为 案例工程/rtl/counter.sv：

```
module counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       enable,
    output logic [3:0] count
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)             count <= 4'd0;
    else if (enable) begin
      if (count == 4'd9)    count <= 4'd0;
      else                  count <= count + 4'd1;
    end
  end
endmodule
```

17.2 完整断言与绑定文件

`history_valid` 在复位后建立历史有效标志。`ap_range` 检查范围，`ap_hold` 检查保持，`ap_increment` 检查普通递增，`ap_wrap` 检查回绕；两条 `cover` 检查最高计数值及回绕场景是否能够出现。末尾 `bind` 将检查器接入同名设计信号，不需要修改 RTL。

保存为 案例工程/formal/counter_props.sv：

```
module counter_props (
    input logic       clk, rst_n, enable,
    input logic [3:0] count
);
  logic history_valid;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) history_valid <= 1'b0;
    else        history_valid <= 1'b1;
```

```
  ap_range: assert property (@(posedge clk) disable iff (!rst_n)
      count <= 4'd9);

  ap_hold: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(!enable) |-> count == $past(count));

  ap_increment: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count < 4'd9)
      |-> count == $past(count) + 4'd1);

  ap_wrap: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count == 4'd9)
      |-> count == 4'd0);

  cp_reaches_nine: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9);
  cp_wrap: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9 ##1 count == 4'd0);
endmodule

bind counter counter_props u_counter_props (.*);
```

17.3 编译文件列表

先列 RTL，再列包含 bind 的属性文件；路径相对于运行 VC Formal 的 案例工程/ 目录，而不是 Tcl 文件所在目录。

保存为 案例工程/filelist_counter.f：

```
./rtl/counter.sv
./formal/counter_props.sv
```

17.4 完整运行脚本

依次选择应用、读入设计与 SVA、定义时钟复位、保存初始状态、检查设置、列出属性并开始证明。初始化步骤需要在实际安装环境核对复位状态，不能把声明复位等同于已经完成复位。

保存为 案例工程/scripts/fpv_counter.tcl：

```
# Run from the example project root with: vcf -f scripts/fpv_counter.tcl -batch
set_fml_appmode FPV
read_file -top counter -sva -format sverilog -vcs {-f filelist_counter.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup -block
report_fv_setup -list
report_fv -list
set_fml_var fml_max_time 1H
set_fml_var fml_max_memory 8GB
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_counter.tcl -batch 2>&1 | tee logs/fpv_counter.log
```

结果阅读。在初始状态正确的前提下，正确版预期满足这些断言，并存在到达 9 和回绕的覆盖轨迹。这不是本次 VC Formal 的实际运行结果。这些断言分别覆盖范围、保持、普通递增和回绕；复位值及运行中重新复位仍需结合初始化模型和专用复位检查，不能仅凭范围属性成立就认定全部功能正确。

18. 工程练习（建议自行运行与审查）

1. 将计数器回绕阈值从 9 改为 10，观察 `ap_range` 的反例；将阈值改回后重复证明。
2. 删除 `history_valid`，分析首个有效采样点 `$past` 规则的差异。
3. 为一个 2 输入仲裁器编写互斥、授权必须对应请求和竞争覆盖三类属性。
4. 为 ready/valid 通道编写背压保持、握手可达与连续握手覆盖。
5. 故意加入约束 `assume(!enable)`，观察覆盖目标为什么无法满足，并撤销该约束。

本章第 17 节已直接列出练习所需的 RTL、SVA、文件列表与完整运行脚本；独立源码文件仅用于方便编辑，不是阅读本章的前置要求。

19. 语法检索与选择

类别	语法或函数	适合描述的问题
角色	`assert`、`assume`、`cover`	设计责任、环境前提和可达性
采样	`@`、`default clocking`	用哪个事件观察信号
失效	`disable iff`	哪些条件中止当前检查尝试
蕴含	重叠、非重叠	前件完成后何时开始后件
延迟	`##n`、范围延迟	事件间的周期关系
重复	连续、非连续、goto	事件需要重复多少次
序列组合	`and`、`or`、`intersect`	多段行为的组合及结束时刻
区间关系	`throughout`、`within`	条件持续或片段包含
匹配控制	`first_match`	选择首个完成匹配
时间终止	`until`、`until_with`	终止事件发生前后的条件保持
强弱要求	`strong`、`weak`	是否必须存在完成的匹配
历史采样	`$past`、`$rose`、`$fell`	过去值与边沿变化
稳定和互斥	`$stable`、`$onehot0`、`$onehot`	数据保持与授权关系
检查器复用	`property`、`sequence`、`let`、`bind`	参数化并接入多个实例

语法选择应从规格问题出发，而不是先选一个看起来简短的运算符。例如“忙期间数据不变”要明确忙区间是否包含最后一拍；“接到请求后响应”要明确请求是电平、脉冲还是握手接受事件；“响应对应请求”还必须说明是否允许多个未完成事务。下面集中列出容易在表格中被误拆的运算符和序列写法，均保持原字符，可直接复制到适当的序列或属性上下文。

```
a |-> b
a |=> b
a ##2 b
a ##[1:3] b
a[*3]
a[=3]
a[->3]
```

这些是语法片段，不是一个完整模块，也不能在顶层源码中连续粘贴后直接编译。完整带时钟、复位、绑定和运行脚本的示例见本章第 17 节。

20. 参考与责任界定

本教程独立编写，对照 IEEE 1800 的常见语法概念以及用户所给 VC Formal User Guide X-2025.06-SP1 §3.7 ~ 3.9、附录 B。工具具体支持范围、强活性处理、采样行为和优化由正式安装版本决定。本文示例用于教学；没有声称在用户实际 VC Formal 授权环境中运行过全部代码。

VC Formal 命令分类与通用命令详解

1. 怎样阅读命令语法

命令不能只按名称记忆，还要同时理解执行阶段、接受的对象、对验证环境的修改以及返回结果。启动命令在操作系统终端执行；会话命令由 VC Formal 的 Tcl 解释器执行；SVA 属性由编译器处理；DPV 的参考函数则属于 C/C++ 源码。把这些层次分清，才能判断某个报错发生在启动、编译、环境设置还是证明阶段。

标记	含义	使用要求
尖括号参数	需要替换的名称、路径或数值	不把尖括号原样输入命令
方括号参数	语法说明中的可选部分	与 Tcl 的命令替换方括号区分
备选值	参数可取的若干互斥值	实际调用选择一个合法值
花括号列表	Tcl 中保持整体的参数或脚本	防止空格和特殊字符被提前解释
对象集合	工具管理的设计或属性对象	使用集合命令，不当成普通字符串列表

语法框用于描述命令的接受形式；示例框给出具体调用。下列第一行表示选项可以省略，第二行才是可直接解释的 Tcl。语法说明中的竖线只表示备选值，不是要发送给命令的字符；完整 SVA 运算符则必须原样保留。

```
report_fv [-status <proven|falsified|inconclusive>] [<properties>]
report_fv -status proven
```

正文以提供的 X-2025.06-SP1 UG 为基线；若 UG 只给出调用示例而将完整参数定义转交给其他参考手册，本章明确标注其范围，不虚构额外选项。下列命令示例通常要求已完成其上游步骤，不能脱离当前设计和任务逐行任意运行。完整可保存的 RTL、SVA、文件列表和 Tcl 工程放在第 5 节，各应用章节也保留自己的完整案例。

2. 按工作阶段选择命令

阶段	要完成的工作	详细说明位置
启动与帮助	选择工作方式、读取本机命令说明	本章第 3 节
编译与环境	读入设计、建立时钟复位和初始状态	本章第 3 节
属性与任务	管理断言、假设、覆盖和任务范围	本章第 4 节
运行与资源	设置计算资源、执行、报告和保存	本章第 4 节
自动规则与未知值	自动提取检查、设置未知值来源及观察点	AEP、FXP
连通性与低功耗	描述端点、电源条件和隔离要求	CC、FLP / LPC
等价与寄存器	建立设计对应关系、加载寄存器规格	SEQ、DPV、FRV
覆盖及属性质量	可达性、形式核心和故障检测能力	FCA、FTA、覆盖与验证验收
安全分析	故障影响、告警及信息传播	FuSa、FSV
结构查询	查询实例、网络、端口和对象属性	设计查询与集合

调试与分解	反例、约束冲突、复杂度和组合证明	调试与证明性能、AIP 与分解证明

这个分类描述的是依赖顺序，而不是互不相干的菜单。例如 AEP 自动生成的目标仍要通过通用属性命令管理；FXP 设置的来源和观察点需要在正确的复位环境下解释；FRV 生成检查器后，还必须确认实际总线访问能够触发检查。应把应用专用命令嵌入完整验证流程，避免只执行“生成”和“报告”而漏掉证明。

3. 命令语法、参数与用例

命令	主要作用
vcf	启动外壳、批处理和图形会话
help	按命令名称查询帮助
man	阅读命令的详细手册页
view_help	打开图形帮助条目
set_fml_appmode	选择形式应用
get_fml_appmode	查询当前应用模式
set_app_var	设置平台应用变量
report_app_var	查询平台变量的值和说明
set_fml_var	设置形式分析变量
report_fml_var	查询形式变量
read_file	一次读入、编译并展开设计
analyze	分析源文件并存入设计库
elaborate	展开顶层并建立验证设计
set_blackbox	把选定模块或实例设为黑盒
report_blackbox	报告黑盒对象
get_blackbox	取得黑盒集合
create_clock	定义时钟及参考时钟
create_reset	定义复位有效值及启动复位
get_resets	查询复位对象
set_constant	同时约束初始化与正式分析中的常量
set_change_at	限制输入允许变化的时钟事件
report_change_at	检查输入变化规则
get_change_at	读取输入变化规则对象
sim_run	运行初始化仿真
sim_force	在初始化仿真中强制信号
sim_set_state	设置寄存器和存储器的初始状态
sim_save_reset	把当前状态保存为形式初始状态
sim_get	查询当前初始化仿真值
fv_setup_config	配置模型准备检查
check_fv_setup	执行模型准备检查
report_fv_setup	报告准备检查的状态与违规
infer_formal_setup	推断时钟等形式设置
formal_setup_config	查询或配置自动设置流程
report_formal_setup	报告自动或显式形式设置

环境设置决定形式模型中哪些初始状态和输入变化被允许。特别注意编译前生效的变量、黑盒规则、时钟复位定义和初始化仿真的先后顺序；在编译后修改前端开关通常不能自动重建已经生成的模型。

3.1 vcf：启动外壳、批处理和图形会话

```
vcf [-batch] [-out_dir <dir_name>]
    [-cmd_log_file <log_name>] [-f <file_name>]
    [-fmode <FPV|COV|SEQ|CC|AEP|FRV|FXP|FSV|FUSA|DPV>]
    [-gui] [-verdi] [-verdi_base] [-verdi_apex]
    [-help] [-id] [-lic_wait <minutes>] [-no_init]
    [-no_restore] [-no_ui] [-output_log_file <log_name>]
    [-read_only] [-reset] [-restore] [-session <session_name>]
    [-use_ipv4] [-use_ipv6] [-x <config_tcl_cmd>]
    [-y <post_config_tcl_cmd>] [-doc]
    [-fml_base] [-fml_ultra] [-fml_elite] [-fml_token]
    [-fml_legacy] [-fml_no_legacy] [-fml_zoix] [-fml_z01x]
    [-fml_auto_restore_level <none|job|engine|host|all>]
    [-scl_span] [-rma_off] [-container] [-noecho]
    [-fml_nxg <0|1|2>] [-echo]
```

参数解释。 `-f` 指定启动后执行的 Tcl 脚本，完整写法为 `-file`；`-batch` 进入非交互批处理，`-out_dir` 指定输出目录，`-session` 指定运行数据库目录。`-cmd_log_file` 保存命令记录，`-output_log_file` 保存控制台输出，两者用途不同。`-fmode` 选择启动应用，省略时为 FPV，启动列表不含 FTA。

`-gui` 和 `-verdi` 打开图形环境；`-verdi_base`、`-verdi_apex` 选择相应的 Verdi 授权。`-fml_base`、`-fml_ultra`、`-fml_elite`、`-fml_token` 选择形式工具授权方式；`-fml_legacy` 使用旧授权查找，`-fml_no_legacy` 不回退到旧方式，两个 FuSa 选项选择分级 FuSa 授权。`-lic_wait` 的单位为分钟，`-scl_span` 允许跨多个服务器获取授权。

`-restore` 恢复会话，`-read_only` 只读恢复；`-no_restore` 删除以前会话并重新开始，`-reset` 清理旧工作会话，使用前应保留需要的数据库。`-no_init` 不读初始化设置文件，`-no_ui` 不启动界面进程。`-x`、`-y` 分别在配置阶段及配置后执行 Tcl 命令。`-fml_auto_restore_level` 控制自动保存会话的恢复层级。`-rma_off` 关闭相应授权模式下自动启用的回归加速，`-container` 用于容器环境；`-use_ipv4`、`-use_ipv6` 选择进程通信协议；`-fml_nxg` 选择命令一致性模式。`-noecho` 不在输出与会话日志中回显命令；`-echo` 只回显环境而不调用可执行程序。`-help` 显示帮助，`-id` 显示构建和环境信息，`-doc` 打开文档。

用法与结果。这是操作系统命令，不是在 `vcf>` 提示符内执行的 Tcl 命令。批处理脚本必须等待需要的证明完成，再导出报告和退出；只加 `-batch` 并不会替脚本中的所有异步命令建立依赖。每个回归任务使用独立会话目录，避免多个进程同时写入同一个数据库。

```
vcf -f scripts/fpv_counter.tcl -batch \
    -session counter_run -output_log_file counter.log
vcf -f scripts/fpv_counter.tcl -verdi
vcf -restore -session counter_run -read_only
```

来源：User Guide，X-2025.06-SP1，第 54–62 页。

3.2 help：按命令名称查询帮助

```
help [<command-pattern>]
```

参数解释。不指定模式时显示帮助入口；模式可包含通配符，例如 `report*` 用于查找报告类命令。这里的星号是命令名称匹配，不是设计对象筛选。

用法与结果。先确认命令是否存在，再看其参数。未知命令与未知参数是不同问题：前者可能来自应用模式、安装包或命令版本，后者通常需要核对具体调用形式。

```
help
help report*
```

来源：User Guide，X-2025.06-SP1，第 46–50；QRG 6 页。

3.3 man：阅读命令的详细手册页

```
man <command-name>
```

参数解释。命令名不带参数。详细页通常给出用途、语法、参数和示例，适合核对超出本文调用形式的选项。

用法与结果。 UG 对部分平台命令明确转引 VC Static Platform Reference Guide，因此不能把 UG 中的一行示例当成命令支持的全部参数。

```
man report_fv
man create_clock
```

来源：User Guide，X-2025.06-SP1，第 46–50；QRG 6 页。

3.4 view_help：打开图形帮助条目

```
view_help <command-name>
```

参数解释。指定需要查看的命令名称。图形帮助依赖可用的文档与界面环境。

用法与结果。它用于阅读说明，不运行该命令，也不改变当前验证状态。批处理环境中应优先保存文本帮助和命令日志。

```
view_help report_fv
```

来源：User Guide，X-2025.06-SP1，第 46–50；QRG 6 页。

3.5 set_fml_appmode：选择形式应用

```
set_fml_appmode <FPV|COV|SEQ|CC|AEP|FRV|FXP|FSV|FUSA|DPV|FTA>
```

参数解释。位置参数是应用模式名。FCA 是覆盖分析应用的名称，本文采用该版 UG 明确列出的 Tcl 模式 `COV`；FTA 可以通过本命令切换，但不能用 `vcf -fmode FTA` 启动。

用法与结果。通常在编译前选定应用。FTA 的例外流程是先在 FPV 中准备并运行属性，再切换到 FTA。切换模式不意味着已加载的专用配置、故障数据库或证明条件自动有效。

```
set_fml_appmode FPV
get_fml_appmode
```

来源：User Guide，X-2025.06-SP1，第 64–65 页。

3.6 get_fml_appmode：查询当前应用模式

```
get_fml_appmode
```

参数解释。无位置参数，返回当前形式应用名。

用法与结果。在公共脚本入口和专用应用切换后查询一次，可以及早发现使用了错误的默认模式。不要用 GUI 页签名称代替实际 Tcl 模式检查。

```
set current_app [get_fml_appmode]
puts "Current application: $current_app"
```

来源：User Guide，X-2025.06-SP1，第 65 页。

3.7 set_app_var：设置平台应用变量

```
set_app_var <variable-name> <value>
```

参数解释。第一个参数是平台登记的应用变量，第二个参数是符合该变量类型和值域的值。变量是否在设计读入前生效，应根据变量说明确定。

用法与结果。本命令不同于 Tcl 的 `set`：后者可以创建普通脚本变量，而本命令针对工具已定义的应用变量。编译开关、前端行为与形式求解变量不能混为一类。

```
report_app_var apply_bind_in_all_units -verbose
set_app_var apply_bind_in_all_units false
```

来源：User Guide，X-2025.06-SP1，第 63–64、119 页。

3.8 `report_app_var`：查询平台变量的值和说明

```
report_app_var [<name-pattern>] [-verbose]
```

参数解释。不指定模式时列出平台变量；模式限制名称范围，`-verbose` 显示详细说明。输出区分当前值、类型、默认值和限制。

用法与结果。报告平台变量不会把用户通过 `set` 创建的所有普通变量都列出。需要审计配置时，应同时保存设置脚本和相关详细报告，而不是只记录修改后的一个数值。

```
report_app_var fml_*
report_app_var fml_composite_trace -verbose
```

来源：User Guide，X-2025.06-SP1，第 63–64 页。

3.9 `set_fml_var`：设置形式分析变量

```
set_fml_var <variable-name> <value> [-global]
```

参数解释。变量名决定控制对象，例如运行时间、内存、空洞性分析或轨迹生成。`-global` 在文档示例中用于把设置应用到所有任务；不是每项变量都可在求解过程中即时修改。

用法与结果。变量可能改变模型，也可能仅改变资源。前者需要重新构建并复核结果，后者也要在任务开始前明确设定。不要把缩短时间限制后留下的未完成目标算作成功。

```
set_fml_var fml_max_time 24H
set_fml_var fml_max_memory 32GB
set_fml_var fml_vacuity_on true
set_fml_var fml_report_profile_data true -global
```

来源：User Guide，X-2025.06-SP1，第 131–132、709–713；QRG 7–8 页。

3.10 `report_fml_var`：查询形式变量

```
report_fml_var [<name-pattern>] [-verbose]
```

参数解释。名称模式支持按前缀查找；`-verbose` 用于查看默认值、当前值与具体说明。

用法与结果。文档基线相同也可能启用了不同安装配置。对影响覆盖生成、初始状态或抽象的变量，先查详细说明，再在正式脚本中显式设置需要的值。

```
report_fml_var fml_max_*
report_fml_var fml_vacuity_on -verbose
```

来源：User Guide，X-2025.06-SP1，第 384、1127–1140；QRG 6 页。

3.11 `read_file`：一次读入、编译并展开设计

```
read_file -format <verilog|sverilog|vhdl> -top <top-name>
          [-sva] [-aep <aep-types>] [-cov <coverage-types>]
          -vcs {<compiler-options-and-files>}
```

参数解释。`-format` 选择语言，`-top` 指定顶层，`-sva` 启用源代码属性处理。`-aep` 和 `-cov` 是对应应用的目标生成选择，不能在任意应用中随意组合。`-vcs` 后的一个 Tcl 参数传给编译前端，可包含文件名、文件列表和前端选项。

用法与结果。这是 UG 展示的一步流程，不是平台参考手册的全部选项。推荐用花括号包住固定的编译

参数串，避免 Tcl 对方括号和美元符号提前替换；需要变量展开时，则使用经过审查的双引号或 `list` 构造。设计文件、属性文件和绑定文件必须全部被编译。

```
set_fml_appmode FPV
read_file -format sverilog -top counter -sva \
    -vcs {-f filelist_counter.f}
```

来源：User Guide，X-2025.06-SP1，第 118–120、371、394–395 页。

3.12 `analyze`：分析源文件并存入设计库

```
analyze -format <verilog|sverilog|vhdl> [-library <library-name>]
        [-vcs {<compiler-options>}] <source-files>
```

参数解释。`-format` 指定前端语言，`-library` 用于区分分析库；源文件既可直接给出，也可使用文档中的 `-vcs` 参数串和文件列表形式。

用法与结果。该步骤只完成前端分析，不等于已建立可证明的顶层设计。混合语言工程可分别分析 VHDL 与 SystemVerilog，再通过 `elaborate` 统一展开；SEQ 常为参考实现和待比较实现使用不同的库。

```
analyze -format sverilog -vcs {-f filelist_counter.f}
elaborate counter -sva
```

来源：User Guide，X-2025.06-SP1，第 119–120、522–523 页。

3.13 `elaborate`：展开顶层并建立验证设计

```
elaborate <top-name> [-sva] [-aep <aep-types>]
          [-cov <coverage-types>] [-vcs {<compiler-options>}]
```

参数解释。顶层名来自已分析的设计单元。`-sva` 处理属性，`-aep`、`-cov` 指定相应自动目标。混合语言绑定有时需要额外前端选项，UG 给出了 `-lca` 与绑定相关示例。

用法与结果。展开后应检查顶层、实例层级、参数值、黑盒以及属性数量。对参数化模块，仅仅成功分析文件并不能保证展开的是要验证的那一个参数配置。

```
analyze -format sverilog -vcs {-f filelist_counter.f}
elaborate counter -sva
report_fv -list
```

来源：User Guide，X-2025.06-SP1，第 119–120、375–376 页。

3.14 `set_blackbox`：把选定模块或实例设为黑盒

```
set_blackbox -designs <module-name-list>
set_blackbox -cells <hierarchical-instance-list>
```

参数解释。`-designs` 依据设计单元名称选择模块，可能影响它的多个实例；`-cells` 选择具体层级实例。层级名称应与当前顶层下的展开结果一致。

用法与结果。黑盒省略内部功能，输出通常成为模型中的自由输入。这样可以减小问题，也可能产生真实实现中不存在的行为。证明通过与失败的解释必须结合抽象方向；安全信息流应用还需要专门的安全黑盒设置，不能直接沿用普通黑盒的推断。

```
set_blackbox -designs {large_memory}
set_blackbox -cells {u_soc.u_analog}
report_blackbox
```

来源：User Guide，X-2025.06-SP1，第 128–131；QRG 7 页。

3.15 `report_blackbox`：报告黑盒对象

```
report_blackbox
```

参数解释。无参数形式输出当前黑盒配置及对象。

用法与结果。在运行前比对预期黑盒与实际黑盒。未解析模块、显式黑盒和工具优化得到的结构，应按来源分别记录；缺少源文件不应被当成合理抽象。

```
report_blackbox
```

来源：User Guide，X-2025.06-SP1，第 128–131；QRG 7 页。

3.16 `get_blackbox`：取得黑盒集合

```
get_blackbox
```

参数解释。返回当前设计中的黑盒对象，供后续 Tcl 查询使用。

用法与结果。返回值是工具对象集合而不是普通文本报告。检查其大小、名称和层级时使用集合命令；不要直接根据控制台截断显示的几项判断黑盒已经全部列出。

```
set boxes [get_blackbox]
puts [sizeof_collection $boxes]
foreach_in_collection box $boxes {
    puts [get_object_name $box]
}
```

来源：User Guide，X-2025.06-SP1，第 128–131；QRG 19 页。

3.17 `create_clock`：定义时钟及参考时钟

```
create_clock -period <period_value> [-name <clock_name>]
             [-waveform <edge_list>] [-add] [-comment <string>]
             [-refclk] [-initial <0|1>] [source_objects]
```

参数解释。`-period` 是时钟周期；`source_objects` 是定义时钟的网络、端口或引脚。`-name` 指定时钟对象名，`-waveform` 给出边沿列表，`-add` 向原时钟补充选项，`-comment` 保存说明。`-refclk` 指定参考时钟；`-initial` 指定时间零的时钟值，本版列出的默认值是 1。

用法与结果。参考时钟用于没有显式时钟的脚本属性、复位仿真及部分深度报告。单时钟设计自动使用该时钟，多时钟默认选择最快时钟，可显式覆盖。相同 `-name` 再次用于不同信号会覆盖原时钟，因此不同真实时钟应有不同名称。周期表达相对时间关系，不应把数值直接当成芯片在真实电压温度下的工作频率证明。

```
create_clock clk -name core_clk -period 100 -refclk
create_clock slow_clk -name peripheral_clk -period 200 \
    -waveform {0 100} -initial 0
```

来源：User Guide，X-2025.06-SP1，第 132–138 页。

3.18 `create_reset`：定义复位有效值及启动复位

```
create_reset [-sync] [-async] [-both] [-type <reset|set|both>]
             [-name <reset_name>] [-sense <low|high|any>]
             [-value <low|high>] [-add] [-tdr]
             [-disable_assertions_db] [-untimed] [source_objects]
```

参数解释。`source_objects` 是实际复位信号，不可只写对象名而不给源信号。`-sense low` 表示低有效，`-sense high` 表示高有效；共享平台帮助推荐用 `-value` 代替 `-sense`，但本版形式流程正文仍用 `-sense` 解释行为。

`-sync`、`-async`、`-both` 描述复位连接类型，缺省为异步；`-type`、`-name`、`-add` 分别描述类别、命名与选项追加。`-tdr` 标记测试或调试复位，`-disable_assertions_db` 关闭对应 AssertionDB 生成，`-untimed` 表示不是时序单元的时钟化输出。UG 特别指出 VC Formal 的关键行为由复位有效电平决定，不能把共享平台的全部标记都理解为额外形式约束。

用法与结果。低有效复位在初始化仿真时被施加为 0，在后续形式分析中被固定为 1；高有效情况相反。因此标准流程验证的是复位后的工作行为，不是运行过程中任意再次复位。要验证动态复位，必须另建允许复位变化的模型和相应属性。`create_reset -name rst` 不等价于 `create_reset rst`，工具不支持用这种写法创建虚拟复位。

```
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
```

来源：User Guide，X-2025.06-SP1，第 138–140 页。

3.19 `get_resets`：查询复位对象

```
get_resets [-filter <filterexp>] [-regexp] [-quiet] [-nocase]
           [-exact] [-of_objects <object-list>] [patterns]
```

参数解释。 `patterns` 匹配复位名称；`-filter` 根据属性筛选，`-regexp` 采用正则表达式，`-exact` 要求精确匹配，`-nocase` 忽略大小写，`-quiet` 抑制信息。`-of_objects` 从端口、引脚、网络或复位根对象查询关联复位。

用法与结果。查询出的对象应与真实复位域逐项对应。设计读入后如果增加了包装层，路径必须相对新的顶层修正；名称看起来相近并不保证选中了实际复位源。

```
get_resets
get_resets rst*
get_resets -filter {name == rst_n}
```

来源：User Guide，X-2025.06-SP1，第 140 页。

3.20 `set_constant`：同时约束初始化与正式分析中的常量

```
set_constant <signal-or-object-list> -value <constant>
```

参数解释。信号对象是要固定的端口或网络，`-value` 给出数值。Verilog 数字可带位宽，例如 `1'b0`，传入脚本时应保持正确的 Tcl 字面值。

用法与结果。常量传播能简化后续逻辑，但它不是一条属性，不能像 `fvassume` 创建的约束那样转换成断言。与普通假设相比，它也参与复位状态计算。只能固定当前功能模式确实恒定的信号；固定模式选择之后，其他模式不在本次结论范围。

```
set_constant scan_en -value 1'b0
set_constant test_mode -value 1'b0
```

来源：User Guide，X-2025.06-SP1，第 70 页。

3.21 `set_change_at`：限制输入允许变化的时钟事件

```
set_change_at -clock <clock-name> [-posedge] [-negedge] <signals>
set_change_at -clock <clock-name> -default
set_change_at -default -remove
```

参数解释。 `-clock` 指定控制输入变化时机的时钟，边沿选项进一步指定允许变化的事件；`-default` 设置默认信号规则，`-remove` 移除对应规则。这里展示 UG 和快速参考出现的调用形式。

用法与结果。在慢时钟上写一个 assume，只保证慢时钟采样点上的条件，并不自动保证两个采样点之间稳定。多时钟验证中，快时钟可能看到中间变化。本命令用于表达真实接口的稳定区间，不能把本来异步变化的输入强行同步化来消除失败。

```
set_change_at -clock clk -default
report_change_at
```

来源：User Guide，X-2025.06-SP1，第 74–76、118；QRG 7 页。

3.22 report_change_at：检查输入变化规则

```
report_change_at
```

参数解释。列出当前设置的允许变化事件及关联信号。

用法与结果。应确认每个外部接口受哪个时钟规则影响，尤其注意默认规则是否覆盖了本来需要异步变化的复位、握手或中断输入。

```
report_change_at
```

来源：User Guide，X-2025.06-SP1，第 75–76 页。

3.23 get_change_at：读取输入变化规则对象

```
get_change_at
```

参数解释。取得已定义的变化时机设置，供脚本进一步查询。UG 在多时钟建模中列出该查询入口，未在正文展开其完整筛选参数。

用法与结果。将它与报告命令区分：报告面向阅读，查询结果用于脚本处理。不能据名称推断未列出的参数；自动化脚本只采用经过当前工具帮助确认的筛选方式。

```
set changes [get_change_at]
puts $changes
```

来源：User Guide，X-2025.06-SP1，第 75–76 页。

3.24 sim_run：运行初始化仿真

```
sim_run -stable
sim_run <cycles> [-clk <clock-name>]
```

参数解释。 `-stable` 运行到用于初始化的稳定状态；数值形式运行给定周期数，`-clk` 指定计数所用的时钟。

用法与结果。本命令服务于形式模型的初始化，不等于覆盖设计全部工作行为的动态验证。运行后用 `sim_get` 检查关键寄存器，再执行 `sim_save_reset` 把所得到的状态交给证明。修改会影响形式设置的条件后，工具可能重新生成复位状态，须避免覆盖手工初值。

```
sim_run -stable
sim_get [get_sequentials *count*]
sim_save_reset
```

来源：User Guide，X-2025.06-SP1，第 141–157；QRG 7 页。

3.25 sim_force：在初始化仿真中强制信号

```
sim_force <signal-name> -apply <value>
```

参数解释。信号名是需要强制的对象，`-apply` 给出施加值。此处展示复位仿真与初始化示例中的施加形式。

用法与结果。强制初始化信号与在正式分析中固定常量不是同一个操作。标准 `create_reset` 会组合初始化强制和正式分析的非复位常量行为；手工写初始化序列时，需要明确每个阶段的信号值，不能假定施加动作已经建立全部后续约束。

```
sim_force rst_n -apply 1'b0
sim_run 3 -clk clk
sim_save_reset
```

来源：User Guide，X-2025.06-SP1，第 139、141–149 页。

3.26 sim_set_state：设置寄存器和存储器的初始状态

```
sim_set_state [-fsdb {<file> <instance> <time>}]
              [-uninit_script <file_name>] [-apply <0|1|x|z>]
              [-exact] [-regexp] [-all] [-uninitialized]
              [-readmemb {<file> <memory> [start_addr [end_addr]]}]
              [-readmemh {<file> <memory> [start_addr [end_addr]]}]
              [-exclude_package] [-top <instance_name>] [-xml <file_name>]
              [<sequential-patterns-or-collection>]
```

参数解释。`-apply` 直接设置状态值；位置参数选择时序对象，`-all` 选择全部状态，`-uninitialized` 只选择未初始化对象，`-exact` 与 `-regexp` 改变名称匹配方式。`-fsdb` 从指定实例、指定时间取状态；`-uninit_script` 输出 FSDB 中缺少的未初始化对象脚本。`-readmemb`、`-readmemh` 分别读取二进制、十六进制存储器文件，可指定地址范围。`-xml` 读取保存的初始状态，`-top` 限定加载实例，`-exclude_package` 排除工具包中的信号。

用法与结果。应在初始化仿真完成后设置，并随后保存状态；先设置再运行复位仿真，手工值可能被覆盖。把一个实际未复位寄存器设为 0 是缩小初始状态集合；把有确定初值的寄存器设成 X 则是初始状态抽象。两者对证明结论的影响不同，均需有明确目的和证据。

```
sim_run -stable
sim_set_state {u_counter.count} -apply x
sim_save_reset

sim_set_state -xml reviewed_reset.xml
sim_save_reset
```

来源：User Guide，X-2025.06-SP1，第 141–145 页。

3.27 sim_save_reset：把当前状态保存为形式初始状态

```
sim_save_reset
```

参数解释。无参数形式把当前初始化状态保存到任务对应的复位状态资料中。

用法与结果。`create_reset` 只定义复位行为，`sim_run` 计算初始化状态，本命令才把当前状态交给后续证明。复制任务或读入复位 XML 后，同样需要明确当前任务采用的初始状态，避免误用其他运行遗留的值。

```
sim_run -stable
sim_save_reset
view_trace -reset
```

来源：User Guide，X-2025.06-SP1，第 141–143、149–157 页。

3.28 sim_get：查询当前初始化仿真值

```
sim_get <signals-or-collection>
sim_get -signals {<signal-patterns>}
```

参数解释。位置参数或 `-signals` 选择信号。返回值来自当前仿真时刻；紧接复位仿真查询时，就是所观察的复位状态。

用法与结果。此值不是所有形式轨迹的共同值，也不是某条失败反例的特定周期值。检查未知初值时应同时记录对象名称和初始化步骤，不能把一次仿真取值直接提升为全时域假设。

```
sim_get [get_sequentials *count*]
sim_get -signals {control_reg*}
```

来源：User Guide，X-2025.06-SP1，第 156–157；QRG 7 页。

3.29 fv_setup_config：配置模型准备检查

```
fv_setup_config [-check <check-names>] [-category <category>]
                [-severity <error|info|warning>]
                [-autorun <checkpoint>] [-disable] [-enable]
                [-list] [-verbosity <none|summary|verbose>]
```

参数解释。-check 支持 clock、comb_loop、glitch、multi_driver、osc_loop、osc_seq、reset。-category 为 all、general 或 modport；非 all 类别要求只指定一种检查，modport 对应多驱动检查。-severity 默认 warning，设为 error 可在检查违规时终止流程。-autorun 的检查点为 design_load；本版组合环自动检查只在设计加载点执行。-disable、-enable 改变检查启用情况，-list 查看配置，-verbosity 默认 summary。

用法与结果。 modport 类别用于区分接口输出 modport 造成的特殊多驱动情形与普通多驱动，例如多个接口声明为输出但实际只有一个非高阻赋值。不要为了让脚本继续运行把真实组合环或时钟问题整体关掉。

```
fv_setup_config -check comb_loop -severity error
fv_setup_config -list
check_fv_setup -block
```

来源：User Guide，X-2025.06-SP1，第 157–159 页。

3.30 check_fv_setup：执行模型准备检查

```
check_fv_setup [-block] [-run_finish <commands>] [-stop]
               [-break <number-of-falsifications>]
               [-check <check-names>] [-category <category>] [-verbose]
```

参数解释。 -block 等待检查结束；缺省非阻塞，可用报告命令查看进度。-run_finish 指定非阻塞结束后执行的命令，-stop 停止运行，-break 达到指定失败数后停止。-check 与 -category 的检查类型同配置命令，-verbose 逐项打印违规。

用法与结果。这是时钟、复位、组合环和多驱动等准备条件检查，不是业务属性证明。若要检查经过电源开关的电源逻辑异常，必须先读 UPF。自动生成了潜在问题目标不等于这些目标已经运行；POTENTIAL_SETUP_ISSUE 提示正是要求进一步执行并阅读准备检查。

```
check_fv_setup -block
report_fv_setup -list
check_fv -block
```

来源：User Guide，X-2025.06-SP1，第 157–160 页。

3.31 report_fv_setup：报告准备检查的状态与违规

```
report_fv_setup [-list] [-limit <integer>]
```

参数解释。 -list 逐条列出违规，-limit 限制列出的网络数量。

用法与结果。异步检查未结束时，报告只是当前进度快照。正式归档应在阻塞检查结束或完成回调后生成，且保留未经截断的原始日志。

```
check_fv_setup -block
report_fv_setup -list -limit 100
```

来源：User Guide，X-2025.06-SP1，第 160–161 页。

3.32 infer_formal_setup：推断时钟等形式设置

```
infer_formal_setup -type <setup-type>
```

参数解释。-type clock 是快速参考展示的时钟推断用法。推断结果需通过 report_formal_setup 查看；其他推断类型不在此调用形式中擅自扩展。

用法与结果。推断减少重复输入，不替代设计意图。门控时钟、测试时钟、多源选择和包装层可能使结构推断与实际工作模式不同，因此应在正式证明前确认并必要时显式定义。

```
infer_formal_setup -type clock
report_formal_setup
```

来源：Quick Reference Guide，X-2025.06-SP1，第 QRG 7 页。此处为快速参考提供的补充调用；UG 未列出完整参数定义。

3.33 `formal_setup_config`：查询或配置自动设置流程

```
formal_setup_config
```

参数解释。快速参考列出无参数入口，用于查看形式设置相关配置。该资料没有在此处展开全部参数。

用法与结果。把该入口用于检查工具当前配置，不应凭名称补写未证实的选项。需要改变自动推断行为时，采用安装版本帮助中明确支持的配置项，并保存修改后的报告。

```
formal_setup_config
infer_formal_setup -type clock
report_formal_setup
```

来源：Quick Reference Guide，X-2025.06-SP1，第 QRG 7 页。此处为快速参考提供的补充调用；UG 未列出完整参数定义。

3.34 `report_formal_setup`：报告自动或显式形式设置

```
report_formal_setup
```

参数解释。无参数入口用于查看当前形式设置报告。

用法与结果。核对时钟、复位和相关建模条件后，再看具体属性状态。报告设置与 `report_fv_setup` 报告检查结果不同，不能因设置表中存在时钟名称就认为时钟检查已通过。

```
report_formal_setup
check_fv_setup -block
report_fv_setup -list
```

来源：Quick Reference Guide，X-2025.06-SP1，第 QRG 7 页。此处为快速参考提供的补充调用；UG 未列出完整参数定义。

4. 命令语法、参数与用例

命令	主要作用
`fvassert`	创建断言或改变已有属性的使用方式
`fvassume`	创建环境前提或改变属性用途
`fvcover`	创建可达性目标并扩展到事务结束
`fvdisable`	停用选定的断言、假设或覆盖
`fvenable`	重新启用已有属性
`fvdelete`	删除脚本属性
`get_props`	按类型、用途和状态查询属性集合
`check_fv`	证明属性或寻找覆盖见证
`report_fv`	读取证明状态、深度和详细结果
`fvlearn_config`	配置回归学习的保存与复用
`set_grid_usage`	配置并行工作进程与计算集群
`set_backup_grid_usage`	配置备用计算资源
`report_fml_hosts`	查看集群主机和工作进程状态

命令	主要作用
report_fml_engines	按目标报告求解引擎表现
report_fml_jobs	按作业报告运行状态与结束原因
fvtask	创建、切换和删除验证任务
get_fvtask	取得当前或指定任务集合
report_fvtask	报告任务状态
save_session	保存设置与运行资料
restore_session	在工具中恢复保存的会话
quit	退出工具
fvcompile	编译延后建立的脚本属性

属性管理改变当前任务中哪些结论需要证明、哪些条件被假设、哪些事件只要求存在一次。运行命令还可能是非阻塞的：脚本继续执行不表示证明已经结束。报告、会话和波形分别保存不同的信息，应共同用于复现结果。

4.1 fvassert：创建断言或改变已有属性的使用方式

```
fvassert [<name>] -expr {<property-expression>} [-clock <clock>]
fvassert [-class <classes>] [-type <types>] [-usage <usages>]
         [-filter <expression>] <property-names-or-collection>
```

参数解释。带 -expr 时创建脚本属性；不带 -expr、直接传入已有属性名称时，将其使用方式设为 assert 并启用。-class 表示来源类别，-type 是声明或生成类型，-usage 是当前用途，三者不是同义词。

用法与结果。表达式含显式采样事件时使用该事件；此时额外写 -clock 会给出警告并优先采用表达式中的时钟。没有事件时使用参考时钟。表达式用花括号保护，避免 $past 和位选择被 Tcl 解释。将假设转成断言是验证环境承诺的方法之一，但必须在承担该承诺的设计范围内证明。

```
fvassert AP_HOLD -expr {
  @(posedge clk) disable iff (!rst_n)
  valid && !ready |=> valid && $stable(data)
}
fvassert assume_input_protocol
```

来源：User Guide，X-2025.06-SP1，第 87–89、106–113 页。

4.2 fvassume：创建环境前提或改变属性用途

```
fvassume [<name>] -expr {<expression>} [-clock <clock>]
fvassume [<name>] -expr {<combinational-expression>} -depth <n>
fvassume [<name>] -env -expr {<environment-expression>}
fvassume <property-names-or-collection>
```

参数解释。普通 -expr 创建证明过程中的假设。-depth n 要求正整数，仅在复位后的前 n 个采样周期施加，默认以参考时钟上升沿计数，可用 -clock 改变；其表达式必须是组合逻辑，不能含 SVA 时序运算符。-env 创建同时参与初始化和正式分析的环境约束，表达式只允许涉及主输入、未驱动网络、截断点和黑盒输出等指定对象，且复位仿真能接受的表达式有额外限制。

用法与结果。 -env 必须在 sim_run 之前建立，否则对已经完成的初始化不生效。假设应表达外部环境确实保证的条件；在同一任务中把待证明的设计输出直接设成假设，会把问题本身删除。已证明断言的复用必须说明证明依赖，不能形成循环。

```
fvassume AS_INPUT_STABLE -expr {
  @(posedge clk) disable iff (!rst_n)
  valid && !ready |=> valid && $stable(data)
```

```
}
fvassume AS_BOOT_MODE -expr {mode == 2'b00} -depth 3
fvassume AS_NO_TEST -env -expr {test_mode == 1'b0}
```

来源：User Guide，X-2025.06-SP1，第 66–76、87–89、106–113 页。

4.3 `fvcover`：创建可达性目标并扩展到事务结束

```
fvcover [<name>] -expr {<sequence-or-expression>}
fvcover -extend_cond {<end-condition>} [-suffix <suffix>]
        <base-cover-property>
fvcover [-usage cover] [-class <class>] [-type <type>]
        -extend_cond {<end-condition>} [-suffix <suffix>]
```

参数解释。 `-expr` 创建要寻找见证的目标。`-extend_cond` 基于已有覆盖追加布尔结束条件，不能与 `-expr` 同时使用；`-suffix` 指定新名字后缀，省略时追加 `_extended`。也可以按 source、script、covergroup 等属性类别或类型选择基础覆盖。

用法与结果。到达一个覆盖点不等于事务已经正常退出。扩展覆盖要求该点之后还能到达空闲或事务清空条件，适合检查“触发以后仍能完成”。它本质上仍在寻找一条轨迹，并不自动证明所有命中该点的轨迹都能完成。

```
fvcover CP_REQUEST -expr {@(posedge clk) req}
fvcover CP_TRANSFER -expr {@(posedge clk) req ##[1:4] ack}
fvcover -extend_cond {idle} -suffix eot CP_TRANSFER
```

来源：User Guide，X-2025.06-SP1，第 87–89、108–110 页。

4.4 `fvdisable`：停用选定的断言、假设或覆盖

```
fvdisable [-class <classes>] [-type <types>] [-usage <usages>]
          [-filter <expression>] [<properties>]
```

参数解释。可按来源、声明类型、当前用途或名称选择属性；不明确区分筛选条件容易影响错误的对象。`-usage assume` 选择当前参与约束的属性，而非所有原始声明为 assume 的属性。

用法与结果。停用断言只是不再检查该断言，停用假设则放宽了允许行为，会使已有证明资料失效或需要重新求解。UG 对 enabled 与 usage 的部分旧例描述并不一致，脚本应查询这两个属性，不依赖旧输出中的 none 文本。

```
fvdisable -class aep -type bounds_check
fvdisable -usage cover
report_fv -list
```

来源：User Guide，X-2025.06-SP1，第 106–113、375 页。

4.5 `fvenable`：重新启用已有属性

```
fvenable [-class <classes>] [-type <types>] [-usage <usages>]
         [-filter <expression>] [<properties>]
```

参数解释。按选择条件启用已有目标，不重新生成 RTL 中未插入的检查。

用法与结果。重新启用与重新声明不是一回事。若使用 `aep_config -dont_generate` 在编译前阻止了生成，这里没有可供启用的对象；需要重新配置并编译。重新启用假设后，应重新检查是否产生约束冲突或空洞。

```
fvenable -class aep -type bounds_check
fvenable AP_HOLD
report_fv -list
```

来源：User Guide，X-2025.06-SP1，第 106–113 页。

4.6 `fvdelete`：删除脚本属性

```
fvdelete <property-names-or-collection>
```

参数解释。指定要删除的属性对象。快速参考给出本命令作为属性管理操作，示例用于可由脚本重新建立的属性。

用法与结果。删除会失去对象本身，停用则保留对象与便于审计的信息。临时试验可删除脚本目标；正式工程更应修改版本受控的属性源码和生成脚本，而不是只在交互会话里删除失败目标。

```
fvcover CP_TEMP -expr {@(posedge clk) req}
fvdelete CP_TEMP
```

来源：Quick Reference Guide，X-2025.06-SP1，第 QRG 8、19 页。此处为快速参考提供的补充调用；UG 未列出完整参数定义。

4.7 `get_props`：按类型、用途和状态查询属性集合

```
get_props [-class <classes>] [-type <types>] [-usage <usages>]
          [-status <statuses>] [-filter <expression>]
          [-regexp] [-exact] [-selected] [<names-or-patterns>]
```

参数解释。`-class` 包括 source、script、aep、connection、coverage、register、security、seq、xprop、fusa 等来源；`-type` 描述具体种类；`-usage` 为 assert、assume、cover。`-status` 按报告状态筛选，`-filter` 根据属性表达式筛选，`-regexp` 和 `-exact` 调整匹配规则；`-selected` 只取界面当前选择的属性。

用法与结果。返回的是工具集合。空集合可能是名称错误、没有生成目标、筛选条件错误或属性被停用，不能直接解释为所有目标已经通过。应查询集合大小并结合未筛选的总报告。

```
set asserts [get_props -usage assert]
puts "Assertions: [sizeof_collection $asserts]"
foreach_in_collection p $asserts {
    puts "[get_object_name $p] [get_attribute $p status]"
}
```

来源：User Guide，X-2025.06-SP1，第 110–114 页。

4.8 `check_fv`：证明属性或寻找覆盖见证

```
check_fv [-block] [-run_finish <commands>] [-stop]
         [-break <number-of-falsifications>]
         [-property <properties-or-collection>]
         [-subtype <property|vacuity|witness>] [-assume]
```

参数解释。`-block` 阻塞直到本次检查结束；默认非阻塞。`-run_finish` 指定非阻塞完成后执行的 Tcl 命令。`-stop` 停止当前运行，`-break` 在达到指定失败数时停止。`-property` 选择目标；`-subtype` 可取一个或多个 property、vacuity、witness。`-assume` 允许把已证明断言及不可达覆盖用于约束。

用法与结果。证明同时服从启用的假设、常量、初始状态、黑盒和截断设置。非阻塞运行期间可以查看报告，但不能并行改变影响运行的设置。完成意味着这次资源限制下的运行结束，不等于全部目标获得确定结果；因此报告中的未完成状态必须单独处理。

```
check_fv_setup -block
check_fv -block
report_fv -list > results.txt

check_fv -property [get_props AP_HOLD] \
    -subtype {property vacuity} \
    -run_finish {report_fv -verbose > hold_results.txt}
```

来源：User Guide，X-2025.06-SP1，第 184–189、202–207 页。

4.9 report_fv：读取证明状态、深度和详细结果

```
report_fv [-class <classes>] [-type <types>] [-usage <usages>]
          [-status <statuses>] [-list] [-verbose] [-no_summary]
          [<property-names-or-patterns>]
```

参数解释。-list 输出逐项列表，-verbose 增加引擎、用时、证明深度和属性详细信息，-no_summary 省略汇总。筛选参数限定报告对象，但不会运行目标或改变其状态。

用法与结果。报告是当前状态的快照。安全深度表示有界检查已经覆盖的范围，不是无界证明；反例深度表示失败位置，两者不能合并比较。筛选后只看到 proven 目标时，必须另查总目标数以及其他状态，防止通过筛选制造“全部通过”的假象。

```
report_fv
report_fv -list -status falsified > failures.txt
report_fv -verbose -no_summary AP_HOLD
```

来源：User Guide，X-2025.06-SP1，第 247–261 页。

4.10 fvlearn_config：配置回归学习的保存与复用

```
fvlearn_config [-local_dir <directory>] [-enable <0|1>]
               [-local_dir_learn <directory>]
               [-local_dir_apply <directory>]
               [-learn_only] [-apply_only] [-global] [-verbose] [-status]
```

参数解释。-local_dir 共用学习和应用目录，也可分别用 -local_dir_learn、-local_dir_apply 设置。-learn_only 只保存当前学习并可能覆盖已有数据；-apply_only 只应用旧数据，不写入本次学习。-enable 开关 RMA，-global 应用于当前各任务，-verbose 输出收益信息，-status 返回开关与数据库路径字典。

用法与结果。应在证明命令之前配置。复用的是学习结果与求解经验，不是免除对修改后设计的检查。准备检查和形式核心计算不积累这类学习。目标数量极大的 COV 任务可能因保存数据库增加成本，必须比较真实端到端耗时，而非只看引擎用时。

```
fvlearn_config -local_dir rma_db
check_fv -block
fvlearn_config -status

fvlearn_config -local_dir rma_db -apply_only
```

来源：User Guide，X-2025.06-SP1，第 171–182 页。

4.11 set_grid_usage：配置并行工作进程与计算集群

```
set_grid_usage [-type <string>] [-control <string>] [-dir <string>]
  [-file <string>] [-memory <string>] [-global] [-default]
  [-job_array] [-prefetch_pct <percent>] [-level <integer>] [-navigator]
```

参数解释。 -type 指定计算集群类型及工作进程数，例如 LSF=8、SLURM=6；-control 是该集群实际使用的提交命令及资源要求。-dir 指定工作进程临时目录，-file 指定提交配置文件。-memory 表示多级资源配置对应的内存容量（GB），-level 指定级别，必须从 0 开始连续编号。-global 作用于全部当前任务，-navigator 作用于 navigator 任务而非普通当前任务；-default 恢复默认设置。-job_array 启用作业数组，-prefetch_pct 控制预取进程比例，范围 0–100。

用法与结果。请求工作进程数不等于实际获得资源，也不是线程数或总内存。各节点必须能访问相同设计、临时目录和授权服务；队列的内存限制与形式引擎的内存限制应一致。提交字符串是集群特定配置，不能直接照搬其他站点名称。

```
set_grid_usage -type rsh=8
check_fv -block
```

```
report_fml_jobs -list

set_grid_usage -type SLURM=6 \
    -control {/nfs/slurm/bin/sbatch -p vcfM5 --wait}
```

来源：User Guide，X-2025.06-SP1，第 190–198 页。

4.12 set_backup_grid_usage：配置备用计算资源

```
set_backup_grid_usage -type <backend=worker-count>
                      -control {<submission-command-and-options>}
```

参数解释。快速参考把此命令列为备用集群配置入口。后端与提交选项必须来自实际部署的队列设置。

用法与结果。备用资源用于运行环境的资源切换，不改变验证目标的逻辑含义。主资源失败时，应通过作业报告区分排队、启动失败、内存耗尽和外部终止，而不是把所有未完成归为证明困难。

```
set_backup_grid_usage -type LSF=4 \
    -control {bsub -q backup -R rusage[mem=8000]}
```

来源：User Guide，X-2025.06-SP1，第 190–198；QRG 7–8 页。

4.13 report_fml_hosts：查看集群主机和工作进程状态

```
report_fml_hosts
```

参数解释。报告实际可用或分配的主机及工作进程信息；用于和资源请求配置比对。

用法与结果。集群请求成功并不代表工作进程已经开始运行。长时间没有证明进展时，先确认机器是否已分配、进程是否启动以及共享目录是否可访问，再调整形式算法。

```
report_fml_hosts
report_fml_jobs -list
```

来源：User Guide，X-2025.06-SP1，第 208–217；QRG 8 页。

4.14 report_fml_engines：按目标报告求解引擎表现

```
report_fml_engines [-no_summary] [-list] [-verbose]
                   [-depthCounts] [-depthVsTime]
                   [-status <statuses>] [-jobId <number>]
                   [-goalId <number>] [-engine <name>]
                   [-subtype <subtypes>] [-jobStatus <statuses>]
```

参数解释。默认汇总当前任务最近一次运行。`-list` 每个目标一行，`-verbose` 每个目标在不同引擎中的状态分别列出。`-depthCounts` 按失败深度和有界深度计数，`-depthVsTime` 查看深度随时间的变化。`-jobId`、`-goalId`、`-engine`、`-subtype`、`-jobStatus` 分别按作业、目标、引擎、目标子类和作业状态筛选。

用法与结果。一个属性可能产生自身、空洞性和见证等多个目标。报告中的累计引擎时间不是墙上时钟经过时间；SEQ 子证明的时间也可能是多个引擎的总和。判断性能改动时，应同时记录端到端时间、完成目标数和安全深度。

```
report_fml_engines -list
report_fml_engines -verbose -status bounded
report_fml_engines -depthCounts
```

来源：User Guide，X-2025.06-SP1，第 209–212 页。

4.15 report_fml_jobs：按作业报告运行状态与结束原因

```
report_fml_jobs [-status <status>] [-reason <reason>] [-list]
                [-totaltime] [-no_summary] [-engine <name>]
                [-host <name>] [-jobId <integer>] [-profile]
```

```
[-memBucket <GB>] [-grid_level <integer>]
```

参数解释。`-status` 筛选 scheduled、running 或 completed；`-reason` 筛选 normal、timeout、memout、orc_killed、sig_killed、crash、start_fail、misc_error 等原因。`-list` 列出每个作业的状态、时间、CPU 利用率、内存、目标数量和机器信息；`-totaltime` 汇总作业时间，`-no_summary` 省略汇总。`-engine`、`-host`、`-jobId` 按对象过滤；`-profile` 按引擎统计时间、内存与完成目标，`-memBucket` 控制内存分组宽度，默认 8 GB；`-grid_level` 筛选资源级别。

用法与结果。timeout 可能只是工具分阶段尝试短时引擎的正常结果；orc_killed 常表示其他引擎已经解决目标；sig_killed 可能来自集群外部限制。它们都不能直接等同于 RTL 错误。CPU 利用率明显低于预期时，应先检查机器争用与资源分配。

```
report_fml_jobs -list
report_fml_jobs -reason memout -list
report_fml_jobs -profile -memBucket 4
```

来源：User Guide，X-2025.06-SP1，第 212–217 页。

4.16 `fvtask`：创建、切换和删除验证任务

```
fvtask [-create] [-delete] [-limit_copy] [-copy <copy_task>]
  [-appcopy <app_name>] [-attributes <name/value-pairs>]
  [-asserts <properties>] [-assumes <properties>] [-covers <properties>]
  [-constants <signals>] [-snips <signals>] [-changeats <signals>]
  [-keep_status] [-keep_task] [<task_name>]
```

参数解释。`-create` 创建任务并通常切换为当前任务；`-delete` 删除非默认任务；仅给任务名表示切换任务。`-copy` 从指定任务复制，`-appcopy` 从指定应用复制。`-asserts`、`-assumes`、`-covers` 选择属性；`-constants`、`-snips`、`-changeats` 选择现有常量、截断信号和变化时刻约束。`-limit_copy` 限制复制范围，要求至少指定一类属性；`-attributes` 显式指定任务属性名和值。`-keep_status` 保留父任务的目标状态；`-keep_task` 创建或修改目标任务但不切换当前任务。

用法与结果。任务隔离允许按功能和难度组织证明，但不自动使任务之间的假设正确。限制复制时特别检查约束、初始状态及任务专属属性，UG 明确提示专属属性不会总是默认复制。新任务缺少关键环境约束可能出现伪反例，多复制错误假设则可能导致不真实的通过。时钟与复位定义是全局设置，不会因为拆成任务而形成独立时钟环境。同一会话同一时刻只有一个任务执行 `check_fv`。语法定义中没有单列 `-abstractions`，即使说明文字提到复制抽象，也不应据此构造不存在于语法框的开关。

```
fvtask -create hold_checks -copy FPV \
    -asserts *hold* -assumes *
check_fv -block
report_fv -list
fvtask FPV
```

来源：User Guide，X-2025.06-SP1，第 313–323 页。

4.17 `get_fvtask`：取得当前或指定任务集合

```
get_fvtask [-filter <attribute-expression>] [tasks]
```

参数解释。无参数取得当前任务，星号取得所有任务；`-filter` 按任务属性筛选。

用法与结果。返回工具集合，适合读取任务属性或构造批量报告。不要把 `get_fvtask` 无参数误认为列出所有任务。

```
set current [get_fvtask]
set all_tasks [get_fvtask *]
puts [sizeof_collection $all_tasks]
```

来源：User Guide，X-2025.06-SP1，第 317 页。

4.18 `report_fvtask`：报告任务状态

```
report_fvtask [-filter <expression>] [-xml] [tasks]
```

参数解释。任务名称或模式限制范围，`-filter` 按属性过滤，`-xml` 采用 XML 输出。

用法与结果。该报告说明任务是否活动以及汇总状态；具体失败属性仍需进入对应任务查看。批量回归应把任务名与设计配置一起记录，不能合并不同前提下的通过计数。

```
report_fvtask *
report_fvtask -xml * > task_report.xml
```

来源：User Guide，X-2025.06-SP1，第 317 页。

4.19 `save_session`：保存设置与运行资料

```
save_session [-session <session-name>]
```

参数解释。省略名称保存当前会话；`-session` 指定会话保存位置或名称。

用法与结果。退出并不保证所需设置和结果已经保存。与源代码版本、Tcl 脚本和日志一起归档，才能在后续调试中知道数据库由哪一组输入生成。保存会话也不是可移植的正式证明证书。

```
check_fv -block
report_fv -list > results.txt
save_session -session reviewed_run
```

来源：User Guide，X-2025.06-SP1，第 324–329 页。

4.20 `restore_session`：在工具中恢复保存的会话

```
restore_session -session <session-name>
```

参数解释。参数指定已保存的会话。也可以从操作系统通过 `vcf -restore -session ...` 启动恢复。

用法与结果。恢复后先查询应用模式、任务和报告，再决定继续求解。恢复旧数据库不会自动把后来修改的源码读入，也不能将旧结果标记为对新设计有效。

```
restore_session -session reviewed_run
get_fml_appmode
report_fvtask *
report_fv
```

来源：User Guide，X-2025.06-SP1，第 324–329 页。

4.21 `quit`：退出工具

```
quit [-force]
```

参数解释。无参数退出当前外壳；`-force` 用于跳过保存相关步骤快速退出。

用法与结果。正式批处理应在证明或指定完成回调结束后生成报告，并按需要显式保存会话，然后退出。不能在非阻塞 `check_fv` 后立即执行 `quit`，否则结果可能尚未产生。

```
check_fv -block
report_fv -list > final_results.txt
save_session
quit
```

来源：User Guide，X-2025.06-SP1，第 185、324–325 页。

4.22 `fvcompile`：编译延后建立的脚本属性

```
fvcompile
```

参数解释。无参数调用，用于在延后属性编译流程中一次编译已生成的脚本属性。

用法与结果。自动生成大量属性时，先设置延后编译、建立各个属性，再统一编译，可避免逐条重复处理。这里的编译针对属性，不取代设计前端的 analyze 和 elaborate。

```
set_fml_var fml_defer_property_compile true
fvassert AP_ONEHOT -expr {@(posedge clk) $onehot0(grant)}
fvcover CP_REQUEST -expr {@(posedge clk) req}
fvcompile
```

来源：User Guide，X-2025.06-SP1，第 379–380 页。

5. 完整计数器工程

上面的命令列表用于按用途检索，其中 `top/filelist.f` 等名称属于参数示意。下面则列出全部实际文件，帮助读者从命令清单进入可保存的教学工程。

5.1 被验证的计数器 RTL

规格：复位后计数为零；使能时从 0 计到 9 后回到 0；没有使能时保持。RTL 文件只包含设计，不把待验证结论写成环境假设。

保存为 案例工程/rtl/counter.sv：

```
module counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       enable,
    output logic [3:0] count
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)             count <= 4'd0;
    else if (enable) begin
      if (count == 4'd9)    count <= 4'd0;
      else                  count <= count + 4'd1;
    end
  end
endmodule
```

5.2 完整断言与绑定文件

`history_valid` 在复位后建立历史有效标志。`ap_range` 检查范围，`ap_hold` 检查保持，`ap_increment` 检查普通递增，`ap_wrap` 检查回绕；两条 `cover` 检查最高计数值及回绕场景是否能够出现。末尾 `bind` 将检查器接入同名设计信号，不需要修改 RTL。

保存为 案例工程/formal/counter_props.sv：

```
module counter_props (
    input logic       clk, rst_n, enable,
    input logic [3:0] count
);
  logic history_valid;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) history_valid <= 1'b0;
    else        history_valid <= 1'b1;

  ap_range: assert property (@(posedge clk) disable iff (!rst_n)
      count <= 4'd9);

  ap_hold: assert property (@(posedge clk) disable iff (!rst_n)
```

```
      history_valid && $past(!enable) |-> count == $past(count));

  ap_increment: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count < 4'd9)
      |-> count == $past(count) + 4'd1);

  ap_wrap: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count == 4'd9)
      |-> count == 4'd0);

  cp_reaches_nine: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9);
  cp_wrap: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9 ##1 count == 4'd0);
endmodule

bind counter counter_props u_counter_props (.*);
```

5.3 编译文件列表

先列 RTL，再列包含 bind 的属性文件；路径相对于运行 VC Formal 的 **案例工程/** 目录，而不是 Tcl 文件所在目录。

保存为 案例工程/filelist_counter.f：

```
./rtl/counter.sv
./formal/counter_props.sv
```

5.4 完整运行脚本

依次选择应用、读入设计与 SVA、定义时钟复位、保存初始状态、检查设置、列出属性并开始证明。初始化步骤需要在实际安装环境核对复位状态，不能把声明复位等同于已经完成复位。

保存为 案例工程/scripts/fpv_counter.tcl：

```
# Run from the example project root with: vcf -f scripts/fpv_counter.tcl -batch
set_fml_appmode FPV
read_file -top counter -sva -format sverilog -vcs {-f filelist_counter.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup -block
report_fv_setup -list
report_fv -list
set_fml_var fml_max_time 1H
set_fml_var fml_max_memory 8GB
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_counter.tcl -batch 2>&1 | tee logs/fpv_counter.log
```

结果阅读。在初始状态正确的前提下，正确版预期满足这些断言，并存在到达 9 和回绕的覆盖轨迹。这不是本次 VC Formal 的实际运行结果。这些断言分别覆盖范围、保持、普通递增和回绕；复位值及运行中重新复位仍需结合初始化模型和专用复位检查，不能仅凭范围属性成立就认定全部功能正确。

FPV：功能属性验证应用使用指南

1. 定位、能力与不适用范围

FPV 通过数学证明验证 RTL 和 SVA 所描述的设计规则。输入包括：待验证 RTL、用户断言、Assertion IP（AIP）断言、环境假设、时钟复位和初始化状态；输出为证明状态、反例、覆盖轨迹和分析报告。它适合局部协议、状态机、控制面、缓冲区、寄存器副作用、异常控制等，可在传统 testbench 完成前投入使用。

FPV 不会自动理解芯片规格，也不自动生成所有应该证明的规则。只有对**已经编写并启用**的属性进行了证明才有相应结论，且环境约束必须可信。若设计缺少软件配置顺序或外部协议模型，可能出现不实际的反例；若过度约束，则可能出现空洞证明。

2. 标准流程与输入输出

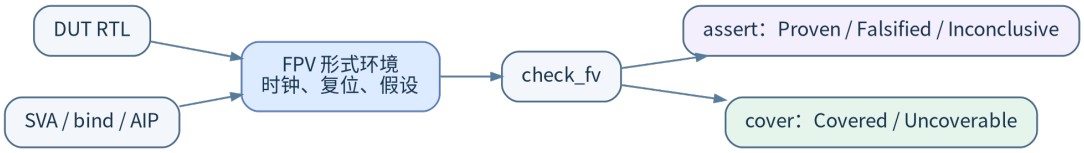


图 6: FPV 工作流程

阶段	任务	命令或文件	核查点
选择应用	设置 FPV	`set_fml_appmode FPV`	模式是否正确
编译	读入 RTL、属性和 bind 文件	`read_file` 或 `analyze/elaborate`	顶层、参数、属性数量
初始化	时钟、复位、常量、初始状态	`create_clock`、`create_reset`、`sim_save_reset`	与真实启动一致
环境审查	检查黑盒和模型问题	`report_blackbox`、`check_fv_setup`	不存在关键环境缺陷
运行	执行验证	`check_fv -block`	任务真正结束
分析	输出证明状态和反例	`report_fv`、`view_trace`	性质与前提一致
回归	保存可复现结果	`save_session`、日志/manifest	版本与运行资源可追踪

流程中的每一步都会改变后续证明问题。编译决定结构和属性是否存在，时钟与复位决定状态更新和初始状态，约束决定允许的输入行为，证明命令才负责求解。报告必须与产生它的设置一起保存；只保留最后的通过数量，无法重现验证结论。

3. 单时钟计数器：完整 RTL、SVA 与运行流程

本节直接给出全部依赖文件。按“保存为”的路径建立目录并保存代码，就能准备对应教学工程，不必再到附件寻找 RTL、断言或脚本。

3.1 被验证的计数器 RTL

规格：复位后计数为零；使能时从 0 计到 9 后回到 0；没有使能时保持。RTL 文件只包含设计，不把待验证结论写成环境假设。

保存为 案例工程/rtl/counter.sv：

```
module counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       enable,
    output logic [3:0] count
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)             count <= 4'd0;
    else if (enable) begin
      if (count == 4'd9)    count <= 4'd0;
      else                  count <= count + 4'd1;
    end
  end
endmodule
```

3.2 完整断言与绑定文件

history_valid 在复位后建立历史有效标志。ap_range 检查范围，ap_hold 检查保持，ap_increment 检查普通递增，ap_wrap 检查回绕；两条 cover 检查最高计数值及回绕场景是否能够出现。末尾 bind 将检查器接入同名设计信号，不需要修改 RTL。

保存为 案例工程/formal/counter_props.sv：

```
module counter_props (
    input logic       clk, rst_n, enable,
    input logic [3:0] count
);
  logic history_valid;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) history_valid <= 1'b0;
    else        history_valid <= 1'b1;

  ap_range: assert property (@(posedge clk) disable iff (!rst_n)
      count <= 4'd9);

  ap_hold: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(!enable) |-> count == $past(count));

  ap_increment: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count < 4'd9)
      |-> count == $past(count) + 4'd1);

  ap_wrap: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count == 4'd9)
      |-> count == 4'd0);

  cp_reaches_nine: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9);
  cp_wrap: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9 ##1 count == 4'd0);
endmodule

bind counter counter_props u_counter_props (.*);
```

3.3 编译文件列表

先列 RTL，再列包含 bind 的属性文件；路径相对于运行 VC Formal 的 案例工程/ 目录，而不是 Tcl 文件所在目录。

保存为 案例工程/filelist_counter.f：

```
./rtl/counter.sv
./formal/counter_props.sv
```

3.4 完整运行脚本

依次选择应用、读入设计与 SVA、定义时钟复位、保存初始状态、检查设置、列出属性并开始证明。初始化步骤需要在实际安装环境核对复位状态，不能把声明复位等同于已经完成复位。

保存为 案例工程/scripts/fpv_counter.tcl：

```
# Run from the example project root with: vcf -f scripts/fpv_counter.tcl -batch
set_fml_appmode FPV
read_file -top counter -sva -format sverilog -vcs {-f filelist_counter.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup -block
report_fv_setup -list
report_fv -list
set_fml_var fml_max_time 1H
set_fml_var fml_max_memory 8GB
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_counter.tcl -batch 2>&1 | tee logs/fpv_counter.log
```

结果阅读。在初始状态正确的前提下，正确版预期满足这些断言，并存在到达 9 和回绕的覆盖轨迹。这不是本次 VC Formal 的实际运行结果。这些断言分别覆盖范围、保持、普通递增和回绕；复位值及运行中重新复位仍需结合初始化模型和专用复位检查，不能仅凭范围属性成立就认定全部功能正确。

3.5 故意错误的 RTL

错误只在回绕阈值：把 9 改为 10。模块名仍为 counter，所以此文件与正确版必须分别编译。属性文件沿用本节已经完整列出的 counter_props.sv。

保存为 案例工程/rtl/counter_bad.sv：

```
// Intentionally buggy teaching variant: roll-over condition is 10 instead of 9.
// Do NOT include with rtl/counter.sv in the same compilation.
module counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       enable,
    output logic [3:0] count
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)             count <= 4'd0;
    else if (enable) begin
```

```
      if (count == 4'd10)   count <= 4'd0;
      else                  count <= count + 4'd1;
    end
  end
endmodule
```

3.6 错误版编译文件列表

只读入错误设计及同一份检查器，不同时包含两个同名设计。

保存为 案例工程/filelist_counter_bad.f：

```
./rtl/counter_bad.sv
./formal/counter_props.sv
```

3.7 错误版完整运行脚本

与正确版采用相同接口和初始化方法，不为了让错误版通过而增加输入限制。

保存为 案例工程/scripts/fpv_counter_bad.tcl：

```
# Intentionally failing counter example. Do not include the good counter at once.
set_fml_appmode FPV
read_file -top counter -sva -format sverilog -vcs {-f filelist_counter_bad.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup -block
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_counter_bad.tcl -batch 2>&1 | tee logs/fpv_counter_bad.log
```

错误版的预期反例是：在允许连续使能的输入下，计数从 9 进入 10，违反范围与回绕规则。查看反例时应区分时钟沿的采样值与非阻塞赋值后的寄存器值；不能把这里描述的预期现象伪装成工具日志。

4. 如何组织断言

建议把每条属性归入四个类别：初始化（复位值和复位后输出）；协议（有效应答、授权、稳定性）；内部安全性（计数范围、状态合法、资源互斥）；进展性（最大等待时间、错误响应）。每个类别不仅包含断言，还要有能触发关键规则的覆盖目标。可将大型属性拆成“输入合法 → 请求被接受 → 内部状态变化 → 输出响应”的小性质，降低定位和求解难度。

4.1 它真正验证了什么

```
ap_range: assert property (@(posedge clk) disable iff(!rst_n)
  count <= 4'd9);

cp_wrap: cover property (@(posedge clk) disable iff(!rst_n)
  count == 4'd9 ##1 count == 4'd0);
```

`ap_range` 证明占用值不超过上界；`cp_wrap` 搜索一次从 9 到 0 的回绕。两者不能互相替代。若设计从未能到达 9，`ap_range` 仍可能通过，但回绕规则并未真正被验证。对错误 RTL，前者应给出反例，

后者是否能够覆盖取决于更新条件。

4.2 使用 bind 与外部属性模块

推荐把待验证 RTL 与检查器分离，方便调试和提交代码：

```
module interface_checker (
  input logic clk, rst_n,
  input logic valid, ready,
  input logic [7:0] data
);
  ap_stable: assert property (@(posedge clk) disable iff(!rst_n)
     valid && !ready |=> valid && $stable(data));
  cp_transfer: cover property (@(posedge clk) disable iff(!rst_n)
     valid && ready);
endmodule
```

在 `formal/bind.sv` 内写 `bind <DUT_type> interface_checker ...`，然后使绑定文件编译进入 `-sva` 的设计。必须确认属性端口与 DUT 的实际时序一致，不要仅凭属性能被编译就认为建模正确。

5. 属性与运行命令详解

操作	命令	核心区别
建立责任和前提	`fvassert`、`fvassume`、`fvcover`	证明、限制与搜索见证是不同逻辑问题
查询目标	`get_props`	返回可继续筛选和处理的属性集合
执行和报告	`check_fv`、`report_fv`	启动证明与读取状态是两个步骤
调整参与范围	`fvdisable`、`fvenable`	禁用目标不是证明通过

下列命令是 FPV 属性工作的核心。编译、时钟复位、任务复制和集群参数的完整说明位于通用命令章节；本章保留最直接影响断言责任和结果解释的命令全文，便于在完整案例旁理解每个操作。

5.1 fvassert：创建断言或改变已有属性的使用方式

```
fvassert [<name>] -expr {<property-expression>} [-clock <clock>]
fvassert [-class <classes>] [-type <types>] [-usage <usages>]
         [-filter <expression>] <property-names-or-collection>
```

参数解释。带 `-expr` 时创建脚本属性；不带 `-expr`、直接传入已有属性名称时，将其使用方式设为 assert 并启用。`-class` 表示来源类别，`-type` 是声明或生成类型，`-usage` 是当前用途，三者不是同义词。

用法与结果。表达式含显式采样事件时使用该事件；此时额外写 `-clock` 会给出警告并优先采用表达式中的时钟。没有事件时使用参考时钟。表达式用花括号保护，避免 `$past` 和位选择被 Tcl 解释。将假设转成断言是验证环境承诺的方法之一，但必须在承担该承诺的设计范围内证明。

```
fvassert AP_HOLD -expr {
  @(posedge clk) disable iff (!rst_n)
  valid && !ready |=> valid && $stable(data)
}
fvassert assume_input_protocol
```

来源：User Guide，X-2025.06-SP1，第 87–89、106–113 页。

5.2 fvassume：创建环境前提或改变属性用途

```
fvassume [<name>] -expr {<expression>} [-clock <clock>]
```

```
fvassume [<name>] -expr {<combinational-expression>} -depth <n>
fvassume [<name>] -env -expr {<environment-expression>}
fvassume <property-names-or-collection>
```

参数解释。普通 `-expr` 创建证明过程中的假设。`-depth n` 要求正整数，仅在复位后的前 n 个采样周期施加，默认以参考时钟上升沿计数，可用 `-clock` 改变；其表达式必须是组合逻辑，不能含 SVA 时序运算符。`-env` 创建同时参与初始化和正式分析的环境约束，表达式只允许涉及主输入、未驱动网络、截断点和黑盒输出等指定对象，且复位仿真能接受的表达式有额外限制。

用法与结果。`-env` 必须在 `sim_run` 之前建立，否则对已经完成的初始化不生效。假设应表达外部环境确实保证的条件；在同一任务中把待证明的设计输出直接设成假设，会把问题本身删除。已证明断言的复用必须说明证明依赖，不能形成循环。

```
fvassume AS_INPUT_STABLE -expr {
  @(posedge clk) disable iff (!rst_n)
  valid && !ready |=> valid && $stable(data)
}
fvassume AS_BOOT_MODE -expr {mode == 2'b00} -depth 3
fvassume AS_NO_TEST -env -expr {test_mode == 1'b0}
```

来源：User Guide，X-2025.06-SP1，第 66–76、87–89、106–113 页。

5.3 `fvcover`：创建可达性目标并扩展到事务结束

```
fvcover [<name>] -expr {<sequence-or-expression>}
fvcover -extend_cond {<end-condition>} [-suffix <suffix>]
        <base-cover-property>
fvcover [-usage cover] [-class <class>] [-type <type>]
        -extend_cond {<end-condition>} [-suffix <suffix>]
```

参数解释。`-expr` 创建要寻找见证的目标。`-extend_cond` 基于已有覆盖追加布尔结束条件，不能与 `-expr` 同时使用；`-suffix` 指定新名字后缀，省略时追加 `_extended`。也可以按 source、script、covergroup 等属性类别或类型选择基础覆盖。

用法与结果。到达一个覆盖点不等于事务已经正常退出。扩展覆盖要求该点之后还能到达空闲或事务清空条件，适合检查“触发以后仍能完成”。它本质上仍在寻找一条轨迹，并不自动证明所有命中该点的轨迹都能完成。

```
fvcover CP_REQUEST -expr {@(posedge clk) req}
fvcover CP_TRANSFER -expr {@(posedge clk) req ##[1:4] ack}
fvcover -extend_cond {idle} -suffix eot CP_TRANSFER
```

来源：User Guide，X-2025.06-SP1，第 87–89、108–110 页。

5.4 `get_props`：按类型、用途和状态查询属性集合

```
get_props [-class <classes>] [-type <types>] [-usage <usages>]
          [-status <statuses>] [-filter <expression>]
          [-regexp] [-exact] [-selected] [<names-or-patterns>]
```

参数解释。`-class` 包括 source、script、aep、connection、coverage、register、security、seq、xprop、fusa 等来源；`-type` 描述具体种类；`-usage` 为 assert、assume、cover。`-status` 按报告状态筛选，`-filter` 根据属性表达式筛选，`-regexp` 和 `-exact` 调整匹配规则；`-selected` 只取界面当前选择的属性。

用法与结果。返回的是工具集合。空集合可能是名称错误、没有生成目标、筛选条件错误或属性被停用，不能直接解释为所有目标已经通过。应查询集合大小并结合未筛选的总报告。

```
set asserts [get_props -usage assert]
puts "Assertions: [sizeof_collection $asserts]"
foreach_in_collection p $asserts {
```

```
    puts "[get_object_name $p] [get_attribute $p status]"
}
```

来源：User Guide，X-2025.06-SP1，第 110–114 页。

5.5 check_fv：证明属性或寻找覆盖见证

```
check_fv [-block] [-run_finish <commands>] [-stop]
         [-break <number-of-falsifications>]
         [-property <properties-or-collection>]
         [-subtype <property|vacuity|witness>] [-assume]
```

参数解释。`-block` 阻塞直到本次检查结束；默认非阻塞。`-run_finish` 指定非阻塞完成后执行的 Tcl 命令。`-stop` 停止当前运行，`-break` 在达到指定失败数时停止。`-property` 选择目标；`-subtype` 可取一个或多个 property、vacuity、witness。`-assume` 允许把已证明断言及不可达覆盖用于约束。

用法与结果。证明同时服从启用的假设、常量、初始状态、黑盒和截断设置。非阻塞运行期间可以查看报告，但不能并行改变影响运行的设置。完成意味着这次资源限制下的运行结束，不等于全部目标获得确定结果；因此报告中的未完成状态必须单独处理。

```
check_fv_setup -block
check_fv -block
report_fv -list > results.txt

check_fv -property [get_props AP_HOLD] \
    -subtype {property vacuity} \
    -run_finish {report_fv -verbose > hold_results.txt}
```

来源：User Guide，X-2025.06-SP1，第 184–189、202–207 页。

5.6 report_fv：读取证明状态、深度和详细结果

```
report_fv [-class <classes>] [-type <types>] [-usage <usages>]
          [-status <statuses>] [-list] [-verbose] [-no_summary]
          [<property-names-or-patterns>]
```

参数解释。`-list` 输出逐项列表，`-verbose` 增加引擎、用时、证明深度和属性详细信息，`-no_summary` 省略汇总。筛选参数限定报告对象，但不会运行目标或改变其状态。

用法与结果。报告是当前状态的快照。安全深度表示有界检查已经覆盖的范围，不是无界证明；反例深度表示失败位置，两者不能合并比较。筛选后只看到 proven 目标时，必须另查总目标数以及其他状态，防止通过筛选制造“全部通过”的假象。

```
report_fv
report_fv -list -status falsified > failures.txt
report_fv -verbose -no_summary AP_HOLD
```

来源：User Guide，X-2025.06-SP1，第 247–261 页。

5.7 fvdisable：停用选定的断言、假设或覆盖

```
fvdisable [-class <classes>] [-type <types>] [-usage <usages>]
          [-filter <expression>] [<properties>]
```

参数解释。可按来源、声明类型、当前用途或名称选择属性；不明确区分筛选条件容易影响错误的对象。`-usage assume` 选择当前参与约束的属性，而非所有原始声明为 assume 的属性。

用法与结果。停用断言只是不再检查该断言，停用假设则放宽了允许行为，会使已有证明资料失效或需要重新求解。UG 对 enabled 与 usage 的部分旧例描述并不一致，脚本应查询这两个属性，不依赖旧输出中的 none 文本。

```
fvdisable -class aep -type bounds_check
fvdisable -usage cover
```

```
report_fv -list
```

来源：User Guide，X-2025.06-SP1，第 106–113、375 页。

5.8 fvenable：重新启用已有属性

```
fvenable [-class <classes>] [-type <types>] [-usage <usages>]
         [-filter <expression>] [<properties>]
```

参数解释。按选择条件启用已有目标，不重新生成 RTL 中未插入的检查。

用法与结果。重新启用与重新声明不是一回事。若使用 `aep_config -dont_generate` 在编译前阻止了生成，这里没有可供启用的对象；需要重新配置并编译。重新启用假设后，应重新检查是否产生约束冲突或空洞。

```
fvenable -class aep -type bounds_check
fvenable AP_HOLD
report_fv -list
```

来源：User Guide，X-2025.06-SP1，第 106–113 页。

6. 四个逐步进阶的用例

用例 A：复位后状态正确

规格：复位退出后，状态机应进入空闲状态。首先明确同步或异步复位和断言采样时刻：

```
ap_reset_release: assert property (
  @(posedge clk) $rose(rst_n) |-> state == IDLE);
```

这段规则只适用于状态在第一次采样到复位高电平的时刻应该已经是 `IDLE` 的实现。若模块采用同步释放后的一个周期才进入有效状态，则必须调整响应周期。为复位前提单独建立初始化流程，不能只靠 `disable iff` 回避复位检查。

用例 B：握手停止期间数据不变

```
ap_stall_hold: assert property (@(posedge clk) disable iff(!rst_n)
  valid && !ready |=> valid && $stable(data));
cp_backpressure: cover property (@(posedge clk) disable iff(!rst_n)
  valid && !ready ##1 ready);
```

如 `ready` 为外部驱动，禁止假设 `ready` 始终为 1；否则背压规则可能很容易被证明，却没有真正触发的测试场景。

用例 C：深层计数器溢出

使用参数化深度为 10 的计数器，故意把最大值条件写错。预期安全性 `count <= 9` 会失败；反例提供每拍 `enable`，可从逐周期变化找到第一次非法值。修复后重新运行，与更早的报告比较应以 RTL 的校验值为前提。

用例 D：多端口仲裁

```
ap_mutex: assert property (@(posedge clk) disable iff(!rst_n)
  $onehot0(grant));
cp_contend: cover property (@(posedge clk) disable iff(!rst_n)
  req == 2'b11);
```

这是最基础的安全性和触发覆盖。若想证明“等待不超过 8 拍”，必须明确持续请求和仲裁器能否接收新请求，并对每一个端口分别写属性。状态空间复杂时，可先证明互斥与请求合法，再证明有界公平性。

6.5 FIFO 控制器：完整代码

这个例子只验证 FIFO 的占用计数、空满标志和请求接受规则，不包含存储阵列，因此不能据此宣称已经证明数据顺序正确。满状态允许同时读写；空状态不能读出，且本例没有空 FIFO 直通功能。

6.5.1 控制器 RTL

`pop_accept` 先排除空读；`push_accept` 在未满或本拍成功读出的条件下接受写入。实际更新依据接受信号，而不是原始 `push/pop` 请求。

保存为 案例工程/rtl/fifo_ctrl.sv：

```
module fifo_ctrl #(
    parameter int DEPTH = 4,
    parameter int CW = $clog2(DEPTH + 1)
) (
    input logic clk, rst_n,
    input logic push, pop,
    output logic full, empty,
    output logic push_accept, pop_accept,
    output logic [CW-1:0] occupancy
);
  assign full       = (occupancy == DEPTH);
  assign empty      = (occupancy == 0);
  assign pop_accept  = pop && !empty;
  assign push_accept = push && (!full || pop_accept);

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) occupancy <= '0;
    else case ({push_accept, pop_accept})
      2'b10: occupancy <= occupancy + 1'b1;
      2'b01: occupancy <= occupancy - 1'b1;
      default: occupancy <= occupancy;
    endcase
  end
endmodule
```

6.5.2 完整 SVA 与参数绑定

属性参数与 RTL 的 `DEPTH/CW` 对应；检查范围、标志一致性以及禁止空读、禁止无空间写入。覆盖目标辅助确认满状态与使用后回到空状态可达。

保存为 案例工程/formal/fifo_ctrl_props.sv：

```
module fifo_ctrl_props #(
  parameter int DEPTH = 4,
  parameter int CW = $clog2(DEPTH + 1)
) (
  input logic clk, rst_n, push, pop,
  input logic full, empty, push_accept, pop_accept,
  input logic [CW-1:0] occupancy
);
  ap_safe_count: assert property (@(posedge clk) disable iff (!rst_n)
    occupancy <= DEPTH);
  ap_flag_full: assert property (@(posedge clk) disable iff (!rst_n)
    full == (occupancy == DEPTH));
  ap_flag_empty: assert property (@(posedge clk) disable iff (!rst_n)
    empty == (occupancy == 0));
  ap_no_empty_pop: assert property (@(posedge clk) disable iff (!rst_n)
    pop_accept |-> !empty);
  ap_no_overflow_push: assert property (@(posedge clk) disable iff (!rst_n)
```

```
    push_accept |-> (!full || pop_accept));
  cp_full: cover property (@(posedge clk) disable iff (!rst_n)
    full);
  cp_empty_after_use: cover property (@(posedge clk) disable iff (!rst_n)
    !empty ##[1:5] empty);
endmodule

bind fifo_ctrl fifo_ctrl_props #(.DEPTH(DEPTH), .CW(CW)) u_fifo_ctrl_props (.*);
```

6.5.3 文件列表

保存为 案例工程/filelist_fifo.f：

```
./rtl/fifo_ctrl.sv
./formal/fifo_ctrl_props.sv
```

6.5.4 完整脚本

保存为 案例工程/scripts/fpv_fifo.tcl：

```
set_fml_appmode FPV
read_file -top fifo_ctrl -sva -format sverilog -vcs {-f filelist_fifo.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup -block
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_fifo.tcl -batch 2>&1 | tee logs/fpv_fifo.log
```

7. 如何分析结果

`Proven`：属性在该模型和假设下成立。需要查触发是否可达与约束是否合理。`Falsified`：工具给出违反路径，首先检查输入是否符合环境规格与时钟复位是否正确。`Inconclusive`：目前未完成，不能当作通过。`Covered`：存在一条满足覆盖目标的路径。`Uncoverable`：该目标在当前模型不可达，需要区分真实死逻辑与过度约束。详见 验证工程与结果判读。

8. 优化证明效率而不损害结论

1. 缩小模块验证范围，必要时划分不同 `fvtask`；不要先大面积假设内部值。
2. 将复杂的一条断言拆成可解释的若干不变量与阶段性结果。
3. 分析影响锥，确认算术、RAM、时钟域和不必要的外围逻辑是否进入证明。
4. 再考虑 `set_fml_var fml_effort high`、资源限制、`set_abstractions`、`snip_driver` 等进阶优化；新抽象可能引入虚假反例。
5. 对长期回归使用符合条件的 `fvlearn_config -local_dir ...`，并区分继承的缓存与新运行获得的真实结果。
6. 使用 `report_fml_engines`、`report_fml_jobs`、`report_fml_hosts` 定位瓶颈。

9. 常见故障与排查

现象	优先检查	不应做的事
所有断言都轻松通过	触发覆盖、空洞性和假设	直接宣布完整验证通过
许多断言在第 0 拍失败	初始化状态与 `$past`	假设所有寄存器为预期值
证明几乎没有进展	影响锥、参数和 RAM	无审查地增加内部假设
只有有界深度结论	是否真正 `Proven`	把展开深度当作无限证明
Verdi 中找不到信号	优化、层级、会话版本	在不知影响的情况下关掉全部优化
频繁得到非法请求反例	环境模型与协议规范	把所有 DUT 失败行为都排除

优先检查会改变验证问题本身的设置，再考虑求解器。没有生成目标、输入被固定、复位释放错误或属性触发条件不可达，都可能造成“运行很快且全部通过”的假象。对任何性能优化，都应比较修改前后的属性数量、环境前提和关键覆盖场景。

10. 进阶主题

FPV 可以与 AIP、覆盖质量评估（FCA）、FTA 的形式测试平台突变分析、Proof Architect 的组合式证明方法等配合。有关无法快速证明的属性，阅读 调试与证明性能；有关交付报告，阅读 覆盖与验证验收。FPV 的物理规模限制随属性影响锥、状态空间及引擎而变化，没有一个对所有设计都有效的固定门数门槛。

11. 参考定位

User Guide §17（FPV，686 页起），§3（SVA/属性，66 页起），§4（初始化，115 页起），§6 ~ 7（运行与调试，183 ~ 312 页），§18 ~ 20（性能、组合式证明与验收）。Quick Reference Guide 的 FPV 页面提供命令速览，但不是全部参数说明。

AEP：自动提取属性应用使用指南

1. AEP 的角色与应用价值

AEP 不需要用户先手工写出每一条 SVA，就能从 RTL 结构和编码方式中自动推导一批易引发功能问题的条件，并将其交给形式引擎检查。适用于新接入 IP、第三方 RTL、复杂状态机、溢出与数组访问风险排查，以及验证项目启动阶段的快速质量筛查。

AEP **不等于自动证明全部功能规格**。它擅长发现通用的实现错误（越界、算术溢出、状态机问题、三态冲突、一些不完整分支等），但不会知道客户要求某个控制寄存器在第几个周期复位，也不会自动理解外部协议的最终响应期限。经 AEP 筛查后仍需补充 FPV 业务属性。

2. 检查对象与生成条件

检查类型	选择值	主要检查对象
数组访问	`bounds_check`	数组索引是否超出声明范围
算术结果	`arith_oflow`	支持的运算是否发生算术溢出
移位结果	`shift_oflow`	左操作数的全部位是否被移出
类型转换	`cast_oflow`	赋值右侧最外层转换的有效位是否丢失
三态悬空	`floating_bus`	是否没有有效三态驱动
三态多驱动	`multi_driver`	是否同时启用多个三态驱动
三态驱动冲突	`conflict_driver`	多个三态驱动是否输出相反值
分支完整性	`full_case`	声明为完整的选择条件是否确实完整
分支互斥性	`parallel_case`	声明为并行的选择条件是否确实互斥
异步置位与复位	`set_reset`	异步置位和复位是否同时有效
显式未知值	`x_assign`	显式 X 赋值是否可被执行
单状态无限等待	`fsm_deadlock`	是否无限停留于一个状态
多状态循环等待	`fsm_livelock`	是否困在多个状态组成的循环
状态机综合锁定	`fsm_unionlock`	用一条状态机属性检查两种锁定

算术、移位与转换不是同一检查。`shift_oflow` 关注全部位被移出，不是只要丢失任意一位就失败。8 位左操作数使用最大为 7 的移位量时，不能仅因高位被丢弃就推断该检查一定失败。`cast_oflow` 的自动提取条件是赋值右侧最外层为 `$signed` 或 `$unsigned`，然后检查参数中最左侧的置位能否存入左侧信号。两者在本版本均为 beta 功能，不能当作通用的数值精度或所有截位问题检查器。

三态多驱动与数值冲突必须区分。两个三态驱动同时开启属于多驱动；若两者都输出相同值，未必触发 `conflict_driver`。冲突检查针对相反值同时驱动的情况，并不为普通的非三态多赋值结构生成同一种目标。悬空检查则关注所有驱动都关闭，三者分别覆盖不同场景。

分支检查依赖设计声明。`full_case` 与 `parallel_case` 检查相应综合指示或 SystemVerilog 的 `unique`、`priority` 等结构所作的承诺，并不等于对每个没有 `default` 的分支都生成相同检查。某个非法编码必须被拒绝、忽略还是返回错误，应由功能规格决定，不能依靠增加综合注释消除设计问题。

状态机检查需要单独启用。`fsm_deadlock` 与 `fsm_livelock` 按状态生成相应目标，`fsm_unionlock` 则为每个状态机生成一条同时考虑死锁和活锁的目标。它们计算代价较高，**不会因为使用 `-aep all` 自动生成**。等待外部响应是否属于错误，还取决于真实系统是否保证最终响应；公平性条件必须来自环境保证，而不是为使目标通过临时添加。

本版本 VHDL 不支持 `parallel_case`、`full_case`、`shift_oflow` 和 `cast_oflow`；三态总线检查还需要额外启用变量。来源：User Guide，第 363–371 页。

3. 入门 AEP 案例：数组越界

3.1 完整设计源码

数组只有两个元素，但两位 `index` 能表示 0、1、2、3。这里故意保留未检查的索引访问；数组内容是否已初始化与本例的索引越界问题应分开分析。

保存为 案例工程/rtl/array_demo.sv：

```
// Deliberately has an index range wider than the 2-element memory.
module array_demo (
  input  logic       clk,
  input  logic [1:0] index,
  output logic [7:0] data_out
);
  logic [7:0] mem [0:1];
  always_ff @(posedge clk)
    data_out <= mem[index];
endmodule
```

如果接口规格确实允许任意两位索引，就应当由 RTL 拒绝非法访问。只有在接口规范明确禁止索引 2、3 时，才可以建立相应环境假设，不能为了消除反例任意限制输入。

3.2 完整文件列表

保存为 案例工程/filelist_aep.f：

```
./rtl/array_demo.sv
```

4. 标准运行步骤

4.1 最小数组检查脚本

先选择 `bounds_check`，再检查属性是否实际生成。没有生成属性不等同于数组安全。

保存为 案例工程/scripts/aep_bounds.tcl：

```
set_fml_appmode AEP
read_file -top array_demo -aep bounds_check -format sverilog -vcs {-f filelist_aep.f}
create_clock clk -period 100
check_fv_setup -block
report_fv -list
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/aep_bounds.tcl -batch 2>&1 | tee logs/aep_bounds.log
```

真实工程有复位的，应加上 `create_reset` 和必要的初始化序列；上述 `array_demo` 刻意只有时钟，突出数组检查的概念。若工具对该语言、数组形式或优化方式没有生成相应属性，先检查 `-aep` 语法与对象是否符合其抽取前提。

4.2 多种 AEP 组合

```
read_file -top my_top -aep fsm_deadlock+fsm_livelock \
  -format sverilog -vcs {-f filelist.f}
```

这里的 + 表示组合多个检查选择值，是指南中提供的一种常见写法；`-aep all` 不包含状态机锁定检查，也不意味着应对每个设计一开始就开启全部普通检查。建议先对高优先级逻辑运行小集合，再逐步扩展，以便降低误报排查成本。

4.3 VHDL 的相关配置

```
set_app_var fml_enable_vhdl_aep true
set_fml_var fml_enable_vhdl_bus_checks true
set_app_var fml_enable_vhdl_cov true
```

前一个是应用变量，后一个是形式变量，分别用 `set_app_var` 和 `set_fml_var` 设置，不能互换。它们用于启用 VHDL 自动检查及三态总线检查。VHDL 的语言结构与 SystemVerilog 不同，不能假定每一种 AEP 的覆盖条件完全相同。

5. 状态机检查：死锁、活锁与公平性

FSM 死锁一般指在给定约束下进入无法合理继续的状态；活锁指能够不停转移却始终不进入完成或期望状态。是否一定算设计缺陷，取决于环境前提：一个等待读取外设的状态如果允许外设永远不响应，可能是系统规格缺失，而非状态机 RTL 必然有错。

```
read_file -top controller -aep fsm_deadlock+fsm_livelock \
  -format sverilog -vcs {-f controller.f}
aep_generate -type fsm_fairness
# 可把候选公平性配置保存为脚本后审查
aep_generate -type fsm_fairness -tcl fairness.tcl
```

生成公平性相关条件时应由系统工程师审查其物理可实现性。例如“不停请求外部总线”并不能保证对方必然授权，除非系统规格明确了调度公平性。

5.1 控制某种状态的自动生成

```
aep_config -dont_generate -type fsm_livelock {state_reg:state_name}
```

该指令适用于已确认特定状态的自动生成规则不适当的情况；必须给出关闭原因和关联设计评审。示例 `state_reg:state_name` 只是占位对象，实际名称应从生成属性和设计状态列表取得。不要以大量关闭属性的方式解决真问题。

6. 命令语法、参数与用例

命令	主要作用
`aep_config`	在读入设计前控制自动检查生成
`aep_generate`	生成状态机公平性及输入变化约束
`enable_aep_goals_in_coi`	只启用指定 FPV 目标相关的 AEP
`fvwaive`	对可豁免目标记录豁免
`fvunwaive`	撤销目标豁免
`save_waiver_file`	保存当前目标豁免
`read_waiver_file`	读入已审核豁免

本组命令按准备、生成、执行与结果处理的关系组织。语法框保留参数名称和可选关系；示例中的实

例、属性和文件名称需要与当前工程一致。生成目标与证明目标是不同阶段，调用生成命令后应检查实际目标数量，再启动证明。

6.1 aep_config：在读入设计前控制自动检查生成

```
aep_config [-dont_generate] [-type <type-list>]
           [-enable_counter_arith_oflow <true|false>]
           [-generate_only_with_fanout <true|false>]
           [<signal-name:state>]
```

参数解释。 `-dont_generate` 阻止所选类别生成；`-type` 可选择数组越界、算术溢出、驱动冲突、悬空、多驱动、分支完整性或互斥、置位复位、X 赋值、三种状态机锁定检查以及 all。位置参数可精确到状态寄存器及某一状态，使用单个冒号连接。`-enable_counter_arith_oflow` 控制自增、自减计数器的算术溢出检查，默认 true；`-generate_only_with_fanout` 只保留目标信号有扇出使用的检查，默认 false。

用法与结果。本命令必须在 `read_file` 之前执行。它改变插入到模型中的目标，而 `fvdisable` 只停用已生成的目标。对于设计本来允许等待的 RESET 状态，应按规格局部排除，不应把全部死锁检查关闭。UG 语法注释中提到的 `-exclude` 与展示选项不一致，示例采用正文明确给出的 `-dont_generate`。

```
set_fml_appmode AEP
aep_config -dont_generate -type fsm_deadlock \
    {u_ctrl.state:RESET}
aep_config -enable_counter_arith_oflow false
read_file -top ctrl -format sverilog \
    -aep bounds_check+fsm_deadlock -vcs {ctrl.sv}
```

来源：User Guide，X-2025.06-SP1，第 375–377 页。

6.2 aep_generate：生成状态机公平性及输入变化约束

```
aep_generate -type <fsm_fairness|change_at>
             [-input_type <in|snip|abstract|bbout>]
             [-inputs <inputs-or-collection>] [-tcl <file_name>]
             [-max_width <integer>] [-bit] [-property <properties>]
```

参数解释。 `-type fsm_fairness` 为有关输入建立公平性前提；`change_at` 建立允许变化边沿规则。`-input_type` 指定主输入、截断点、抽象点或黑盒输出；`-inputs` 指定对象，不给时使用属性影响范围内的输入。`-tcl` 把生成结果写到文件以供审核。`-max_width` 限制总线位宽：未指定 inputs 时默认阈值 2，指定 inputs 时默认 10；超限会警告，没有生成任何公平性约束会报错。`-bit` 按位生成，`-property` 限定相关属性。

用法与结果。先建立时钟、复位和常量，再生成。总线公平性可能要求遍历其全部可能值，按位公平性则与总线组合公平性不同；这些都是额外环境承诺。建议先输出 Tcl，逐条确认外部环境真的保证这种变化，之后才 `source`。不能用公平性强迫设计内部卡住的状态“总会变化”。

```
create_clock clk -period 100
create_reset rst_n -sense low
aep_generate -type fsm_fairness \
    -inputs {ready response_valid} -max_width 2 \
    -tcl fairness_review.tcl
```

来源：User Guide，X-2025.06-SP1，第 377–380 页。

6.3 enable_aep_goals_in_coi：只启用指定 FPV 目标相关的 AEP

```
enable_aep_goals_in_coi [-type <types>] [-status <statuses>]
                        [-filter <expression>] [-regexp] [-exact]
                        [<names-ids-or-property-collections>]
```

参数解释。位置参数指定 FPV 目标；`-type` 限制 AEP 类别，`-status` 限定状态，`-filter` 施加属性过滤，`-regexp` 或 `-exact` 改变名称匹配。

用法与结果。该命令在 FPV 应用中工作，会启用所选目标影响范围内的 AEP，并停用其他 AEP。它用于聚焦相关结构，不等于其他结构已经验证过。使用后应记录 AEP 总数和启用数量，避免在汇总报告中忽略未检查部分。

```
fvassert AP_OUTPUT_NONZERO -expr {out > 0}
enable_aep_goals_in_coi AP_OUTPUT_NONZERO
report_fv -class aep -list
```

来源：User Guide，X-2025.06-SP1，第 373–375 页。

6.4 `fvwaive`：对可豁免目标记录豁免

```
fvwaive <property-name>
```

参数解释。指定已经审核、需要保留为例外的目标。AEP 豁免流程要求读入设计前设置 `fml_aep_enable_waiver_flow`。

用法与结果。豁免不把目标变成已证明。UG 明确说明该机制不支持 assert 类型 AEP，因此不能把本命令当成通用的断言免检方式。FCA 还有覆盖目标豁免，应区分不同应用和目标类型。

```
set_fml_var fml_aep_enable_waiver_flow true
# 在设计读入和目标审核完成后执行：
fvwaive reviewed_waivable_goal
```

来源：User Guide，X-2025.06-SP1，第 383–384、424–428 页。

6.5 `fvunwaive`：撤销目标豁免

```
fvunwaive <property-name>
```

参数解释。指定已有豁免的目标名称。

用法与结果。规格、RTL 或输入约束变化后，应撤销不再有依据的旧豁免并重新分析；只重新导入旧排除文件会继承过时结论。

```
fvunwaive reviewed_waivable_goal
report_fv -list
```

来源：User Guide，X-2025.06-SP1，第 383–384、424–428 页。

6.6 `save_waiver_file`：保存当前目标豁免

```
save_waiver_file -file <filename>
```

参数解释。 `-file` 指定输出文件；AEP 用法保存可复用的 XML 豁免内容。

用法与结果。文件扩展名不能决定其真实格式。保存时应同时保存豁免理由、适用设计版本和审核人；工具文件通常不足以表达全部工程依据。

```
save_waiver_file -file aep_waivers.xml
```

来源：User Guide，X-2025.06-SP1，第 384 页。

6.7 `read_waiver_file`：读入已审核豁免

```
read_waiver_file -elfiles <filename>
```

参数解释。 `-elfiles` 在本版 AEP 用法中指定先前保存的豁免文件。

用法与结果。应在目标已经建立、正式检查之前导入，并确认匹配的目标数量。名称变化造成豁免未匹配，与目标无需再检查，是不同情况。

```
read_waiver_file -elfiles aep_waivers.xml
```

```
report_fv -list
check_fv -block
```

来源：User Guide，X-2025.06-SP1，第 384 页。

7. 常见问题与处理方案

问题 A：数组越界属性失败。查索引值、数组实际维度、索引来源、是否为非法但允许输入；若设计应该防御非法访问，则修复输入检查和返回行为。**问题 B：`full_case` 告警**。检查组合分支是否覆盖所有合法编码；若某些编码应视为非法，先明确可达性，不能仅靠注释 `full_case` 掩盖。**问题 C：状态机活锁告警**。查看反例循环，确认是否需要外设公平假设。**问题 D：属性数为零**。检查 AEP 开关、RTL 类型和提取范围，不应推断“RTL 不存在问题”。

用例：审查豁免

```
fvwaive <property_name>
save_waiver_file -file reviewed_aep.el
```

建议豁免记录独立写明编号、原始现象、为何不是设计缺陷、所依据的系统规格、最后审查日期与责任人。`fvwaive` 只是结果管理，不会使设计行为变正确。

8. 推荐的 AEP + FPV 双阶段工作方法

先在整段 RTL 范围运行 AEP 发现基础实现风险；其次针对关键模块编写 FPV 属性以覆盖具体的时序与功能规格。AEP 发现的真缺陷，应补充成可长期回归的手工断言（如果合适）。AEP 发现的假阳性，应检查真实前提并保留说明；不可简单添加 `assume` 让它消失。

对于大型 SoC，可以先针对地址解码、寄存器文件索引、FIFO 状态机、宽窄运算及三态总线使用 AEP。对于自动生成属性无法证明的情况，可单独分组并提升资源，而不是影响所有检查的分析时间。

9. 官方参考

User Guide §10：第 362 ~ 385 页；§3.5 属性 class 与 type；§7 反例与状态；Quick Reference Guide 的 AEP 章节。本文检查类型的简述为独立归纳，并非逐页复制官方表格。

FXP：形式未知值传播验证使用指南

1. 为什么未知值传播值得单独验证

RTL 仿真中的 X 可能表示：寄存器尚未复位、未驱动、赋值了 X、模块未定义、数组越界、选择条件不确定等。真实电路并不存在一种独立的 “X 电压值”；它表示形式或仿真层面对确定性的缺失。在安全控制、启动流程、电源域切换、复位释放和错误路径中，一个无法确定的控制值可能在时序上被传播或放大。

FXP 的核心目标是寻找：**指定的未知值来源是否可能影响不应该出现未知值的观察信号**。它通过生成 X 注入点与观察点、建立检查并使用形式引擎寻找违反条件的轨迹。普通 FPV 的二值断言不能自动等同于这种 X 传播分析。



图 7: FXP 注入源和观察点

2. 注入点与观察点的概念

注入点是允许引入未知值的位置，例如未初始化寄存器、显式 X 赋值、未驱动信号、黑盒输出或人为指定的关键节点。**观察点**是要求未知值不应到达的位置，例如顶层功能输出、关键状态机控制端、黑盒输入和被指定为需要稳定可判定值的寄存器。

形式检查回答的是：在当前复位和输入条件下，能否找到一条路径，让注入的不确定性传播到观察点？它不是统计 X 出现概率，也不等于电路可靠性故障率分析。

3. 自动提取对象：来源与观察点

选择值	注入来源	观察位置
`uninit`	未初始化寄存器	否
`xassign`、`undriven`、`undef`	显式未知值、未驱动、未定义运算	否
`in`、`bbout`、`unresout`	顶层输入、黑盒输出、未解析模块输出	否
`out`、`bbin`、`unresin`	否	顶层输出、黑盒输入、未解析模块输入
`oba`	越界引入未知值的位置	越界相关观察位置
`snip`、`abs`	截断与抽象引入的来源	相应截断与抽象观察位置
截断驱动端类别	否	截断网络的驱动端

来源描述不确定性如何进入模型，观察位置描述哪里必须保持确定。它们是两个独立选择维度：只指定未初始化寄存器，不自动等于已经为所有关键输出建立检查；只建立输出观察目标，也不能保证所有需要分析的未知值来源都已注入。应在生成后同时检查来源数量和目标数量。

例如分析 “未初始化缓存是否影响输出”，可以自动选择 `uninit` 与 `out`，或者选择 `uninit` 后用 `fxp_assert` 明确指定输出。本章完整案例采用后一种方式，避免把自动输出目标与手工输出目标重复生成。未初始化缓存存在并不必然构成缺陷；当 `valid` 可靠屏蔽缓存，且下游在无效期间只看到确定值时，不确定性可能不会到达要求确定的观察位置。

UG 的类型表将截断驱动端写作 snip_drv，语法列表另有 snipdrv 写法；本章不将这处差异猜成两种都有效。实际需要选择该类别时，应核对安装版本帮助。其余类别及双重角色来自 UG 第 447–448 页的类型定义。

4. 标准脚本与运行顺序

```
set_fml_appmode FXP
read_file -top soc_top -format sverilog -vcs {-f filelist.f}
create_clock clk -period 100
create_reset rst_n -sense low
set_constant test_mode -value 1'b0
sim_run -stable
sim_save_reset
check_fv_setup -block
fxp_generate
report_fv -list
check_fv -block
report_fv -verbose
```

fxp_generate 使用默认类型自动提取检查。若没有复位的原型模块，应删除不适用的 create_reset，不能输入一个不存在的复位信号。任何被 set_constant 固定的控制输入都应有真实工程依据，否则可能人为切断未知值传播路径。

4.1 显式选择未初始化寄存器类型

```
fxp_generate -name uninitialized_checks -type {uninit}
```

只关注某类来源有利于快速建立第一批可解释的结果。UG 中还给出了将某个范围及若干类型一起配置的方式；例如只针对某个子模块，可传入对应的 scope 列表（需要真实实例路径）。

5. 命令语法、参数与用例

命令	主要作用
fxp_generate	按实例范围和类别生成 X 检查
fxp_assume	控制 X 来源或排除已知确定的位置
fxp_assert	定义条件化的 X 观察点
fxp_compute_rootcause	计算失败 X 检查的上游来源
fxp_report_rootcause	报告 X 来源计算结果

本组命令按准备、生成、执行与结果处理的关系组织。语法框保留参数名称和可选关系；示例中的实例、属性和文件名称需要与当前工程一致。生成目标与证明目标是不同阶段，调用生成命令后应检查实际目标数量，再启动证明。

5.1 fxp_generate：按实例范围和类别生成 X 检查

```
fxp_generate [<instance-scopes>] [-name <prefix>]
             [-type <type-list>]
```

参数解释。范围使用实例名而不是模块定义名；嵌套实例使用相对顶层的层级路径，多个实例写入 Tcl 列表。-name 作为生成属性的共同前缀。-type 选择来源与观察类别，省略时使用默认组合；auto 对应文档列出的默认组合。常见类别包括 uninit、oba、xassign、undef、undriven、in、out、bbin、bbout、unresin、unresout、abs 和 snip。

用法与结果。默认类别并不是“给所有输入都注入 X”：主输入 in、黑盒输出 bbout 和未解析模块输出

unresout 在属性类型表中不是默认。语法列表中的 snipdrv 与表中的 snip_drv 拼写存在差异，教程不把其中任意一个扩展成通用可运行选项。最小案例显式给出 uninit 与 out，确保既有来源又有观察点。

```
fxp_generate -name reset_x -type {uninit out}
fxp_generate {u_cpu u_uart.regs} \
    -name io_x -type {uninit xassign out}
report_fv -class xprop -list
```

来源：User Guide，X-2025.06-SP1，第 447–451 页。

5.2 fxp_assume：控制 X 来源或排除已知确定的位置

```
fxp_assume -injectx [-name <name>] [-reset] [-oba] [-xassign]
           [-undef] [-condition <expression>] [-scope <scope>] <signal>
fxp_assume -nox [-name <name>] [-reset] [-condition <expression>] <signal>
```

参数解释。 `-injectx` 建立 X 来源，`-nox` 从指定点排除 X。`-name` 命名生成目标；`-reset` 用于时序单元的出复位状态。`-oba`、`-xassign`、`-undef` 分别对应越界、显式 X 和未定义行为，作用于整个范围时必须配合 `-scope`；`-condition` 是控制该操作的组合条件。

用法与结果。已经用 `create_clock` 或 `create_reset` 定义的时钟、复位信号不能作为 X 注入源。`-nox` 是模型前提，不会给 RTL 添加复位或屏蔽逻辑；只在规格或外部电路保证确定时使用。UG 的 nox 语法行漏写了 condition 前的连字符，参数解释和快速参考均采用 `-condition`，此处采用后者并保留该差异说明。

```
fxp_assume -injectx -name cold_data -reset {u_buf.data_q}
fxp_assume -nox -name known_mode -condition {cfg_valid} {mode}
fxp_assert -name output_defined -condition {out_valid} {out_data}
```

来源：User Guide，X-2025.06-SP1，第 451–453 页。

5.3 fxp_assert：定义条件化的 X 观察点

```
fxp_assert [-name <name>] [-condition <expression>] <signal>
```

参数解释。 `-name` 指定检查名称，`-condition` 决定在哪些组合条件成立时要求目标信号确定；位置参数为观察信号。

用法与结果。无有效数据的周期允许数据寄存器未初始化时，应只在 out_valid 为真时要求数据确定。同时必须用 FPV 检查 out_valid 不会提前置位，否则错误的屏蔽条件可能让 FXP 看不到真实问题。

```
fxp_assert -name data_when_valid \
    -condition {out_valid} {out_data}
check_fv -block
report_fv -class xprop -list
```

来源：User Guide，X-2025.06-SP1，第 453 页。

5.4 fxp_compute_rootcause：计算失败 X 检查的上游来源

```
fxp_compute_rootcause [-property <property-names>] [-stop] [-block]
```

参数解释。 `-property` 指定失败目标，`-block` 等待定位结束，`-stop` 停止相关计算并清理待计算属性列表。

用法与结果。先有失败属性和轨迹，再执行来源分析。它区分未初始化、显式 X、越界等来源，减少从下游逐级人工回溯的工作，但还需要核对该路径的有效性控制。自动模式由 `fxp_compute_rootcause_auto` 控制，默认 false。

```
fxp_compute_rootcause -property {output_defined} -block
fxp_report_rootcause -property {output_defined} -list
```

来源：User Guide，X-2025.06-SP1，第 456–457 页。

5.5 fxp_report_rootcause：报告 X 来源计算结果

```
fxp_report_rootcause [-property <property-names>] [-list] [-no_summary]
```

参数解释。 `-property` 选择目标，`-list` 按属性逐行输出，`-no_summary` 不显示来源类型汇总。

用法与结果。只执行报告命令不会自动补齐缺失的来源计算。看到某寄存器未初始化后，应进一步判断它是否应有复位、复位是否被门控、工具是否识别了复位以及下游是否错误使用了无效数据。

```
fxp_compute_rootcause -block
fxp_report_rootcause -list
```

来源：User Guide，X-2025.06-SP1，第 456–457 页。

5.6 什么时候允许 -nox

只有当硬件结构、外部协议或正式初始化流程**保证该信号在指定阶段已知**时，才允许消除其 X 来源。例如某寄存器由上电复位明确赋值，且复位时序确实执行；或者某输入在完成训练后才被采样，且存在可证明的有效标志。如果将所有未知数据都用 `-nox` 约束掉，FXP 就失去实际作用。

6. 教学用例 A：未复位的数据寄存器

```
module uninit_demo (
  input  logic clk, rst_n, write,
  input  logic [7:0] in_data,
  output logic [7:0] out_data
);
  logic [7:0] cache;
  always_ff @(posedge clk) begin
    if (write)
      cache <= in_data;
  end
  always_comb out_data = cache;
endmodule
```

`cache` 不受 `rst_n` 复位，且复位后在第一次写入之前可以作为任意初值输出。若 `out_data` 必须始终有效，FXP 应提示风险。修复方式有两类：一是复位 `cache`，二是在 `cache` 内容有效前通过 `valid` 等硬件逻辑阻止下游采样，或输出一个确定值。**仅添加形式假设让 `write` 在第一拍发生，不一定等同于真实硬件有此保证**。

6.1 更合理的有效性控制

```
logic initialized;
always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n) initialized <= 1'b0;
  else if (write) initialized <= 1'b1;
end
assign out_data = initialized ? cache : 8'h00;
```

当 `cache` 尚未被写入时，输出为确定值；写入后使用缓存数据。FXP 仍要检查选择条件以及其他传播分支是否可能携带未知值。

6.2 本节完整工程文件

上面的片段展示问题原理。以下是同一类未初始化缓存案例的完整文件，故意保留复位端口却不复位缓存，用来说明“端口存在”并不意味着缓存有已知初值。

6.2.1 被验证的完整 RTL

保存为 案例工程/rtl/uninit_demo.sv：

```
// Purposefully uninitialized cache to illustrate FXP, not recommended RTL.
module uninit_demo (
  input logic clk, rst_n, write,
  input logic [7:0] in_data,
  output logic [7:0] out_data
);
  logic [7:0] cache;
  always_ff @(posedge clk)
    if (write) cache <= in_data;
  assign out_data = cache;
endmodule
```

6.2.2 完整文件列表

保存为 案例工程/filelist_fxp.f：

```
./rtl/uninit_demo.sv
```

6.2.3 完整 FXP 脚本

`fxp_generate` 选择未初始化对象，`fxp_assert` 将输出作为必须已知的观察位置。输入 `write` 没有被假设为始终有效，因此必须考虑尚未写入的路径。

保存为 案例工程/scripts/fxp_uninit.tcl：

```
set_fml_appmode FXP
read_file -top uninit_demo -format sverilog -vcs {-f filelist_fxp.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
fxp_generate -name uninitialized_checks -type {uninit}
fxp_assert -name output_must_be_known out_data
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fxp_uninit.tcl -batch 2>&1 | tee logs/fxp_uninit.log
```

预期观察是：未发生有效写入时，缓存的不确定性可能到达输出。分析失败应查看工具实际选择的注入点和反例，不应通过把 `write` 固定为 1 来掩盖初始化缺陷。这里没有真实 FXP 运行报告。

7. 教学用例 B：显式 X 赋值与越界

```
always_comb begin
  unique case (opcode)
    2'b00: result = a + b;
    2'b01: result = a ^ b;
    default: result = 'x;
  endcase
end
```

对未定义操作码，显式 X 可能被后级直接使用；`-type {xassign}` 可以帮助定位。若协议规定未支

持操作码必须返回错误码或忽略写入，则直接赋 X 不是完整实现，应修改 RTL 和相应 SVA。如果设计规则明确规定非法操作码永远不会输入，则需要有可审查的上游保证，不能只对当前检查添加无限制输入假设。

对数组 `mem[index]` 的非法访问则属于 `oba` 等相关检查。在一些 RTL 里，越界索引导致的 X 传播既可被 AEP 发现，也可通过 FXP 分析其实际影响的观察点，两者结论不同、相互补充。

8. 教学用例 C：有意屏蔽不确定数据

```
assign packet_out = valid ? raw_packet : '0;
```

如果 `valid` 的生成完全可靠且只有 `valid=1` 时 `raw_packet` 才被使用，则未初始化的 `raw_packet` 未必造成危险输出。FXP 应在考虑 `valid` 的实际来源和复位行为后判断其是否能够影响 `packet_out`；不要只凭 `raw_packet` 未复位就断定必须增加复位网络。对于大面积数据寄存器，**有效位优先**的方案往往比强行复位所有触发器更合理，但前提是证明有效位本身可靠，且其控制可完全屏蔽未知数据。

9. 调试：由下游症状回溯到上游来源

```
check_fv -block
report_fv -list
view_trace -property <failed_property>
fxp_compute_rootcause <failed_property>
fxp_report_rootcause
```

定位时先找观察点在何时受影响，然后从该周期往前追踪经过的条件选择、多路复用器、寄存器和复位逻辑。重点确认是否存在：复位未覆盖、输出在有效位之前被采样、黑盒建模不当、未知模式输入未处理、未驱动网络或隐含时钟门控。必须把“真正产生不确定性”和“最终使不确定性可见”分开分析。

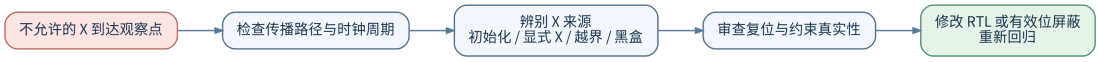


图 8: FXP 调试排查流程

10. 设计规模很大时怎么做

先针对关键输出与重要控制节点建立一批观察点；其次根据报告只针对相关模块开启额外的注入类型；最后扩大到全模块或全芯片。全信号扫描往往生成大量属性，可能包含没有工程意义的中间节点和普通数据寄存器，因此不推荐把 `get_signals` 的全部对象直接无差别传给 `fxp_assert`，尤其不推荐在没有容量预算和报告分类时执行。若需批量生成对象列表，应该过滤时钟、复位、扫描测试、非功能占位和已知允许变化的信号，并在最终报告中清晰列出排除规则。

10.1 与复位覆盖工作的关系

FXP 可帮助识别“实际运行时需要已知，但没有完整初始化”的重要状态，但**不能单独替代复位规格验证**。建议同时使用 FPV 证明复位之后关键控制寄存器为规定的常量。这样发现的 X 传播问题可以分成：状态未初始化、控制路径泄漏、协议保护失效、或仿真 X 语义不同。

11. 常见问题

现象	可能原因	改进方向
复位后一拍输出为 X	数据寄存器未复位或有效位错误	修改 RTL 或协议保护
某黑盒输出引起所有下游失败	模型过于自由	提供正确的黑盒约束或可综合模型
所有属性都通过但已知存在 X 问题	未将关键输出列为观察点或注入范围不足	核对 `fxp_generate` 类型及属性数量
生成属性太多，内存不足	扫描范围过宽	按重要模块、输出和原因分批
`-injectx` 报错	选中时钟/复位或不支持对象	查该命令对象与 scope 限制
人工 `-nox` 后全部通过	可能屏蔽真实传播	审查硬件依据，保留原始对照结果

FXP 反例是观察点在特定条件下可能不确定的证据，不应直接把所有未复位寄存器加上复位。先确认下游是否在数据有效前采样，再核对屏蔽条件和复位时序；只有规格要求上电即确定时，才需要相应的初始化实现。

12. FXP 与 AEP、FPV 的分工

AEP 检查数组越界或危险表达式等自动规则；FXP 检查不确定性是否影响重要观察点；FPV 则证明指定输入规范下的功能值和时序关系。同一处越界访问可以同时涉及三种问题，但三者的目标和通过条件不同。

例如越界读导致数据可能不确定，三个应用依次能够定位越界原因、传播风险、以及实际功能规则失败。联合使用更全面，但三者仍依赖正确的模型和约束。

13. 官方依据与版本提醒

User Guide 第 12 章（444 ~ 457 页），Quick Reference Guide FXP 部分。`set_blackbox` 的开关应以安装版本帮助为准，部分旧笔记中存在 `-design` 与 `-designs` 混用；不要把未核对的选项直接写入生产脚本。源码形式验证与真实芯片的亚稳态、模拟电压和物理故障不是同一个模型。

CC：形式连通性检查使用指南

1. 这项应用解决的问题

大规模 SoC 有大量中断线、复位线、测试模式信号、控制总线、调试线和模块间的数据连接。人工核对顶层 RTL 不仅费时，也难以覆盖条件选择、多级复用和逻辑变换。CC 通过预先声明源、目的、使能及延迟条件，验证设计的结构或功能上是否存在符合要求的传播关系。

结构连通性关注从源到目的的候选有向路径是否存在；**功能连通性**进一步检查在规定使能、初始状态和时序条件下，目的值是否满足规格。结构上有路径并不能证明功能上始终正确；功能检查还需具体考虑控制条件。

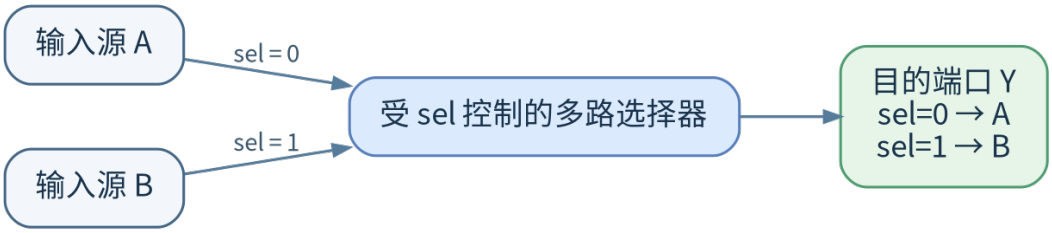


图 9: CC 的源、选择控制与目的端

2. 连通性规格包含哪些信息

User Guide §13.2 将一个基本连通性要求分解为：名称、源 `src`、目的 `dest`、使能表达式、路径延迟 `path_delay`、使能保持时间 `enable_hold`、使能延迟 `enable_delay`、起始偏移 `offset_delay` 和时钟。常用连接的延迟为 0，并不代表全部 SoC 连接都零延迟。

字段	要解决的具体问题	示例说明
name	这一连接叫什么	`cc_irq0`
src	信号从哪里来	`irq_in[0]`
dest	到哪里去	`irq_out[0]`
enable	何时应该生效	`sel == 0`
path_delay	源到目的要经过几个时钟	0 或固定延迟
enable_hold	使能需要保持的周期范围	与数据到达时机有关
enable_delay	使能相对源信号何时有效	可为 0 或负偏移
offset_delay	距离初始化的检查起点	排除合法启动窗口
clock	延迟检查采用哪个时钟	`clk`

一条连接规格至少应明确源、目的和成立条件。有模式选择时，必须同时描述选择条件与传播关系；有寄存器或流水级时，还必须明确允许的延迟。别名只用于统一路径书写，不改变电路行为。导入后先检查识别出的连接数量和方向，再运行证明，避免拼写错误导致规格没有真正成为待验证目标。

不能用“检查有线路连接”概括含使能的连通性：例如一个由 `sel` 控制的多路选择器，在 `sel=0` 时源 `a` 应到 `y`，在 `sel=1` 时源 `b` 应到 `y`，这属于两条具有不同使能的规格。

3. 最小实例：多路选择器

3.1 完整多路选择器源码

此处没有寄存器，输出完全由当前 sel/a/b 决定。

保存为 案例工程/rtl/mux2.sv：

```
module mux2 (
  input logic a, b, sel,
  output logic y
);
  assign y = sel ? b : a;
endmodule
```

对应两条连接规格：当 sel=0 时 a 与 y 相同；当 sel=1 时 b 与 y 相同。可以在 Tcl 中创建：

```
set_fml_appmode CC
read_file -top mux2 -format sverilog -vcs {./rtl/mux2.sv}
add_cc -src a -dest y -enable {!sel}
add_cc -src b -dest y -enable {sel}
check_fv -block
report_fv -list
```

add_cc 的 -enable 语法应以当前安装帮助为准；如果本版本对复杂表达式格式有限制，可改成 CSV 规格导入。**该脚本是展示连接语义的教学骨架，运行前须按安装版本补充完整的编译和检查配置。**

4. CSV 导入：大规模 SoC 的推荐方法

CSV 可以由规格表维护，从模块接口说明或端口表生成。依据 UG 第 461 ~ 464 页，默认字段顺序从属性名、源、目的、使能、路径延迟开始。一个简单 CSV 如下：

4.1 完整 CSV 连接规格

本例使用名称、源、目的、使能、路径延迟五列。第一行是说明性注释；是否接受这一注释形式仍需按安装版本核对。两条连接覆盖两个选择条件。

保存为 案例工程/cc_connections.csv：

```
// format: name,src,dest,enable,path_delay
cc_mux_a,a,y,!sel,0
cc_mux_b,b,y,sel,0
```

4.2 CSV 方式的完整运行脚本

直接读入上节 RTL 和本节 CSV；不需要隐藏的 filelist.f。先查看 report_load_cc，确认两条连接均成功加载，再分析形式结果。

保存为 案例工程/scripts/cc_mux.tcl：

```
set_fml_appmode CC
read_file -top mux2 -format sverilog -vcs {./rtl/mux2.sv}
load_cc -format csv cc_connections.csv
report_load_cc
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
```

```
vcf -f scripts/cc_mux.tcl -batch 2>&1 | tee logs/cc_mux.log
```

对于跨组织的大规模规格，建议保留完整的九个核心字段并明确表头与默认值。`load_cc -default_param_order` 对应默认列顺序，`load_cc_set_param` 可定义自定义列对应关系，如 `load_cc_set_param csv_src "%2%"`；如果使用自定义格式必须随规格表一起提交，防止 CSV 列移动造成检查错位。

4.3 延迟连接的限制

CC 支持与时钟关联的连接延迟；路径中可能有寄存器，也可能由使能控制时序组合。验证之前需要指定准确时钟并检查 `path_delay` 的解释。不要把源和目的之间的**物理布线延迟**误认为 `path_delay=1`；这里描述的是形式模型中按采样时钟计算的功能延迟。多时钟跨域连接不能简单用一条普通单时钟延迟规格代替 CDC 分析。

5. 源目的别名与设计层级

实际 SoC 规格名称通常与 RTL 层级名不同，可以使用别名处理：

```
set_cc_alias CPU_IRQ u_top.u_cpu.irq_in
report_cc_alias
# 必要时删除已失效的别名
delete_cc_alias CPU_IRQ
```

别名使规格表与层级变动解耦，但必须建立审查规则：别名应对应单一、明确的对象或合法总线片段；层级优化后应重新检查别名的解析结果。

6. 命令语法、参数与用例

命令	主要作用
`add_cc`	建立一条条件连通性检查
`load_cc`	从 CSV 或表格式导入连接规格
`load_cc_set_param`	定义 CSV 字段的位置和组合方式
`report_load_cc`	报告连接规格导入和结构检查
`set_cc_alias`	定义连接规格中的层级别名
`report_cc_alias`	查看连接别名
`delete_cc_alias`	删除旧连接别名
`generate_cc`	从设计提取可能的连接及使能条件
`compute_cc_cov`	分析已证明连接的翻转覆盖
`save_cc_cov_results`	保存连接覆盖和排除资料

本组命令按准备、生成、执行与结果处理的关系组织。语法框保留参数名称和可选关系；示例中的实例、属性和文件名称需要与当前工程一致。生成目标与证明目标是不同阶段，调用生成命令后应检查实际目标数量，再启动证明。

6.1 `add_cc`：建立一条条件连通性检查

```
add_cc -src <source-signals> -dest <destination-signals>
       [-enable <expression>] [-path_delay <min:max|latency>]
       [-enable_hold <cycles>] [-enable_delay <cycles>]
       [-offset_delay <cycles>] [-clock <clock-expression>]
       [-name <name>]
```

参数解释。 `-src`、`-dest` 是有顺序的信号列表，支持名称模式，顺序决定对应关系。`-enable` 是连

接成立条件，默认 true；`-path_delay` 是源到目的延迟，默认 0。`-enable_hold` 指定使能需要保持多久，`-enable_delay` 指定使能相对源值需要提前多久成立，默认均为 0。`-offset_delay` 指定复位后多少周期才开始检查。`-clock` 给出采样表达式，默认使用已定义时钟的上升沿；`-name` 为检查命名。

用法与结果。普通同拍连接可不含时序延迟；有延迟时必须有有效时钟。源与目的名字正确但顺序错误、位序颠倒或使能提前量不对，都可能造成失败。对经过寄存器的路径，不能只通过增加 path_delay 随意消除反例，必须以接口时序规格为依据。

```
add_cc -name data0_to_y -src {a} -dest {y} -enable {!sel}
add_cc -name data1_to_y -src {b} -dest {y} -enable {sel}
check_fv -block
report_fv -class connection -list
```

来源：User Guide，X-2025.06-SP1，第 469–479 页。

6.2 `load_cc`：从 CSV 或表格式导入连接规格

```
load_cc [-format <csv|table>] [-quiet] <filename>
        [-default_param_order]
```

参数解释。默认格式是 csv；table 是工具定义的增强表格式，可把使能条件分散在多列。`-quiet` 抑制进度和结构检查汇总，但仍报告错误与警告。`-default_param_order` 让本次读取按默认字段顺序解释，不覆盖已保存的自定义字段配置。

用法与结果。 CSV 的列顺序是规格的一部分，不能只凭第一行标题猜测工具解释方式。table 格式需要两行表头，条件通常是多个位或字的相等判断的合取，不能把任意 SVA 写在表格里。导入后先看 `report_load_cc` 的成功条数和错误，再进行功能连通性证明。

```
load_cc -format csv cc_connections.csv
report_load_cc -verbose
check_fv -block
```

来源：User Guide，X-2025.06-SP1，第 461–469 页。

6.3 `load_cc_set_param`：定义 CSV 字段的位置和组合方式

```
load_cc_set_param <parameter-name> <value>
```

参数解释。字段位置写成 `%1%`、`%2%` 等占位符；如 csv_src、csv_dest、csv_prop_name、csv_enable。层级字段可以拼接，例如 `%3%.%4%`。csv_start_line、csv_end_line 用于行范围，csv_comment_pattern 指定行首注释标志。其余延迟、时钟和使能字段应与连接定义一致。

用法与结果。列编号从 1 开始。拼接时空字段会省略，相应多余层级分隔符也会删除。它改变后续文件的解释规则，不修改 CSV 本身；多个不同格式文件在同一会话导入时，要显式切换配置或使用默认字段顺序选项。

```
load_cc_set_param csv_prop_name "%1%"
load_cc_set_param csv_src "%2%"
load_cc_set_param csv_dest "%3%"
load_cc_set_param csv_enable "%4%"
load_cc_set_param csv_start_line "2"
load_cc cc_connections.csv
```

来源：User Guide，X-2025.06-SP1，第 464–468 页。

6.4 `report_load_cc`：报告连接规格导入和结构检查

```
report_load_cc [-list] [-verbose] [-no_summary] [-error]
               [-warning] [-connected] [-disconnected]
               [-not_applicable] [filename]
```

参数解释。 `-list` 按文件列出每个目标的状态、行号和连接名；`-verbose` 还显示详细消息；`-no_summary` 去掉汇总。其余状态选项分别筛选错误、警告、结构已连接、结构不连接和不适用的目标；位置文件名限制来源。

用法与结果。 connected 在本报告中是导入阶段的结构信息，不是使能和延迟条件下的完整功能证明。not_applicable 可能来自禁用或常量源等情况，不能当成已通过。正式报告应同时包含导入条数和 `check_fv` 的结果。

```
report_load_cc -error -verbose
report_load_cc -disconnected -list
report_load_cc -verbose cc_connections.csv
```

来源：User Guide，X-2025.06-SP1，第 468–469 页。

6.5 `set_cc_alias`：定义连接规格中的层级别名

```
set_cc_alias [-nest] <alias> <replacement>
```

参数解释。别名只能使用字母、数字和下划线，点号保留给层级分隔。`-nest` 允许递归解析别名；replacement 可以是层级路径或用已有别名组成的表达式。

用法与结果。别名是路径书写便利，不改变电路。带位选择的替换值应用花括号包住，防止 Tcl 把方括号当成命令替换。递归别名应避免循环，并在导入目标后核对实际源目的。

```
set_cc_alias core chip_top.u_core
set_cc_alias irq {chip_top.irq_bus[0]}
set_cc_alias -nest core_irq core.irq_o
report_cc_alias
```

来源：User Guide，X-2025.06-SP1，第 479 页。

6.6 `report_cc_alias`：查看连接别名

```
report_cc_alias [<alias-pattern>]
```

参数解释。不带模式查看定义列表，模式用于查找单个或一组别名。

用法与结果。在 IP 复用和顶层层级调整后，先确认别名仍指向当前对象。旧别名没有自动随 RTL 的层级重命名更新。

```
report_cc_alias
report_cc_alias core*
```

来源：User Guide，X-2025.06-SP1，第 479 页。

6.7 `delete_cc_alias`：删除旧连接别名

```
delete_cc_alias <alias-name>
```

参数解释。指定要删除的别名。

用法与结果。删除影响后续名字解析；不能假定已经生成的属性会随别名删除而重建。改变别名后需要重新确认规格导入结果。

```
delete_cc_alias core_irq
report_cc_alias
```

来源：User Guide，X-2025.06-SP1，第 479 页。

6.8 `generate_cc`：从设计提取可能的连接及使能条件

```
generate_cc [-src <objects>] [-dest <objects>] [-control <objects>]
            [-run] [-preserve] [-clearall] [-file <path>]
            [-limit <integer>] [-verbose]
```

参数解释。源、目的和控制对象可为网络、端口或实例。先用 `-src`、`-dest` 积累对象对，`-run` 才执行提取。`-control` 指定使能表达式追溯到哪些控制对象；`-preserve` 保留源目的供后续运行，`-clearall` 清除已设源、目的和控制列表。`-file` 输出 Tcl，否则写入属性数据库；`-limit` 限制每对对象提取的路径数量，默认 1；`-verbose` 输出细节。

用法与结果。提取的是当前 RTL 已存在的连接，不是独立规格。它适合辅助生成待审核连接清单，但不能把由错误 RTL 自己提取的连接再证明一遍就称为满足原设计需求。多对对象一次运行通常比逐对运行更高效。

```
generate_cc -src {a} -dest {y}
generate_cc -src {b} -dest {y}
generate_cc -run -file extracted_connections.tcl
```

来源：User Guide，X-2025.06-SP1，第 502–506 页。

6.9 `compute_cc_cov`：分析已证明连接的翻转覆盖

```
compute_cc_cov [-par_task <task>] [-endpoint] [-net]
               [-structural] [-property <properties>] [-block]
               [-run_finish <commands>] [-enable]
```

参数解释。`-par_task` 指定父 CC 任务，默认当前任务；`-endpoint` 计算源和目的翻转，`-net` 计算路径上网络，`-enable` 包含使能表达式及其到选择位置的路径。`-property` 指定启用且已证明的连接。`-structural` 直接把证明连接对应的覆盖目标标为已覆盖，不执行覆盖分析。`-block` 等待完成；本页对 `-run_finish` 写有只在阻塞模式有效的限制，不能套用 check_fv 的非阻塞回调规则。

用法与结果。必须在读设计时生成翻转目标，并启用连通性覆盖流程。`-structural` 的直接标记与实际翻转可达性证明不是同一种证据；报告中必须区分。库文件缺省可能不生成覆盖，需根据工程使用 `-cm_libs vy`。

```
compute_cc_cov -endpoint -net -block
save_cc_cov_results -covdb_name cc_toggle.vdb
```

来源：User Guide，X-2025.06-SP1，第 510–515 页。

6.10 `save_cc_cov_results`：保存连接覆盖和排除资料

```
save_cc_cov_results [-covdb_name <database>]
                    [-exclude_disabled] [-elfile]
```

参数解释。`-covdb_name` 指定输出数据库，缺省 `cc_tgl_cov.vdb`；`-exclude_disabled` 把禁用或无关目标写入排除文件；`-elfile` 保存不可达翻转目标的排除文件，并沿用数据库基本名称。

用法与结果。保存结果不会重新执行覆盖计算。启用严格排除模式后，已覆盖目标在后续运行中保留已覆盖标记，因此更需要控制适用的设计版本和约束条件，不能跨不相容配置沿用。

```
compute_cc_cov -endpoint -block
save_cc_cov_results -covdb_name cc_toggle.vdb -elfile
```

来源：User Guide，X-2025.06-SP1，第 511–515 页。

7. 三种常见用例

用例一：复位信号分发。检查顶层复位能否经过相关复位控制逻辑到达每个子模块的对应复位端；如经过同步复位器，不要错误地要求零拍直接等值。**用例二：中断选择器**。按每个路由配置值检查中断来源与目的的条件连接，另外独立检查未使能中断不应错误出现在输出端。**用例三：测试模式控制**。检查正常模式 `test_mode=0` 时功能信号未被测试逻辑截断，同时要避免把受硬件控制的测试状态误假设成永久关闭。

8. 失败分类与排查

结构不连通：检查层级、拼写、位选择、网表优化、黑盒以及真正是否有从源到目的的组合/时序路径。**结构有路径但功能失败**：检查控制使能、复位时序、源目的位宽、数据翻转关系、被插入的反相器或优先级。**不可适用或无意义的连接**：确认目的是否可观察、是否被固定常量、规则是否超出所支持的延迟/时钟形式。

不会因为报告中某条连接显示 `Connected` 就证明整个 SoC 的功能行为。CC 只覆盖已经列出的连接规则，规格完整性需要接口所有者审核。

9. 对大型项目的建议

按接口类别分 CSV 文件（时钟、复位、中断、扫描、调试、寄存器地址控制等），为每行保留规格编号及责任模块。批处理时针对真实的错误分类输出 `report_load_cc` 与 `report_fv`。将结构不连通与功能不符合分开处理；以保留全部输入列表的方式生成导出报告，便于发现某个连接根本没有被加载的情况。

10. 参考

User Guide §13（458 ~ 516 页），尤其 §13.2（460 页起）、CSV 格式说明（461 ~ 465 页）、连接延迟与覆盖章节；Quick Reference Guide CC 章节。低功耗场景另见 FLP_LPC.md。

FCA：形式覆盖率分析使用指南

1. FCA 与 FPV 的区别

FPV 证明某一条属性是否成立；FCA 侧重理解覆盖目标的可达性，帮助分析仿真中未覆盖的代码和功能点是否在现有约束下真正不可能触发。典型情况：随机回归得到某分支从未执行，是因为测试不够、环境限制过严，还是这个分支从设计初始状态根本不可到达？FCA 可以在适当的逻辑模型中研究可达性，并为覆盖率排除提出依据。

FCA 的结果与 VCS 的覆盖数据通常相配合，常涉及 `simv.vdb`、排除文件 `.el` 和 `save_covdb` 产生的覆盖数据库。**不是所有未覆盖都应该豁免**，覆盖目标在形式模型下不可到达，也不代表目标一定不对应产品真实行为；模型和假设可能过度限制。

2. 覆盖对象与每种结果的含义

对象	要求出现的行为	不能据此推断的结论
代码行	相应可执行语句能够被执行	该语句计算结果正确
分支	某个判断分支或选择分支被采用	所有子条件组合均已出现
布尔条件	覆盖模型指定的条件取值或组合出现	所有接口时序均已检查
信号翻转	指定位出现规定方向的取值变化	数据内容和先后关系正确
状态与转移	状态机进入指定状态或完成指定转移	状态机不会死锁或饥饿
功能覆盖目标	覆盖组或用户目标描述的场景出现	该场景下所有功能正确

行、分支与条件覆盖的观察层次不同。下面的赋值语句可以执行，只说明控制流进入了对应分支。两个分支都执行过，也不表示 `req`、`ready` 和 `cancel` 的各种有效组合都被检查。具体条件覆盖项取决于所导入的数据库以及编译时启用的覆盖配置，不应把“条件覆盖”笼统写成对任意布尔表达式的全部组合穷举。

```
always_comb begin
  accepted = 1'b0;
  if (req && ready && !cancel)
    accepted = 1'b1;
end
```

翻转覆盖与数据正确性不同。数据总线的每一位都出现过 0 和 1，并不能说明一次写入后读出的字节顺序正确，也不能说明校验位与数据一致。翻转覆盖适合发现被固定的控制位、没有活动的接口和配置遗漏；数据关系仍要通过断言或参考模型比较。

状态覆盖与进展性不同。一个状态能够到达，不等于可以从它离开；所有状态都到达过，也不等于不存在无限循环。FCA 的状态与转移目标需要结合 AEP 的状态机检查或手工活性属性，才能回答“请求是否最终完成”这类问题。

覆盖类型必须在编译或数据库导入阶段明确。原生覆盖可通过 `read_file -cov ...` 提取，仿真覆盖可以由 `read_covdb` 导入。分支等类别可能需要对应的配置变量在编译前启用。报告必须说明类型、设计范围、初始状态和环境假设，再给出可达、不可达和未完成的数量。来源：User Guide，第 386–443 页。

3. 第一个 FCA 用例：输入条件与代码分支可达性

3.1 完整设计源码

本例没有限制 mode，因此四种输入组合都允许出现。分析其可达性不是预先宣布某个分支不可到达。

保存为 案例工程/rtl/coverage_demo.sv：

```
// FCA exercise: branch conditions and reachability.
// mode is intentionally unconstrained, unless the verification environment
// supplies an explicitly reviewed operating-mode assumption.
module coverage_demo (
  input  logic       clk,
  input  logic       rst_n,
  input  logic [1:0] mode,
  output logic [1:0] result
);
  always_comb begin
    result = 2'd0;
    case (mode)
      2'd0: result = 2'd1;
      2'd1: result = 2'd2;
      2'd2: result = 2'd3;
      default: result = 2'd0;
    endcase
  end
endmodule
```

如果真实系统规范要求 mode[1] 恒为零，另外两个模式在该约束下才不可达；必须记录约束出处，并区分真实限制与为了提高覆盖数字而新增的假设。组合分支能否保留并产生检查，还受编译优化和覆盖类型配置影响。

3.2 完整文件列表

保存为 案例工程/filelist_fca.f：

```
./rtl/coverage_demo.sv
```

4. 主要流程：完整 Tcl

4.1 完整运行脚本

本版本使用 COV 模式。脚本保留检查设置和结果报告；没有在未审查结果时自动把所有未完成目标导出为排除项。

保存为 案例工程/scripts/fca_coverage.tcl：

```
# Formal Coverage Analyzer uses COV appmode in X-2025.06-SP1 User Guide.
# Run from example project root; treat result as tool-dependent.
set_fml_appmode COV
read_file -top coverage_demo -format sverilog -cov all -vcs {-f filelist_fca.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup -block
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
```

```
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fca_coverage.tcl -batch 2>&1 | tee logs/fca_coverage.log
```

需要导出经过审查的不可达候选时，再按实际会话执行：

```
save_cov_exclusion -file formal_unreachable.el
```

`-cov all` 的覆盖类型仍须核对版本，某些类型有单独的启用变量。应先确认生成了哪些目标、目标数量以及实际完成状态，再解释覆盖率或排除文件。

5. 从仿真数据库开始

```
set_fml_appmode COV
read_covdb -cov_input simv.vdb -cov_dut tb.dut
read_file -top dut -format sverilog -cov all -vcs {-f filelist.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv -block
save_covdb -status covered -name formal_result.vdb
save_cov_exclusion -file candidate_exclusion.el
```

`-cov_dut` 必须是仿真数据库中的真实 DUT 实例路径。若层级不匹配，生成的排除记录可能关联错误。UG 第 395 页说明，无仿真数据库时的排除对象与有仿真数据库时的层级来源不同；最终在覆盖可视化工具中导入 `.el` 之前需要确认层级。

6. 命令语法、参数与用例

命令	主要作用
`read_covdb`	导入仿真覆盖数据库
`save_cov_exclusion`	导出覆盖排除文件
`save_covdb`	保存形式覆盖分析结果
`view_coverage`	查看数据库与排除项
`cov_config`	配置自动分解覆盖分析
`compute_cov`	执行自动分解后的覆盖任务

本组命令按准备、生成、执行与结果处理的关系组织。语法框保留参数名称和可选关系；示例中的实例、属性和文件名称需要与当前工程一致。生成目标与证明目标是不同阶段，调用生成命令后应检查实际目标数量，再启动证明。

6.1 `read_covdb`：导入仿真覆盖数据库

```
read_covdb -cov_input <database_path> -cov_dut <simulation_instance>
           [-fv_dut <formal_instance>]
```

参数解释。`-cov_input` 指向仿真生成的覆盖数据库；`-cov_dut` 是数据库内的 DUT 实例路径，不是 RTL 模块名。形式环境另加包装模块时，用 `-fv_dut` 指定形式设计内的 DUT 路径，使两个层次中的覆盖对象正确对应。

用法与结果。通常先设定 COV 应用模式、导入数据库，再读取设计和生成覆盖目标。没有仿真数据库时可以直接从 RTL 开始不可达分析。导入后主要分析仿真尚未覆盖的目标；不能把仿真的实例路径直接当作形式包装中的路径。以下列出 UG 使用流程中展示的选项，不扩展未在该流程中解释的平台参

数。

```
set_fml_appmode COV
read_covdb -cov_input simv.vdb -cov_dut simTB.i1.DUT \
           -fv_dut formal_top.u_dut
read_file -top formal_top -format sverilog -vcs "-f formal.f"
```

来源：User Guide，X-2025.06-SP1，第 395–399 页。

6.2 save_cov_exclusion：导出覆盖排除文件

```
save_cov_exclusion -file <filename> [-append] [-missing_else]
  [-cov <metric_list>] [-line_intent] [-no_line_intent]
  [-filter <expression>] [-targets <goals>]
  [-enable_top_mod_excl] [-mod_excl] [-mod_excl_scope <modules>]
  [-strict] [-scope <module_or_instance>]
```

参数解释。-file 指定文件，-append 追加。-missing_else 纳入隐含的缺失 else 分支。-cov 以加号连接指标，支持 line、cond、toggle、fsm_state、fsm_trans、branch、cg、assert、cover、all；all 指结构覆盖类型，assert 和 cover 应显式指定。语法将 -cov 标为可选，原文同段又出现 “mandatory”，与省略它的示例矛盾，正文采用其明确给出的省略示例，不把该句解释为必填。

-line_intent 只保留与代码意图相关的目标，-no_line_intent 排除这些目标；-filter 按属性筛选，-targets 显式指定目标集合。-enable_top_mod_excl 对顶层使用模块排除、对子层使用实例排除；-mod_excl 求同一模块各实例共有的不可达项，-mod_excl_scope 限定模块。-scope 限定导出层次。-strict 写出供 URG/Verdi 严格检查使用的排除文件，以便发现 “数据库中已覆盖、排除文件却排除” 的冲突。

用法与结果。默认导出已判为不可达的目标，成功返回 1，失败返回 0。显式 -targets 可以改变导出对象，因此仅有排除文件不等于每一项都有不可达证明；审查时同时保留约束、状态和日志。

```
check_fv -block
save_cov_exclusion -file unr.el -strict
save_cov_exclusion -file selected_unr.el \
  -targets [get_props -status uncoverable]
```

来源：User Guide，X-2025.06-SP1，第 401–403 页。

6.3 save_covdb：保存形式覆盖分析结果

```
save_covdb -name <covdb_name> [-update] [-overwrite]
  [-cov <metric_list>] [-line_intent] [-no_line_intent]
  [-status <status_list>]
```

参数解释。 -name 给出目标数据库；-update 作为新测试结果更新已有数据库，-overwrite 删除旧内容并重建，两种策略不要混用。-cov 筛选指标，-line_intent 与 -no_line_intent 筛选代码意图相关目标。-status 支持 covered、uncoverable。assert 和 cover 不会因为省略 -cov 而自动全部保存，需明确选择。

用法与结果。应先保存数据库，再调用 view_coverage。原文特别提醒：未使用 -status covered 时，不可达目标可能直接从数据库移除，Verdi Exclusion Manager 不一定显示它们为 “被排除的目标”。需要可审计排除记录时，另存 .el 文件，不能只保留最终百分比。

```
save_covdb -name formal_result.vdb -overwrite -status covered
save_cov_exclusion -file formal_unr.el -strict
view_coverage -cov_input formal_result.vdb -elfile formal_unr.el
```

来源：User Guide，X-2025.06-SP1，第 404–405 页。

6.4 view_coverage：查看数据库与排除项

```
view_coverage -cov_input <saved_database> -elfile <exclusion_file>
```

参数解释。 -cov_input 选择已保存的覆盖数据库，-elfile 叠加不可达排除文件。这是 UG 展示的查看流程，而不是声明这两个选项构成该命令的全部平台语法。

用法与结果。打开 Verdi Coverage 后，在 Exclusion Manager 中审查排除项及其来源。检查每个不可达结论是否依赖设计模式、绑常量或环境限制，再决定能否用于仿真覆盖验收。查看窗口本身不执行证明，也不改变已有约束。

```
save_covdb -name fca_result.vdb -overwrite -status covered
view_coverage -cov_input fca_result.vdb -elfile unr.el
```

来源：User Guide，X-2025.06-SP1，第 405 页。

6.5 cov_config：配置自动分解覆盖分析

```
cov_config [-auto_scale]
  [-line_task <n>] [-cond_task <n>] [-branch_task <n>]
  [-tgl_task <n>] [-fsm_task <n>] [-cg_task <n>]
  [-limit <percentage>] [-line_limit <n>] [-branch_limit <n>]
  [-tgl_limit <n>] [-fsm_limit <n>] [-cond_limit <n>]
  [-module <module>] [-inst_list <instances>]
  [-start_task_cmd <commands>] [-end_task_cmd <commands>]
  [-start_line_task_cmd <commands>] [-start_cond_task_cmd <commands>]
  [-start_branch_task_cmd <commands>] [-start_tgl_task_cmd <commands>]
  [-start_fsm_task_cmd <commands>] [-start_cg_task_cmd <commands>]
  [-exclude_assumes_covdb] [-exclude_bb_signals]
cov_config -debug
```

参数解释。 -auto_scale 开启自动分解；各 *_task 指定对应指标的并行任务数。-limit 是模块已有总覆盖率阈值，默认 70；高于阈值的模块跳过。各 *_limit 则是未覆盖目标数量阈值，低于该数量的模块跳过，不能把两类阈值都理解为百分比。-module 指定形式设计起点，-inst_list 指定输入覆盖数据库中的实例起点。

-start_task_cmd 在每个任务创建后、证明前执行，-end_task_cmd 在该任务证明结束后执行；各 -start_*_task_cmd 仅作用于对应指标。-exclude_assumes_covdb 保存约束对应的排除信息，-exclude_bb_signals 排除数据库中的黑盒信号。-debug 输出时间和内存消耗信息，是后文使用示例补充的开关。

用法与结果。必须先开启 -auto_scale，再配置其他自动分解选项，或者在同一条命令中设置。阈值和层次选择项必须在 read_file 或 elaborate 之前使用；读取设计后再修改，不会按预期重新决定覆盖插装范围。此流程用 compute_cov 启动，而不是简单替换几个 check_fv 参数。

```
set_fml_appmode COV
read_covdb -cov_input simv.vdb -cov_dut tb.u_dut
cov_config -auto_scale -limit 80 -tgl_task 4
cov_config -start_task_cmd {set_fml_var fml_max_time 10M}
read_file -top dut -format sverilog -vcs "-f rtl.f"
compute_cov
```

来源：User Guide，X-2025.06-SP1，第 435–438 页。

6.6 compute_cov：执行自动分解后的覆盖任务

```
compute_cov
```

参数解释。该用法不需要位置参数；输入是 cov_config 建立的自动分解配置以及已读取的 RTL、覆盖数据库和环境。

用法与结果。命令负责按指标和层次生成并执行任务，报告各任务和总体结果。每个任务可以产生自

已的排除文件，流程还合并产生 `unreachables.el`。任务被筛选跳过不等于目标已证明不可达，应将自动筛选范围与最终结论一起记录。

```
cov_config -auto_scale
cov_config -tgl_task 4
# 完成数据库、RTL、时钟和复位设置后执行。
compute_cov
```

来源：User Guide，X-2025.06-SP1，第 438–440 页。

7. Auto Scale 及资源策略

当覆盖目标数量巨大时，FCA 可自动划分多个覆盖类型和模块任务。先运行一个小范围测试估计内存与每类目标的数量，再考虑 `cov_config -auto_scale`、`-line_task`、`-fsm_task`、`-tgl_task` 等参数。不要在不了解网格队列和许可证预算的情况下照搬快速指南中几十或上百个 worker 的示例。汇总结果时，应说明每一类型完成、不可达和未完成的数量。

8. 排除文件需要审查

“不可达” 至少可能来自三种情形：设计逻辑确实使其不可达；复位/环境约束使其不可达；模型存在缺失使其无法触发。前者可能是冗余 RTL，也可能是特定配置下合法不可达；后两者则不能简单纳入永久覆盖排除。建议每条排除记录带上规格版本、环境限制和审批人。

一个完整的报告应区分仿真原始覆盖率、形式分析发现的不可达目标、审查后允许排除的目标、仍需增加激励的可达目标。不能把形式分析结果当成单纯提高覆盖率百分比的手段。

9. 常见排查

现象	检查事项
无覆盖属性	`-cov` 类型、模式、文件展开、分支额外开关
导出的排除文件和仿真层级不一致	`-cov_dut`、顶层名、编译优化
许多目标显示不可达	复位、环境常量、黑盒与约束
计算耗时非常长	按模块、覆盖类型拆分任务；使用 Auto Scale
`.el` 无法被查看器识别	数据库版本和导入路径

覆盖目标缺失首先属于模型构建问题，而不是目标已经不可达。数据库应与当前层级、参数和编译设置一致；排除文件应保留来源。得到不可达结果以后，还应撤去非必要约束或检查对应输入场景，以区分设计死代码与验证环境过度限制。

10. 命名差异和官方参考

X-2025.06-SP1 版本的 Quick Reference Guide 使用 `set_fml_appmode FCA`，但 User Guide §2（65 页）及 §11.2.2（395 页）使用 `set_fml_appmode COV`。本手册选择 User Guide 的 `COV` 作为示例；安装环境如存在别名或不同版本行为，以 `set_fml_appmode -help` 和 `get_fml_appmode` 确认为准。资料：User Guide 第 11 章及 Quick Reference Guide FCA 页。

SEQ：顺序等价检查使用指南

1. 为什么不能只比较组合逻辑

RTL 优化常常改变状态寄存器数量、编码、流水线或时钟门控。如果参考版本和实现版本内部结构不同，但对外部可观察输入输出仍符合约定，则二者可能功能等价。SEQ 比较两个顺序系统：参考模型（Specification）和实现模型（Implementation），通过寻找状态对应关系、输入关系和输出关系来检查其等价性。

SEQ 可用于 RTL 重构、状态机重编码、寄存器重定时、时钟门控优化以及某些微架构变化。需要明确：**等价的定义离不开观测点、初始状态、输入条件与时间对应关系**。如果实现比参考多一拍输出延迟，未经处理的逐周期相等检查可能失败，而根据允许的事务延迟重写比较规则后则可能成立。

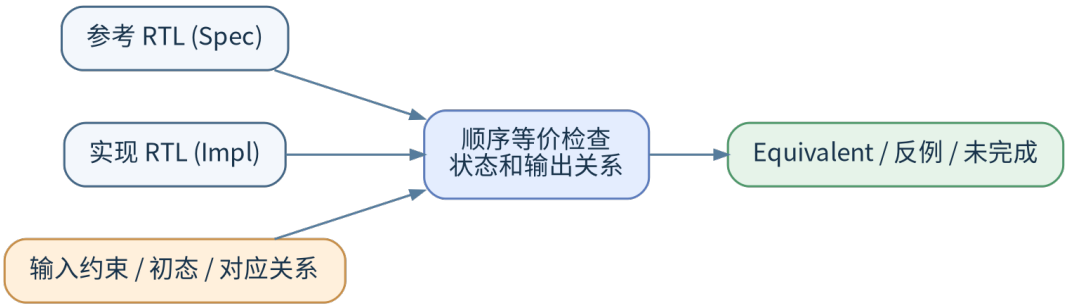


图 10: SEQ 两个版本的比较

2. 初学者例子：两种计数器写法

两边都是四位模 16 计数器；不是 FPV 章的模 10 计数器。接口相同，代码组织不同。

2.1 参考实现完整源码

使能时加一，不使能时通过未赋值保持状态。

保存为 案例工程/rtl/counter_spec.sv：

```
module counter_spec(input logic clk, rst_n, en, output logic [3:0] q);
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) q <= 4'd0;
    else if (en) q <= q + 4'd1;
endmodule
```

2.2 待比较实现完整源码

将使能扩展为四位加数；为零时相当于保持，为一时相当于递增。

保存为 案例工程/rtl/counter_impl.sv：

```
module counter_impl(input logic clk, rst_n, en, output logic [3:0] q);
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) q <= 4'd0;
    else        q <= q + {3'b000,en};
endmodule
```

2.3 故意不等价版本

同样使用 counter_impl 作为模块名，但使能时加二。正确版与错误版只能选一个加入本次比较。

保存为 案例工程/rtl/counter_impl_bad.sv：

```
// 故意错误版本：模块名仍为 counter_impl；不要与正确版本同时编译。
module counter_impl(input logic clk, rst_n, en, output logic [3:0] q);
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) q <= 4'd0;
    else        q <= q + (en ? 4'd2 : 4'd0);
endmodule
```

在合法初始化和对应关系下，前两种写法预期等价，第三种则可在使能后产生差异。预期结果不代替实际的工具证明。

3. 编译两个独立设计库并运行

3.1 正确实现的完整 SEQ 脚本

参考设计放入 spec 库，待比较设计放入 impl 库。此脚本直接列出 RTL 路径，没有额外未展示的文件列表。map_by_name 后必须查看对应结果，不能认为名字相同就完成了初始化审查。

保存为 案例工程/scripts/seq_counter.tcl：

```
set_fml_appmode SEQ
analyze -format sverilog -library spec -vcs {./rtl/counter_spec.sv}
analyze -format sverilog -library impl -vcs {./rtl/counter_impl.sv}
elaborate_seq -spectop counter_spec -impltop counter_impl
create_clock clk -period 100
create_reset rst_n -sense low
map_by_name
report_seq_mappings
check_fv_setup -block
check_fv -block
report_fv -list
```

3.2 错误实现的完整 SEQ 脚本

只替换待比较侧的源文件，参考侧和环境条件不变。

保存为 案例工程/scripts/seq_counter_bad.tcl：

```
set_fml_appmode SEQ
analyze -format sverilog -library spec -vcs {./rtl/counter_spec.sv}
analyze -format sverilog -library impl -vcs {./rtl/counter_impl_bad.sv}
elaborate_seq -spectop counter_spec -impltop counter_impl
create_clock clk -period 100
create_reset rst_n -sense low
map_by_name
report_seq_mappings
check_fv_setup -block
check_fv -block
report_fv -list
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/seq_counter.tcl -batch 2>&1 | tee logs/seq_counter.log
```

```
vcf -f scripts/seq_counter_bad.tcl -batch 2>&1 | tee logs/seq_counter_bad.log
```

这里保留原有教学脚本的初始化框架；实际项目应检查两个设计的复位状态和初始状态关系，不能把 `create_reset` 或名称对应当作“任意初态自动一致”。如需额外复位激励，应按 SEQ 的本地运行流程补充并记录。

4. 对应关系是什么

匹配可基于相同名称的输入、输出和部分状态，常用 `map_by_name`，其结果可通过 `report_seq_mappings` 审查；更复杂时可用 `seqmap`、`sequnmap` 或读入人工编写的匹配文件。不能仅因为“映射出来很多对象”就认为匹配一定正确；应特别检查复位极性、寄存器位宽、符号扩展、被优化掉的状态和不同流水线阶段的关系。

4.1 未初始化位的影响

若参考版和实现版的寄存器初态均任意，两者不一定要求从任意**独立**初态开始都立刻一致：合理的等价证明需要定义可接受的初始状态关系。将所有未初始化位置零的 `seq_config -map_uninit -map_x zero` 是该版本示例中的一种选择，但只在硬件上电模型允许时使用，否则可能忽略真实启动差异。

5. 顺序优化类型与对应关系

改动	主要挑战	需要关注的比较条件
组合表达式重写	表达式结构变化	相同输入下输出是否同拍一致
状态机重新编码	状态变量命名与编码变化	状态关系、输出一致性
加入时钟门控	等效使能与采样行为	门控使能关闭时是否保持
寄存器重定时	状态位置与延迟变化	正确建立跨状态对应
流水线深度改变	输入到输出周期可能变化	确认规格是否允许变化
微架构重构	内部状态数量可能不同	匹配可观察事务行为

等价检查必须比较相同的外部行为，而不能只依赖内部寄存器名字相似。表达式重写通常可按同拍值比较；状态重编码可能需要状态关系；流水级调整则必须明确输入接受与输出有效之间的周期关系。建立对应关系是为了构造正确的比较问题，不是允许任意把不一致信号强制认为相等。

6. 命令语法、参数与用例

命令	主要作用
`elaborate_seq`	建立参考与实现两个设计
`map_by_name`	按名称建立两侧对应关系
`seqmap`	增加人工指定的对应关系
`sequnmap`	移除或禁止自动对应
`seq_config`	配置比较策略、未知值与调试
`get_seq_mappings`	返回可编程查询的对应集合
`report_seq_mappings`	报告对应对象与未对应原因
`get_seq_equivalences`	查询内部等价检查状态
`seq_debug`	计算失败比较的诊断信息
`seq_debug_report`	输出比较失败分析

本组命令按准备、生成、执行与结果处理的关系组织。语法框保留参数名称和可选关系；示例中的实例、属性和文件名称需要与当前工程一致。生成目标与证明目标是不同阶段，调用生成命令后应检查实际目标数量，再启动证明。

6.1 elaborate_seq：建立参考与实现两个设计

```
elaborate_seq -spectop <reference_top> -impltop <implementation_top>
              [-speclib <reference_library>] [-impllib <implementation_library>]
elaborate_seq -spectop <top> -same_design
```

参数解释。 `-spectop`、`-impltop` 分别指定参考和实现顶层。分析时使用非默认 spec、impl 库名，需要相应设置 `-speclib`、`-impllib`。`-same_design` 从同一个已分析设计建立两份实例，不必重复 `analyze`，也不再要求 `-impltop`。

用法与结果。两个版本模块重名时，用库隔离，不能把两个定义放进同一个普通工作库后指望工具自动区分。`-same_design` 适合同一 RTL 在不同配置下的比较，不会自动建立配置差异；差异仍由两侧的常量、模式和环境设置表达。

```
set_fml_appmode SEQ
analyze -format sverilog -library spec ref.sv
analyze -format sverilog -library impl impl.sv
elaborate_seq -spectop ref_top -impltop impl_top
```

来源：User Guide，X-2025.06-SP1，第 520–521 页。

6.2 map_by_name：按名称建立两侧对应关系

```
map_by_name [-exclude <signals>] [-blackbox] [-undriven] [-abstraction]
  [-infer_unresolved_ports] [-map_type <types>]
  [-align_mismatch_bitwidth <alignment>] [-clock <clock>]
  [-edge <posedge|negedge|edge>]
  [-spec_latency <cycles>] [-impl_latency <cycles>]
  [-condition <expression>]
```

参数解释。 `-exclude` 排除不应自动对应的信号；`-blackbox` 包括同名黑盒和编译存储器抽象；默认还要求其底层模块名相同。`-undriven` 对应未驱动信号，`-abstraction` 对应抽象对象。`-map_type` 限制对象类别，例如只比较 output。`-align_mismatch_bitwidth` 指定不同位宽按 lsb 或 msb 对齐，使用前确认哪一部分具有规格意义。

`-clock` 与 `-edge` 决定比较采样事件，默认边沿为 posedge。`-spec_latency`、`-impl_latency` 设置两侧比较延迟；`-condition` 限定相等检查成立的条件。未解析模块的端口方向可以由 `-infer_unresolved_ports` 辅助推断。此处展开 UG 在使用章节逐项说明的选项，不将其误标为独立平台命令参考的全部选项。

用法与结果。应在 `sim_run` 之前建立对应。初始化之后才执行可能报错；即使降低错误等级，生成的约束种类也可能从环境约束变成普通属性约束，初始状态因此不再等价。输入和黑盒输出之间的相等通常形成假设；输出和黑盒输入之间的相等形成待证明目标，二者不能混淆。

```
seqmap -port spec.request impl.req
map_by_name -exclude {debug_out}
report_seq_mappings -list
# 输出存在明确的采样时钟时，可单独建立带延迟的检查。
map_by_name -map_type {output} -clock spec.clk \
  -spec_latency 3 -impl_latency 3
```

来源：User Guide，X-2025.06-SP1，第 522–528 页。

6.3 seqmap：增加人工指定的对应关系

```
seqmap [-name <name>] [-port] [-exact] <spec_expression> <impl_expression>
```

参数解释。两个位置参数可以是信号、实例或匹配表达式。实例形式在两侧实例下按寄存器名称寻找对应；通配形式只接受符合类别和位宽规则的对象。`-name` 给这条规则命名，多对匹配会产生带序号的派生名称；名称不能重复。`-port` 用于端口对象，`-exact` 表示精确信号对，而不是前缀搜索。

用法与结果。寄存器匹配主要为求解器提供提示，并不意味着未经证明就把内部寄存器强制设成相等。端口对应则可生成输入假设和输出断言，必须按接口规格审查。默认表达式匹配中，寄存器与未驱动信号不能跨类别对应。

```
seqmap -name state_pair -exact spec.state impl.state_reg
seqmap -port spec.req impl.request
map_by_name
report_seq_mappings -list
```

来源：User Guide，X-2025.06-SP1，第 558–560 页。

6.4 `sequnmap`：移除或禁止自动对应

```
sequnmap [-port] [-exact] <spec_expression> <impl_expression>
sequnmap -name <mapping_name>
```

参数解释。按信号或实例表达式移除一组关系，或仅用 `-name` 指定已命名关系。`-name` 不能与两个位置表达式一起使用。`-port` 作用于端口对应，`-exact` 避免前缀匹配扩大范围。

用法与结果。移除端口对应可能删除已经生成的属性；移除输入约束会使依赖旧约束的 proven、vacuous、uncoverable 结果失效。被显式取消的对应在后续 `map_by_name` 或 `check_fv` 中可能继续被跳过，因此不能把该命令理解为仅清除报告中的一行。

```
sequnmap -name state_pair
sequnmap -port spec.debug_in impl.debug_in
report_seq_mappings -type not_mapped -list
```

来源：User Guide，X-2025.06-SP1，第 559–561 页。

6.5 `seq_config`：配置比较策略、未知值与调试

```
seq_config [-specprefix <prefix>] [-implprefix <prefix>]
  [-recipe <recipe>] [-map_uninit] [-map_x <zero|one|safe|bestmatch|none>]
  [-map_counters] [-extended_mapping] [-latch_scope <spec|impl|none>]
  [-auto_failure_debug <ON|OFF>] [-enable_rca_path <ON|OFF>]
  [-map_diff_bb_designs <ON|OFF>] [-enable_clock_coi_analysis <true|false>]
  [-infer_unresolved_ports <ON|OFF>]
  [-enable_infer_blackbox_port_direction <true|false>]
```

参数解释。`-specprefix`、`-implprefix` 默认 spec、impl。`-recipe` 可选 orch_generic_easy、orch_generic_low、orch_generic_medium、orch_generic_hard，默认 medium。`-map_uninit` 在复位时按名称对应未初始化寄存器；`-map_counters` 对应抽象计数器；`-extended_mapping` 扩展人工对应的对象搜索范围。`-latch_scope` 选择对哪一侧进行锁存器处理。

`-map_x` 默认 none：zero/one 将工具产生的未定义值固定为 0/1，safe 对应同名但位宽不同的信号，bestmatch 优先按名称对应否则置零，none 保留未知行为。它不处理 snip；切断的两侧信号需另行建立关系。`-auto_failure_debug` 控制失败分析；`-enable_rca_path` 默认 ON。`-map_diff_bb_designs` 默认 OFF，开启后允许同名但底层模块不同的黑盒对应。未解析端口推断默认 ON，所有黑盒端口方向推断默认 false；`-enable_clock_coi_analysis` 默认 false。

用法与结果。这些选项不只是运行速度设置。尤其把 X 固定为常量、对应未初始化状态、允许不同黑盒对应，都改变了比较问题或其抽象方式。应在报告中保留设置及理由，并先确认正常初始化下的行为，不能为了消除失败而直接使用 bestmatch。

```
seq_config -recipe orch_generic_medium
seq_config -map_x none
seq_config -auto_failure_debug ON
```

```
seq_config -map_diff_bb_designs OFF
```

来源：User Guide，X-2025.06-SP1，第 562–564 页。

6.6 get_seq_mappings：返回可编程查询的对应集合

```
get_seq_mappings [-type <mapped|not_mapped>]
  [-map_type <types>] [-skip_map_type <types>] [-clear_mem]
```

参数解释。-map_type 支持 input、output、bbin、bbout、absin、absout、sequential、undriven、unloaded、explicit_x、undefined_x、net，以及 auto、all；-skip_map_type 排除具体类别。-type 区分已对应和未对应，-clear_mem 清理之前该命令建立的集合，避免长期脚本累积查询对象。

用法与结果。在 map_by_name 后调用。返回值是工具集合，不是普通 Tcl 字符串列表；应用 foreach_in_collection 和 get_attribute 读取属性。not_mapped_reason 比单看未对应数量更有意义，能够区分名称不存在、位宽不同、方向不同等问题。

```
set missing [get_seq_mappings -type not_mapped -map_type input]
foreach_in_collection obj $missing {
  puts "[get_attribute $obj full_name]: [get_attribute $obj not_mapped_reason]"
}
```

来源：User Guide，X-2025.06-SP1，第 545–549 页。

6.7 report_seq_mappings：报告对应对象与未对应原因

```
report_seq_mappings [-type <mapped|not_mapped>]
  [-map_type <types>] [-skip_map_type <types>]
  [-backend] [-list] [-verbose] [-no_summary]
  [-sort <ascending|descending>]
```

参数解释。对象类别与 get_seq_mappings 相同。-list 每对一行，-verbose 展开细节，-no_summary 去掉汇总；-sort 按位宽升序或降序排列。-backend 查看求解后端实际采用的对应，不可同时使用 -map_type、-list、-verbose、-no_summary。

用法与结果。此命令说明“哪些对象建立了关系”，不是“哪些比较已经证明成功”。默认新流程先给类别汇总，需显式加 -list 才逐项显示。最终等价结论仍由输出比较和必要的约束审查决定。

```
report_seq_mappings -map_type all -list
report_seq_mappings -type not_mapped -verbose
report_seq_mappings -backend
```

来源：User Guide，X-2025.06-SP1，第 550–558 页。

6.8 get_seq_equivalences：查询内部等价检查状态

```
get_seq_equivalences [-status <success|failed|bounded|no_status>]
                     [-id <integer>]
```

参数解释。 -status 筛选内部等价点状态；-id 选择报告中指定的内部等价编号。success 是内部检查成功，failed 是内部不等价，bounded 是有界结果，no_status 可能表示该提示未被使用。

用法与结果。内部寄存器不等价不必然导致输出不等价，例如重编码或不同流水组织可能改变内部状态而保持接口功能。该查询服务于调试和分析，不可用“所有提示必须 success”替代最终输出检查标准。

```
set failed [get_seq_equivalences -status failed]
foreach_in_collection eq $failed {
  puts [get_attribute $eq full_name]
}
```

来源：User Guide，X-2025.06-SP1，第 564–566 页。

6.9 seq_debug：计算失败比较的诊断信息

```
seq_debug [-stop] [-block] [<property_list>]
```

参数解释。省略目标列表时对当前适用的失败属性计算；提供属性名则限制范围。`-block` 等待分析完成，`-stop` 停止正在进行的分析。

用法与结果。先运行比较并获得反例，再计算诊断。信息区分第一处不一致、驱动关系不一致、未对应、未驱动、切断、抽象或黑盒影响等类型。这些类型说明反例如何形成，不自动判定是 RTL 错误还是环境设置错误。

```
check_fv -block
seq_debug -block
seq_debug_report -detail verbose
```

来源：User Guide，X-2025.06-SP1，第 584–588 页。

6.10 seq_debug_report：输出比较失败分析

```
seq_debug_report [-detail <list|summary|verbose>]
  [-format <text|xml>] [-group_failure <criteria>] [<property_list>]
```

参数解释。`-detail` 默认 summary；`-format` 默认 text；`-group_failure` 可按 mismatch、depth、rootcause_type、rootcause_signal 分组。属性列表限制报告范围。

用法与结果。必须先有诊断计算结果。按最早失败深度和共同原因信号分组，可以避免逐条重复检查同一个接口配置错误；XML 格式适合后续脚本汇总，但应保留原属性名称和反例索引。

```
seq_debug -block
seq_debug_report -detail verbose -format text
seq_debug_report -group_failure {depth rootcause_type}
```

来源：User Guide，X-2025.06-SP1，第 587–589 页。

7. 如何解释不等价结果

首先判断是否为真实对外行为差异，然后才检查内部状态。推荐按如下顺序：确定失败的输出或检查点；核对两设计的输入在同一采样时刻是否一致；检查复位起始状态；检查 X 与未初始化值策略；检查流水线延迟或时钟门控；最后追踪内部状态分歧最早发生的位置。

三**种不同结论不能混同：**一是明确找到了两者输出行为不一致的反例；二是当前资源下无法证明；三是当前对应关系或抽象造成局部失败。若初始状态和外部协议前提建错，需要修复比较环境而不是简单修改 RTL。

8. 高级优化策略

对于寄存器重定时、多时钟门控和大型算术逻辑，可考虑 `seq_config -recipe orch_generic_medium`、`set_fml_var fml_effort high`、`set_seq_abstractions ...` 和分组证明。抽象可能缩小计算量，也可能引入新反例，需要再验证其可实现性。

对复杂关系建议分步骤：证明复位和普通模式的基础等价，确认输入输出映射，再处理新增加的内部状态及特殊操作模式。不要将所有复杂证明一次交给自动命名对应而忽略潜在误配。

9. 教学练习

1. 证明 `en ? q+1 : q` 与 `q+en` 在指定位宽下是否一致；注意表达式位宽。
2. 修改实现版，使输出晚 1 拍，观察直接逐拍比较为什么失败。
3. 在实现版增加一个无功能影响的寄存器，分析映射报告和求解结果。

4. 把初始状态改为未定义，比较是否仍成立，并说明真正的硬件上电规格。

10. 参考

User Guide §14（517 ~ 611 页），尤其设计编译、映射与调试部分；§18 的复杂性分析；Quick Reference Guide SEQ 章节。对比较多周期事务而非逐拍行为的需求，也可对照 DPV.md。

FRV：形式寄存器验证使用指南

1. 典型目标和输入资料

SoC 中寄存器字段数量多，容易出现复位值错误、读写权限错误、写一清零行为错误、保留位处理错误、地址别名或总线包装接口问题。FRV 通过寄存器说明文件（IP-XACT XML、RALF 等）提取字段规则，产生并运行形式检查；其价值在于减少重复手工编写针对每个寄存器位的测试与断言。

输入至少包括真实 RTL、寄存器规格文件、总线协议适配及形式环境。特别注意：寄存器文件包含的**理想行为**必须与设计真实的事务接口含义一致；仅有 XML 不会使工具自动猜到哪一条总线读写应对应哪个寄存器。

2. 核心概念

寄存器属性	预期行为	应验证的例子
Reset value	复位后默认值	配置字段复位为零
RW	可读可写	写入已接受后读回一致
RO	只读	总线写入不能改变状态
WO	只写	读取行为应符合规格定义
W1C	写 1 清零	写 1 清对应位，写 0 不清
W1S	写 1 置位	写 1 置位，其他位不变
Reserved	保留位	写入是否被忽略、读取返回什么
Volatile	硬件可以自主更新	不要求所有周期保持软件写入值
Side effect	读取或写入有副作用	读清除、中断状态操作

访问类型决定一次总线事务应当如何改变寄存器，而复位值规定启动状态，两者需要分开验证。以写一清零字段为例，写入一的位应清除，写入零的位应保持；字节使能关闭的位也应保持。若硬件事件与软件写入同拍发生，还必须在规格中给出优先级，否则工具无法从地址和访问类型推导唯一预期结果。

字段策略由寄存器规范决定。不要把没有发生读写握手的周期当作已接受事务，也不能把硬件会自动变化的计数器字段视作永远保持的软件配置值。

2.1 自动断言检查哪条路径

断言类别	对照对象	含义
首次读回与写后读回	总线读取值与复位值或最近写入值	检查外部可见寄存器行为
后门初值	内部字段与规定初始值	检查复位后首个周期的内部状态
后门读取	总线读回与指定周期的内部字段	检查读路径及延迟对应
后门写入	总线写入与随后内部字段	检查写路径及更新延迟
地址块以外读取	块外访问的返回值	检查规格定义的越界访问处理

总线检查回答软件看到的数据是否正确，后门检查直接观察内部字段，两者可以同时用于区分“寄存器没有更新”和“寄存器更新了但读路径错误”。是否生成某条检查还受访问类型、是否提供初始值、是否提供后门信号以及 `COMPOSITE` 等参数影响。地址块外访问若设计返回错误响应，数据检查可能空通过；这种接口行为应按规格配置，而不是为了增加目标数量强行要求返回普通数据。

自动生成名称中的 <attr> 是字段访问属性相关占位内容。UG 列出的命名形式如下，生成后的真实属性名应从报告中取得：

```
ast_<attr>_read_data_check
ast_<attr>_initial_read_check
ast_<attr>_write_read_check
ast_<attr>_bdr_initial_check
ast_<attr>_bdr_read_check
ast_<attr>_bdr_write_check
ast_out_adblk_read_check
```

2.2 覆盖目标按访问顺序编排

分组	覆盖对象	要确认的场景
本寄存器读写历史	首次读、写后读、再次读	初始化和写入后的访问均可发生
交错访问	写入后插入其他地址访问再读回	周边访问不阻止目标序列出现
首次访问前的其他访问	其他地址访问后首次读或写	非空历史下仍能访问目标字段
访问权限	向只读字段写、从只写字段读	禁止方向的请求没有被环境直接排除
用户指定地址	指定地址与目标字段的交错顺序	特殊副作用和地址关系能够触发
有效字节通道	字节使能覆盖目标	字段的有效字节通道确实更新

本寄存器的四种基本历史区分第一次读是否发生在写之前，以及再次读之前是否已发生写入。覆盖这些场景有助于发现环境把首次访问固定为写入，从而掩盖复位值检查的问题。

```
cov_first_read_before_write
cov_first_read_after_write
cov_read_after_read_before_write
cov_read_after_read_and_write
```

交错访问的目标要求先写目标寄存器，在中间访问其他地址，再读回目标寄存器。其中 diff_addr 表示其他寄存器地址，out_addr 表示地址块以外，user_addr 表示用户指定地址。它们只证明访问顺序可出现，数据是否保持仍由相应断言证明。

```
cov_diff_addr_write_between_write_read
cov_diff_addr_read_between_write_read
cov_out_addr_write_between_write_read
cov_out_addr_read_between_write_read
cov_user_addr_write_between_write_read
cov_user_addr_read_between_write_read
```

首次目标访问之前的历史有四种组合：先读其他地址再首次写目标，先写其他地址再首次读目标，先写其他地址再首次写目标，以及先读其他地址再首次读目标。每种组合分别对应普通其他地址、地址块外地址和用户指定地址。

```
cov_diff_addr_read_before_first_write
cov_out_addr_read_before_first_write
cov_user_addr_read_before_first_write
cov_diff_addr_write_before_first_read
cov_out_addr_write_before_first_read
cov_user_addr_write_before_first_read
cov_diff_addr_write_before_first_write
cov_out_addr_write_before_first_write
cov_user_addr_write_before_first_write
cov_diff_addr_read_before_first_read
cov_out_addr_read_before_first_read
cov_user_addr_read_before_first_read
```

权限与字节使能场景不应被理解为要求非法操作成功。只读字段允许收到写请求，但规格可能要求忽略写入或报错；该请求应能够到达，随后由断言检查正确处理。字节通道目标则检查目标字段的有效字节通道是否发生更新，不能代替逐字节的保持和更新规则。

```
cov_read_only_write
cov_write_only_read
cov_valid_byte_lane_updated
```

以上按行为分组列出 UG 的 25 个覆盖名称，而不是把每个名称拆成孤立小段。UG 明确将其中一组用户地址目标列为默认不生成，后续版本增加的目标也仍受参数和适用条件控制；不能假定每个字段默认产生全部 25 项。来源：User Guide，第 622–625 页。

3. 寄存器规格与 RTL 的对照

例如有 `CTRL`（地址 0x00，位 0 使能，RW）、`STATUS`（地址 0x04，位 0 完成，W1C）。典型教学实现是：接受对 `CTRL` 地址的写入后更新使能位；`STATUS.done` 由硬件事件置位、软件向对应位写 1 清除；异步复位全部清零。对应检查至少覆盖默认值、RW 正常写入、W1C 保持和清零，以及无效地址的处理。

FRV 的自动检查可以缩小手工工作量，但对多写入源冲突、硬件置位和软件清零同周期优先级等复杂语义，通常仍需要补充明确的 SVA。假定硬件事件优先级高于 W1C，需要实现与规格一致的条件判断。

3.1 完整教学实现与手写属性

以下把上述规格写成可以独立保存的 RTL 和 SVA。接口不是 APB/AXI：`wr_en=1` 的上升沿立即接受写入，读取是由 `rd_en` 控制的组合输出；无字节使能，无等待周期。这样可以先理解 RW、W1C 与硬件置位优先级，再接入真实总线。

3.1.1 寄存器 RTL

保存为 案例工程/rtl/reg_block_demo.sv：

```
// 单周期教学寄存器接口，不是 APB/AXI 适配器。
// 上升沿 wr_en=1 即接受写入；rd_en=1 时组合读出。
module reg_block_demo (
  input  logic        clk, rst_n,
  input  logic        wr_en, rd_en,
  input  logic [7:0]  addr,
  input  logic [31:0] wdata,
  input  logic        hw_done,
  output logic [31:0] rdata,
  output logic        ctrl_enable, status_done
);
  localparam logic [7:0] CTRL_ADDR   = 8'h00;
  localparam logic [7:0] STATUS_ADDR = 8'h04;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      ctrl_enable <= 1'b0;
      status_done <= 1'b0;
    end else begin
      // CTRL[0] 为 RW，其他位忽略写入。
      if (wr_en && addr == CTRL_ADDR)
        ctrl_enable <= wdata[0];

      // 硬件置位优先于软件写一清零；写零保持原状态。
      if (hw_done)
        status_done <= 1'b1;
```

```
      else if (wr_en && addr == STATUS_ADDR && wdata[0])
        status_done <= 1'b0;
    end
  end

  always_comb begin
    rdata = 32'd0;
    if (rd_en) begin
      case (addr)
        CTRL_ADDR:   rdata = {31'd0, ctrl_enable};
        STATUS_ADDR: rdata = {31'd0, status_done};
        default:     rdata = 32'd0;
      endcase
    end
  end
endmodule
```

3.1.2 完整检查器

W1C 检查只有在上一拍没有硬件事件时才要求清零；如果硬件事件和软件写清除同时发生，本例明确要求硬件置位优先。

保存为 案例工程/formal/reg_block_demo_props.sv：

```
module reg_block_demo_props (
  input logic clk, rst_n, wr_en, rd_en, hw_done,
  input logic [7:0] addr,
  input logic [31:0] wdata, rdata,
  input logic ctrl_enable, status_done
);
  logic history_valid;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) history_valid <= 1'b0;
    else        history_valid <= 1'b1;

  // 当前采样值应等于上一拍已接受写事务的结果。
  ap_rw_write: assert property (@(posedge clk) disable iff (!rst_n)
    history_valid && $past(wr_en && addr == 8'h00)
      |-> ctrl_enable == $past(wdata[0]));
  ap_rw_hold: assert property (@(posedge clk) disable iff (!rst_n)
    history_valid && !$past(wr_en && addr == 8'h00)
      |-> ctrl_enable == $past(ctrl_enable));
  ap_hw_set_priority: assert property (@(posedge clk) disable iff (!rst_n)
    history_valid && $past(hw_done) |-> status_done);
  ap_w1c: assert property (@(posedge clk) disable iff (!rst_n)
    history_valid && $past(!hw_done && wr_en && addr == 8'h04 && wdata[0])
      |-> !status_done);
  ap_status_hold: assert property (@(posedge clk) disable iff (!rst_n)
    history_valid && $past(!hw_done && !(wr_en && addr == 8'h04 && wdata[0]))
      |-> status_done == $past(status_done));

  ap_read_ctrl: assert property (@(posedge clk) disable iff (!rst_n)
    rd_en && addr == 8'h00 |-> rdata == {31'd0, ctrl_enable});
  ap_read_status: assert property (@(posedge clk) disable iff (!rst_n)
    rd_en && addr == 8'h04 |-> rdata == {31'd0, status_done});
  ap_read_default: assert property (@(posedge clk) disable iff (!rst_n)
    (!rd_en || (addr != 8'h00 && addr != 8'h04)) |-> rdata == 32'd0);

  // 不使用 disable iff，单独说明采样到复位时的后继状态。
  ap_reset: assert property (@(posedge clk)
```

```
    !rst_n |=> !ctrl_enable && !status_done);
  cp_hw_sw_conflict: cover property (@(posedge clk) disable iff (!rst_n)
    hw_done && wr_en && addr == 8'h04 && wdata[0]);
endmodule

bind reg_block_demo reg_block_demo_props u_reg_props (.*);
```

3.1.3 文件列表

保存为 案例工程/filelist_fpv_register_demo.f:

```
./rtl/reg_block_demo.sv
./formal/reg_block_demo_props.sv
```

3.1.4 手写属性的 FPV 运行脚本

这是 FRV 专章中的配套 FPV 练习，不调用 frv_load，不会生成 FRV 自动检查器。

保存为 案例工程/scripts/fpv_register_demo.tcl:

```
# 从案例工程目录执行；结果必须以本机 VC Formal 日志为准。
set_fml_appmode FPV
read_file -top reg_block_demo -sva -format sverilog -vcs {-f
 ↪  filelist_fpv_register_demo.f}
create_clock clk -period 100
create_reset rst_n -sense low
# 先核验实际复位状态，再保存初始状态。
sim_run -stable
sim_save_reset
check_fv_setup -block
report_fv -list
check_fv -block
report_fv -verbose
```

运行命令（Bash）:

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_register_demo.tcl -batch 2>&1 | tee logs/fpv_register_demo.log
```

检查范围。本节源码足以说明并验证这个简化接口下的寄存器规则，但不能替代 FRV 对 IP-XACT/RALF 的加载、总线适配和自动生成属性。原生 FRV 工程仍须由本节之后的规格输入及接口桥接组成。

4. 一个最小脚本模板

```
set_fml_appmode FRV
# 二选一：实际工程提供寄存器描述文件
frv_load -ral registers.ralf -top block:REG_BLOCK -auto_load
# frv_load -ipxact registers.xml -auto_load
read_file -top reg_block -sva -format sverilog \
  -vcs {-f filelist_reg.f} bind_frv.sv
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup -block
report_fv -class register -list
check_fv -block
frv_report
```

```
report_fv -class register -verbose
```

bind_frv.sv 必须实现与实际总线的接口对应关系，不能把此占位文件视作工具自动生成的所有环境组件。在上述 -top block:REG_BLOCK 中，REG_BLOCK 是 RALF 定义的地址块名称，**不是随意从 RTL 顶层抄来的实例名**；如果使用其他层级，可指定 comp:、mmap: 或 block:，详见 UG 第 626 ~ 630 页。IP-XACT 和 RALF 的可支持字段、命名空间及访问类型以该版本 User Guide 16.4 ~ 16.11 为准。

4.1 原生 FRV 模板完整源码

它保留防误执行的 error；RALF/IP-XACT、生成检查器接口以及真实总线绑定并未伪造成已经存在的文件。补齐输入并审查后才能启用相应命令。

保存为 案例工程/templates/frv_ralf.tcl：

```
# FRV：RALF/IP-XACT 寄存器检查模板。绝不能原样执行。
# 替换 RTL 顶层、RALF、总线桥接与复位时序，并审核寄存器访问与检查器绑定。
error " 此文件是 FRV 教学模板：请先替换设计、寄存器定义、接口桥接和复位序列"
set_fml_appmode FRV
# frv_load -ral /project/csr/regs.ralf -top block:REG_BLOCK -auto_load
# read_file -top reg_top -sva -format sverilog -vcs {-f filelist_reg.f} bind_frv.sv
# create_clock clk -period 100
# create_reset rst_n -sense low
# sim_run -stable
# sim_save_reset
# check_fv -block
# frv_report
```

5. 命令语法、参数与用例

命令	主要作用
frv_load	从寄存器规格生成并装入检查器
frv_config	控制寄存器属性生成范围
frv_report	检查规格解析与属性生成情况

本组命令按准备、生成、执行与结果处理的关系组织。语法框保留参数名称和可选关系；示例中的实例、属性和文件名称需要与当前工程一致。生成目标与证明目标是不同阶段，调用生成命令后应检查实际目标数量，再启动证明。

5.1 frv_load：从寄存器规格生成并装入检查器

```
frv_load [-ipxact <file>] [-ral <file>] [-csv <file>]
  [-bind <bind_file>] [-top <[comp|mmap|block]:name>]
  [-checker_file <file>] [-include <type_list>]
  [-auto_load] [-parameters <parameter_list>] [-verbose]
```

参数解释。-ipxact、-ral、-csv 选择相应格式的寄存器规格，工程应明确选择自己的输入格式，不把三种规格混在一次调用中。-bind 给出总线连接文件；-checker_file 指定生成的检查器文件名，默认 frv.sv。-top 限定 component、memoryMap 或 addressBlock；不写类型时依次按 comp、mmap、block 查找，同名对象可使用完整层次。

-include 可选 unmapped、unmapped_reg、unmapped_field、reserved、reserved_reg、reserved_field。包含保留或未分配空间后，还需设置 RSVD_MODE：0 不检查，1 期望全零，2 期望全一，3 使用 snps:outAddrValue 自定义值。-auto_load 装入生成的检查器、绑定文件和依赖库。-parameters 是 Tcl 参数列表，例如 {RSVD_MODE=1 COVER_READ=1}；支持的检查器参

数还包括 ADDR_WIDTH、DATA_WIDTH、COMPOSITE、BUS_LATENCY、WR_LATENCY、RD_LATENCY、COVER_TYPE、COVER_WRITE、COVER_BYTE_UPDATE、OUTADDR_EN、LATENCY_MD、OUTABLK_MD、SNPS_AD_ABS、COVER_ALLOW_ERROR。`-verbose` 显示不支持的结构，而不是静默忽略。

用法与结果。该命令生成检查器，不代表已经完成证明。总线包装必须把协议事件转换成检查器所需的读写事件；APB/AHB/AXI 包装在安装目录的 Frv 包中，其他协议需要检查并适配通用包装。原文示例部分用 `-parameter`，正式语法用 `-parameters`，正文统一使用后者，不依赖工具对缩写的接受行为。

```
frv_config -generate_covers true
frv_load -ipxact regs.xml -bind regs_bind.sv \
  -top block:soc.main.regs -include {reserved_field} \
  -parameters {RSVD_MODE=1 COVER_READ=1 COVER_WRITE=1} \
  -auto_load -verbose
frv_report
```

来源：User Guide，X-2025.06-SP1，第 626–630 页。

5.2 `frv_config`：控制寄存器属性生成范围

```
frv_config [-generate_backdoor_checks <boolean>]
  [-generate_covers <boolean>]
  [-generate_out_of_block_range_checks <true|false>:<0|1>]
  [-ignore <type_list>] [-disable_backdoor_checks <reg_array|all>]
  [-generate_wo_read_check <boolean>]
  [-generate_wo_read_check_value <value>]
  [-generate_bdrsig_from_name] [-enable_fiv <true|false>]
  [-enable_volatile_check <none|with_hw_update|all>]
```

参数解释。反向观察检查与 cover 生成默认开启；地址块之外的检查默认关闭。`-generate_out_of_block_range_checks` 同时给出开关及期望读值，期望值只能为 0 或 1，省略时默认 0。`-ignore` 当前列出 unknown_access；`-disable_backdoor_checks` 可禁用寄存器数组或全部反向观察检查。

`-generate_wo_read_check` 默认关闭，开启后再用 `-generate_wo_read_check_value` 指定只写字段被读时的期望值，顺序不能颠倒。`-generate_bdrsig_from_name` 根据规格的 name 属性生成 RTL 信号路径，只有名称确实对应 RTL 时才可用。`-enable_fiv` 默认 false；`-enable_volatile_check` 默认 none，with_hw_update 只对带硬件更新描述的易变字段生成读数据检查，all 对所有此类字段生成。

用法与结果。必须在 `frv_load` 之前配置，因为这些选项决定生成哪些属性，而不是仅在结果报告时过滤。无参数调用打印当前默认配置，适合保存到运行日志。boolean 选项按原文支持 true/false 或 1/0；对明确只列 true/false 的选项保持该拼写。

```
frv_config -generate_covers true
frv_config -generate_wo_read_check true
frv_config -generate_wo_read_check_value 0
frv_config -enable_volatile_check with_hw_update
frv_config
frv_load -ipxact regs.xml -auto_load
```

来源：User Guide，X-2025.06-SP1，第 630–632 页。

5.3 `frv_report`：检查规格解析与属性生成情况

```
frv_report [-hide_fields] [-no_summary]
  [-top <[comp|mmap|block]:name>]
  [-vendor_extensions <[comp|mmap|block]:hierarchical_name>]
  [-filter <expression>]
```

参数解释。 `-hide_fields` 隐藏字段细节，`-no_summary` 隐藏汇总。`-top` 限定规格对象，类型省略时按 comp、mmap、block 查找。`-vendor_extensions` 展开指定寄存器或字段的厂商扩展信息。`-filter` 支持 regAccess、fieldAccess，以 `==` 或 `!=` 比较访问类型。

用法与结果。应在证明前检查寄存器数量、地址、属性数量与预期是否一致。缺少寄存器或字段检查往往来自规格格式、访问类型、包含范围或参数设置，不能用“所有已生成断言通过”来掩盖漏生成。

```
frv_report
frv_report -filter {regAccess == rw}
frv_report -top block:soc.main.regs -hide_fields
frv_report -vendor_extensions block:soc.main.regs.CTRL.EN
```

来源：User Guide，X-2025.06-SP1，第 632–633 页。

5.4 FRV 检查器和总线桥接的实际组成

UG 第 627 页说明，`frv_load` 可以生成专用的 SVA 检查器文件（默认 `frv.sv`），并可用 `-auto_load` 自动装入生成的检查器、绑定文件及库文件。对于常见的 AMBA 总线，安装目录 `$VC_STATIC_HOME/packages/Frv/` 还包含 APB、AHB、AXI 的预制适配模块，以及用于简单非 AMBA 接口的通用适配模块。

典型路径	用途
`snps_frv_apb_wrapper.sv`	APB 总线寄存器访问抽象
`snps_frv_ahb_wrapper.sv`	AHB 总线寄存器访问抽象
`snps_frv_axi_wrapper.sv`	AXI 总线寄存器访问抽象
`snps_frv_gen_wrapper.sv`	一般寄存器读写接口示例

桥接模块主要负责把总线请求、响应、地址和有效数据准确提供给属性检查器。**提供预制适配模块并不意味着可以免除总线配置检查**：例如 AXI 的 `ready/valid` 接受时刻、超时和响应对应关系仍需核实。若 IP 自定义了特殊握手，需要编写自己的适配器并检查覆盖目标是否生成。

5.5 关于 `-parameter` / `-parameters` 的说明

同一版 UG 第 626 页的语法框将选项写成 `-parameters`，随后第 630 页示例写成 `frv_load -parameter {...}`。正文语法统一采用语法框中的 `-parameters`；不推断两种拼写均被 Tcl 接受。移植已有脚本时，使用安装版本的命令帮助确认是否支持示例中的简写。

6. 结果分析的方法

如果复位值属性失败：检查该寄存器是不是在真实复位域内、复位序列是否正确、字段是否含上电未知值。若读回失败：先确定读请求是否被接受以及返回数据何时有效，再检查寄存器地址、字节使能与读数据对齐。若 W1C 失败：检查写入掩码、硬件事件与软件写清除的优先级。若很多属性都没被生成：核对寄存器描述路径与映射，不要直接把“没有失败”理解为“寄存器全部正确”。

7. 与 UVM RAL、仿真和 FPGA/SoC 寄存器校验的配合

软件寄存器模型适合生成仿真激励和软件驱动期望值；FRV 可以对寄存器行为进行更系统的逻辑证明。两者应复用同一权威寄存器描述，避免手工维护两份不一致的默认值。对异步硬件更新、跨时钟寄存器和只在某个模式下可写的寄存器，还应有专用接口前提和 SVA 约束。

8. 工程清单

- 确认 IP-XACT/RALF 字段地址、宽度、复位和访问类型均与规格一致。
- 确认总线读写接受点、字节使能、错误响应、超时与复位行为。
- 确认已经报告并审查 FRV 自动生成属性的数量及类别。
- 对 W1C/W1S/volatile/自清除等字段增加必要的手工检查。
- 对无法证明、豁免和未支持的字段单独汇总，而不是将其遗漏。

9. 参考资料

User Guide §16（619 ~ 685 页）及附录中的相关设计查询命令；工具安装目录 `<VC_STATIC_INSTALL>/doc/vcst/examples/FRV` 可寻找官方示例（若随本机安装）。

FTA：形式测试平台分析使用指南

1. FTA 解决的不是 RTL 正确性，而是属性环境的质量

形式验证通常给出很多 Proven，但这些证明是否足够覆盖真实设计风险？某个功能错误是否会被当前断言检测到？FTA 使用**故障/RTL 修改（mutation）注入**的方法分析形式测试平台的检错能力。如果注入一个功能变化后仍没有任何断言检测到异常，说明当前检查集合可能存在薄弱位置。它是一项验证质量评估方法，而不是再运行一次普通 FPV。

例如，RTL 中 `grant[0]` 的组合条件被修改。如果已有的互斥属性仍全部通过，但缺少“请求必须得到相应授权”的规则，那么 FTA 可能发现这一类变更不易被现有属性检测。



图 11: FTA 故障注入与属性检出

2. 工作过程

1. 在 FPV 模式编译 DUT 和属性，建立正确的时钟、复位与环境前提。
2. 审查现有属性状态，确认值得评估的断言范围，区分断言与覆盖目标。
3. 根据当前软件版本配置可用的故障类型和注入策略。
4. 切换到 FTA，运行激活、传播与检测分析。
5. 汇总可被检出的故障、无法检出的故障以及尚无结论的故障。
6. 对未检测区域补充有意义的设计属性，再运行 FTA 和 FPV 回归。

3. 不能直接从 `-fmode FTA` 启动

User Guide 第 65 页明确提醒：该版本通常从 FPV 模式建立工作区，再使用 `set_fml_appmode FTA` 切换，`vcf -fmode FTA` 不属于支持的直接启动方式。快速指南有独立的 FTA 命令行，但并不意味着可以跳过准备好的 FPV 环境。

4. 示范 Tcl：多阶段流程的轮廓

```
# 第一阶段：FPV 环境和属性分析
set_fml_appmode FPV
read_file -top core -sva -format sverilog -vcs {-f filelist.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup -block
check_fv -block
```

```
report_fv -list

# 第二阶段：FTA（故障设置根据具体版本和设计调整）
set_fml_appmode FTA
fta_init
configure_fta_props -par_task FPV -status {proven}
compute_fta -par_task FPV
fta_report
```

这个脚本描述学习顺序，**不是一个对任何 FTA 许可证和故障模型都立即可运行的完整正式工程**。尤其 `fta_init`、`configure_fta_props` 和 `compute_fta` 的参数需与当前会话的 FPV task、故障数据库及流程类型对应。阅读 User Guide 第 923 ~ 941 页完整流程再部署生产项目。

4.1 FTA 工程配置模板

`error` 明确阻止把缺少已验证属性集合或故障配置的脚本当成真实 FTA 分析。

保存为 案例工程/templates/fta_mutation.tcl：

```
# FTA：Formal Testbench Analyzer。须基于真实的 FPV 工程与属性集合。
# 不能仅用 vcf -fmode FTA 直接启动，详见 UG §21 和手册 FTA.md。
error " 此文件是 FTA 教学模板：请先完成真实 FPV 环境、属性集合和故障策略"
# set_fml_appmode FPV
# read_file -top dut -sva -format sverilog -vcs {-f filelist.f}
# create_clock clk -period 100
# create_reset rst_n -sense low
# sim_run -stable
# sim_save_reset
# check_fv -block
# set_fml_appmode FTA
# fta_init
# configure_fta_props -par_task FPV -status {proven}
# compute_fta -par_task FPV
# fta_report
```

5. 故障分类与结果含义

场景	工程解释	后续建议
故障被属性检出	当前断言对该类改变敏感	检查检出的属性是否符合预期
故障未检出	可能是死代码、影响不可达、属性缺失或环境屏蔽	先审查可达性和影响路径
注入无法激活	故障点在现有环境下不被触发	检查约束及故障定义
结果未完成	当前证明资源不足	提升策略或缩小任务
被审查豁免	有外部充分证据认定不需要该项	记录依据并保留原结果

故障结果必须结合分析前的属性状态和所选故障模型解释。基准设计本身已经失败时，不能把同一失败计为新增故障被检出；故障无法激活或传播时，也不能据此宣称断言已经检查了对应功能。应把未检出的有效故障回到规格和属性正文中分析。

检出率与业务规格覆盖完整程度相关，但不能直接等同于“芯片功能正确的百分比”。

6. 命令语法、参数与用例

命令	主要作用
fta_init	限定故障注入设计范围
configure_fta_props	选择检测故障所使用的属性
compute_fta	执行形式测试环境质量分析
get_fta_faults	按实例与状态选取故障
fta_report	输出实例级故障统计
read_fta_waiver	重新应用已审查的故障豁免
read_faultdb	导入仿真测试环境故障数据库
save_faultdb	保存形式故障分析结果
cluster_fta_faults	把相似未检测故障分组
report_fta_fault_clusters	查看未检测故障分组
debug_fta	判断未检测故障能否传播到输出

本组命令按准备、生成、执行与结果处理的关系组织。语法框保留参数名称和可选关系；示例中的实例、属性和文件名称需要与当前工程一致。生成目标与证明目标是不同阶段，调用生成命令后应检查实际目标数量，再启动证明。

6.1 fta_init：限定故障注入设计范围

```
fta_init -top <top_module>
fta_init -scope <module_list>
```

参数解释。 -top 指定故障注入的顶层，分步 analyze/elaborate 流程尤其要先设置；-scope 限定参与故障注入的模块列表。两种写法分别出现在 UG 的流程说明中，不等于所有模式都必须同时提供两个选项。

用法与结果。 FTA 应从已有的干净 FPV 环境开始，随后在读入或展开设计时使用 -inject_fault。未提供专用 Certitude 文件时工具可为所有模块自动生成注入配置；这会显著扩大故障集合。显式配置文件中的 FaultSet 与注入开关存在优先级，不能只看一条 Tcl 判断实际故障范围。

```
set_fml_appmode FPV
fta_init -top controller
fta_init -scope {controller}
read_file -top controller -sva -inject_fault RTL \
  -format sverilog -vcs "-f formal.f"
```

来源：User Guide，X-2025.06-SP1，第 923–925 页。

6.2 configure_fta_props：选择检测故障所使用的属性

```
configure_fta_props -par_task <fpv_task>
  [-fault_class <classes>] [-type <types>]
  [-usage <usages>] [-status <statuses>]
  [-min_proof_depth <depth>]
```

参数解释。 -par_task 指定父 FPV 任务。-fault_class 在已插入的故障内进一步选择类别，例如 SynchronousControlFlow、ComboLogicControlFlow；它不能补插编译时没有生成的故障。-type、-usage、-status 选择参与检测的属性，默认只使用 proven 属性。-min_proof_depth 为纳入的未完成属性设定最小有界证明深度；未显式给出时，工具取所选未完成属性中最小的证明深度。

用法与结果。选择 inconclusive 属性时要同时报告其有界深度，不能让最终故障检测率看起来与全部使用完整证明的配置相同。原文一处示例误拼成 -par_tack，其他定义和示例使用 -par_task；本教程使用后一拼写。

```
configure_fta_props -par_task FPV \
  -fault_class {SynchronousControlFlow ComboLogicControlFlow}
```

```
configure_fta_props -par_task FPV -status {proven}
```

来源：User Guide，X-2025.06-SP1，第 925–926 页。

6.3 compute_fta：执行形式测试环境质量分析

```
compute_fta -par_task <fpv_task> [-property <property_collection>]
```

参数解释。 `-par_task` 指明提供规格断言和约束的父任务。`-property` 可以显式选取用于检测故障的属性；省略时使用配置及默认选择规则，通常为 proven 属性。

用法与结果。先在全部故障被内部关闭的情况下重跑 FPV，确认注入后的设计模型仍满足原基线，再运行 FTA。故障未被检测并不自动证明故障没有影响：可能是断言漏写、约束屏蔽了激活条件、故障不可传播，或求解尚无结论。需结合分类和反例继续分析。

```
check_fv -block
configure_fta_props -par_task FPV -status {proven}
compute_fta -par_task FPV
# 在 FTA 分析完成后查看结果。
fta_report -instance {controller}
```

来源：User Guide，X-2025.06-SP1，第 925–927，941 页。

6.4 get_fta_faults：按实例与状态选取故障

```
get_fta_faults <instance_list> [-status <status_list>]
```

参数解释。位置参数限定设计实例；`-status non_detected` 可筛选未检测故障。返回工具集合，可交给 `report_fv`、`fvdisable`、`fvenable`、`fvwaive`。

用法与结果。是否区分同模块不同实例中的故障，受编译前的 `fml_fta_inst_qual` 设置影响。大量共用模块场景下，应先确认实例级故障是否独立，再比较每个实例的检测率。禁用或豁免集合只是在本次分析中排除目标，不等于已经检测。

```
set nd [get_fta_faults controller.u_fifo -status non_detected]
report_fv $nd -list
report_fv $nd -verbose
```

来源：User Guide，X-2025.06-SP1，第 927–930 页。

6.5 fta_report：输出实例级故障统计

```
fta_report -instance <instance_list>
fta_report <instance_name> -list
fta_report <instance_name> -verbose
```

参数解释。 `-instance` 指定一个或多个实例，`-list` 和 `-verbose` 分别用于逐项和详细报告。这里保留 UG 中分别出现的调用形式；不把示意性的 `-list/-verbose` 连写成一个可执行开关。

用法与结果。先看总量与状态分布，再看未检测故障对应的 RTL 和规格断言。报告应注明故障类、实例范围、被排除故障和参与检测的属性集合，否则不同运行的百分比不可直接比较。

```
fta_report -instance {controller.u_fifo controller.u_arbiter}
fta_report controller.u_fifo -verbose
```

来源：User Guide，X-2025.06-SP1，第 928–930 页。

6.6 read_fta_waiver：重新应用已审查的故障豁免

```
read_fta_waiver [-elfiles <exclusion_file_list>]
```

参数解释。 `-elfiles` 指定一个或多个通过 `save_waiver_file -fta` 生成的文件。故障键包含类别、原 RTL 构造及其他唯一属性，而不是只依赖易变化的显示序号。

用法与结果。设计修改导致原故障键不再匹配时，不应强行沿用旧豁免；应审查失配项。工作目录中的 COI.el、FC.el 可能被相关 FTA/高强度汇总流程自动使用，运行归档时要同时收集这些文件，避免隐藏的排除来源。

```
fvwaive [get_fta_faults controller.u_debug]
save_waiver_file -fta -file reviewed_faults.el
read_fta_waiver -elfiles {reviewed_faults.el}
```

来源：User Guide，X-2025.06-SP1，第 926–927 页。

6.7 `read_faultdb`：导入仿真测试环境故障数据库

```
read_faultdb -name <database_path>
```

参数解释。 `-name` 指向 Testbench Quality Assurance/Certitude 数据库，而不是普通 VCS 代码覆盖 `.vdb`。

用法与结果。必须在 `read_file` 或 `elaborate` 之前使用。已在仿真数据库中判为检测到的故障可以从形式分析目标中排除，使形式工具集中处理其余故障。合并统计之前仍需确认两个流程的设计版本、故障定义和配置一致。

```
set_fml_appmode FPV
read_faultdb -name simulation_faults
read_file -top controller -inject_fault RTL -sva \
  -format sverilog -vcs "-f formal.f"
```

来源：User Guide，X-2025.06-SP1，第 938–939 页。

6.8 `save_faultdb`：保存形式故障分析结果

```
save_faultdb [-name <database_path>]
```

参数解释。 `-name` 指定保存目录或数据库路径。结果保存到 Testbench Quality Assurance 数据库，供独立工具合并和查看。

用法与结果。这是 FTA 的数据库接口，不是 FuSa 的 `fusa_save`，也不是覆盖率的 `save_covdb`。三者的数据对象和后续消费工具不同，文件名相似不能互换命令。

```
save_faultdb -name formal_fault_results
```

来源：User Guide，X-2025.06-SP1，第 939 页。

6.9 `cluster_fta_faults`：把相似未检测故障分组

```
cluster_fta_faults
```

参数解释。 UG 在本节给出的调用不带参数；它对未检测故障进行相似性分组。

用法与结果。分组用于减少重复调试。修复组内一个代表故障暴露的检查缺失后，应重跑整组，而不是直接把其他成员改成“已检测”。相似只是分析提示，不是逻辑等价证明。

```
cluster_fta_faults
report_fta_fault_clusters
```

来源：User Guide，X-2025.06-SP1，第 955 页。

6.10 `report_fta_fault_clusters`：查看未检测故障分组

```
report_fta_fault_clusters
```

参数解释。输入是先前 `cluster_fta_faults` 计算的分组结果；本节未列额外参数。

用法与结果。优先选择影响可观察输出、关联关键功能且覆盖属性较少的分组做代表性调试。保留组成员清单，便于在断言补充后核实整组状态变化。

```
cluster_fta_faults
report_fta_fault_clusters
```

来源：User Guide，X-2025.06-SP1，第 955 页。

6.11 debug_fta：判断未检测故障能否传播到输出

```
debug_fta [-property <fault_property>]
debug_fta -dump_setup
debug_fta -run_user_setup
```

参数解释。`-property` 限定目标故障；不提供时使用流程默认目标。`-dump_setup` 导出 fta_ndf.tcl；修改该模板后用 `-run_user_setup` 运行用户设置。三个形式是不同阶段，不应把导出与运行开关无条件堆在同一调用中。

用法与结果。命令借助 SEQ 比较正常设计与带目标故障的设计，并自动打开带 Verdi 的独立会话，因此批处理 FTA 也可能需要图形显示与 SEQ 许可。它会调整工作目录中的 Certitude 配置文件；回到原 FPV-FTA 流程前应按原文要求恢复。初始化时不启用目标故障，故障在复位阶段的影响不属于这个默认调试问题。

```
debug_fta -dump_setup
# 审阅并修改生成的 fta_ndf.tcl 后，另行执行：
debug_fta -run_user_setup
```

来源：User Guide，X-2025.06-SP1，第 955–958 页。

7. 两个评估示例

例一：FIFO 缺少溢出断言。只写了 FIFO 空闲和复位属性，注入“满时仍接受写入”的 RTL 改变，可能不被检出。补充 `occupancy <= DEPTH`、接受写入时机等断言，重新比较检测结果。**例二：状态机错误状态转换**。只检查状态编码合法性，状态从 `WAIT` 错误跳到 `IDLE` 仍可能保持编码合法，却违反协议。FTA 可以促使增加状态转移规则与完成覆盖。

7.1 用完整源码观察断言是否能发现设计变化

下面采用计数器的正确版与错误版进行手工对照，帮助理解“改变设计后，原有属性是否敏感”。这一练习使用 FPV；它不是 FTA 自动生成故障或计算检出率的替代品。为便于单独阅读本章，相关文件再次全文列出。

7.1.1 被验证的计数器 RTL

规格：复位后计数为零；使能时从 0 计到 9 后回到 0；没有使能时保持。RTL 文件只包含设计，不把待验证结论写成环境假设。

保存为 案例工程/rtl/counter.sv：

```
module counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       enable,
    output logic [3:0] count
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)             count <= 4'd0;
    else if (enable) begin
      if (count == 4'd9)    count <= 4'd0;
      else                  count <= count + 4'd1;
    end
  end
```

```
endmodule
```

7.1.2 完整断言与绑定文件

history_valid 在复位后建立历史有效标志。ap_range 检查范围，ap_hold 检查保持，ap_increment 检查普通递增，ap_wrap 检查回绕；两条 cover 检查最高计数值及回绕场景是否能够出现。末尾 bind 将检查器接入同名设计信号，不需要修改 RTL。

保存为 案例工程/formal/counter_props.sv：

```
module counter_props (
    input logic       clk, rst_n, enable,
    input logic [3:0] count
);
  logic history_valid;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) history_valid <= 1'b0;
    else        history_valid <= 1'b1;

  ap_range: assert property (@(posedge clk) disable iff (!rst_n)
      count <= 4'd9);

  ap_hold: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(!enable) |-> count == $past(count));

  ap_increment: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count < 4'd9)
      |-> count == $past(count) + 4'd1);

  ap_wrap: assert property (@(posedge clk) disable iff (!rst_n)
      history_valid && $past(enable && count == 4'd9)
      |-> count == 4'd0);

  cp_reaches_nine: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9);
  cp_wrap: cover property (@(posedge clk) disable iff (!rst_n)
      count == 4'd9 ##1 count == 4'd0);
endmodule

bind counter counter_props u_counter_props (.*);
```

7.1.3 编译文件列表

先列 RTL，再列包含 bind 的属性文件；路径相对于运行 VC Formal 的 案例工程/ 目录，而不是 Tcl 文件所在目录。

保存为 案例工程/filelist_counter.f：

```
./rtl/counter.sv
./formal/counter_props.sv
```

7.1.4 完整运行脚本

依次选择应用、读入设计与 SVA、定义时钟复位、保存初始状态、检查设置、列出属性并开始证明。初始化步骤需要在实际安装环境核对复位状态，不能把声明复位等同于已经完成复位。

保存为 案例工程/scripts/fpv_counter.tcl：

```
# Run from the example project root with: vcf -f scripts/fpv_counter.tcl -batch
set_fml_appmode FPV
read_file -top counter -sva -format sverilog -vcs {-f filelist_counter.f}
```

```
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup -block
report_fv_setup -list
report_fv -list
set_fml_var fml_max_time 1H
set_fml_var fml_max_memory 8GB
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_counter.tcl -batch 2>&1 | tee logs/fpv_counter.log
```

结果阅读。在初始状态正确的前提下，正确版预期满足这些断言，并存在到达 9 和回绕的覆盖轨迹。这不是本次 VC Formal 的实际运行结果。这些断言分别覆盖范围、保持、普通递增和回绕；复位值及运行中重新复位仍需结合初始化模型和专用复位检查，不能仅凭范围属性成立就认定全部功能正确。

7.1.5 故意错误的 RTL

错误只在回绕阈值：把 9 改为 10。模块名仍为 `counter`，所以此文件与正确版必须分别编译。属性文件沿用本节已经完整列出的 `counter_props.sv`。

保存为 案例工程/rtl/counter_bad.sv：

```
// Intentionally buggy teaching variant: roll-over condition is 10 instead of 9.
// Do NOT include with rtl/counter.sv in the same compilation.
module counter (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       enable,
    output logic [3:0] count
);
  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)             count <= 4'd0;
    else if (enable) begin
      if (count == 4'd10)   count <= 4'd0;
      else                  count <= count + 4'd1;
    end
  end
endmodule
```

7.1.6 错误版编译文件列表

只读入错误设计及同一份检查器，不同时包含两个同名设计。

保存为 案例工程/filelist_counter_bad.f：

```
./rtl/counter_bad.sv
./formal/counter_props.sv
```

7.1.7 错误版完整运行脚本

与正确版采用相同接口和初始化方法，不为了让错误版通过而增加输入限制。

保存为 案例工程/scripts/fpv_counter_bad.tcl：

```
# Intentionally failing counter example. Do not include the good counter at once.
set_fml_appmode FPV
read_file -top counter -sva -format sverilog -vcs {-f filelist_counter_bad.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup -block
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_counter_bad.tcl -batch 2>&1 | tee logs/fpv_counter_bad.log
```

错误版的预期反例是：在允许连续使能的输入下，计数从 9 进入 10，违反范围与回绕规则。查看反例时应区分时钟沿的采样值与非阻塞赋值后的寄存器值；不能把这里描述的预期现象伪装成工具日志。

8. FTA 与故障仿真的区别

FTA 使用形式分析寻找能够触发和传播注入差异的逻辑证据；故障仿真通常依赖已准备的激励序列。两者都可用来研究测试平台质量，但对状态空间、收敛条件、故障模型和资源需求的依赖不同。FTA 的故障并不等同于制造测试的 stuck-at / transition ATPG 故障模型。

9. 参考

User Guide 第 21 章，尤其多阶段运行、故障注入、结果调试与性能部分；User Guide 第 20 章的最终验证质量评估方法。

FuSa：功能安全形式分析使用指南

1. 场景与关键概念

面向汽车、工业控制或其他要求功能安全的芯片，故障不仅要判断“是否存在”，还需分析在规定运行状态下可能带来什么影响，是否可以通过硬件检测机制观察到。FuSa 与故障数据库和 Z01X 等工具协同，可按故障可控性、可观察性、检测能力分类，为 FMEDA（失效模式、影响及诊断分析）提供支持。

可控性：是否可能在允许的输入/状态下触发相关故障影响；**可观察性**：故障差异是否可能传播到指定观察点；**可检测性**：预定诊断机制是否能检测到故障。三者相关但不同。不能把“故障可观察”直接写成“已经被安全机制检测”。

2. 前置要求和故障输入

据 UG 第 1015 ~ 1016 页，准备 FuSa 前应先具备无关键设置问题的 FPV 环境；故障定义可以来自 Standard Fault Format（SFF）或统一故障数据库（FDB），还需提供观察点、检测点及有关常量前提。工具版本和故障数据库版本应保持兼容。

典型资料包括：RTL、参数配置、故障清单、正确的时钟复位与初始化、合法运行模式的定义、观察/检测节点列表。若观察点设错，即使故障在真实系统中危险也可能被错误分类。

3. 四类分析

分析类别	目标	可能的误解
结构分析（Structural）	初步查看故障与观察/检测点的结构关系	有结构路径不代表一定可传播
可控性分析（Control）	检查是否能激活相应故障效应	不可控可能受环境约束影响
可观察性分析（Observe）	研究故障效应是否到达观察端	到达观察端不代表诊断会报警
可检测性分析（Detection）	判断指定检测机制是否响应	取决于指定检测点和时序规则

四类分析形成逐步细化的证据：结构关系只说明存在可能的传播路径；可控性研究故障是否能被激活；可观测性研究故障是否能影响指定功能输出；检测分析研究安全机制是否能在要求的条件和时间内报警。一个故障不能被观察到，与一个故障已经被安全机制检测到，是完全不同的结论。

UG 23.1.1 给出结构、可控性、可观察性及检测分析。不同模式的输出状态不应混合汇总，尤其不能把“结构排除”当成已经对完整工作状态进行了所有逻辑分析。

4. 标准脚本结构

```
set_fml_appmode FUSA
# 先配置故障清单，后读入设计
fusa_config -sff faults.sff
# 或者结合 FDB 设置，具体数据库名与路径按项目确定
# fusa_config -fdb campaign -fdb_path /project/faultdb
read_file -top safety_top -format sverilog -vcs {-f filelist.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
```

```
fusa_observation -add safety_output
fusa_detection -add alarm
fusa_generate
set_fml_var fusa_run_mode structural
check_fv -block
fusa_report
```

在这个教学骨架中，`faults.sff`、`safety_output`、`alarm` 是必须由实际故障清单和 RTL 替换的对象。正式项目应先在 FPV 环境完成 `check_fv_setup -block` 等准备核查，并了解 UG 关于正式 FuSa 重复执行这些检查的性能提示。

4.1 原生 FuSa 模板完整源码

真实 SFF/FDB 文件和系统安全故障定义属于必要输入，不能用下面的教学异或掩码代替这些输入来声称完成 FuSa。模板默认停止执行。

保存为 案例工程/templates/fusa_safety.tcl：

```
# FuSa：功能安全分析，需要 SFF/FDB、真实的安全检测点和失效定义。
error " 此文件是 FuSa 教学模板：请先提供 SFF/FDB、观察点、检测点、RTL 和安全场景"
# set_fml_appmode FUSA
# fusa_config -sff /project/faults/campaign.sff
# read_file -top safety_top -format sverilog -vcs {-f filelist.f}
# create_clock clk -period 100
# create_reset rst_n -sense low
# sim_run -stable
# sim_save_reset
# fusa_observation -add {safety_top.data_out safety_top.done}
# fusa_detection -add {safety_top.error_alarm}
# fusa_generate
# set_fml_var fusa_run_mode structural
# check_fv -block
# fusa_report
```

5. 逐步提高分析精度

```
set_fml_var fusa_run_mode structural
check_fv -block
set_fml_var fusa_run_mode control
check_fv -block
set_fml_var fusa_run_mode observe
check_fv -block
```

执行不同分析阶段时，需要使用每阶段生成的故障类别并保存结果。某些版本或项目中还需专用的检测配置，不能仅凭这三条命令推断完整 FMEDA 已经生成。阅读该安装版本的 `fusa_config -help`、`fusa_generate -help`、`fusa_report -help`。

6. 命令语法、参数与用例

命令	主要作用
`fusa_config`	读取安全故障清单与数据库
`fusa_clear`	清理本轮故障与生成属性
`fusa_map_status`	指定内部结果的外部状态编码
`fusa_blackbox`	为安全分析建立可传播的抽象模块
`fusa_observation`	设置功能观察点

命令	主要作用
fusa_detection	设置安全检测点
fusa_generate	生成安全故障及分析属性
fusa_get_ignored_signals	查看未参与故障生成的信号
fusa_report	检查安全故障、分析模式与传播抽象
fusa_save	导出安全故障分析结果

本组命令按准备、生成、执行与结果处理的关系组织。语法框保留参数名称和可选关系；示例中的实例、属性和文件名称需要与当前工程一致。生成目标与证明目标是不同阶段，调用生成命令后应检查实际目标数量，再启动证明。

6.1 fusa_config：读取安全故障清单与数据库

```
fusa_config [-sff <files>] [-fdb <database_name>]
  [-fdb_path <local_project_path>] [-fdb_server <server:port>]
  [-fdb_campaign <campaign>] [-status <statuses>]
  [-dut_path <dut_path>] [-save_fdb_incr <interval>]
```

参数解释。至少提供 -sff 或 -fdb 所代表的故障输入。-sff 接受 SFF 文件列表；-fdb 指定统一故障数据库。-fdb_path 是本地数据库路径，不能同时与 -sff、-fdb_server 使用；-fdb_server 选择服务端，-fdb_campaign 选择故障分析活动。-status 选择待分析状态或状态组，状态组在 fusa_generate 时展开。-dut_path 对齐故障文件与当前设计的顶层路径。

-save_fdb_incr 控制运行期间增量保存，单位 M 或 H，默认 1H，非零时最小 10M，设为 0 关闭。SFF 流程重配前需要 fusa_clear，并且 SFF 输入应在读设计前完成。

用法与结果。统一安全流程涉及的工具版本需要匹配。UG 的“本地 FDB”示意代码有重复远程选项的排版问题，因此本教程以 -fdb_path 的明确定义区分本地和服务端用法。数据库可连接不代表其故障对象都能在当前设计中找到，随后必须检查生成与忽略报告。

```
set_fml_appmode FUSA
fusa_config -sff {faults.sff}
# FDB 工程可改用下面这一形式，而不是与上面的 SFF 调用连续混用。
# fusa_config -fdb safety_db -fdb_path /project/fdb \
#   -fdb_campaign main_campaign -dut_path controller
```

来源：User Guide，X-2025.06-SP1，第 1016–1019 页。

6.2 fusa_clear：清理本轮故障与生成属性

```
fusa_clear
```

参数解释。不带参数；清除已读故障和相关属性，使后续允许重新配置。

用法与结果。 FDB 流程可以在保留设计的情况下切换故障配置，但必须先 clear。SFF 文件更换受读设计时机限制：读设计后再提供新的 SFF 会报错，旧式 SFF 流程在读设计后连 clear 也不允许。它不是普遍适用的“任意重启本应用”命令。

```
fusa_config -sff first.sff
fusa_clear
fusa_config -sff second.sff
# 然后才读取并展开设计。
```

来源：User Guide，X-2025.06-SP1，第 1024 页。

6.3 fusa_map_status：指定内部结果的外部状态编码

```
fusa_map_status [-sff_status <status_code>] [<internal_status>]
```

参数解释。位置参数是内部状态，例如 FNR、SIN、SSF、SNO、SND、CNC、CCC、CIN、ONO、OOB、OIN、DND、DDT、DIN、NOD、NON、ODT、OND；`-sff_status` 指定输出编码。

用法与结果。状态编码有版本差异，不能仅凭两三个字母判断分析证据。内部“不可观察”“可检测”“无结论”是不同维度；自定义外部编码后仍要保留原内部状态及分析模式。以下示例显式重申原文列出的 FNR→NA 关系，不擅自改写安全分类。

```
fusa_map_status -sff_status NA FNR
```

来源：User Guide，X-2025.06-SP1，第 1019–1021 页。

6.4 `fusa_blackbox`：为安全分析建立可传播的抽象模块

```
fusa_blackbox [-module <modules>] [-all]
  [-all_auto_path] [-no_auto_path] [-seq_path] [-comb_path]
  [-ignore_unresolved] [-data_path_only] [-remove]
  [-task <failure_mode_task>] [<instances>]
```

参数解释。 `-module` 或实例列表限定对象；`-all` 作用于全部结构黑盒。`-all_auto_path` 默认自动创建输入到输出路径，`-no_auto_path` 不自动创建；`-seq_path` 和 `-comb_path` 分别采用时序和组合传播。`-ignore_unresolved` 跳过未解析模块；`-data_path_only` 不沿多路选择器的选择控制传播，但允许数据分支传播。`-remove` 撤销安全抽象。`-task` 仅在多个 failure mode 时指定所属任务。

用法与结果。在设计展开后、`fusa_generate` 前使用。普通结构黑盒会断开传播路径，可能错误证明故障不可观察；安全抽象保持保守传播，但也可能添加实际上不成立的路径，形成需要精化的失败。生成前的报告不一定列出最终安全黑盒。

```
fusa_blackbox -all -seq_path
fusa_generate
fusa_report -blackbox -path
```

来源：User Guide，X-2025.06-SP1，第 1021–1022 页。

6.5 `fusa_observation`：设置功能观察点

```
fusa_observation [-add] [-delete] [-file <filename>] [<signals>]
```

参数解释。 `-add` 增加，`-delete` 删除；信号可直接给出为 Tcl 列表，也可由 `-file` 读取。对象应为输出数据、事务响应或其他用于判断功能影响的信号。

用法与结果。观察点与检测点应分别从系统规格确定，不应为了提高检测率把所有功能输出都当作告警。FDB/SFF 中已有的点可能自动导入，Tcl 用于补充或修订；生成后用报告核实最终集合。

```
fusa_observation -add {controller.data_out controller.done}
fusa_report -observation
```

来源：User Guide，X-2025.06-SP1，第 1022–1023 页。

6.6 `fusa_detection`：设置安全检测点

```
fusa_detection [-add] [-delete] [-file <filename>] [<signals>]
```

参数解释。 `-add` 增加，`-delete` 删除；信号可直接给出为 Tcl 列表，也可由 `-file` 读取。对象应为错误标志、告警或其他安全机制用于报告故障的信号。

用法与结果。观察点与检测点应分别从系统规格确定，不应为了提高检测率把所有功能输出都当作告警。FDB/SFF 中已有的点可能自动导入，Tcl 用于补充或修订；生成后用报告核实最终集合。

```
fusa_detection -add {controller.error_alarm}
fusa_report -detection
```

来源：User Guide，X-2025.06-SP1，第 1022–1023 页。

6.7 fusa_generate：生成安全故障及分析属性

```
fusa_generate [-scopes <scopes>] [-excludes <scopes>] [-exact]
  [-fault_type <st0|st1|both|transient|transient_0|transient_1|all>]
  [-analysis <analysis_list>] [-verbose] [-types <node_types>]
```

参数解释。按已导入的 SFF/FDB 清单运行时，使用无参数形式。主动生成或筛选对象时，`-scopes` 指定范围，`-excludes` 排除子范围，`-exact` 避免部分名称匹配。`-fault_type` 选择固定故障或瞬态故障；`-analysis` 选择 structural、control、observe、detect；`-types` 选择 WIRE、PORT、VARI、ARRY；`-verbose` 展开生成信息。

用法与结果。生成属性之后仍需设置运行模式并执行 `check_fv`。该版本对瞬态故障的支持范围不能由允许的枚举值推断：UG 的支持矩阵只列其结构分析支持。时钟和复位故障也有专门的结构分析限制，不能把全部注入对象纳入同一个动态检测率分母。

```
fusa_generate
set_fml_var fusa_run_mode structural
check_fv -block
fusa_report -fault_category
fusa_get_ignored_signals
```

来源：User Guide，X-2025.06-SP1，第 1017–1018，1023–1027 页。

6.8 fusa_get_ignored_signals：查看未参与故障生成的信号

```
fusa_get_ignored_signals [-clock] [-reset] [-skipped]
```

参数解释。 `-clock` 只显示被忽略的时钟，`-reset` 只显示复位，`-skipped` 显示其他被跳过的信号；不加筛选时查看整体。

用法与结果。该报告是故障范围审查的一部分。信号出现在 RTL 并不表示其故障参与了全部分析模式，尤其不要把时钟、复位的结构分析支持误说成可控、可观察和检测分析均支持。

```
fusa_get_ignored_signals -clock
fusa_get_ignored_signals -reset
fusa_get_ignored_signals -skipped
```

来源：User Guide，X-2025.06-SP1，第 1025 页。

6.9 fusa_report：检查安全故障、分析模式与传播抽象

```
fusa_report [-list] [-verbose] [-no_summary]
  [-observation] [-detection] [-fault_id <id>]
  [-blackbox] [-fdb_report] [-file <filename>]
  [-path] [-path_config] [-col] [-with_top]
  [-instance <instances>] [-fault_category] [-fdb_version]
```

参数解释。`-list` 每个故障一行，`-verbose` 输出详细记录，`-no_summary` 去掉汇总；`-fault_id` 定位故障，`-instance` 按实例筛选，`-with_top` 保留顶层前缀。`-observation`、`-detection` 输出两类点。`-blackbox` 查看安全黑盒，`-path` 查看传播路径，`-path_config` 查看用户路径配置，`-col` 返回对应实例集合。`-fdb_report` 输出数据库报告，`-file` 指定该报告的 SFF 文件；`-fault_category` 展示分类，`-fdb_version` 查看数据库版本。

用法与结果。报告中故障状态与分析模式都应保留。例如结构上无通路、动态不可激活和未求得结论不能合并成一个“通过”。需要复核检测能力时同时查看检测点是否正确导入，而不只读取 observed 数量。

```
fusa_report -list
fusa_report -observation
fusa_report -detection
fusa_report -blackbox -path
fusa_report -fault_category
```

来源：User Guide，X-2025.06-SP1，第 1027–1028 页。

6.10 fusa_save：导出安全故障分析结果

```
fusa_save [-sff <filename>] [-fault_db <database_path>]
  [-test <test_name>] [-status <statuses>] [-all_failure_modes]
```

参数解释。 `-sff` 保存 SFF；`-fault_db` 指定数据库；`-test` 指定结果所属测试名；`-status` 只保存所选状态；`-all_failure_modes` 保存全部 failure mode 的当前结果。已有本地 FDB 配置时可无参数保存到当前数据库。

用法与结果。输出格式应与输入流程一致：SFF 输入导出 SFF，FDB 输入保存 FDB，不把该命令当作通用格式转换器。它保存当前证据，不补做未完成的分析；汇总报告应明确未知状态仍有哪些。

```
fusa_save -sff formal_safety_result.sff
# FDB 工程使用 fusa_save 或明确的 -fault_db/-test 配置。
```

来源：User Guide，X-2025.06-SP1，第 1029 页。

7. 案例：带奇偶校验的安全寄存器

一个重要配置寄存器使用奇偶校验位，写入时更新校验；读取/使用时，如果校验与数据不一致则拉高 `alarm`。可以指定寄存器数据寄存器及校验位中的故障为分析对象，把 `alarm` 作为检测点，关键输出作为观察点。分析时区分：故障是否真正改变了数据；改变是否到达输出；奇偶校验是否在规定时延内报警。即使一个故障未影响输出，也不能把其他故障的检测结果套用在它身上。

7.1 完整代码：观察值、报警与漏检场景

为把“输出受影响”与“报警检测到”分开，以下只在读取路径增加教学用异或掩码。数据寄存器和校验寄存器正常更新；`fault_mask` 改变读出的数据位，`fault_parity` 改变读出校验位。这个模型既不是晶体管故障模型，也不是故障数据库文件。

7.1.1 完整 RTL

保存为 案例工程/rtl/safety_reg_demo.sv：

```
// 教学故障模型：用异或掩码改变读取路径的数据/校验位。
// 不是 SFF/FDB 注入器，也不代表真实硬件故障的全部类型。
module safety_reg_demo (
  input  logic       clk, rst_n, write_en,
  input  logic [7:0] write_data,
  input  logic [7:0] fault_mask,
  input  logic       fault_parity,
  output logic [7:0] safety_output,
  output logic       alarm
);
  logic [7:0] stored_data;
  logic       stored_parity;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      stored_data   <= 8'd0;
      stored_parity <= 1'b0;
    end else if (write_en) begin
      stored_data   <= write_data;
      stored_parity <= ^write_data;
    end
  end
```

```
  assign safety_output = stored_data ^ fault_mask;
  assign alarm = (^safety_output) ^ stored_parity ^ fault_parity;
endmodule
```

7.1.2 完整 SVA

除了单比特错误报警，还覆盖两位数据同时翻转但不报警的情况。这有助于识别奇偶校验的能力限制，不能把单个场景的结果推广为全部故障检出率。

保存为 案例工程/formal/safety_reg_demo_props.sv：

```
module safety_reg_demo_props (
  input logic clk, rst_n,
  input logic [7:0] fault_mask,
  input logic fault_parity, alarm
);
  ap_no_fault_no_alarm: assert property (@(posedge clk) disable iff (!rst_n)
    fault_mask == 8'd0 && !fault_parity |-> !alarm);
  ap_single_bit_detected: assert property (@(posedge clk) disable iff (!rst_n)
    $onehot(fault_mask) && !fault_parity |-> alarm);
  ap_parity_bit_detected: assert property (@(posedge clk) disable iff (!rst_n)
    fault_mask == 8'd0 && fault_parity |-> alarm);
  // 两位数据同时翻转不改变奇偶性，用覆盖目标寻找漏检场景。
  cp_two_bit_undetected: cover property (@(posedge clk) disable iff (!rst_n)
    fault_mask == 8'h03 && !fault_parity && !alarm);
endmodule

bind safety_reg_demo safety_reg_demo_props u_safety_props (
  .clk(clk), .rst_n(rst_n), .fault_mask(fault_mask),
  .fault_parity(fault_parity), .alarm(alarm)
);
```

7.1.3 文件列表

保存为 案例工程/filelist_fpv_safety_demo.f：

```
./rtl/safety_reg_demo.sv
./formal/safety_reg_demo_props.sv
```

7.1.4 配套 FPV 脚本

保存为 案例工程/scripts/fpv_safety_demo.tcl：

```
# 从案例工程目录执行；结果必须以本机 VC Formal 日志为准。
set_fml_appmode FPV
read_file -top safety_reg_demo -sva -format sverilog -vcs {-f
 ↪  filelist_fpv_safety_demo.f}
create_clock clk -period 100
create_reset rst_n -sense low
# 先核验实际复位状态，再保存初始状态。
sim_run -stable
sim_save_reset
check_fv_setup -block
report_fv -list
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
mkdir -p logs
```

```
set -o pipefail
vcf -f scripts/fpv_safety_demo.tcl -batch 2>&1 | tee logs/fpv_safety_demo.log
```

在该教学模型中，单个数据位或单独校验位改变预期会报警；两位数据同时翻转的覆盖目标预期可达。只有将真实故障清单、观察点、检测点和系统失效定义接入原生 FuSa 流程后，才能解释该流程的实际故障分类。

8. 与 ISO 26262 / FMEDA 的关系

FuSa 分析结果可作为硬件随机故障指标评估的一部分，但不能仅凭工具中的一个百分比就认定达到某 ASIL 等级。功能安全还涉及故障模型、故障率来源、诊断间隔、残余风险、独立性、系统级失效影响和安全机制完整性。所有故障分类以及能够被排除的对象应由功能安全负责人审查。

9. 常见错误

- 故障数据与 RTL 层级不一致，使部分故障没有真正进入分析。
- 未先校验时钟复位及模式常量，造成大量虚假的不可控。
- 观察点或检测点过少，使报告过于乐观或不完整。
- 将可观察与已检测、未完成与不可达混同。
- 把结构分析结果视作所有故障的最终证明。

10. 官方资料

User Guide §23（1009 ~ 1039 页）、§4（初始化）、§6（结果控制）；如涉及数据库交互，应同时参照 Z01X / FDB 的相同软件版本文档。

FSV：形式安全信息流验证使用指南

1. 目标：检查本不应出现的信息传播

数字系统可能包含安全域与普通域。安全域中的密钥、认证状态、受保护计数器等信息不能在未授权条件下影响普通域输出；普通域输入也不能未经允许改变关键安全状态。FSV 用形式化的来源、目的及传播条件描述这些安全信息流要求。

与一般 CC 不同，FSV 不只是检查“需要的连接是否存在”，还要检查**禁止的信息传播是否可能发生**。示例：安全密钥不允许直接或间接影响公开的调试端口；普通外设不能未经权限检查更新可信启动状态。可见信息流分析并不自动涵盖时间侧信道、功耗侧信道以及模拟泄漏。



图 12: FSV 安全域隔离

2. 什么是信息流与非干扰

假设存在高安全级输入 `secret` 和低安全级输出 `public_out`。一种非干扰（noninterference）性质要求：当普通级输入和合法初态相同时，改变 `secret` 不得改变 `public_out`，除非系统允许某种已定义的解密或公开处理。它是一种关于两种输入情况的关系，而不只是单条波形上某个值等于零。

FSV 通过源和目的定义生成信息流属性，并在形式环境中检查合法传播范围。对于含授权条件、故障检测条件或某些模式允许公开的数据，必须建模对应条件，不能只无条件禁止全部传播。

3. 典型命令结构

```
set_fml_appmode FSV
read_file -top security_top -format sverilog -vcs {-f filelist.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup -block
fsv_generate -src secret_key -dest debug_out
check_fv -block
fsv_report
report_fv -list
```

脚本中信号名和复位过程是教学占位，真正的形式模型必须包含完整的普通输入、授权模式及初态。`fsv_generate` 的实际语法支持一个或多个源、一个或多个目的，以及 `-src_condition`、`-dest_condition`、`-exclude`、`-name`、`-zone` 等选项（见 User Guide 第 966 页）。

3.1 面向真实系统的完整模板

此模板用于替换成真实安全域层级；第 5 节另有完整的小型设计与运行脚本。这里保留保护性 `error`，不会把尚未审查的系统安全策略自动启用。

保存为 案例工程/templates/fsv_secure_flow.tcl：

```
# FSV：信息流安全。请先完成 source、destination 与授权策略审查。
error " 此文件是 FSV 教学模板：必须指定真正的安全源与公开输出"
# set_fml_appmode FSV
# read_file -top secure_top -format sverilog -vcs {-f filelist.f}
# create_clock clk -period 100
# create_reset rst_n -sense low
# sim_run -stable
# sim_save_reset
# fsv_generate -src u_core.secret -dest u_dbg.out
# check_fv -block
# fsv_report
```

4. 可控的允许传播

有些设计需要在合法授权后输出秘密信息的某种变换结果，例如密钥派生操作的授权输出。此时不能简单禁止所有相关数据依赖，而应制定允许传播的安全策略。FSV 的 `-src_condition`、`-dest_condition` 等选项可帮助表达仅对特定工作条件分析的信息传播。任何白名单范围都需要安全审查，避免授权信号可由不可信输入随意置位。

5. 案例：保护调试输出的完整工程

策略是：无论调试是否开启，公开输出都不得依赖 `secret_key`。允许调试输出普通状态，但不允许输出秘密数据。本节不用“调试永久关闭”这样的假设掩盖泄漏。

5.1 正确版本 RTL

保存为 案例工程/rtl/debug_guard.sv：

```
// 安全策略：公开调试输出永远不得依赖 secret_key。
module debug_guard (
  input  logic       clk, rst_n, debug_enable,
  input  logic [7:0] secret_key, normal_status,
  output logic [7:0] debug_out
);
  always_comb begin
    if (debug_enable) debug_out = normal_status;
    else              debug_out = 8'h00;
  end
endmodule
```

5.2 故意泄漏的完整 RTL

两份文件模块名相同，必须分别读入。坏版只把普通状态替换为秘密数据，让读者能看清被检查的差异。

保存为 案例工程/rtl/debug_guard_bad.sv：

```
// 故意泄漏版本；模块名与正确版相同，必须分开编译。
module debug_guard (
  input  logic       clk, rst_n, debug_enable,
  input  logic [7:0] secret_key, normal_status,
```

```
  output logic [7:0] debug_out
);
  always_comb begin
    if (debug_enable) debug_out = secret_key;
    else              debug_out = 8'h00;
  end
endmodule
```

5.3 正确版文件列表

保存为 案例工程/filelist_fsv_good.f：

```
./rtl/debug_guard.sv
```

5.4 正确版完整 FSV 脚本

本例有明确的源、目的与实际 RTL，不再只使用 `security_top/filelist.f` 占位名称。仍须核对本机 `fsv_generate` 的接口及对象解析结果。

保存为 案例工程/scripts/fsv_debug_good.tcl：

```
# 读取一个版本；先检查源/目的是否成功解析，不能把零属性当作通过。
set_fml_appmode FSV
read_file -top debug_guard -format sverilog -vcs {-f filelist_fsv_good.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup -block
fsv_generate -src secret_key -dest debug_out
report_fv -list
check_fv -block
fsv_report
report_fv -verbose
```

5.5 错误版文件列表

保存为 案例工程/filelist_fsv_bad.f：

```
./rtl/debug_guard_bad.sv
```

5.6 错误版完整 FSV 脚本

保存为 案例工程/scripts/fsv_debug_bad.tcl：

```
# 读取一个版本；先检查源/目的是否成功解析，不能把零属性当作通过。
set_fml_appmode FSV
read_file -top debug_guard -format sverilog -vcs {-f filelist_fsv_bad.f}
create_clock clk -period 100
create_reset rst_n -sense low
sim_run -stable
sim_save_reset
check_fv_setup -block
fsv_generate -src secret_key -dest debug_out
report_fv -list
check_fv -block
fsv_report
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
```

```
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fsv_debug_good.tcl -batch 2>&1 | tee logs/fsv_debug_good.log
vcf -f scripts/fsv_debug_bad.tcl -batch 2>&1 | tee logs/fsv_debug_bad.log
```

结果核查。正确版源端不参与输出计算，错误版在调试开启时有明确依赖。工具可能对未使用的源做优化，必须先确认属性生成、源目的解析及报告分类；零属性或对象找不到不能当作通过。示例结果仍待本机 VC Formal 执行确认。

6. 信息流安全与普通功能断言相互补充

FPV 可证明 `debug_enable==0` 时 `debug_out==0`，但这不一定足以证明“只要安全源变化，任何普通输出都不发生可识别的变化”。尤其如果信息通过状态机控制、地址、错误响应或其他共享资源间接传播，普通值检查可能漏掉依赖关系。FSV 帮助发现这些关联，但它也需要明确的观察范围和安全模型。

7. 黑盒与抽象的风险

`fsv_blackbox <instance>` 用于安全验证中指定特殊黑盒对象；它的抽象行为会影响信息流路径和可证明性。把加密模块全部黑盒化后，若未正确描述安全输出与普通输出的可见关系，结果可能既不保守也不完整。建议先建立最小原型，比较带/不带黑盒的属性数量与失效情况，再决定大型 SoC 的建模策略。

8. 命令语法、参数与用例

命令	主要作用
`fsv_generate`	定义禁止的信息传播
`fsv_blackbox`	建立保持传播的安全抽象
`fsv_report`	审查安全抽象及传播路径

本组命令按准备、生成、执行与结果处理的关系组织。语法框保留参数名称和可选关系；示例中的实例、属性和文件名称需要与当前工程一致。生成目标与证明目标是不同阶段，调用生成命令后应检查实际目标数量，再启动证明。

8.1 `fsv_generate`：定义禁止的信息传播

```
fsv_generate -src <signals> [-src_condition <expression>]
  -dest <signals> [-dest_condition <expression>]
  [-exclude <signals_or_instances>] [-name <prefix>]
  [-zone <zone_name>] [-condition_cover]
```

参数解释。 `-src`、`-dest` 必填，可为输入输出、寄存器、网络或实例；每个目的对象生成一个目标。`-src_condition` 只要求信息开始传播的那一拍成立；`-dest_condition` 只要求观察传播的那一拍成立，它们不要求同时或一直成立。`-exclude` 排除本条检查中的合法传播路径；`-name` 给属性和约束加前缀。`-zone` 将检查分组，同组源标记共同作用于该组断言，但单条源/目的条件不会自动共享。`-condition_cover` 增加条件覆盖检查，便于识别不可达条件。

用法与结果。该命令证明的是给定模型中不存在指定传播，不是加密强度、所有侧信道或完整系统安全。把源条件换成始终成立的普通假设会改变问题，可能造成错误证明。时钟和复位不能作为普通标记源；被 create_clock/create_reset 声明的对象会被自动忽略并给出警告。

```
fsv_generate -src {key_reg} -dest {debug_data} \
  -name key_to_debug -condition_cover
fsv_generate -src {secret_data} -src_condition {secure_mode} \
  -dest {public_data} -dest_condition {public_valid} \
  -exclude {u_encrypt} -name bypass_encrypt
```

来源：User Guide，X-2025.06-SP1，第 966–973 页。

8.2 `fsv_blackbox`：建立保持传播的安全抽象

```
fsv_blackbox [-all] [-all_auto_path] [-no_auto_path]
  [-add_path] [-remove_path] [-src <input_ports>] [-dest <output_ports>]
  [-add_input_assertion] [-seq_path] [-comb_path]
  [-ignore_unresolved] [-data_path_only] [-remove] [<instance>]
```

参数解释。`-all` 处理全部结构黑盒；默认 `-all_auto_path` 自动建路径；`-no_auto_path` 创建无自动路径模型。`-add_path` 与 `-remove_path` 结合 `-src`、`-dest` 增减指定端口间传播；`-add_input_assertion` 检查安全信息能否到达黑盒输入。`-seq_path` 引入一拍传播，帮助避免组合环；`-comb_path` 使用组合传播。`-ignore_unresolved` 跳过未解析模块，`-data_path_only` 限制控制选择信号的传播，`-remove` 撤销安全抽象。

用法与结果。直接对有 RTL 的模块做安全抽象时，自动路径可来自 RTL 的结构关系；先用结构黑盒删除内部逻辑后再加安全抽象，只能保守连接所有输入与输出，可能制造更多失败。反过来，普通结构黑盒切断传播可能制造错误通过。人工删除路径必须有设计或规格依据。

```
fsv_blackbox -seq_path -all_auto_path u_cipher_ctrl
fsv_report -blackbox -path
# 只有确认 ack_i 到 req_o 不存在功能传播时才移除该路径。
fsv_blackbox -remove_path -src ack_i -dest req_o u_cipher_ctrl
```

来源：User Guide，X-2025.06-SP1，第 978–982 页。

8.3 `fsv_report`：审查安全抽象及传播路径

```
fsv_report -blackbox [-list] [-path_config] [-path] [-col]
```

参数解释。`-blackbox` 选择安全抽象报告，`-list` 列出对象，`-path_config` 显示用户配置，`-path` 展开当前输入到输出路径，`-col` 返回实例集合。这里只列 UG 对安全黑盒报告明确说明的形式。

用法与结果。用户配置不等于最终路径集合；自动建路径和人工增删后，应查看 `-path`。路径缺失会导致假通过，过多保守路径会导致假失败，二者处理方向相反。

```
fsv_report -blackbox -list
fsv_report -blackbox -path_config
fsv_report -blackbox -path
```

来源：User Guide，X-2025.06-SP1，第 981–982 页。

9. 常见错误及处理

- 忽略普通域的间接数据依赖，只检查显式赋值。
- 未考虑复位或切换安全模式期间的合法输入，导致假阳性。
- 把未知授权条件设置为恒真，从而屏蔽真实泄漏。
- 把“没有找到信息传播反例”当成“已经没有所有侧信道风险”。
- 将安全策略与实现混在同一套强制约束中，无法审查真正的安全责任。

10. 官方参考

User Guide §22（961~1008 页），尤其 §22.2 的源目的定义和 `fsv_generate`；Quick Reference Guide FSV 部分。

DPV：数据路径验证与 C/C++—RTL 事务等价使用指南

1. 解决的问题与 SEQ 的区别

高性能加法器、乘法器、DSP、编码器和浮点运算单元通常先建立不含硬件周期细节的 C/C++ 算法参考模型，后将其实现为多级流水 RTL。DPV 使用 HECTOR 技术对照高层事务模型和硬件实现的结果，目标是证明相同事务输入产生正确的输出结果，即便两边的执行阶段不同。

SEQ 一般比较两个顺序设计及其相关状态；**DPV** 强调事务级数据变换的对照，例如参考模型在一个函数中完成计算，RTL 则经过 4 个周期才输出。它并不把两个模型天真地逐拍对齐，而是需要声明输入与输出的**事务阶段关系**。

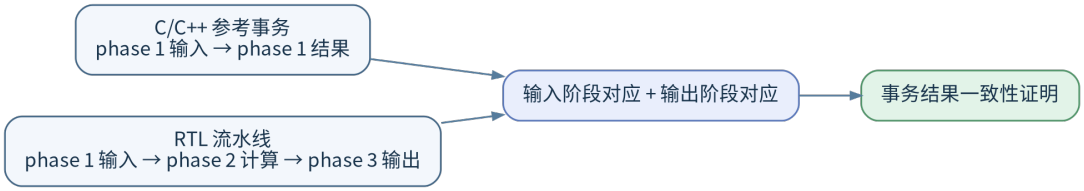


图 13: DPV 的事务阶段对照

2. 典型应用示例：8 位加法器

参考 C++ 函数计算 `uint8_t y = (uint8_t)(a+b)`；RTL 可能有两拍流水线：第 1 拍接收 `a,b`，第 2 拍计算，第 3 拍向输出寄存器提交。若直接比较参考函数返回值与 RTL 每拍 `out`，即使算法正确也可能失败，因为没有处理有效标志和延迟。

需要定义：哪个事件接受一个输入事务；何时读取参考模型参数；RTL 在哪个 phase 输出结果；是否允许同时在流水线中有多个事务；操作是模 256 加法还是需要保留进位位。输入位宽、C++ 整数提升和有符号规则都必须统一。

3. C++ 事务接口的结构

User Guide 第 1067 页示例提到 `Hector::registerInput`、`Hector::registerOutput` 以及 `Hector::beginCapture`、`Hector::endCapture` 用于定义事务输入输出及捕获范围。实际完整 C++ 包装代码需要适配安装提供的 HECTOR 头文件与编译前端；**本教程不会凭空编写一个未经验证的完整 HECTOR API 程序并声称可执行**。下列普通 C++ 函数表达数学规格，可以作为普通 C++ 代码编译，但它不是 HECTOR wrapper：

```
#include <cstdint>
uint8_t add_mod256(uint8_t a, uint8_t b) {
    return static_cast<uint8_t>(a + b);
}
```

编写正式 DPV C++ spec 时还应处理整型符号、移位、未定义行为、数组访问和宿主编译器的优化规则，否则会出现“高层规范自身就不精确”的问题。

3.1 完整 C++ 源文件与穷举自测

下面两个文件可以用普通 C++ 编译器构建，用于先确认参考函数的模 256 运算。它们不包含、也不伪装成 HECTOR 的接口包装；运行成功不等同于 DPV 已经证明 C++ 与 RTL 等价。

3.1.1 参考函数完整源码

保存为 案例工程/cpp/add_mod256.cc：

```
#include <cstdint>

// 普通 C++ 参考函数；此文件没有调用 HECTOR 接口。
std::uint8_t add_mod256(std::uint8_t a, std::uint8_t b) {
    const unsigned sum = static_cast<unsigned>(a)
                       + static_cast<unsigned>(b);
    return static_cast<std::uint8_t>(sum & 0xffu);
}
```

3.1.2 完整自测程序

枚举 256×256 组输入，遇到不一致返回非零退出状态；检查的是参考函数，不是 RTL。

保存为 案例工程/cpp/add_mod256_selftest.cc：

```
#include <cstdint>
#include <iostream>

std::uint8_t add_mod256(std::uint8_t a, std::uint8_t b);

int main() {
    unsigned tested = 0;
    for (unsigned a = 0; a < 256; ++a) {
        for (unsigned b = 0; b < 256; ++b) {
            const auto actual = add_mod256(
                static_cast<std::uint8_t>(a),
                static_cast<std::uint8_t>(b));
            const unsigned expected = (a + b) % 256u;
            if (static_cast<unsigned>(actual) != expected) {
                std::cerr << "Mismatch: a=" << a << " b=" << b << '\n';
                return 1;
            }
            ++tested;
        }
    }
    std::cout << "PASS: " << tested << " C++ input pairs\n";
    return 0;
}
```

普通 C++ 自测命令（从 案例工程/ 目录执行）：

```
mkdir -p build
g++ -std=c++11 -Wall -Wextra -Werror \
  cpp/add_mod256.cc cpp/add_mod256_selftest.cc \
  -o build/add_mod256_selftest
./build/add_mod256_selftest
```

4. 创建双设计环境

User Guide §25.3 给出基本过程：创建设计、分析源代码、可选展开、编译数据流图。下面分别展示这两个阶段的命令（具体接口及参数仍需按现有工程修改）：

```
set_fml_appmode DPV

proc compile_spec {} {
  create_design -name spec -top hector_wrapper
  cppan model.cc
  compile_design spec
}

proc compile_impl {} {
  create_design -name impl -top alu_pipe -clock clk -reset rst_n -negReset
  vcs -sverilog alu_pipe.sv
  compile_design impl
}

compile_spec
compile_impl
```

`create_design` 将 C++ 包装函数与 RTL 顶层分别定义成可比较的设计。`cppan` 分析 C++，`vcs/vlogan/vhdlan` 处理 RTL，`compile_design` 构造工具所需的内部结构。不能把 `compile_design` 与普通 FPV 的 `read_file` 混为一谈。

4.1 软件版本差异：C++11 前端开关

上传的快速参考稿写了 `set_hector_comp_use_new_flow true`，但 User Guide 第 1069 页给出的 Tcl 示例是：

```
set _hector_comp_use_new_flow true
```

两者不是同一条 Tcl 命令，且 User Guide 指出 C++11 前端可能需要额外许可证。因此本教程以 User Guide 形式提示版本差异，正式脚本应在实际安装环境核验支持的变量及其配置方式，避免照抄未经核实的快速笔记。

4.2 原生 DPV 流程模板完整源码

此模板仍需要正式 HECTOR 包装函数和由实际捕获过程确定的阶段关系。不能把普通 `add_mod256` 函数直接重命名为 `hector_wrapper` 后宣称完成对接。

保存为 案例工程/templates/dpv_hector.tcl：

```
# DPV：C/C++ 与 RTL 计算结果对照。需要 HECTOR 事务包装与阶段定义。
error " 此文件是 DPV 教学模板：补齐真实 HECTOR C++ wrapper 和 spec/impl 阶段定义"
# set_fml_appmode DPV
# set _hector_comp_use_new_flow true
# create_design -name spec -top hector_wrapper
# cppan /project/ref/model.cc
# compile_design spec
# create_design -name impl -top alu_pipe -clock clk -reset rst_n -negReset
# vcs -sverilog -f filelist_impl.f
# compile_design impl
# proc ual {} {
#   map_by_name -inputs -specphase 1 -implphase 1
#   map_by_name -outputs -specphase 1 -implphase 3
# }
# set_user_assumes_lemmas_procedure ual
# compose
# solveNB p
# puts [get_task_runtime p]
```

5. 通过阶段对应建立事务关系

```
proc ual {} {
  map_by_name -inputs  -specphase 1 -implphase 1
  map_by_name -outputs -specphase 1 -implphase 3
}
set_user_assumes_lemmas_procedure "ual"
```

示例将参考模型输入第 1 阶段与 RTL 第 1 阶段相匹配，同时将参考模型输出第 1 阶段与 RTL 第 3 阶段相匹配。两边所使用的相位编号由实际 C++ wrapper/RTL 捕获和阶段定义决定；“第 3 阶段”只是这个例子的合法约定，不能机械用于所有流水线。

`map_by_name` 根据名称批量形成输入假设及输出对照关系；名称不同的信号需要显式指定。处理加法器输出时还要确保 C++ 与 SystemVerilog 的截断规则、运算宽度及溢出处理相同。

5.1 完整 RTL 与独立的时序检查

以下 RTL 有输入采集、求和、结果提交三个寄存器阶段。由于并发 SVA 在上升沿使用更新前的采样值，本例用接受输入后的第 3 个后继采样沿检查输出。这个 SVA 采样间隔不等于 HECTOR 阶段编号，不能由 `##3` 直接推导 DPV 的 `-implphase`。

5.1.1 完整流水线 RTL

保存为 案例工程/rtl/alu_pipe_demo.sv：

```
// 三个寄存器阶段：输入采集、求和、结果提交。
// 不含 ready/stall；in_valid=1 的每个上升沿都接受一项输入。
module alu_pipe_demo (
  input  logic       clk, rst_n, in_valid,
  input  logic [7:0] a, b,
  output logic       out_valid,
  output logic [7:0] out
);
  logic [7:0] a_q, b_q, sum_q;
  logic       valid_q, sum_valid;

  always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
      a_q       <= 8'd0;
      b_q       <= 8'd0;
      sum_q     <= 8'd0;
      out       <= 8'd0;
      valid_q   <= 1'b0;
      sum_valid <= 1'b0;
      out_valid <= 1'b0;
    end else begin
      valid_q   <= in_valid;
      sum_valid <= valid_q;
      out_valid <= sum_valid;
      if (in_valid) begin
        a_q <= a;
        b_q <= b;
      end
      if (valid_q)   sum_q <= a_q + b_q;
      if (sum_valid) out   <= sum_q;
    end
  end
endmodule
```

5.1.2 完整 SVA

属性局部变量为每个并发事务保存自己的八位预期值；另一条属性检查有效标志的延迟关系。本例不含停止、反压或冲刷。

保存为 案例工程/formal/alu_pipe_demo_props.sv：

```
module alu_pipe_demo_props (
  input logic clk, rst_n, in_valid, out_valid,
  input logic [7:0] a, b, out
);
  // expected 只属于当前这一次属性匹配，连续事务不会共用一个值。
  property p_add_result;
    logic [7:0] expected;
    @(posedge clk) disable iff (!rst_n)
      (in_valid, expected = a + b)
      |-> ##3 (out_valid && out == expected);
  endproperty
  ap_add_result: assert property (p_add_result);

  logic [2:0] history_valid;
  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) history_valid <= 3'b000;
    else history_valid <= {history_valid[1:0], 1'b1};

  ap_valid_delay: assert property (@(posedge clk) disable iff (!rst_n)
    history_valid[2] |-> out_valid == $past(in_valid, 3));
  cp_consecutive_results: cover property (@(posedge clk) disable iff (!rst_n)
    out_valid ##1 out_valid ##1 out_valid);
endmodule

bind alu_pipe_demo alu_pipe_demo_props u_alu_props (.*);
```

5.1.3 文件列表

保存为 案例工程/filelist_fpv_alu_demo.f：

```
./rtl/alu_pipe_demo.sv
./formal/alu_pipe_demo_props.sv
```

5.1.4 配套 FPV 脚本

这是检查 RTL 事务行为的独立练习，不会自动构成 C++—RTL 的 DPV 证明。

保存为 案例工程/scripts/fpv_alu_demo.tcl：

```
# 从案例工程目录执行；结果必须以本机 VC Formal 日志为准。
set_fml_appmode FPV
read_file -top alu_pipe_demo -sva -format sverilog -vcs {-f filelist_fpv_alu_demo.f}
create_clock clk -period 100
create_reset rst_n -sense low
# 先核验实际复位状态，再保存初始状态。
sim_run -stable
sim_save_reset
check_fv_setup -block
report_fv -list
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
cd 案例工程
```

```
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_alu_demo.tcl -batch 2>&1 | tee logs/fpv_alu_demo.log
```

6. 证明与报告

```
# 在属性已经创建、设计已经编译的前提下运行
check_fv
proofwait
listproofs
listassumes
listlemmas
listcovers
```

对于特定证明对象，DPV 也支持 `solveNB proofName`、`solveNB_init proofName` 等命令；在工程中需要先明确 proof 的名称。调试可以查看对应的反例及 `simcex`，但 C++ 参考模型和 RTL 的相位关系必须正确才能解释差异。

7. 完整可验证事务的六个核对项

1. 输入采集的条件是什么，例如 `in_valid && in_ready`。
2. 每个输入事务是否有唯一的 ID 或输出顺序保证。
3. 参考模型与 RTL 对同一个事务使用完全一致的参数。
4. 输出可比较时的有效标志和阶段编号是否正确。
5. 输出整数截断、饱和、舍入和异常处理是否一致。
6. 复位、气泡、停顿、反压、流水线冲刷等情况是否定义在两边的行为中。

若第 1 阶段输入有效但后续遇到 `stall`，事务完成时间可能不是固定三拍，那么应进一步建模阶段条件或使用其他机制，不能持续使用不成立的固定延迟前提。

8. 命令语法、参数与用例

命令	主要作用
`create_design`	创建数据路径参考或实现设计
`cppan`	分析 C 或 C++ 参考模型
`vcs`	分析或展开 DPV 的 RTL 实现
`vlogan`	分析混合语言中的 Verilog 文件
`vhdlan`	分析混合语言中的 VHDL 文件
`compile_design`	将已分析设计编译为数据流图
`set_user_assumes_lemmas_procedure`	注册事务对应与限制的 Tcl 过程
`map_by_name〔DPV〕`	为 DPV 建立事务阶段对应
`assume`	在 DPV 中声明输入关系或合法值
`lemma`	在 DPV 中声明输出等价目标
`compose`	组合参考和实现及其事务关系
`solveNB`	启动 DPV 证明
`get_task_runtime`	查询 DPV 任务运行时间
`proofwait`	等待已启动的证明计算
`listproofs`	列出证明结果
`listassumes`	列出当前假设
`listlemmas`	列出待证明引理

命令	主要作用
`listcovers`	列出覆盖目标

本组说明的是 UG 给出的入门工作流程。DPV 的完整前端参数、HECTOR 事务包装和高级证明命令在专用 Data Path Validation User Guide 中另行定义；本次提供的参考文件不包含那份手册，因而不把命令示例补写成没有来源支持的完整参数全集。这里的 C/C++ 编译、设计组合和证明启动属于不同阶段，不能把过程定义成功当作等价证明成功。

8.1 `create_design`：创建数据路径参考或实现设计

```
create_design -name <design_name> -top <top_function_or_module>
              [-clock <clock_port>] [-reset <reset_port>] [-negReset]
```

参数解释。 `-name` 建立后续编译引用的设计名，通常为 spec、impl；`-top` 对 C++ 是捕获包装函数，对 RTL 是模块。`-clock`、`-reset` 描述 RTL 端口；`-negReset` 表示低有效复位，省略时为高有效。

用法与结果。这是 DPV/HECTOR 设计创建接口，不能用普通 `create_clock`、`create_reset` 的语法替代这些选项。C++ 包装须通过 `Hector::registerInput`、`Hector::registerOutput` 和 capture 区间定义事务；只有普通 C++ 函数而未声明输入输出，不能自动得到正确比较问题。

```
proc compile_spec {} {
  create_design -name spec -top hector_wrapper
  cppan spec.cc
  compile_design spec
}
proc compile_impl {} {
  create_design -name impl -top adder -clock clk -reset rst_n -negReset
  vcs impl.v
  compile_design impl
}
```

来源：User Guide，X-2025.06-SP1，第 1067–1069 页。

8.2 `cppan`：分析 C 或 C++ 参考模型

```
cppan <options> <source_files>
```

参数解释。位置参数是源码文件列表，选项由 DPV 的 C/C++ 前端解释。UG 本章没有逐一列出所有编译开关，因此保留实际展示的最小形式，不把未经核对的编译器参数假定为兼容。

用法与结果。先 `create_design` 再分析，之后用 `compile_design` 转换为数据流图。该基线同时说明旧 C++99 和新 C++11 前端；切换新前端涉及配置与额外许可。不能由本机 g++ 成功编译推断 DPV 前端也接受全部现代 C++ 特性。

```
create_design -name spec -top hector_wrapper
cppan spec.cc
compile_design spec
```

来源：User Guide，X-2025.06-SP1，第 1068–1069 页。

8.3 `vcs`：分析或展开 DPV 的 RTL 实现

```
vcs <options> [<source_files>]
```

参数解释。纯 Verilog 流程可直接提供源码完成分析与展开；混合语言先用 `vlogan` 和 `vhdlan` 分析，再用 `vcs` 展开。这里的调用处在 DPV Tcl 中，而不是在示例脚本内启动普通仿真测试。

用法与结果。语法中的 `<options>` 是前端选项占位，不能原样复制运行。选项和语言支持应以配套 DPV 与前端参考为准；本教程给出的最小调用与 UG 的可见示例一致。

```
create_design -name impl -top adder -clock clk -reset rst_n -negReset
vcs impl.v
compile_design impl
```

来源：User Guide，X-2025.06-SP1，第 1068–1069 页。

8.4 `vlogan`：分析混合语言中的 Verilog 文件

```
vlogan <options> <source_files>
```

参数解释。输入为 Verilog/SystemVerilog 源码列表，分析选项由相应前端解释。UG 本章只给出这种一般调用形式，没有列出完整开关清单。

用法与结果。混合语言分析完成后仍要展开并生成数据流图。库、编译顺序和顶层绑定必须相容；分析阶段通过不能代替模型建立成功。

```
vlogan impl.v
```

来源：User Guide，X-2025.06-SP1，第 1068 页。

8.5 `vhdlan`：分析混合语言中的 VHDL 文件

```
vhdlan <options> <source_files>
```

参数解释。输入为 VHDL 源码列表，分析选项由相应前端解释。UG 本章只给出这种一般调用形式，没有列出完整开关清单。

用法与结果。混合语言分析完成后仍要展开并生成数据流图。库、编译顺序和顶层绑定必须相容；分析阶段通过不能代替模型建立成功。

```
vhdlan arith_pkg.vhd arith.vhd
```

来源：User Guide，X-2025.06-SP1，第 1068 页。

8.6 `compile_design`：将已分析设计编译为数据流图

```
compile_design <design_name>
```

参数解释。位置参数必须对应先前 `create_design -name` 建立的名称，例如 spec 或 impl。

用法与结果。此步骤把参考或实现设计编译为后续比较使用的数据流图。它还没有建立两侧输入、输出及事务阶段关系；这些关系由 compose 阶段的用户过程提供。

```
create_design -name spec -top hector_wrapper
cppan spec.cc
compile_design spec
```

来源：User Guide，X-2025.06-SP1，第 1068–1069 页。

8.7 `set_user_assumes_lemmas_procedure`：注册事务对应与限制的 Tcl 过程

```
set_user_assumes_lemmas_procedure <procedure_name>
```

参数解释。参数是用户自行定义的 Tcl 过程名，而不是一条直接进行证明的断言。compose 阶段调用此过程来创建输入假设和输出比较目标。

用法与结果。不要把示例中的 ual、compile_spec、compile_impl 当作工具内置命令；它们是教程或工程定义的过程。普通 Tcl `proc` 的过程体仍需完整写在脚本中。

```
proc ual {} {
  map_by_name -inputs -specphase 1 -implphase 1
  map_by_name -outputs -specphase 1 -implphase 3
  assume legal_opcode = impl.in_opcode(1) == 1'b0
}
set_user_assumes_lemmas_procedure ual
```

来源：User Guide，X-2025.06-SP1，第 1069–1070 页。

8.8 map_by_name〔DPV〕：为 DPV 建立事务阶段对应

```
map_by_name -inputs -specphase <phase> -implphase <phase>
map_by_name -outputs -specphase <phase> -implphase <phase>
```

参数解释。 `-inputs` 为同名输入生成相等假设，`-outputs` 为同名输出生成待证明 lemma。`-specphase`、`-implphase` 分别指定参考和实现的事务阶段，它们不是 SEQ 的 `-spec_latency`、`-impl_latency`。

用法与结果。一个 C++ 事务结果可能对应流水 RTL 第三阶段的输出，因此不能仅按名称直接同拍比较。阶段对应还必须与 valid、使能、复位后起始事务一致；错误阶段常使算术本身正确的设计出现反例。

```
proc ual {} {
  map_by_name -inputs -specphase 1 -implphase 1
  map_by_name -outputs -specphase 1 -implphase 3
}
set_user_assumes_lemmas_procedure ual
```

来源：User Guide，X-2025.06-SP1，第 1070 页。

8.9 assume：在 DPV 中声明输入关系或合法值

```
assume <name> = <expression>
```

参数解释。名称标识假设，等号后是 HECTOR 表达式；`impl.in_a(1)` 表示实现设计指定事务阶段的信号，不是普通 SystemVerilog 函数调用。

用法与结果。用于建立两侧输入一致和真实合法输入限制。它与 `fvassume -expr` 属于不同命令接口，不应跨应用混抄。所有假设仍需要规格依据，否则可以把实际不等价的输入排除。

```
proc ual {} {
  assume same_a = spec.in_a(1)[7:0] == impl.in_a(1)[7:0]
  assume legal_op = impl.in_opcode(1) == 1'b0
}
set_user_assumes_lemmas_procedure ual
```

来源：User Guide，X-2025.06-SP1，第 1070 页。

8.10 lemma：在 DPV 中声明输出等价目标

```
lemma <name> = <expression>
```

参数解释。名称标识待证明目标；表达式通常比较两侧指定阶段的输出。它不是无条件成立的输入假设。

用法与结果。在输入对应和事务条件已建立后证明 lemma。需要多个输出或多个操作模式时，分别命名，有助于区分位宽、符号扩展、溢出和流水时刻的问题。

```
proc ual {} {
  assume same_a = spec.in_a(1)[7:0] == impl.in_a(1)[7:0]
  assume same_b = spec.in_b(1)[7:0] == impl.in_b(1)[7:0]
  lemma same_sum = spec.out_sum(1) == impl.out_sum(3)
}
set_user_assumes_lemmas_procedure ual
```

来源：User Guide，X-2025.06-SP1，第 1070 页。

8.11 compose：组合参考和实现及其事务关系

```
compose
```

参数解释。使用已编译的 spec、impl 设计以及已注册的假设/lemma 过程，构造双设计比较环境；本章介绍的是无参数流程入口。

用法与结果。必须在两侧编译成功之后执行。看到双设计层次不等于输入已经正确对应，仍需检查生成的假设、lemma 和阶段关系。UG 把更详细的 compose 配置交给独立 DPV 手册，本教程不补造未提供的完整语法。

```
compile_spec
compile_impl
compose
```

来源：User Guide，X-2025.06-SP1，第 1071 页。

8.12 `solveNB`：启动 DPV 证明

```
solveNB <proof_name>
```

参数解释。参数为证明任务名称；UG 文字说明用该命令启动 lemma 证明，并可由图形界面查看相应 proof/subproof。

用法与结果。这是非阻塞求解入口。运行后应等待任务实际结束再解释状态或保存最终报告，不能把 Tcl 返回解释为已证明。其完整等待、脚本选择和资源接口应与配套 DPV 参考一起核对，不能套用 `check_fv -block` 的选项。

```
compose
solveNB adder_equivalence
```

来源：User Guide，X-2025.06-SP1，第 1070–1071 页。

8.13 `get_task_runtime`：查询 DPV 任务运行时间

```
get_task_runtime <task_name>
```

参数解释。参数是 GUI Task List 或脚本中建立的任务名，返回总运行时间，格式为 HOURS:MM:SS。

用法与结果。该时间可以用于记录资源实验，但不能单凭时间判定证明状态。比较性能时同时记录硬件、脚本、阶段关系、前端设置以及最终任务结果。

```
get_task_runtime adder_equivalence
```

来源：User Guide，X-2025.06-SP1，第 1071 页。

8.14 `proofwait`：等待已启动的证明计算

```
proofwait
```

参数解释。快速参考给出无参数的等待调用。它用于等待此前已经启动的证明，不负责创建设计、建立对应关系或生成引理。

用法与结果。等待返回之后仍须读取证明状态，不能用“等到结束”代替“已经证明”。UG 第 1071 页另行强调，在 DPV 中不能依赖 `proofwait -nokill` 禁止自动终止无进展任务；应使用该流程规定的 `set_vcf_auto_job_kill` 变量配置。此处不补写未在提供资料中展开的其他等待选项。

```
check_fv
proofwait
listproofs
```

来源：Quick Reference Guide，X-2025.06-SP1，第 30 页。

8.15 `listproofs`：列出证明结果

```
listproofs
```

参数解释。快速参考给出这里的无参数调用形式。命令用于相应的 DPV 证明环境；提供的 UG 未逐项

列出其完整可选参数，因此本条不构造额外过滤开关。

用法与结果。检查证明对象是否实际创建、是否有未完成或失败结果。证明列表不是用户引理文本本身，也不保证所有预期事务均已纳入范围。与其他报告一起保留当前设计版本和阶段对应设置，避免把不同事务模型下的报告混在一起。

```
listproofs
```

来源：Quick Reference Guide，X-2025.06-SP1，第 30 页。

8.16 `listassumes`：列出当前假设

```
listassumes
```

参数解释。快速参考给出这里的无参数调用形式。命令用于相应的 DPV 证明环境；提供的 UG 未逐项列出其完整可选参数，因此本条不构造额外过滤开关。

用法与结果。确认参考设计与实现之间的输入关系，以及用户附加的运行前提。特别检查有没有把要比较的输出直接写成相等假设。与其他报告一起保留当前设计版本和阶段对应设置，避免把不同事务模型下的报告混在一起。

```
listassumes
```

来源：Quick Reference Guide，X-2025.06-SP1，第 30 页。

8.17 `listlemmas`：列出待证明引理

```
listlemmas
```

参数解释。快速参考给出这里的无参数调用形式。命令用于相应的 DPV 证明环境；提供的 UG 未逐项列出其完整可选参数，因此本条不构造额外过滤开关。

用法与结果。核对每条输出关系的参考阶段、实现阶段和有效条件。列出引理只表示目标存在，不能视为该目标已经证明。与其他报告一起保留当前设计版本和阶段对应设置，避免把不同事务模型下的报告混在一起。

```
listlemmas
```

来源：Quick Reference Guide，X-2025.06-SP1，第 30 页。

8.18 `listcovers`：列出覆盖目标

```
listcovers
```

参数解释。快速参考给出这里的无参数调用形式。命令用于相应的 DPV 证明环境；提供的 UG 未逐项列出其完整可选参数，因此本条不构造额外过滤开关。

用法与结果。覆盖目标用于查看某个操作码、有效事务或完成条件是否可以发生。覆盖成功只提供存在性证据，不代替所有输入下的事务等价。与其他报告一起保留当前设计版本和阶段对应设置，避免把不同事务模型下的报告混在一起。

```
listcovers
```

来源：Quick Reference Guide，X-2025.06-SP1，第 30 页。

9. 数据路径证明困难时的策略

分清是**规格定义错误**（C++ 的有符号性/未定义行为）、**阶段设置错误**（错误的输入输出相位）、**RTL 功能错误**，还是**求解规模困难**（乘法/除法/浮点运算）。先用较小位宽和少量操作码验证模型，再推广到完整位宽。必要时按指令类别切分证明目标，或利用 DPV 特有的重写与求解脚本；引擎调优应晚于正确的事务接口设置。

10. 官方参考

User Guide §25（1066 ~ 1080 页）、Quick Reference Guide DPV 部分。更深入的 HECTOR 包装函数、内部 phase、定制求解策略应以单独的 VC Formal Data Path Validation User Guide 为准，不能由通用 UG 的十余页简介替代。

FLP / LPC：低功耗连通性形式验证使用指南

1. 核心问题

低功耗 SoC 通过不同电源域、电源关断、隔离单元、保持寄存器和电源控制器降低功耗。在普通 RTL 连通性检查中，信号可能看起来已经正确相连，但电源域关断后仍可能把未知或无效数据送到始终通电的逻辑；隔离控制接反，可能导致不同电源状态下的功能不符合规格。

FLP CC 将 RTL 与 UPF（Unified Power Format，IEEE 1801）电源意图结合，验证在相应电源状态及使能条件下，指定源与目的的连通性规则是否成立。它不只是检查 UPF 文件能否成功解析，还要分析电源意图对 RTL 的功能影响。



图 14: 低功耗域和隔离输出

2. 重要对象

对象	解释	例子
Power domain	电源控制范围	可关断图像处理域
Supply net / set	电源网络与电压状态	`VDD_CORE`
Isolation	电源关断时对跨域输出进行限制	下游锁定为 0
Retention	保存关断前需要恢复的状态	睡眠前的配置位
Power state table	组合描述电源合法状态	工作、待机、关断
Power controller	控制电源和隔离时序	关断前先使能隔离

电源意图描述设计在哪些供电条件下工作，隔离和保持策略分别保护跨域输出与需要保存的状态。常供电控制器必须在被控制电源域关闭后仍能驱动正确的控制信号。验证时应分别覆盖正常供电、掉电、恢复三个阶段，并核对隔离控制相对于供电变化的先后关系。

要求明确：何时隔离、生效电平是什么、钳位值为何、是否需要恢复保存状态。普通 CC 的组合连接模型不足以单独表达所有电源关闭的效果。

3. 该版本的工作模式

Quick Reference Guide 的低功耗流程不是直接输入 `set_fml_appmode FLP`，而是：

```
set_fml_appmode CC
fvlp_instrument -type "iso pnm"
set_fml_var fvlp_enable_pst_constraint true
read_file -top lp_top -format sverilog -vcs {-f filelist.f}
```

```
read_upf power_intent.upf
create_clock clk -period 100
create_reset rst_n -sense low
```

`fvlp_instrument` 启用低功耗语义插桩，`-type` 可根据设计需要选择 `iso`、`pnm`、`ret` 等，具体支持类型参照 UG 1043 页。插入低功耗支持、读入 RTL/UPF 的顺序需要与安装版本示例一致。`read_upf` 中的路径必须指向该工程真实 UPF 文件。

3.1 原生低功耗检查模板完整源码

UPF、电源状态及隔离策略必须来自实际项目。本模板的省略对象属于明确的待配置输入，不是已提供的完整低功耗工程。

保存为 案例工程/templates/flp_low_power.tcl：

```
# 低功耗 CC：必须有 RTL、UPF、电源状态表、真正的电源域信号。
# 不支持普通 CC 的 delayed connectivity check（UG 第 1041 页）。
error " 此文件是 FLP 教学模板：须先提供 UPF、电源控制与隔离策略"
# set_fml_appmode CC
# fvlp_instrument -type "iso pnm"
# set_fml_var fvlp_enable_pst_constraint true
# read_file -top lp_top -format sverilog -vcs {-f filelist.f}
# read_upf /project/upf/power_intent.upf
# create_clock clk -period 100
# create_reset rst_n -sense low
# add_cc -src ... -dest ... -lpa_type ... ;# 填写真实端点与类型
# check_fv -block
# report_fv -verbose
```

4. 最小系统案例

上游处理单元运行于可关断电源域 `PD_A`，下游控制单元在始终通电的 `PD_AON`。上游输出 `data_o` 经隔离单元到下游 `data_i`。规格：当 `PD_A` 供电正常且隔离关闭时，`data_i` 应等于 `data_o`；当 `PD_A` 关断时，`data_i` 应被钳位为零。

功能连接与隔离钳位应分别检查。普通端点关系通过低功耗条件下的连通性检查定义；钳位检查则直接指定端口或引脚、隔离单元所在位置以及应保持的钳位值。`fvlp_check_clamp_value` 不接受示例旧稿中混入的 `-lpa_type` 或 `-enable`，这些参数不能从其他命令移植过来。

```
fvlp_check_clamp_value u_aon/data_i -location parent -clamp_value 0
```

该调用要求已经读入实际 RTL 与 UPF，并完成低功耗模型配置；`u_aon/data_i` 必须替换为展开设计中真实的端口或引脚。电源域、隔离使能和供电关系应由 UPF 与设计共同确定，不由这条命令凭空创建。

4.1 完整源码：先理解逻辑隔离关系

为了不把 UPF 环境和普通逻辑混在一起，以下单独给出理想隔离控制的 RTL 与 SVA。这里的 `source_power_good` 只是布尔输入，不会使仿真器或形式工具真的把某个电源域关断，也没有建立掉电后的未知值模型。

4.1.1 完整逻辑模型

源电源不正常时，理想控制模型立即使能隔离；真实系统的电源控制时序、跨时钟处理和故障行为必须另行验证。

保存为 案例工程/rtl/lp_isolation_demo.sv：

```
// 仅表示理想的逻辑隔离关系，不模拟电源关断产生的未知值。
```

```
module lp_isolation_demo (
  input  logic       clk, rst_n,
  input  logic       source_power_good, isolation_request,
  input  logic [7:0] source_data,
  output logic       isolation_enable,
  output logic [7:0] destination_data
);
  // 理想控制模型：源电源不正常时强制隔离；真实 PMC 必须另行验证。
  assign isolation_enable = !source_power_good || isolation_request;
  assign destination_data = isolation_enable ? 8'h00 : source_data;
endmodule
```

4.1.2 完整 SVA

保存为 案例工程/formal/lp_isolation_demo_props.sv：

```
module lp_isolation_demo_props (
  input logic clk, rst_n,
  input logic source_power_good, isolation_request, isolation_enable,
  input logic [7:0] source_data, destination_data
);
  ap_isolation_clamp: assert property (@(posedge clk) disable iff (!rst_n)
    isolation_enable |-> destination_data == 8'h00);
  ap_power_off_clamp: assert property (@(posedge clk) disable iff (!rst_n)
    !source_power_good |-> destination_data == 8'h00);
  ap_normal_connection: assert property (@(posedge clk) disable iff (!rst_n)
    source_power_good && !isolation_request
      |-> destination_data == source_data);
  cp_off_on_sequence: cover property (@(posedge clk) disable iff (!rst_n)
    !source_power_good ##1 source_power_good && !isolation_request);
endmodule

bind lp_isolation_demo lp_isolation_demo_props u_lp_props (.*);
```

4.1.3 文件列表

保存为 案例工程/filelist_fpv_lp_logic_demo.f：

```
./rtl/lp_isolation_demo.sv
./formal/lp_isolation_demo_props.sv
```

4.1.4 普通 FPV 练习脚本

保存为 案例工程/scripts/fpv_lp_logic_demo.tcl：

```
# 从案例工程目录执行；结果必须以本机 VC Formal 日志为准。
set_fml_appmode FPV
read_file -top lp_isolation_demo -sva -format sverilog -vcs {-f
 ↪  filelist_fpv_lp_logic_demo.f}
create_clock clk -period 100
create_reset rst_n -sense low
# 先核验实际复位状态，再保存初始状态。
sim_run -stable
sim_save_reset
check_fv_setup -block
report_fv -list
check_fv -block
report_fv -verbose
```

运行命令（Bash）：

```
# 从包含“案例工程”目录的上一级目录开始。
```

```
cd 案例工程
mkdir -p logs
set -o pipefail
vcf -f scripts/fpv_lp_logic_demo.tcl -batch 2>&1 | tee logs/fpv_lp_logic_demo.log
```

与原生 FLP 的区别。上述练习说明隔离钳位和正常连接的布尔关系，并不验证 UPF 插入位置、电源网络、保持单元或电源关断行为。真正的 FLP 检查还需要第 3 节列出的 UPF 和工具低功耗配置，不能用本节 FPV 结果替代。

5. UPF 可信性与 PMC

UPF 应由电源架构负责人确认电源域划分、电源状态、隔离和保持策略；必要时还需为真实电源管理控制器（PMC）建立环境模型。UG §24.2 提到可为不存在于形式 RTL 的 PMC 建模其外部行为，不能把这种模型误当作 RTL 自身需要被证明的功能。错误地假设所有电源域永久打开，将直接屏蔽很多低功耗缺陷。

6. FLP CC 和普通 CC 的重要区别

User Guide 第 1041 页明确说明：FLP CC 不支持连通性延迟检查。因此不能直接把普通 CC 中的 `path_delay` 用例复制到低功耗检查环境。对于寄存器延迟、时序握手或跨时钟的低功耗交互，需要以另一套适当的功能属性或等价流程验证，而非宣称 FLP CC 已证明它们。

7. 命令语法、参数与用例

命令	主要作用
`fvlp_instrument`	选择低功耗模型插装内容
`read_upf`	将电源意图装入当前设计
`get_fvlp_free_supply_nets`	查询未驱动的自由电源
`fvlp_set_supply_aon`	将指定电源设置为常开
`fvlp_set_supply_onoff_net`	用 RTL 控制信号驱动电源开关状态
`fvlp_check_clamp_value`	检查端口或引脚的隔离方式
`flp_cc_vacuity_gen_split`	拆分低功耗空洞通过原因
`flp_cc_vacuity_report`	解释低功耗属性的空洞状态
`fvlp_cc_print_path`	显示连通性属性经过的电源相关对象
`fvlp_cc_trace_fanin`	追踪供电引脚的驱动来源

本组命令按准备、生成、执行与结果处理的关系组织。语法框保留参数名称和可选关系；示例中的实例、属性和文件名称需要与当前工程一致。生成目标与证明目标是不同阶段，调用生成命令后应检查实际目标数量，再启动证明。

7.1 `fvlp_instrument`：选择低功耗模型插装内容

```
fvlp_instrument [-type <type_list>]
```

参数解释。`-type` 接受 iso、ret、pnm、bind_generics、ret1。iso 表示隔离策略，ret 表示保持策略，pnm 表示供电网络模型；后两类名称在本节作为可选插装类型列出，但没有足够解释可把它们当成通用推荐配置。典型低功耗连通性设置使用 `{iso pnm ret}`。

用法与结果。在建立 FLP CC 流程时先选择 CC 应用模式并声明插装类型，再读取 RTL 与 UPF。缺少隔离或供电模型会改变被分析对象，不能把有 UPF 文件名但没有相应模型的运行认作完成低功耗检查。

```
set_fml_appmode CC
fvlp_instrument -type {iso pnm ret}
read_file -top top -format sverilog -vcs "-f rtl.f"
read_upf top.upf
```

来源：User Guide，X-2025.06-SP1，第 1043–1044 页。

7.2 `read_upf`：将电源意图装入当前设计

```
read_upf <upf_file>
```

参数解释。位置参数是 UPF 文件路径；文件中的对象名称和作用域必须与已经读入的设计对应。UG 在本节使用此简单形式，没有在本章展开平台所有 UPF 读入选项。

用法与结果。读取 UPF 后工具建立电源网络、隔离、保持及掉电相关模型。额外的形式包装层可能需要自己的电源域并设置 DUT 作用域，不能直接把原 SoC UPF 顶层路径照搬到另一层包装中。此命令只装入电源意图，不替代时钟、复位和缺省 PMC 行为的建模。

```
fvlp_instrument -type {iso pnm ret}
read_file -top formal_top -format sverilog -vcs "-f rtl.f"
read_upf formal_top.upf
```

来源：User Guide，X-2025.06-SP1，第 1043–1046 页。

7.3 `get_fvlp_free_supply_nets`：查询未驱动的自由电源

```
get_fvlp_free_supply_nets
```

参数解释。本节展示无参数查询，其输出是供电模型中没有被驱动的自由 supply 列表。原文把命令与示例返回列表排在同一行，列表不是必须追加的输入参数。

用法与结果。自由电源不是自动“始终上电”。未建模的供电状态可能任意变化，从而形成不符合系统设计的掉电路径；也可能暴露实际缺少电源约束的问题。先查询，再根据真实电源控制设计决定哪些常开、哪些受 RTL 控制。

```
get_fvlp_free_supply_nets
```

来源：User Guide，X-2025.06-SP1，第 1058 页。

7.4 `fvlp_set_supply_aon`：将指定电源设置为常开

```
fvlp_set_supply_aon <supply_list>
```

参数解释。位置参数为 Tcl 列表，例如 `{VDD_AON VDD_RET}`；不要把原文展示列表中的逗号复制进正常的空白分隔 Tcl 列表。

用法与结果。只把架构上确实常开的电源设为 AON。为了减少失败而把所有可掉电域都常开，会删除隔离和保持最需要检查的情形。UG 的示例日志给出工具采用的供电值，不代表可用该命令描述任意模拟电压或地网。

```
get_fvlp_free_supply_nets
fvlp_set_supply_aon {VDD_AON VDD_RET}
```

来源：User Guide，X-2025.06-SP1，第 1058–1059 页。

7.5 `fvlp_set_supply_onoff_net`：用 RTL 控制信号驱动电源开关状态

```
fvlp_set_supply_onoff_net -netname <rtl_signal> <supply_list>
```

参数解释。 `-netname` 指定 RTL 中的开关控制信号，末尾列表指定要受它控制的 supply。

用法与结果。该关联使供电状态随控制信号变化，但它不自动实现完整的电源管理时序协议。隔离先后顺序、保持控制和复位恢复仍需由 PMC 或形式环境描述，并用相应覆盖目标确认能到达关电、上电场景。

```
fvlp_set_supply_onoff_net -netname pmc.core_power_on {VDD_CORE}
```

来源：User Guide，X-2025.06-SP1，第 1059 页。

7.6 fvlp_check_clamp_value：检查端口或引脚的隔离方式

```
fvlp_check_clamp_value <port_or_pin>
  [-location <parent|self|default>]
  [-clamp_value <0|1|LATCH|BASIC>]
```

参数解释。位置参数选择端口或引脚；`-location` 限定隔离位置；`-clamp_value` 指定期望隔离行为。0/1 检查固定钳位，LATCH 对应保持原值，BASIC 创建 LPA_BASIC 检查以验证没有定义隔离。

用法与结果。原 UG 命令名是 `fvlp_check_clamp_value`，不能因快速参考或旧笔记缩写而删去前缀中的 v。它用于针对隔离模型生成检查，不只是打印 RTL 引脚当前电平。

```
fvlp_check_clamp_value u_core/data_out \
  -location parent -clamp_value 0
```

来源：User Guide，X-2025.06-SP1，第 1046 页。

7.7 flp_cc_vacuity_gen_split：拆分低功耗空洞通过原因

```
flp_cc_vacuity_gen_split [<property_name>]
```

参数解释。省略属性时处理默认的空洞属性集合；提供名称则只拆分该属性。生成 src、dest、iso、src_dest、src_iso、dest_iso 六类辅助 cover，分别检查供电和隔离条件的单独及联合可达性。

用法与结果。辅助 cover 在本流程中不一定显示在普通属性表里，需要执行证明后通过专门报告查看。某个条件单独可达不意味着它与另一个条件能同时成立，联合 cover 正是用于找出这种环境互斥。

```
flp_cc_vacuity_gen_split cc_core_to_bus
check_fv -block
flp_cc_vacuity_report cc_core_to_bus
```

来源：User Guide，X-2025.06-SP1，第 1047–1048 页。

7.8 flp_cc_vacuity_report：解释低功耗属性的空洞状态

```
flp_cc_vacuity_report [<property_name>]
```

参数解释。读取拆分生成并运行后的辅助 cover 状态；提供属性名限制范围。

用法与结果。当源电源可开、目的电源可开但隔离条件不可达时，报告会把这些情况区分开。修复应针对 UPF、PMC 或使能规格，而不是直接关闭 vacuity 检查或把全部电源强制常开。

```
flp_cc_vacuity_report cc_core_to_bus
```

来源：User Guide，X-2025.06-SP1，第 1048 页。

7.9 fvlp_cc_print_path：显示连通性属性经过的电源相关对象

```
fvlp_cc_print_path <property_name>
```

参数解释。位置参数为已经生成的低功耗连通性属性名称；报告列出其源与目的之间的对象。

用法与结果。适合检查中间电源域、隔离单元、开关及连接。路径存在仅是结构信息，仍需结合反例中的电源状态和隔离使能判断为何检查失败。

```
fvlp_cc_print_path cc_core_to_bus
```

来源：User Guide，X-2025.06-SP1，第 1064 页。

7.10 fvlp_cc_trace_fanin：追踪供电引脚的驱动来源

```
fvlp_cc_trace_fanin -prop <property_name> -dest <destination> [-incr]
```

参数解释。`-prop` 指定分析属性，`-dest` 指定待追踪节点。发现并行驱动时，把其中一条驱动作为下一次 `-dest`，加 `-incr` 继续追踪。

用法与结果。它主要帮助分析 PG 引脚和 supply 的连接，不能与普通 RTL 数据扇入报告混为一谈。每次迭代保留前一次列出的节点，直到找到真实供电端口、开关或未连接位置。

```
fvlp_cc_trace_fanin -prop cc_core_to_bus -dest PD_CORE/PSW/VDD
fvlp_cc_trace_fanin -prop cc_core_to_bus -dest PD_CORE/VDDA -incr
```

来源：User Guide，X-2025.06-SP1，第 1064 页。

8. 三个需要分别检查的场景

正常通电：隔离关闭时功能数据能够到达下游；**关断阶段：**隔离在失去电源前启动、钳位值正确并持续；**恢复阶段：**电源回到合法状态后，对隔离解除、复位和保持恢复的顺序进行验证。第三阶段常涉及跨周期时序，可能需要 FPV 与电源控制器的状态属性配合。

9. 调试方法

从失败的低功耗连接属性开始，检查 UPF 的源目的供电网络、`iso_en` 及电源状态，再查看波形或 Power Browser。特别注意电源状态表是否包含不实际的组合、隔离使能是否被形式环境错误固定、以及低功耗插桩是否真正参与了编译。不能仅因为正常供电情况下路径连接正确就结束验证。

10. 参考

User Guide §24（1040 ~ 1065 页），Quick Reference Guide Low Power Connectivity Checking 部分；涉及 UPF 语法时还应参考项目采用的 IEEE 1801 版本与电源意图开发规范。

设计查询、对象集合与属性筛选

1. 查询对象不是修改 RTL

VC Formal 展开设计后，脚本使用带有类型的对象表示设计、实例、端口、引脚和网络。查询命令返回对象集合；集合中的成员指向展开模型，而不是从源代码截取出的文字。筛选和排序只改变后续脚本使用的对象范围，不会修改寄存器、电路连接或模块参数。

对象类别	表示什么	常见查询
design	模块或库设计定义	`get_designs`
cell	展开后的实例	`get_cells`
port	当前顶层的输入输出端口	`get_ports`
pin	实例上的接口引脚	`get_pins`
net	连接驱动端与接收端的网络	`get_nets`
lib_cell、lib_pin	库单元及其引脚定义	`get_lib_cells`、`get_lib_pins`

例如两个计数器实例可以引用同一个模块定义，但分别具有不同的层次路径、复位连接和内部状态。查询 `design` 只能确定模块定义；给某个实例施加黑盒或追踪某条信号的影响范围，通常需要实例或网络对象。把对象类别选错，常见结果是查询为空或后续命令拒绝该集合，而不是设计中真的没有这段逻辑。

2. 名称、集合与 Tcl 列表

普通 Tcl 列表保存字符串等值，对象集合保存工具管理的设计对象。`get_object_name` 用于输出可读名称，`sizeof_collection` 用于计数，`foreach_in_collection` 用于逐对象遍历。不要用 `llength` 判断对象数量，也不要把 `query_objects` 的打印结果当作集合返回值。

模式查询通常使用星号和问号；准确匹配含特殊字符的名称时，既要处理查询规则，也要处理 Tcl 的求值规则。花括号可避免 Tcl 提前解释美元符号、方括号和反斜杠。正则表达式模式与普通通配符不是同一语法，是否启用正则模式应保持显式一致。

```
set cells [get_cells -hierarchical *]
puts " 对象数: [sizeof_collection $cells]"
foreach_in_collection cell $cells {
    puts [get_object_name $cell]
}
```

层次查询还要区分“逐层匹配叶名”和“指定完整路径”。脚本应在应用约束或生成检查之前打印命中数量及少量对象名称；本来应有几千个对象却只命中零个，必须作为设置问题停止处理，不能继续产生一份看似通过的空报告。

3. 命令语法、参数与用例

命令	主要作用
`get_command_option_values`	读取命令的默认参数和最近参数
`get_cells`	按名称、层次和属性查询实例
`get_designs`	查询设计定义
`get_ports`	查询顶层端口

命令	主要作用
get_pins	查询实例引脚
get_nets	查询网络
get_lib_cells	查询库单元
get_lib_pins	查询库引脚
get_object_name	把对象转换成可读名称
query_objects	显示集合中的对象
filter_collection	按属性筛选已有集合
foreach_in_collection	逐对象遍历集合
add_to_collection	创建加入对象后的集合
remove_from_collection	创建删除指定对象后的集合
sort_collection	按一个或多个属性排序
sizeof_collection	计算集合对象数
compare_collections	比较两个集合是否相等
copy_collection	复制集合容器
index_collection	按下标取得集合成员
get_attribute	读取对象属性
list_attributes	列出某类对象可查询的属性

以下命令覆盖 UG 附录 A 中的基本查询、集合操作和属性访问。UG 对部分查询只介绍作用，没有给出完整平台选项；这些条目仅列出能够核实的基本形式。详细示例都以已完成设计展开为前提。

3.1 get_command_option_values：读取命令的默认参数和最近参数

```
get_command_option_values [-default] [-current] -command <command>
```

参数解释。-command 指定命令名称；-default 读取已登记的默认值，-current 读取最近使用的值。两个选项均未写出时，优先返回当前值，没有当前值时使用默认值。返回值是“参数名、参数值”交替排列的 Tcl 列表，可以交给 array set。

用法与结果。该查询适合把交互操作中的选项保存到脚本中，但不是所有命令都会登记全部选项。返回空列表只表示没有可查询的记录，不能据此判断命令不支持参数。读取参数不会重新执行原命令。

```
set opts [get_command_option_values -current -command report_fv]
array set report_options $opts
parray report_options
```

来源：User Guide，X-2025.06-SP1，第 1085–1086 页。

3.2 get_cells：按名称、层次和属性查询实例

```
get_cells [-hierarchical] [-quiet] [-regexp] [-nocase] [-exact]
          [-filter <expression>] <patterns>
get_cells [-quiet] [-filter <expression>] -of_objects <objects>
```

参数解释。<patterns> 是实例名称或模式组成的 Tcl 列表；-hierarchical 搜索层次中的实例；-quiet 抑制未匹配对象的提示。-regexp 使用正则表达式，-exact 将模式作为准确名称，两者互斥；-nocase 忽略大小写。-filter 对匹配实例继续进行属性筛选。-of_objects 从给定对象取得所属实例，不与名称模式同时使用。

用法与结果。返回的是实例对象集合，不是模块定义名组成的字符串列表。用 get_object_name 取得可打印名称，用 foreach_in_collection 遍历对象。名称模式决定先搜索哪些实例，属性表达式决定哪些实例最终保留，因此名称查询为空和属性筛选为空应分开排查。

```
set regs [get_cells -hierarchical * -filter {is_sequential == true}]
```

```
puts " 实例数: [sizeof_collection $regs]"
foreach_in_collection cell $regs {
    puts [get_object_name $cell]
}
```

来源：User Guide，X-2025.06-SP1，第 1085–1088、1091–1094 页。

3.3 get_designs：查询设计定义

```
get_designs <patterns>
```

参数解释。 <patterns> 指定对象名称或匹配模式。这里列出 UG 介绍的基本查询形式；各对象类型的其他选项由 VC Static Platform 命令参考定义，不把 get_cells 的全部选项机械套用到这些命令。

用法与结果。查询已经读入并展开的设计定义，而不是某个层次下的实例。多个实例可以引用同一个设计定义。返回值是对象集合，可用于后续筛选、属性读取或计数。先检查集合是否为空，再对具体对象执行需要非空输入的操作。

```
set designs [get_designs *]
query_objects $designs
```

来源：User Guide，X-2025.06-SP1，第 1083–1088 页。

3.4 get_ports：查询顶层端口

```
get_ports <patterns>
```

参数解释。 <patterns> 指定对象名称或匹配模式。这里列出 UG 介绍的基本查询形式；各对象类型的其他选项由 VC Static Platform 命令参考定义，不把 get_cells 的全部选项机械套用到这些命令。

用法与结果。查询当前顶层设计的端口。端口与实例引脚不是同一类对象；查询模块内部接口时，应先判断需要的是层次引脚还是顶层端口。返回值是对象集合，可用于后续筛选、属性读取或计数。先检查集合是否为空，再对具体对象执行需要非空输入的操作。

```
set ports [get_ports *]
foreach_in_collection p $ports {
    puts "[get_object_name $p] [get_attribute $p direction]"
}
```

来源：User Guide，X-2025.06-SP1，第 1083–1088 页。

3.5 get_pins：查询实例引脚

```
get_pins <patterns>
```

参数解释。 <patterns> 指定对象名称或匹配模式。这里列出 UG 介绍的基本查询形式；各对象类型的其他选项由 VC Static Platform 命令参考定义，不把 get_cells 的全部选项机械套用到这些命令。

用法与结果。查询展开设计中的实例引脚。实例路径和引脚名共同标识一个引脚，不应把同名顶层端口当作同一对象。返回值是对象集合，可用于后续筛选、属性读取或计数。先检查集合是否为空，再对具体对象执行需要非空输入的操作。

```
set pins [get_pins {u_counter/*}]
query_objects $pins
```

来源：User Guide，X-2025.06-SP1，第 1083–1088 页。

3.6 get_nets：查询网络

```
get_nets <patterns>
```

参数解释。 <patterns> 指定对象名称或匹配模式。这里列出 UG 介绍的基本查询形式；各对象类型的其他选项由 VC Static Platform 命令参考定义，不把 get_cells 的全部选项机械套用到这些命令。

用法与结果。取得设计中连接驱动端和接收端的网络对象。网络对象、驱动它的引脚和接收它的引脚分别具有不同属性。返回值是对象集合，可用于后续筛选、属性读取或计数。先检查集合是否为空，再对具体对象执行需要非空输入的操作。

```
set nets [get_nets {data*}]
query_objects $nets
```

来源：User Guide，X-2025.06-SP1，第 1083–1088 页。

3.7 get_lib_cells：查询库单元

```
get_lib_cells <patterns>
```

参数解释。`<patterns>` 指定对象名称或匹配模式。这里列出 UG 介绍的基本查询形式；各对象类型的其他选项由 VC Static Platform 命令参考定义，不把 `get_cells` 的全部选项机械套用到这些命令。

用法与结果。取得库中的单元定义；与 `get_cells` 返回的设计实例不同。其可见范围取决于当前已加载的库。返回值是对象集合，可用于后续筛选、属性读取或计数。先检查集合是否为空，再对具体对象执行需要非空输入的操作。

```
set library_cells [get_lib_cells *]
query_objects $library_cells
```

来源：User Guide，X-2025.06-SP1，第 1083–1088 页。

3.8 get_lib_pins：查询库引脚

```
get_lib_pins <patterns>
```

参数解释。`<patterns>` 指定对象名称或匹配模式。这里列出 UG 介绍的基本查询形式；各对象类型的其他选项由 VC Static Platform 命令参考定义，不把 `get_cells` 的全部选项机械套用到这些命令。

用法与结果。取得库单元的引脚定义。设计实例引脚可能关联一个库引脚，但两者不能在脚本中任意互换。返回值是对象集合，可用于后续筛选、属性读取或计数。先检查集合是否为空，再对具体对象执行需要非空输入的操作。

```
set library_pins [get_lib_pins *]
query_objects $library_pins
```

来源：User Guide，X-2025.06-SP1，第 1083–1088 页。

3.9 get_object_name：把对象转换成可读名称

```
get_object_name <objects>
```

参数解释。`<objects>` 是对象或对象集合，返回相应对象名称。名称适合打印、保存到文本或构造报告，但不能代替对象集合完成所有集合运算。

用法与结果。集合在 Tcl 中以句柄表示。直接 `puts $objects` 可能打印句柄，而不是用户想要的实例路径。为了保持对象类型和属性，内部计算保留集合，只在输出时转换名称。

```
set cells [get_cells *]
puts [get_object_name $cells]
```

来源：User Guide，X-2025.06-SP1，第 1083、1089–1090 页。

3.10 query_objects：显示集合中的对象

```
query_objects <collection>
query_objects [-verbose] -class <class> <pattern>
```

参数解释。`<collection>` 是待显示集合；`-class` 选择对象类别，`<pattern>` 指定名称模式；`-verbose` 可显示类别信息，便于辨别混合了端口、引脚等对象的集合。

用法与结果。该命令将对象打印到控制台，但返回空字符串。因此不要用 `set x [query_objects $cells]` 保存查询结果；这样得到的 `x` 不是原集合。需要保存集合时直接保存 `get_cells` 等查询的返回值。

```
set p [get_ports *]
query_objects $p
query_objects -class port *
```

来源：User Guide，X-2025.06-SP1，第 1089–1090、1097 页。

3.11 `filter_collection`：按属性筛选已有集合

```
filter_collection <collection> <expression> [-regexp] [-nocase]
```

参数解释。`<expression>` 是对象属性表达式。`-regexp` 让名称匹配使用正则表达式；`-nocase` 忽略大小写。筛选产生新集合，不修改输入集合。

用法与结果。表达式中的属性必须适用于该对象类别。平台的复合表达式不能简单照搬其他语言的优先级假设，多个逻辑条件应显式加括号。若查询命令本身支持 `-filter`，直接查询并筛选通常比先构造大集合再筛选更合适。

```
set all_cells [get_cells -hierarchical *]
set regs [filter_collection $all_cells {is_sequential == true}]
puts " 全部: [sizeof_collection $all_cells]; 时序实例: [sizeof_collection $regs]"
```

来源：User Guide，X-2025.06-SP1，第 1092–1095 页。

3.12 `foreach_in_collection`：逐对象遍历集合

```
foreach_in_collection <iter_var> <collections> <body>
```

参数解释。`<iter_var>` 是循环变量名称，`<collections>` 是待遍历的集合，`<body>` 是每次执行的 Tcl 脚本。每次迭代时变量表示一个仅含当前对象的集合。

用法与结果。不能把对象集合当作普通 Tcl 列表交给 `foreach`。普通 `foreach` 可能只遍历到集合句柄，后续名称和属性操作便不再对应真实对象。循环体使用花括号，使变量在每次迭代时求值。

```
foreach_in_collection p [get_ports *] {
    set n [get_object_name $p]
    set d [get_attribute $p direction]
    puts "$n : $d"
}
```

来源：User Guide，X-2025.06-SP1，第 1095 页。

3.13 `add_to_collection`：创建加入对象后的集合

```
add_to_collection <collection> <object_spec> [-unique]
```

参数解释。`<object_spec>` 可以是对象名称、模式或集合；`-unique` 用于避免重复对象。返回包含原对象及新增对象的新集合，原集合保持不变。

用法与结果。添加对象不会在 RTL 中创建硬件，只改变查询结果。需要把结果保存回原变量时必须明确赋值。混合类别的集合只适合能同时接受这些类别的下游命令，不能因为加入成功便假定所有属性都通用。

```
set selected [get_ports {clk rst_n}]
set selected [add_to_collection $selected [get_ports {enable}] -unique]
query_objects $selected
```

来源：User Guide，X-2025.06-SP1，第 1096–1097 页。

3.14 remove_from_collection：创建删除指定对象后的集合

```
remove_from_collection <collection> <object_spec>
```

参数解释。 <object_spec> 指定要移除的名称、模式或集合。返回剩余对象构成的新集合；若结果为空，则返回空集合表示。

用法与结果。这不是删除设计对象，也不会删除端口或逻辑。它仅改变后续操作使用的对象范围。原集合不变，便于保留全集并分别构造时钟端口、控制端口和数据端口的子集。

```
set all_ports [get_ports *]
set data_ports [remove_from_collection $all_ports [get_ports {clk rst_n}]]
query_objects $data_ports
```

来源：User Guide，X-2025.06-SP1，第 1096 页。

3.15 sort_collection：按一个或多个属性排序

```
sort_collection [-descending] <collection> <criteria>
```

参数解释。 <criteria> 是排序属性的有序列表，优先按第一个属性排序，值相同时再比较后续属性。默认升序； -descending 使用降序。布尔属性升序时 false 在 true 前面。

用法与结果。排序适合生成稳定、便于版本比较的报告。对于某些对象缺失的稀疏属性，平台还有专门的缺失值排列规则；不要用排序位置推断该对象是否具备硬件特征，应读取属性本身。

```
set sorted [sort_collection [get_ports *] {direction full_name}]
foreach_in_collection p $sorted {
    puts [get_object_name $p]
}
```

来源：User Guide，X-2025.06-SP1，第 1097 页。

3.16 sizeof_collection：计算集合对象数

```
sizeof_collection <collection>
```

参数解释。 <collection> 是待计数集合，返回整数。空字符串作为空集合时返回 0。

用法与结果。集合计数应使用本命令，不能用 llength 计算句柄字符串的长度。命令返回 0 时，先检查名称、层次和筛选条件，不要把空集合直接解释为设计中不存在对应功能。

```
set regs [get_cells -hierarchical * -filter {is_sequential == true}]
if {[sizeof_collection $regs] == 0} {
    error " 未找到时序实例，请检查展开结果与对象属性"
}
```

来源：User Guide，X-2025.06-SP1，第 1098 页。

3.17 compare_collections：比较两个集合是否相等

```
compare_collections <collection1> <collection2> [-order_dependent]
```

参数解释。默认只比较对象成员； -order_dependent 同时要求排列顺序一致。返回值为 0 表示相等，非零表示不同。

用法与结果。返回值并不是“1 表示相等”的布尔接口。若脚本把非零值当作相等，就会反向判断集合变化。可用于检查筛选规则变化是否改变了测试对象范围。

```
set a [get_ports *]
set b [sort_collection $a full_name]
if {[compare_collections $a $b] == 0} {
    puts " 成员相同，排序未改变检查范围"
}
```

来源：User Guide，X-2025.06-SP1，第 1098 页。

3.18 copy_collection：复制集合容器

```
copy_collection <collection>
```

参数解释。返回独立的集合副本。副本中的成员仍是同一设计中的对象，不复制实例、引脚或网络。

用法与结果。普通 `set b $a` 复制的是集合句柄的引用。本命令适合明确要求独立集合容器的场合，但不应误认为复制后拥有第二份可以独立修改的 RTL 设计。

```
set original [get_cells *]
set snapshot [copy_collection $original]
puts [sizeof_collection $snapshot]
```

来源：User Guide，X-2025.06-SP1，第 1099 页。

3.19 index_collection：按下标取得集合成员

```
index_collection <collection> <index>
```

参数解释。`<index>` 从 0 开始。返回指定位置的对象集合。不是所有集合都支持按下标访问，应优先使用集合遍历。

用法与结果。按位置选择对象通常不如按准确名称或属性可靠；展开或排序变化可能改变对象顺序。确需选择第一项时先检查集合非空，并在报告中打印选中对象名称。

```
set ports [sort_collection [get_ports *] full_name]
if {[sizeof_collection $ports] > 0} {
    puts [get_object_name [index_collection $ports 0]]
}
```

来源：User Guide，X-2025.06-SP1，第 1099 页。

3.20 get_attribute：读取对象属性

```
get_attribute <object> <attribute>
```

参数解释。`<object>` 是要查询的对象，`<attribute>` 是大小写敏感的属性名。常见属性包括 `object_class`、`full_name`、`direction`、`ref_name` 和 `is_sequential`，但并非每类对象都支持全部属性。

用法与结果。对象属性描述当前展开模型，不应脱离对象类别解释。例如实例的 `ref_name` 是所引用的设计或库单元，端口的 `direction` 描述顶层接口方向。应用属性通常只读，不应尝试通过改属性代替修改 RTL 或设置验证环境。

```
foreach_in_collection p [get_ports *] {
    puts "[get_attribute $p full_name] : [get_attribute $p direction]"
}
```

来源：User Guide，X-2025.06-SP1，第 1100–1105 页。

3.21 list_attributes：列出某类对象可查询的属性

```
list_attributes -application -class <class>
```

参数解释。`-application` 包含工具应用定义的属性；`-class` 指定对象类别，例如 `cell`、`port`、`pin`、`net`、`design`、`lib_cell`、`lib_pin`。

用法与结果。先查看属性目录，再编写筛选表达式，可以避免把其他工具或其他类别的属性直接套用过来。目录给出属性名和类型；具体对象是否具备某项属性，还应结合该对象的实际查询结果判断。

```
list_attributes -application -class cell
list_attributes -application -class port
```

来源：User Guide，X-2025.06-SP1，第 1100–1101 页。

4. 将查询结果用于验证范围审查

在大规模工程中，查询规则本身也是需要审查的配置。先保留全集，再构造子集，并用计数和名称报告验证筛选范围。只有在确认实例类别和设计层次后，才把集合交给黑盒、连接提取或复杂度分析命令。若集合内混有不同对象类别，属性读取和下游命令必须同时支持这些类别。

```
set all_cells [get_cells -hierarchical *]
set blackboxes [filter_collection $all_cells {is_black_box == true}]
puts " 总实例: [sizeof_collection $all_cells]"
puts " 黑盒实例: [sizeof_collection $blackboxes]"
foreach_in_collection cell $blackboxes {
    puts [get_object_name $cell]
}
```

这段脚本只报告当前模型中的黑盒实例，不证明黑盒模型正确。最终需要把实例清单与功能规格、使用的替代模型以及受其影响的属性一起审查。来源：User Guide，第 1081–1109 页。

Assertion IP 与分解证明

1. 两类复用方式

Assertion IP 复用协议检查知识，Proof Architect 复用经过组织的子证明。前者把接口规则转成属性；后者规定哪些局部证明可以作为其他证明的前提。两者都不能免除环境审查：加载了协议检查器不代表接口角色和时序配置正确，划分了多个子任务也不代表这些任务之间的依赖一定合法。

方法	输入	解决的问题
Assertion IP	协议、接口角色、参数和绑定	减少重复编写标准协议规则
分解证明	子任务、局部结论和依赖关系	用较小证明支持较大结论
独立覆盖检查	场景条件及环境前提	判断任务或协议事务是否真正可达

协议检查器需要区分控制方与响应方。例如一个模块主动发出请求时，对它自己负责的协议关系应生成断言；外部响应方承担的协议条件可以成为环境前提。角色配置颠倒，可能把设计本应保证的规则变成假设，使本来错误的实现被排除在搜索范围之外。

2. 命令语法、参数与用例

命令	主要作用
`aip_load`	预装协议断言库

加载检查库只是准备阶段。仍需将检查器绑定到实际接口，设置协议版本、位宽、角色和合法事务范围，再检查断言、假设、覆盖目标各自的数量。

2.1 `aip_load`：预装协议断言库

```
aip_load [-protocol <protocols>] [-aip_src_path <directory>]
```

参数解释。`-protocol` 支持 APB、AHB、AXI3、AXI4、AXI5、AXI_STREAM、ACE、ACE5、CHI、LPI、GIC、RISCV、I2C、SPI、SPMI、FSCB、all。省略时预装全部支持协议。`-aip_src_path` 指定自定义库目录，否则使用安装包中的 AIP 目录。

用法与结果。该命令在展开前为分析预装库文件，不自动知道 DUT 扮演的协议角色，也不自动完成用户绑定。主设备与从设备需要分别配置哪些规则是环境假设、哪些是 DUT 断言。调试以实际接口信号和协议事件为中心，不必首先深入库的内部辅助信号。

```
set_fml_appmode FPV
aip_load -protocol APB
analyze -format sverilog -vcs "-f rtl_and_bind.f"
elaborate -sva top
```

来源：User Guide，X-2025.06-SP1，第 614–615 页。

3. 组合证明的逻辑要求

假设要证明最终性质 P，先证明局部性质 L，再在第二个任务中以 L 为前提证明 P。只有当 L 在与第二个任务兼容的真实环境下成立，组合才有意义。不能让 “证明 L” 依赖尚未证明的 P，再让 “证明 P” 依赖 L；这会形成互相担保的循环，而不是对原设计的证明。

对情况拆分同样需要完整性。例如把状态分为模式 0、1、2，就必须说明模式 3 为什么不属于合法行为，或者额外建立模式 3 的证明任务。遗漏一种输入情况不能由其余任务全部通过来弥补。保存任务依赖、使用的假设和每个任务状态，与保存最终属性状态同样重要。

4. 命令语法、参数与用例

命令	主要作用
`fv_proof_arch`	创建、生成与审查分解证明策略
`fv_proof_arch_report`	查看分解关系和结果传播安全性

使用 Proof Architect 前，先准备被引用的任务和属性集合。策略报告用于核对任务依赖和组合状态；它不允许把循环依赖或遗漏的情况自动视为已完成证明。

4.1 `fv_proof_arch`：创建、生成与审查分解证明策略

```
fv_proof_arch -name <name> [-type <type>] [-subtype <subtype>]
  [-append] [-generate] [-property <properties>] [-assumes <properties>]
  [-ignore_status] [-unsafe] [-propagate <true|false>]
  [-distribute_max_tasks <n>] [-distribute_max_props <n>]
  [-distribute_custom] [-model_summary] [-model_ref <strategy>]
  [-audit] [-signal <signal>] [-values <values>] [-enumerate <mode>]
```

参数解释。 `-name` 全局唯一且必填。`-type` 可选 copy、assume_guarantee、abstraction、over_constraint、case_split。assume_guarantee 的 `-subtype` 可选 acyclic（默认）、chain、grouped_induction；case_split 默认 subspace，event 另属该版本的 Beta 说明范围。`-append` 添加字段，`-generate` 根据当前定义生成并锁定子任务；`-property` 选择待证明目标，`-assumes` 选择辅助假设。

`-distribute_max_tasks` 限定任务数，`-distribute_max_props` 限定每任务目标数，`-distribute_custom` 允许自定义分配。`-ignore_status` 调整按状态纳入目标的限制；原文描述与相邻默认筛选说明不够一致，使用时应核对生成任务，而不假定所有状态都被接受。`-unsafe` 标为不安全策略；`-propagate` 控制结果是否向父任务传播。`-model_summary` 比较模型，`-model_ref` 指定参考策略，`-audit` 做一致性审查。case_split 用 `-signal`、`-values` 指定分支对象与取值，`-enumerate` 默认 custom。

用法与结果。先设置 `fv_proof_architect true`。假设—保证分解不能让两个未独立成立的目标循环互相假设；chain 中工具禁止这种循环。抽象、增加限制和子空间分解有不同的结果传播规则，因此所有子任务显示成功仍需通过策略完整性审查。

```
set_fml_var fv_proof_architect true
fv_proof_arch -name split_goals -type copy -distribute_max_props 20
fv_proof_arch -name split_goals -property [get_props -usage assert] -generate
fv_proof_arch_report -name split_goals
check_fv -proof_arch split_goals -block
fv_proof_arch -name split_goals -audit
```

来源：User Guide，X-2025.06-SP1，第 805–810 页。

4.2 `fv_proof_arch_report`：查看分解关系和结果传播安全性

```
fv_proof_arch_report [-name <strategy>] [-format <text|xml|collection>]
```

参数解释。 `-name` 限定策略，省略为全部；`-format` 默认 text，也可选择 XML 或工具集合。

用法与结果。报告应检查策略类型、所属任务、Safe、Propagate、锁定状态及子任务中的 assert/assume 配置。Safe 是工具在该策略规则下的审查结果，不免除用户对规格、环境和允许的建模变更

的责任。

```
fv_proof_arch_report -name split_goals
fv_proof_arch_report -format xml
```

来源：User Guide，X-2025.06-SP1，第 810–812 页。

反例调试、证明性能与复杂性控制

1. 为什么形式验证会困难

大设计的原始状态空间往往呈指数增长。真正决定计算规模的不是 RTL 总行数，而是某一条属性需要考虑的状态、时钟、存储结构、算术逻辑、输入自由度和时间深度。极大规模的 SoC 中，一个受局部状态机约束的输出属性可能很容易，而一个 64 位除法运算或需要跨上千周期的罕见序列可能十分困难。

这意味着：**先理解问题与所需观察状态，再投入更多机器资源**。

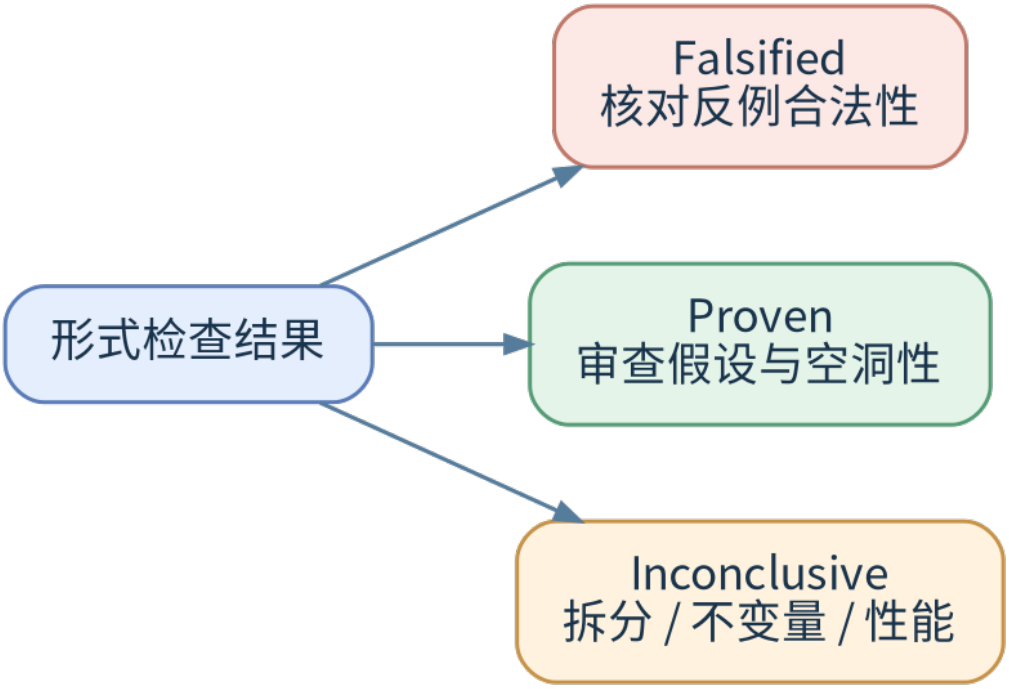


图 15: 失败、空洞与未完成的分流

2. 报告状态决定不同的调试路径

状态	首先需要回答的问题	典型措施
Falsified	是否是真正可实现的反例	查看波形、复位、合法输入和抽象
Proven	规则是否完整且非空洞	`cover` 触发、审查假设及性质范围
Inconclusive	计算难点在哪里	复杂性报告、拆分、抽象、策略调整
Uncoverable	目标为什么无法到达	审查环境约束与初始状态
No properties	属性是否编译和启用	检查 `-sva`、`bind`、模式和任务

先按结果类型分流，再决定是否调整资源。明确失败应先分析反例；空洞通过和约束冲突应先修正模型；只有在环境可信而目标仍未完成时，才进入复杂度、抽象和证明策略分析。对所有问题统一增加运行时间，既浪费资源，也可能延迟发现建模错误。

3. 失败属性：逐周期回溯

```
report_fv -list
report_fv -verbose
view_trace -property <actual_property_name>
```

波形中应同时查看触发输入、内部状态、相关有效标志、输出及复位；不能只截取失败周期最后一帧。回溯时先从断言后件不满足的位置定位，然后反推输入是否启动了前件检查。许多“错误”属于 SVA 错把 `|=>` 写成 `|->`、忽略非阻塞赋值采样、没有在 `$past` 前验证历史有效等问题。

3.1 错误类型诊断表

典型反例	建议核查
复位刚释放就失败	复位模型、SVA 失效条件、前一拍采样
外部输入给出非法值	协议是否允许；合法约束是否缺失
某内部寄存器突然跳变	是否黑盒、抽象或部分初始状态任意
只有一条复杂断言失败	拆成数条检查以确定错误具体阶段
重试之后“失败消失”	模型或约束是否变化，是否仅换了证明任务

调试应沿时间顺序定位第一次偏离规格的状态，而不是只看最后一拍断言变红。将导致该状态的输入、前一拍寄存器值、复位和使能条件同时展开，再判断错误属于 RTL、属性、环境还是抽象。修改时只改变已定位的原因，并保留原始反例用于回归。

4. 约束检查：最应优先防止的错误

形式环境常见问题是**过度约束**：为了让属性收敛，把本应由 DUT 产生的某个输出/内部状态假设为正确；或者把关键请求置为恒零，使断言从不触发。另一类是**约束不足**：允许非法输入状态，导致反例不符合系统协议。

对每个 assume 保存三个信息：自然语言前提、规格或接口出处、可以用于反驳不合理假设的覆盖目标。跨时钟约束尤其要区别输入保持、边沿采样和真实时钟独立性。若存在多组假设，可分组关闭并比较覆盖目标的可达性。

5. 查找状态空间问题

```
report_fv_complexity
report_fml_engines
report_fml_jobs
report_fml_hosts
```

复杂性报告帮助识别大影响锥和特殊数据结构。面对大型存储器，可以考虑：将具体数据证明缩减为一个或少量符号地址、抽象未直接参与控制的 RAM 内容、将读写控制与数据正确性分别验证；但必须清楚抽象是否覆盖真实冲突规则和初始化语义。面对大型算术逻辑，可能需要拆分操作码、证明进位或截断的局部不变量。

6. 改善证明性能的合理顺序

1. 首先排除环境错误和空洞的属性。
2. 查找关键的状态和输入，拆分长时序规则。
3. 增加正确的辅助不变量，而不是任意假设结论。
4. 按功能建立独立任务 `fvtask`，先单独证明基础性质。
5. 选择合适的引擎努力程度与运行时间，例如 `set_fml_var fml_effort high`。

6. 对仍存在规模问题的属性评估抽象/分解；出现新反例时回原始设计验证其可行性。
7. 根据主机内存和许可证数配置网格，并对比实际加速效果。

7. fvtask：按性质组织工作

```
fvtask -create reset_task
fvtask -create datapath_task
# 选择工作任务
fvtask reset_task
get_fvtask
```

不同应用对复制任务或传递属性集合的选项支持可能不同，实际可使用 `fvtask new_task -copy source_task` 或查看 `help fvtask`。为了便于回归，不应在不记录的情况下频繁切换任务并把不同任务的结果拼成一份“同一配置完成”的报告。

8. 抽象及裁剪驱动器

```
# 进阶操作，仅在确认目标和影响后使用
snip_driver u_dut.internal_net
set_abstractions -construct {mult=16 count=4}
report_abstractions
```

裁剪部分网络的驱动或用抽象代替复杂数据结构后，原设计的行为集合可能改变。这可能加速求解，但也可能出现物理上不可能的反例或把某些条件与设计脱离。任何用于最终证明的抽象都需要解释其正确性及覆盖范围。

9. 困难证明的辅助分析

指令	说明
`compute_reduced_constraints -property ...`	分析对某条证明真正相关的约束子集
`report_reduced_constraints`	输出与证明有关的约束信息
`compute_formal_core -property ...`	分析证明关注的关键逻辑部分
`report_formal_core`	输出核心逻辑关系
`compute_proof_assist -property ...`	获取部分证明辅助信息
`report_proof_assist`	输出辅助结果
`fvlearn_config -local_dir ...`	按版本配置回归加速数据

这些是深入调优工具，而不是新手应默认启用的选项；应先明确每次调优对结果是否有实质影响。

10. Proof Architect：组合式证明思路

当一个大型属性难以一次性证明时，可以选择辅助不变量、局部切分和多阶段验证。组合式证明（Compositional Verification）思路是：将大问题分解成较容易的子目标，在逻辑上确保子目标联合能支持原结论。Proof Architect 是 VC Formal 提供的相关能力之一，详见 User Guide 第 19 章。

正确分解需要保证：局部证明使用的假设可由其他模块实际保证；模块间不会形成未经处理的循环假设；最终合成结论对应原始设计。这不是“把大模块拆成几份后，每份单独通过就自动保证整体”。

11. 有规律地建立证明性能实验

每次实验应只改变一到两个有解释的因素。保存：属性数量、最长/最难属性、证明状态分布、CPU 时间、总运行时间、峰值内存、worker 数、工具版本、输入哈希值。面对证明效率提升，要区分是确实证明了更多属性，还是把一部分属性禁用或过度约束造成的虚假改善。

12. 命令语法、参数与用例

命令	主要作用
`fvtrace_config`	配置反例中不确定来源的检查
`fvtrace_check`	检查已有轨迹中的初始化与未定义行为
`fvtrace_report`	报告轨迹来源分析
`view_trace`	打开属性或内部等价点的波形
`compute_reduced_constraints`	寻找某个证明实际需要的较小约束集合
`report_reduced_constraints`	显示证明依赖的约束
`get_reduced_constraints`	返回较小约束集合供脚本处理
`report_fv_complexity`	分析影响属性的设计结构
`get_fv_complexity`	返回影响范围中的结构集合
`compare_coi`	比较两个属性或集合的影响范围
`is_in_fanin`	查询指定信号是否在目标扇入中
`set_abstractions`	用较简单模型代替复杂结构
`report_abstractions`	核查实际生效的抽象
`compute_proof_assist`	定位证明代价随深度增长的原因
`report_proof_assist`	解释性能分析与建议
`bug_hunting_config`	配置深层错误搜索
`report_fml_bug_hunting`	读取错误搜索日志
`fvorder`	为错误搜索提供辅助目标顺序
`check_constraints`	检查环境约束之间的冲突
`report_constraints`	列出约束检查结果和冲突路径
`fvtrace`	把已有轨迹导出为 FSDB
`snip_driver`	切断内部网络的驱动关系

先检查约束和反例，再检查结构复杂度，最后才改变抽象或求解策略。诊断命令大多返回报告或对象集合，不直接形成证明结论。配置命令会改变后续计算方式；比较优化前后的结果时，必须保留相同的设计、假设和属性范围。

12.1 `fvtrace_config`：配置反例中不确定来源的检查

```
fvtrace_config [-check <checks>] [-type <types>]
  [-autorun <checkpoint>] [-severity <severity>]
  [-verbosity <summary|verbose>] [-enable] [-disable]
```

参数解释。 `-check` 选择 initialstate、undriven、aepnode、loop_cutpoint 等检查类别；`-type` 进一步选择子类，例如未初始化的 design/sampled、未驱动的 usersnip/userblackbox、未定义行为的 bound/explicitx/divbyzero。省略类别或子类时选择适用的全部项。`-severity` 设置严重程度，`-verbosity` 控制输出详略；`-enable`、`-disable` 控制指定检查。

用法与结果。 `-autorun` 在打开轨迹时自动检查，none 关闭。原文定义提到 view_trace，示例却写 viewtrace，存在拼写差异；为避免依赖该差异，以下示例使用显式 `fvtrace_check`。该分析揭示反例依赖哪些不确定来源，不等于自动判定反例为设计错误。

```
fvtrace_config -check initialstate -severity low -enable
fvtrace_config -check aepnode -severity high -enable
fvtrace_check p_data_correct
```

来源：User Guide，X-2025.06-SP1，第 168–169 页。

12.2 `fvtrace_check`：检查已有轨迹中的初始化与未定义行为

```
fvtrace_check <property_name> [-check <checks>] [-type <types>]
              [-vacuity] [-witness] [-clear] [-quiet]
```

参数解释。属性必须已经有可用轨迹。`-check`、`-type` 限制检查范围，省略时使用已启用的类别；`-vacuity`、`-witness` 选择对应辅助轨迹；`-clear` 清除该轨迹的分析结果；`-quiet` 抑制执行时输出。

用法与结果。未初始化寄存器、被切断的驱动器、黑盒输出、数组越界和显式 X 都可能贡献自由选择。检查应围绕真正影响失败的来源进行，而不是看到任何 X 就把反例忽略。

```
fvtrace_check p_data_correct -check {initialstate undriven aepnode}
fvtrace_report p_data_correct -list
```

来源：User Guide，X-2025.06-SP1，第 169–170 页。

12.3 `fvtrace_report`：报告轨迹来源分析

```
fvtrace_report [<property_name>] [-vacuity] [-witness]
  [-check <checks>] [-type <types>] [-list] [-limit <n>]
  [-xml] [-config] [-no_summary]
```

参数解释。属性和 `-vacuity`/`-witness` 确定轨迹；`-check`、`-type` 筛选类别。`-list` 展开检查项，`-limit` 限制对象数量，`-xml` 输出 XML；`-config` 显示配置而不是结果；`-no_summary` 隐藏汇总。

用法与结果。 UG 的语法行还出现 `-noSummary` 和缺少横线的 xml，参数解释则明确使用 `-no_summary`、`-xml`；本教程采用参数解释的形式，不重复排版错误。报告之前应已运行 `fvtrace_check`。

```
fvtrace_report p_data_correct -list -limit 50
fvtrace_report -config
```

来源：User Guide，X-2025.06-SP1，第 170 页。

12.4 `view_trace`：打开属性或内部等价点的波形

```
view_trace <property_name> [-rcfile <signal_group_file>]
view_trace -inteq <internal_equivalence_id>
view_trace -deadendID <deadend_id>
```

参数解释。普通属性形式查看对应反例或轨迹；`-rcfile` 复用信号分组设置。`-inteq` 选择 SEQ 内部等价点编号；`-deadendID` 选择约束分析报告中的死路编号。这里汇总 UG 在不同章节展示的调用形式，而非臆造统一的完整平台语法。

用法与结果。死路由约束导致没有合法后继，与 RTL 死锁、活锁不是同一概念。查看死路波形时工具还会显示最后合法状态之后的一段非法变化，用于解释是哪条限制阻断后继；不要把这一段直接作为合法设计反例。

```
view_trace p_data_correct
view_trace p_data_correct -rcfile data_check.rc
# 已获得内部等价编号时可另行使用 view_trace -inteq 3。
```

来源：User Guide，X-2025.06-SP1，第 279，302，565 页。

12.5 `compute_reduced_constraints`：寻找某个证明实际需要的较小约束集合

```
compute_reduced_constraints -property <properties>
  [-subtype <vacuity|witness>] [-block] [-stop]
```

参数解释。`-property` 选择已有证明的属性；`-subtype` 可改为分析空洞性或 witness 组件；`-block` 等待完成，`-stop` 停止当前计算。

用法与结果。结果是在保留该证明时可使用的较小集合，不保证全局最少约束。`set_constant` 和 `fvassume -env` 不在此分析支持范围，因此“报告没有需要的约束”也不意味着问题完全无约束。发现某限制导致空洞后，应回到规格审查，而不是直接把剩余限制都删掉。

```
compute_reduced_constraints -property p_data_correct -block
report_reduced_constraints -property p_data_correct
```

来源：User Guide，X-2025.06-SP1，第 303–304 页。

12.6 `report_reduced_constraints`：显示证明依赖的约束

```
report_reduced_constraints [-property <properties>]
                           [-subtype <vacuity|witness>]
```

参数解释。选择属性和可选子类型，报告先前计算得到的约束集合。

用法与结果。该报告说明当前计算中保留了哪些普通约束，不说明每条都唯一必要。应与环境约束、常量和初始化脚本共同审查，避免遗漏隐式限制。

```
report_reduced_constraints -property p_data_correct
report_reduced_constraints -property p_data_correct -subtype vacuity
```

来源：User Guide，X-2025.06-SP1，第 303–304 页。

12.7 `get_reduced_constraints`：返回较小约束集合供脚本处理

```
get_reduced_constraints [-property <properties>]
```

参数解释。给出属性时返回对应集合；省略时返回已有结果的所有属性之约束并集。

用法与结果。集合可用于统计或审查，但不应直接在原任务中批量禁用集合之外的约束并沿用旧结论。做删减实验应复制任务并重跑证明，明确新结论适用的环境。

```
set needed [get_reduced_constraints -property p_data_correct]
query_objects $needed
```

来源：User Guide，X-2025.06-SP1，第 304 页。

12.8 `report_fv_complexity`：分析影响属性的设计结构

```
report_fv_complexity [-stopif <expression>] [-stops <objects>]
  [-list] [-verbose] [-col] [-limit <n>] [-level <n>] [-xml <file>]
  [-filter <expression>] [-regexp] [-nocase] [-bit]
  [-disable_abstraction] [-design] [-instrumented] [-include <types>]
  [-complement] [-property <properties>] [-output_mode <mapped|unmapped>]
  [-subtype <vacuity|property|witness|all>]
  [-usage <assert|assume|cover|all>] [<objects>]
```

参数解释。`-property` 或位置对象指定起点；多个起点按影响集合并集统计。`-stops` 指定停止对象，`-stopif` 按属性表达式停止搜索。`-list` 展开对象，`-verbose` 增加位宽和源码位置，`-col` 返回集合；`-limit` 默认报告 100 项，-1 不限制，作用于这些展开形式。`-level` 只用于按层次迭代的报告，不能与显式起点混用。`-xml` 保存结构化报告，`-gui` 是原文另列的图形显示开关。

`-filter` 筛选，`-regexp` 和 `-nocase` 控制模式匹配。默认按字统计，`-bit` 更精确但更慢。`-design` 默认包含设计对象，`-instrumented` 包含工具插装对象，`-include` 可纳入 leaf_module、constraint、latch_loop、combo_loop。`-complement` 查看影响范围之外的对象，`-disable_abstraction` 关闭复杂算子抽象以检查真实结构。`-output_mode` 仅适用于 SEQ；`-subtype` 默认 property，`-usage` 默认 all。

用法与结果。影响范围大说明可能困难，但不是证明所需逻辑的精确集合。结构识别依赖 RTL 和综合形

式：门级搭出的加法器不一定被识别为加法算子。调优时应把复杂度与实际求解时间、深度和 formal core 一起看。

```
report_fv_complexity -property p_data_correct -verbose -limit 100
report_fv_complexity -property p_data_correct -bit
report_fv_complexity -level 1 -xml complexity.xml
```

来源：User Guide，X-2025.06-SP1，第 719–734 页。

12.9 `get_fv_complexity`：返回影响范围中的结构集合

```
get_fv_complexity [-stopif <expression>] [-stops <nets>]
  [-list] [-verbose] [-limit <n>] [-level <n>]
  [-filter <expression>] [-regexp] [-nocase] [-bit]
  [-disable_abstraction] [-design] [-instrumented] [-include <types>]
  [-complement] [-output_mode <mapped|unmapped>]
  [-property <list-or-collection-of-properties>]
  [-subtype <vacuity|property|witness|all>]
  [-usage <assert|assume|cover|all>] [<objects>]
```

参数解释。筛选、层次、位级、抽象和对象种类与复杂度报告中的相应概念一致。这里的 `-limit` 默认不限制，而 `report_fv_complexity` 展开报告默认 100 项，不能按同一个默认值解释。`-level` 同样不能与显式起点混用。`-property` 指定属性列表或集合；`-subtype` 选择属性本体、空通过目标或见证目标，默认 `property`；`-usage` 选择断言、假设、覆盖或全部用途，默认 `all`。末尾对象可以是名称、编号或集合。

用法与结果。返回集合用于自动化分析；需要纯文字报告时使用 report 命令。对象返回数量变化还可能来自位级/字级选择和工具插装对象是否包含，比较两次实验前应固定这些设置。

```
set cone [get_fv_complexity p_data_correct]
query_objects $cone
```

来源：User Guide，X-2025.06-SP1，第 727–731 页。

12.10 `compare_coi`：比较两个属性或集合的影响范围

```
compare_coi [-prop1 <property>] [-set1 <collection>]
  [-props <properties>] [-set2 <collection>] [-prop2 <property>]
  [-list] [-verbose] [-col] [-bit] [-limit <n>] [-filter <expression>]
```

参数解释。第一侧用 `-prop1` 或 `-set1`；第二侧用 `-props` 比较一组属性，或用 `-prop2`、`-set2` 比较单个对象/集合。`-list`、`-verbose`、`-col` 决定显示或返回方式，`-bit` 使用位级分析；`-limit` 默认 1000，-1 不限制，`-filter` 进一步筛选。

用法与结果。用于判断共享逻辑、包含关系和互不相交的结构范围，帮助选择任务分组或解释某约束为什么可能影响多个断言。影响范围相交不等于逻辑上必要依赖，仍需结合实际证明或较小约束集合分析。

```
compare_coi -prop1 p_data_correct -props [get_props -usage assume]
compare_coi -prop1 p_data_correct -prop2 p_count_range -list
```

来源：User Guide，X-2025.06-SP1，第 731–733 页。

12.11 `is_in_fanin`：查询指定信号是否在目标扇入中

```
is_in_fanin [-signals <signals>] -lookUpSignals <lookup_signals>
  [-property <properties>] [-seqDepth <depth>] [-col] [-bit]
  [-coi_match <any|all>] [-file <filename>]
```

参数解释。`-signals` 或 `-property` 指定搜索起点，`-lookUpSignals` 是必须寻找的信号。`-seqDepth` 限定时序深度，`-bit` 位级查找，`-col` 返回找到的信号集合。`-coi_match any` 默认只需在任一起点扇入出现，all 要求在所有起点都出现；`-file` 保存报告。

用法与结果。适合确认属性是否真正关联目标数据、复位或控制信号。信号不在结构扇入中常提示检查绑定错误或规格漏写，但信号在扇入中也不保证被充分约束或验证。

```
is_in_fanin -property p_data_correct \
  -lookUpSignals {u_dut.write_data u_dut.write_enable}
is_in_fanin -signals u_dut.result -lookUpSignals rst_n -seqDepth 2
```

来源：User Guide，X-2025.06-SP1，第 733–734 页。

12.12 `set_abstractions`：用较简单模型代替复杂结构

```
set_abstractions -construct <construct_thresholds>
  [-using <abstraction_model>] [-filter <expression>] [-remove] [-global]
```

参数解释。`-construct` 指定结构及尺寸阈值，例如 `{mult=8 div=8 count=8}`；常见结构还包括 add、sub、mod、array、vector、array_comb、vector_comb。默认 snip 用自由变量代替复杂结果。`-using keyvals` 将计数器分成关键值区间；`+linearize` 保留递增/递减方向，仍允许区间内跨步。可增加用户关键值，或使用 user/range 相关组合；具体组合以本章展示的支持形式为限。`-filter` 限定对象，`-remove` 撤销，`-global` 应用于所有任务。

用法与结果。必须在读入设计之后使用，因此主要改善形式模型和求解，不减少 RTL 读入时间。保守抽象扩大行为集合：证明成功通常可支持原问题，但失败可能是抽象引入的伪反例。添加约束精化抽象时，必须保证没有删除原设计的合法行为。

```
set_abstractions -construct {mult=16}
set_abstractions -construct count=4 -using keyvals+linearize
report_abstractions -verbose
# 对照实验需要恢复时：
set_abstractions -construct {mult=16} -remove
```

来源：User Guide，X-2025.06-SP1，第 744–749 页。

12.13 `report_abstractions`：核查实际生效的抽象

```
report_abstractions [-list] [-verbose] [-construct <construct_thresholds>]
```

参数解释。无参数汇总已抽象结构数量；`-list` 给出结构、尺寸、模型和被驱动网络；`-verbose` 进一步列出输入、关键值等；`-construct` 只报告指定类型与阈值。

用法与结果。该报告用于确认抽象是否真正命中目标结构。计数器位宽较小、结构未被识别或过滤名称不对，都可能使设置未起预期作用。返回 1 表示报告命令成功，不是形式证明通过。

```
report_abstractions
report_abstractions -list
report_abstractions -verbose -construct mult=16
```

来源：User Guide，X-2025.06-SP1，第 748–750 页。

12.14 `compute_proof_assist`：定位证明代价随深度增长的原因

```
compute_proof_assist [-block] [-stop] [-property <properties>]
  [-depth <n>] [-minimize <regs>] [-profiler <p0|p1>]
  [-ignore_status] [-live_update <core|depth|all>] [-config <settings>]
```

参数解释。`-property` 选择目标；`-depth` 默认 1000，通常需要目标已有至少一层检查结果，未完成或失败属性所选深度应与已有结果相容。`-ignore_status` 允许不按原状态限制启动分析。`-minimize regs` 尝试减少核心寄存器；p0 收集核心、深度和时间，p1 还收集求解工作量且为默认。`-live_update` 选择实时更新内容。`-config` 用 `{profiler=p1,depth=100}` 设置后续默认值，空列表查询当前值。

用法与结果。此功能在该版本明确支持 FPV 和 FRV。它是性能诊断，不是把未完成属性直接转成证明；建议先看哪一深度引入了存储器、长计数或昂贵属性逻辑，再决定改写还是抽象。

```
compute_proof_assist -property p_data_correct \
  -depth 20 -profiler p1 -ignore_status -block
report_proof_assist -last_depth -verbose -advisor
```

来源：User Guide，X-2025.06-SP1，第 752–754 页。

12.15 report_proof_assist：解释性能分析与建议

```
report_proof_assist [-xml] [-verbose] [-depth <n>]
  [-constraints] [-comb] [-bit] [-target <registers>] [-last_depth]
  [-top_signals <n>] [-last_cores <n>] [-explicit_cycles]
  [-property <property>] [-coi] [-note_truncation <on|off>]
  [-advisor] [-advisor_level <0|1|2|3>] [-report_cutoff <n>]
  [-engine <engine>] [-model <model>] [-intent <intent>]
  [-mode <mode>] [-config <settings>]
```

参数解释。`-depth`、`-last_depth`、`-last_cores` 限定深度或核心序列，`-explicit_cycles` 不合并相似周期。`-constraints` 包含约束，`-comb` 包含组合逻辑，`-bit` 位级显示；`-target` 聚焦寄存器，`-top_signals` 和 `-report_cutoff` 限制显示数量。`-coi` 还显示指定深度内不在核心中的影响对象。`-xml`、`-verbose` 控制输出格式和细节。

`-advisor` 添加建议；`-note_truncation` 默认 on，按原文截到 80 字符。`-advisor_level` 是两位开关，最低位控制复杂度说明，第二位控制下一步 Tcl 建议，默认 2。`-engine` 可用 all、deepest 或 fc1–fc5，默认 deepest；`-model` 可用 all、def、deepest 或模型名。`-intent` 为 bug-hunting、coverage、proof、all，默认 all；`-mode` 为 core-buildup 或 comparative。`-config` 支持相应默认设置，空列表只查询。

用法与结果。最高代价信号只是候选原因。若昂贵部分主要来自属性本身，应先检查不必要的深层 `$past`、复杂匹配或参考模型；若来自真实数据通路，再考虑保持规格正确性的拆分和抽象。不要未经复核直接执行自动建议中的限制。

```
report_proof_assist -last_depth -verbose -comb -constraints
report_proof_assist -advisor -advisor_level 3 -intent proof
report_proof_assist -mode comparative -depth 20 -engine all
```

来源：User Guide，X-2025.06-SP1，第 754–756 页。

12.16 bug_hunting_config：配置深层错误搜索

```
bug_hunting_config [-random <0|1>] [-saveDir <directory>]
```

参数解释。`-random` 默认 0，开启后不同运行可能有不同搜索结果；`-saveDir` 保存配置日志，原文说明该日志由 s7 引擎写出，默认不保存。

用法与结果。先将 `fml_effort` 设为 bug_hunting，再配置。错误搜索有助于找到深层反例，但在资源限制内未找到反例不等于完整证明。固定随机设置和日志目录可以帮助对比搜索实验。

```
set_fml_var fml_effort bug_hunting
bug_hunting_config -random 0 -saveDir bh_logs
check_fv -block
```

来源：User Guide，X-2025.06-SP1，第 240–241 页。

12.17 report_fml_bug_hunting：读取错误搜索日志

```
report_fml_bug_hunting [-jobId <id>] [-status <0|1>]
                       [-nlines <n>] [-pos <head|tail|all>]
```

参数解释。`-jobId` 选择作业，省略为全部；`-status` 默认 1；`-nlines` 指定显示行数，默认 0；`-pos` 默认 tail，可选择开头、末尾或全部。

用法与结果。只有配置了日志保存才有相应内容可读。报告用于理解搜索进展，不应把“作业结束”当

作属性 proven。

```
report_fml_bug_hunting -nlines 40 -pos tail
```

来源：User Guide，X-2025.06-SP1，第 241 页。

12.18 `fvorder`：为错误搜索提供辅助目标顺序

```
fvorder [-clear] [-print] <properties>
```

参数解释。位置参数为属性名称、编号或集合；多次调用增量添加偏序关系。`-clear` 删除当前任务的顺序，`-print` 打印合成后的顺序信息。

用法与结果。只支持 bug hunting 中的 assert 和 cover，不允许循环顺序。工具将其作为搜索引导，并非每次严格遵守；它不建立逻辑假设，也不保证后一个目标能由前一个推出。

```
set_fml_var fml_effort bug_hunting
fvorder {c_init_done c_first_request}
fvorder {c_first_request c_full_fifo}
fvorder {c_full_fifo p_no_data_loss} -print
```

来源：User Guide，X-2025.06-SP1，第 242 页。

12.19 `check_constraints`：检查环境约束之间的冲突

```
check_constraints [-block] [-ccDepth <depth>]
```

参数解释。`-block` 等待检查结束；`-ccDepth` 指定该次约束检查的展开深度。此处是 UG 给出的调用形式，不把它扩展成未提供的完整平台命令定义。

用法与结果。约束冲突意味着环境在某个初始状态或某段执行后没有合法的后续行为。即使没有用户断言，也可以执行约束检查。有限深度内没有发现冲突不代表所有无限执行都存在；约束造成的无后续路径也不能直接当作 RTL 状态机死锁。

```
check_constraints -block -ccDepth 4
report_constraints -list -verbose
```

来源：User Guide，X-2025.06-SP1，第 298–300 页。

12.20 `report_constraints`：列出约束检查结果和冲突路径

```
report_constraints [-list] [-verbose] [-no_summary]
                   [-allDeadends] [-deadendId <id>] [-dict]
```

参数解释。`-list` 列出问题，`-verbose` 增加详细信息，`-no_summary` 不输出汇总。默认按代表性冲突归类，`-allDeadends` 列出全部已发现的无后续路径；`-deadendId` 选定其中一项。`-dict` 返回适合 Tcl 处理的字典。

用法与结果。字典字段包含 `cc_status`、`cc_depth`、`cc_constraints` 以及路径数量和长度信息 `de_num`、`de_len_min`、`de_len_max`、`de_len_avg`。检查脚本应读取状态及检查深度，不应仅以日志没有报错判断环境可满足。

```
report_constraints -list -verbose
set result [report_constraints -dict]
puts $result
```

来源：User Guide，X-2025.06-SP1，第 299–300 页。

12.21 `fvtrace`：把已有轨迹导出为 FSDB

```
fvtrace [-reset] [-property <propName>] [-vacuity <propName>]
        [-witness <propName>] [-inteq <id>] [-file <fileName>]
        [-composite] [<propName>]
```

参数解释。 `-reset` 导出初始化复位轨迹；`-property` 导出属性轨迹；`-vacuity` 和 `-witness` 分别选择空通过分析轨迹和见证轨迹；`-inteq` 选择 SEQ 内部等价项。`-file` 指定输出文件名，默认使用属性名和时间戳。`-composite` 合并复位轨迹与属性轨迹。

用法与结果。命令导出已生成的轨迹，不会把没有反例或见证的属性自动变成可导出的波形。需要启用 Verdi 调试，并保存相应的编译设计。组合轨迹默认不受支持；UG 要求在启动会话前设置环境变量 `SNPS_VCST_DISABLE_VF=1`。不能在普通示例中忽略这项前置条件。

```
set_app_var fml_mode_on true
set_app_var enable_verdi_debug true
set_app_var verdi_export_dir ./debug_export
fvtrace -property top.ap_count -file count_failure.fsdb
```

来源：User Guide，X-2025.06-SP1，第 305 页。

12.22 `snip_driver`：切断内部网络的驱动关系

```
snip_driver <net>
```

参数解释。 `<net>` 是展开设计中的网络。这里列出 UG 与快速参考中出现的基本调用，不据此推测完整平台选项。切断驱动后，下游不再由原有逻辑唯一确定该网络的值。

用法与结果。切断通常扩大下游可见的行为集合，可能使证明更简单，也可能出现原始设计不可能产生的反例。对组合过程内“先赋值、后读取”的信号，UG 还提示可能产生 RAW_SIGNALS_FOUND 告警并影响结果；应检查 `raw_signals` 任务属性，而不是忽略告警。切断前保存原任务，完成后审查相关属性及约束是否仍对应规格。

```
fvtask -create cutpoint_experiment -copy FPV
snip_driver u_datapath.result
puts [get_attribute [get_fvtask] raw_signals]
check_fv -block
report_fv -verbose
```

来源：User Guide，X-2025.06-SP1，第 121–122、323；Quick Reference Guide 第 8 页页。

13. 官方参考

User Guide 第 6 ~ 8、18、19 章，尤其第 714 页起的复杂性分析和第 788 页起的组合式证明；Quick Reference Guide 通用性能部分。

覆盖率评估、验证质量与结果验收

1. 为什么不能只报告“全部属性通过”

形式验证的前提之一是必须有人编写有意义的属性。若一个仲裁器只检查“不存在两路同时授权”，这条性质即使被完整证明，也无法证明“有请求时必须给予授权”。因此验证验收需同时审查三个层面：属性是否正确、属性是否涵盖规格、证明是否完成。

1.1 四种互补的检查手段

方法	回答的问题	局限性
断言证明（FPV）	已编写规则是否永不违反	无法覆盖未编写的规格
触发/行为覆盖（SVA cover）	关键事件是否可达	一条路径可达不代表全部情况正确
代码覆盖与可达性（FCA）	未覆盖代码是否真的不可达	模型与约束可能过度限制
属性敏感性（FTA）	注入错误能否被断言检测	故障集合有限、求解可能未完成

断言证明、可达性分析、属性对设计的关联分析以及故障敏感性分析，对应不同的验证盲点。断言全部通过时，仍应确认关键场景实际可达、关键寄存器受到约束，以及设计行为被有意义地改变后断言会失败。任何单一指标都不足以替代规格评审。

结合这些方法可以发现验证薄弱区域，但不能把它们的百分比相加为“验证完成度”。

2. 建立规格—属性对应表

规格项	属性名称	覆盖目标	输入前提
FIFO 上溢保护	`AP_FIFO_DEPTH`	`CP_FIFO_FULL`	`push` 为请求
请求应答最多 3 拍	`AP_REQ_ACK`	`CP_REQ_ACCEPT`	合法请求被受理
复位后中断清零	`AP_RST_IRQ`	`CP_RESET_RELEASE`	正确复位时序

实际工程中，一条规格可以对应多条断言和多个覆盖目标。记录应保留这种一对多关系，并为每条前提注明依据。比如 FIFO 的满保护既需要检查计数不超范围，也需要检查满且没有同时读出时写指针不移动；“FIFO 已经到满”则是独立的可达性目标，不应拿断言证明状态替代。

记录还应包含：类型、证明状态、评审状态。本例中的运行状态和评审状态均为待完成，不能作为实际验证结果。

这是交付模板，不是某次工具运行产生的真实状态。真实报告不应把未运行的规则写成 `Proven`。当规格中某项只能通过仿真、人工审查或系统测试验证时，要明确标注验证方式，而非制造一个看似覆盖它的空洞断言。

3. 验收需要保留哪些文件

```
release/
├── manifest.yaml          # 设计与工具版本、校验和、资源配置
├── requirements.csv       # 规格与属性对应
├── assumptions.md         # 环境约束的来源和证明影响
├── properties.sv          # 提交时实际编译的 SVA
├── run.tcl                # 运行与初始化脚本
```

```
├── report_fv.txt          # 原始工具输出
├── summary.csv            # 属性状态汇总
├── waivers.md             # 豁免理由与审批
└── known_limits.md        # 未完成目标与后续验证计划
```

使用工具本身的 `save_session` 可保存调试数据，但不能替代可读的人类审核记录。运行结果需保留对应的源码版本和相关中间配置，避免以后找不到相同的证明环境。

4. 对“Proven”的独立复核

每条关键属性应检查：前件是否在某条合法路径上发生；`disable iff` 是否意外关闭主要检查阶段；假设有没有直接限制被验证的 DUT 内部状态；证明对象是否因黑盒化而改变；目标时钟和复位是否正确。建议将首次建立的属性在仿真中加入一两个故意错误版本，确认它确实能够检测到预期缺陷。

5. 不完成或豁免的处理要求

情况	是否允许直接归类“通过”	必须补充的信息
`Falsified`	否	反例分析、RTL/SVA/约束归因
`Inconclusive`	否	当前资源、证明难点、替代验证方案
`Disabled`	否	禁用依据及是否存在未覆盖规格
`Waived`	不能自动等同于通过	责任人、规格依据、审核和到期条件
`Uncoverable`	不是功能证明	约束可达性及覆盖目标定义

豁免是带理由保留的例外，不是对证明状态的改名。不可达目标应记录约束条件和证明证据；未完成目标应记录分析深度、资源限制及剩余风险；明确不属于当前产品配置的目标，应记录配置依据。修改 RTL、规格或约束后，原豁免必须重新评估。

6. 组合使用 FCA 与 FTA

FCA 可优先帮助识别没有在正常运行配置下被触发的代码或状态；FTA 可进一步研究即便某区域可达，目前的断言是否能够检测改变。比如一个状态机的两个分支都可达，但错误地交换两路输出后当前断言仍不能发现，这显示需要更有针对性的输出行为属性。

7. 质量评审问题

1. 验证的 DUT 顶层和参数是否就是准备交付的版本?
2. 所有必检属性数量与规格表是否一致?
3. 属性是否均具有时钟、复位和输入前提说明?
4. 有无过度约束或从未触发的关键断言?
5. 所有失败与未完成是否已分类处理?
6. 自动生成的属性、覆盖目标及豁免是否经过审核?
7. 更改初始化脚本或工具版本后是否重新运行?
8. 有无保留完整的报告、运行命令及可重现文件?

8. 自动化结果摘要

如果需要脚本自动解析 `report_fv`，应优先采用该工具版本明确支持的机器可读输出，避免仅靠模糊匹配报告文本中的 `pass` 或 `fail`；结果里可能同时包含不同应用下的覆盖和断言状态。发布检查脚本 `脚本与工具/validate_publication.py` 检查文件结构、引用路径和代码示例，不是 VC Formal 报

告解析器。

9. 命令语法、参数与用例

命令	主要作用
compute_formal_core	计算某个证明使用的形式核心
report_formal_core	报告证明涉及和未涉及的设计对象
get_formal_core	返回形式核心对象集合
snip_formal_core_complement	创建切断核心外部寄存器的诊断任务
check_config	在证明过程中同步计算形式核心

形式核心描述证明实际依赖的结构，覆盖与验收仍须结合规格责任、环境合理性和不可达原因。没有进入证明核心的逻辑不自动等同于无用逻辑；只有明确分析对象与条件，结果才能用于判断属性集是否存在遗漏。

9.1 compute_formal_core：计算某个证明使用的形式核心

```
compute_formal_core [-block] [-stop] [-property <properties>]
  [-subtype <vacuity|witness>] [-depth <n>]
  [-goalIds <ids>] [-proof <proof_name>] [-minimize <regs>]
```

参数解释。 -property 选择属性，-subtype 选择辅助组件，-depth 请求有界核心。-goalIds 与 -proof 只适用于 SEQ；-minimize regs 尝试减少寄存器，代价可能增加，不能理解为找到全局最小核心。-block 等待完成，-stop 停止。

用法与结果。核心必须与它支持的结果种类一起解释：完整证明核心、有界核心以及计算未完成不是同一种证据。该章节对默认增量行为、无 -depth 的状态选择和示例注释有不一致叙述，因此批处理示例显式选择 proven 属性；分析有界结果时显式给出深度并核对报告。

```
compute_formal_core -property [get_props -status proven] -block
report_formal_core -union -verbose
```

来源：User Guide，X-2025.06-SP1，第 862–864 页。

9.2 report_formal_core：报告证明涉及和未涉及的设计对象

```
report_formal_core [-subtype <subtypes>] [-property <properties>]
  [-list] [-no_summary] [-verbose] [-complement] [-union]
  [-runStatus <completed|scheduled>] [-status <proven|bounded|no_status>]
  [-exclude_constraints] [-goalIds <ids>] [-proof <proof_name>]
  [-comb] [-bit]
```

参数解释。 -union 求所选属性核心并集，-complement 查看其外部对象；-property 限定属性。-list、-verbose、-no_summary 控制显示；-runStatus 筛选计算进度，-status 筛选核心证据状态。-exclude_constraints 排除仅由约束贡献的寄存器/输入；SEQ 可用 -goalIds、-proof。-comb 包含组合逻辑，-bit 按位统计，两者不应同时使用。

用法与结果。环境约束与 set_constant 是隐式背景，不会全部显式列出；加密区域的寄存器也不一定显示。核心之外不代表功能无需验证，核心之内也不代表所有功能已检查。位级与字级统计分母不同，不能直接比较百分比。

```
report_formal_core -union -verbose
report_formal_core -complement -verbose
report_formal_core -comb -union
```

来源：User Guide，X-2025.06-SP1，第 864–868 页。

9.3 get_formal_core：返回形式核心对象集合

```
get_formal_core [-property <property>] [-subtype <subtype>]
  [-complement] [-registers] [-inputs] [-constraints]
```

参数解释。必须选择 -registers、-inputs、-constraints 中至少一种。-property 限定属性，-subtype 可选择 vacuity、witness 或普通组件；-complement 返回核心之外的对象。

用法与结果。使用工具集合函数查询结果。该接口不是获取“所有已验证代码行”，不能把寄存器集合直接当成需求覆盖率。

```
set outside [get_formal_core -property p_data_correct -registers -complement]
query_objects $outside
```

来源：User Guide，X-2025.06-SP1，第 869–870 页。

9.4 snip_formal_core_complement：创建切断核心外部寄存器的诊断任务

```
snip_formal_core_complement -property <property>
  [-par_task <parent_task>] [-new_task <new_task>]
```

参数解释。 -property 必填，指定已有核心结果的目标；-par_task 指定父任务，-new_task 命名新任务。

用法与结果。它用于判断核心之外的大量状态是否拖慢证明。切断会增加自由行为，出现失败可能是伪反例；该实验不能直接代替原模型的验收。应保留父任务不变，并审查新任务的抽象及结果传播关系。

```
compute_formal_core -property p_data_correct -block
snip_formal_core_complement -property p_data_correct \
  -par_task FPV -new_task core_only_experiment
```

来源：User Guide，X-2025.06-SP1，第 868–869 页。

9.5 check_config：在证明过程中同步计算形式核心

```
check_config [-formal_core <off|proven|inconclusive|both>]
```

参数解释。 -formal_core 选择对已证明、有界未完成、两者或都不计算。必须在 check_fv 前设置才会作用于这轮运行。

用法与结果。这是 check_config 的形式核心配置用法，不应与 check_fv_setup 或 fv_setup_config 混淆。它增加或调整分析证据的收集，不改变用户断言的规格含义。

```
check_config -formal_core proven
check_fv -block
report_formal_core
```

来源：User Guide，X-2025.06-SP1，第 870 页。

10. 参考

User Guide §20（形式验收/Signoff），§11（FCA），§21（FTA）。本章阐述通用工程验收方法，不应被解读为 Synopsys 或某行业标准的强制验收条件。

术语表、应用选择与官方资料定位

1. 术语表

英文	中文名称	准确理解
Formal verification	形式验证	对数字设计数学模型进行属性或等价性检查
DUT	待验证设计	被检查的 RTL/网表模块或系统
RTL	寄存器传输级描述	主要使用 Verilog/SystemVerilog/VHDL 编写的数字硬件模型
SVA	SystemVerilog 断言	描述布尔与时序规则的语言结构
Property	属性	可由工具管理的 assert、assume 或 cover 目标
Assertion	断言	DUT 必须满足的规定行为
Assumption	假设	对外部输入与初始条件等的合法性限制
Cover	覆盖目标	搜索某事件/状态是否能够发生
Counterexample	反例	可在当前模型下违反属性的输入/状态路径
Witness	见证	使某目标或性质成立的示范路径
Vacuity	空洞成立	属性成立但关键前提可能从未触发
COI	影响锥	能影响某个目标的逻辑和状态集合
BMC	有界模型检查	展开有限深度搜索违反属性的路径
PDR / IC3	属性引导可达性	用可达集合和归纳不变量等方法完成证明
Induction	数学归纳证明	基础情形与递推情形共同建立全时域性质
Abstraction	抽象	简化设计模型，可能扩大允许行为集合
Proof	证明	在指定模型和前提下建立属性的正确性
Inconclusive	未完成	当前资源或策略尚无明确结论
Noninterference	非干扰	高安全级输入不应影响未授权低安全级输出
Transactional equivalence	事务等价	按事务输入输出阶段对应比较两个实现
Waiver	豁免	经过审查的结果管理，不改变电路正确性
FMEDA	失效模式、影响及诊断分析	对故障后果、诊断能力等的系统分析
UPF	统一电源格式	描述电源域、隔离、保持与供电状态的文件
IP-XACT	IP 结构和寄存器描述规范	供工具交换接口与寄存器规范的数据格式
RALF	寄存器抽象层格式	寄存器定义及访问属性的数据格式

阅读报告时应保持术语层次一致：属性描述要证明的规则，目标是交给引擎的具体求解对象，约束限定允许行为，反例和覆盖见证则是具体轨迹。断言的证明状态与代码覆盖率属于不同对象，不应混合计数。

2. 遇到问题先选哪个应用

你的问题	推荐优先应用	常用补充应用
“这个状态机会不会进入非法状态?”	FPV	AEP
“第三方 RTL 有无明显结构性危险?”	AEP	FPV
“这个中断线是否按配置接到目的端口?”	CC	FPV
“为什么这个信号在启动后可能为 X?”	FXP	AEP / FPV
“重写的 RTL 是否行为一致?”	SEQ	FPV
“寄存器默认值、W1C 行为是否正确?”	FRV	FPV
“未覆盖分支是测试不够还是不可达?”	FCA	FPV
“现有断言能否发现故意注入的 RTL 错误?”	FTA	FPV
“安全故障能否被诊断发现?”	FuSa	故障仿真
“密钥是否可能影响普通调试输出?”	FSV	FPV
“C++ 算法和多拍流水线 RTL 是否一致?”	DPV	SEQ
“电源关断后隔离是否正确?”	FLP / LPC	FPV / CC

先依据验证问题选择应用，再考虑共享的编译、时钟、复位和报告命令。同一模块经常需要多种应用：例如寄存器模块可以使用 FRV 检查访问规则，用 FPV 检查业务副作用，再用 FXP 分析复位后未知值是否外泄。

3. 官方 UG 章节与页码

UG 章	英文主题	页码	本教程对应文件
1	About VC Formal	31–52	形式验证
2	Setting up VC Formal Design Environment	53–65	平台与环境
3	Managing Constraints and Properties	66–114	SVA
4	Compiling Designs and Verifying Formal Setup	115–170	工程与结果
5	Regression Mode Accelerator	171–182	调试与性能
6	Performing and Configuring Checks	183–246	工程与结果
7	Debugging Results	247–312	调试与性能
8–9	Work Environment; Navigator	313–361	平台与环境
10	Automatically Extracted Properties	362–385	AEP
11	Formal Coverage Analyzer	386–443	FCA
12	Formal X-Propagation	444–457	FXP
13	Connectivity Checking	458–516	CC

UG 章	英文主题	页码	本教程对应文件
14	Sequential Equivalence Checking	517–611	SEQ
15	Supported Assertion IPs	612–618	AIP 与分解证明
16	Formal Register Verification	619–685	FRV
17	Formal Property Verification	686–713	FPV
18–19	Convergence; Proof Architect	714–835	调试与性能
20	Obtaining Formal Signoff	836–913	覆盖与验收
21	Formal Testbench Analyzer	914–960	FTA
22	Security Verification	961–1008	FSV
23	Functional Safety	1009–1039	FuSa
24	Formal Low Power	1040–1065	FLP/LPC
25	Data Path Validation	1066–1080	DPV
26	查询命令和集合	1081–1109	设计查询与集合
27–33	SVA、示例、变量与调试附录	1110–1168	通用命令详解

页码均指提供的 X-2025.06-SP1 文档。命令章节将原始调用形式、参数含义和适用条件分开说明；没有在该文档中给出完整选项的命令，只整理已经出现的用法，不据命令名称推测额外参数。

4. 确认过的重要版本差异

1. FCA 是应用名称；该版本 User Guide 使用 `set_fml_appmode COV`，Quick Reference Guide 写为 `FCA`。
2. FTA 一般先在 FPV 会话准备，再通过 `set_fml_appmode FTA` 切换；User Guide 第 65 页说明不能从 `vcf -fmode FTA` 直接启动。
3. DPV 的新 C++ 前端变量写法在两份参考资料中不同，User Guide 第 1069 页给出 `set _hector_comp_use_new_flow true`，需要核验安装环境。
4. 普通 CC 的连接延迟能力不能照搬到 FLP CC：User Guide 第 1041 页明确指出 FLP CC 不支持延迟连接检查。
5. 用户原始笔记包含部分具备本地部署假设的命令，整理后的正式手册在这些位置增加了“教学模板/版本核验/输入文件必要条件”的说明。

5. 文档范围与版权说明

本文为根据用户提供的资料、公开硬件验证概念和作者独立说明编写的原创中文学习材料。它没有声称完整翻译 Synopsys 的 1,168 页官方文档，也没有代替受许可证保护的命令参考书。两份 PDF 是 Synopsys 所有的受保护材料，请在许可证约定范围内保管与使用；本次交付不重新打包原始 PDF。具体命令以实际安装版本的 `-help`、相关专用参考指南和厂商支持为准。